代码随想录知识星球精华（最强八股文）第四版（计算机基础篇））

代码随想录知识星球精华（最强八股文）第四版为九份PDF，分别是：

- 代码随想录知识星球八股文概述
- C++篇
- go篇
- Java篇
- 前端篇
- 算法题篇
- 计算机基础篇
- 问答精华篇
- 面经篇

本篇为最强八股文之计算机基础篇。

计算机网络

OSI 七层参考模型

七层模型，亦称 OSI（Open System Interconnection）参考模型，即开放式系统互联。参考模型是国际标准化组织（ISO）制定的一个用于计算机或通信系统间互联的标准体系，一般称为 OSI 参考模型或七层模型。

它是一个七层的、抽象的模型体，不仅包括一系列抽象的术语或概念，也包括具体的协议。可以简称为“物数网传会表应”。

1. **物理层**：主要定义物理设备标准，如网线的接口类型、光纤的接口类型、各种传输介质的传输速率等。它的主要作用是传输比特流（就是由 1、0 转化为电流强弱来进行传输，到达目的地后再转化为 1、0，也就是我们常说的数模转换与模数转换）。这一层的数据叫做比特。
2. **数据链路层**：建立逻辑连接、进行硬件地址寻址、差错校验等功能。定义了如何让格式化数据以帧为单位进行传输，以及如何控制对物理介质的访问。将比特组合成字节进而组合成帧，用 MAC 地址访问介质。
3. **网络层**：进行逻辑地址寻址，在位于不同地理位置的网络中的两个主机系统之间提供连接和路径选择。Internet 的发展使得从世界各站点访问信息的用户数大大增加，而网络层正是管理这种连接的层。
4. **传输层**：定义了一些传输数据的协议和端口号（ WWW 端口 80 等），如：TCP（传输控制协议，传输效率低，可靠性强，用于传输可靠性要求高，数据量大的数据），UDP（用户数据报协议，与 TCP 特性恰恰相反，用于传输可靠性要求不高，数据量小的数据，如 QQ 聊天数据就是通过这种方式传输的）。 主要是将从下层接收的数据进行分段和传输，到达目的地址后再进行重组。常常把这一层数据叫做段。
5. **会话层**：通过传输层（端口号：传输端口与接收端口）建立数据传输的通路。主要在你的系统之间发起会话或者接受会话请求。
6. **表示层**：数据的表示、安全、压缩。主要是进行对接收的数据进行解释、加密与解密、压缩与解压缩等（也就是把计算机能够识别的东西转换成人能够能识别的东西（如图片、声音等）。
7. **应用层**：网络服务与最终用户的一个接口。这一层为用户的应用程序（例如电子邮件、文件传输和终端仿真）提供网络服务。

在浏览器中输入网址后发生了什么

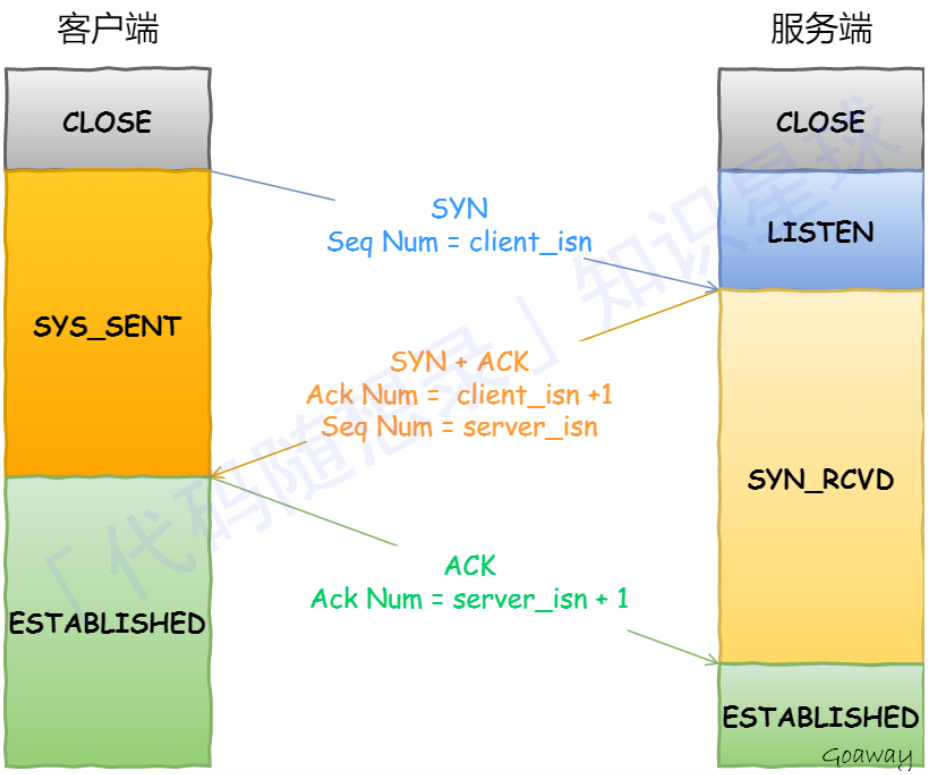
1. 解析URL
2. 生成HTTP请求消息
3. DNS查询
4.

三次握手、四次挥手（TCP 传输层）

> 重要程度：⭐⭐⭐

三次握手

三次握手：服务端先从close到listen。



1、第一个SYN报文：

客户端随机初始化序列号client_isn，放进TCP首部序列号段，然后把SYN置1。把SYN报文发送给服务端，表示发起连接，之后客户端处于SYN-SENT状态。

2、第二个报文SYN+ACK报文：

服务端收到客户端的SYN报文，把自己的序号server_isn放进TCP首部序列号段，确认应答号填入client_isn + 1，把SYN和ACK置1。把SYN+ACK报文发送给客户端，然后进入SYN-RCVD状态。

3、第三个报文ACK：

客户端收到服务端报文后，还要向服务端回应最后一个应答报文。首先该应答报文 TCP 首部 ACK 标志位置为 1，其次「确认应答号」字段填入 server_isn + 1，最后把报文发送给服务端，这次报文可以携带客户到服务器的数据，之后客户端处于 ESTABLISHED 状态。

服务器收到客户端的应答报文后，也进入 ESTABLISHED 状态。

为什么需要三次握手?

总结：

1. 三次握手才可以阻止重复历史连接的初始化(主因)
2. 三次握手才可以同步双方的初始序列号
3. 三次握手才可以避免资源浪费

解释：

1、阻止重复历史连接的初始化（主因）

1. 当旧的SYN报文先到达服务端，服务端回一个ACK+SYN报文
2. 客户端收到后可以根据自身的上下文，判断这是一个历史连接（序列号过期或超时），那么客户端就会发送 RST

报文给服务端，表示中止这一次连接。

两次握手在收到服务端的响应后开始发生数据，不能判断当前连接是否是历史连接。

三次握手可以在客户端**准备发送第三次报文时**，客户端因有足够的上下文来判断当前连接是否是历史连接。

2、同步双方的初始序列号

TCP 协议的通信双方， **都必须维护一个「序列号」**， 序列号是可靠传输的一个关键因素。

- 接收端可以去除重复数据。
- 接收端可以按照序列号顺序接收。
- 标识发送的数据包，哪些已经被收到。

过程

1. 客户端发送第一个报文，携带客户端初始序列号的SYN报文。
2. 服务器发送第二个报文，携带服务器初始序列号的ACK + SYN的应答报文，表示收到客户端的SYN报文。
3. 客户端发送第三个报文，携带服务器的ACK应答报文。

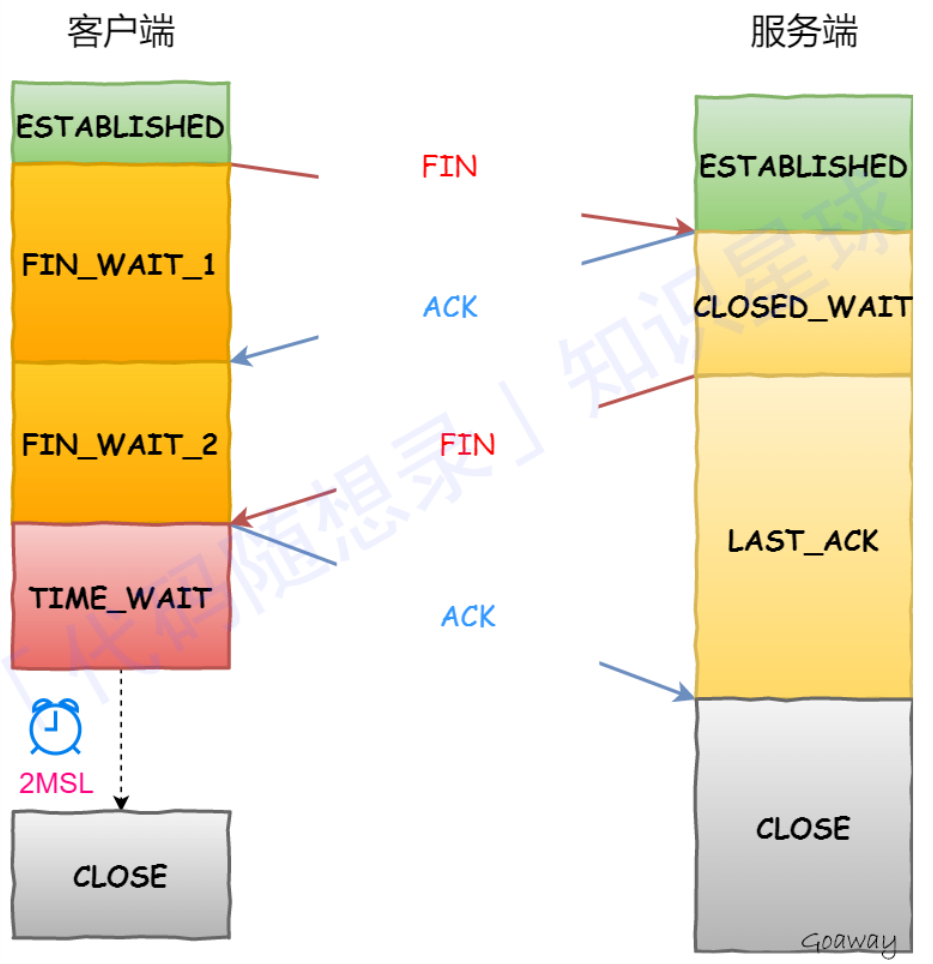
这样一来一回，才能确保双方的初始序列号能被可靠的同步。

两次握手只保证了一方的初始序列号能被对方成功接收，没办法保证双方的初始序列号都能被确认接收。

3、避免资源浪费。

1. 两次握手会造成消息滞留情况下，服务器重复接受无用的连接请求 SYN 报文，而造成重复分配资源。
2. 只有两次握手时，如果客户端的SYN请求连接在网络中阻塞，客户端没有收到服务端的ACK报文，会重新发送SYN。
3. **由于没有第三次握手，服务器不清楚客户端是否收到了自己发送的建立连接的 ACK 确认信号，所以每收到一个 SYN 就只能先主动建立一个连接。**

四次挥手



1、断开过程

1. 假设客户端打算关闭连接，发送一个TCP首部FIN被置1的FIN报文给服务端。
2. 服务端收到以后，向客户端发送ACK应答报文。
3. 等待服务端处理完数据后，向客户端发送FIN报文。
4. 客户端接收到FIN报文后回一个ACK应答报文。
5. 服务器收到ACK报文后，进入close状态，服务器完成连接关闭。
6. 客户端在经过 2MSL 一段时间后，自动进入close状态，客户端也完成连接的关闭。

2、为什么挥手需要四次?

关闭连接时，客户端发送FIN报文，表示其不再发送数据，但还可以接收数据。

客户端收到FIN报文，先回一个ACK应答报文，服务端可能还要数据需要处理和发送，等到其不再发送数据时，才发送FIN报文给客户端表示同意关闭连接。

从上面过程可知:

1. 服务端通常需要等待完成数据的发送和处理，所以服务端的ACK和FIN一般都会分开发送，从而比三次握手导致多了一次。
2. 第一次ACK应答报文可以省略，因为下一个报文段携带了ACK信息，ACK是否出现取决于延迟确认特性。
3. 延迟确认：即接收方收到包后，如果暂时没有内容回复给发送方，则延迟一段时间再确认，假如在这个时间范围内刚好有数据需要传输，则和确认包一起回复。这种也被称为数据捎带。延迟确认只是减轻网络负担，未必可以提升网络性能，有些情况下反而会影响性能。

3、为什么 TIME_WAIT 等待的时间是 2MSL?

1. MSL是 Maximum Segment Lifetime，报文最大生存时间，它是任何报文在网络上存在的最长时间，超过这个时间报文将被丢弃。
2. 等待MSL两倍：网络中可能存在发送方的数据包，当这些发送方的数据包被接收方处理后又会向对方发送响应，所以一来一回需要等待 2 倍的时间。
3. 2MSL 的时间是从客户端接收到 FIN 后发送 ACK 开始计时的。如果在 TIME-WAIT 时间内，因为客户端的 ACK 没有传输到服务端，客户端又接收到了服务端重发的 FIN 报文，那么 2MSL 时间将重新计时。

4、为什么需要 TIME_WAIT 状态?

主动发起关闭连接的一方，才会有 TIME-WAIT 状态。

需要 TIME-WAIT 状态，主要是两个原因：

1. 防止具有相同「四元组」的「旧」数据包被收到
2. 保证「被动关闭连接」的一方能被正确的关闭，即保证最后的 ACK 能让被动关闭方接收，从而帮助其正常关闭

有相同端口的 TCP 连接被复用后，被延迟的相同四元组的数据包抵达了客户端，那么客户端是有可能正常接收这个过期的报文，这就会产生数据错乱等严重的问题。

经过 2MSL 这个时间，**足以让两个方向上的数据包都被丢弃，使得原来连接的数据包在网络中都自然消失，再出现的数据包一定都是新建立连接所产生的**。

最后的ACK如果丢失，客户端直接进入close，服务端一直在等待ACK状态。当客户端发起建立连接的SYN请求，服务端会发送RST报文回应，连接建立会关闭。

如果 TIME-WAIT 等待足够长的情况就会遇到两种情况：

1. 服务端正常收到四次挥手的最后一个 ACK 报文，则服务端正常关闭连接。
2. 服务端没有收到四次挥手的最后一个 ACK 报文时，则会重发 FIN 关闭连接报文并等待新的 ACK 报文。

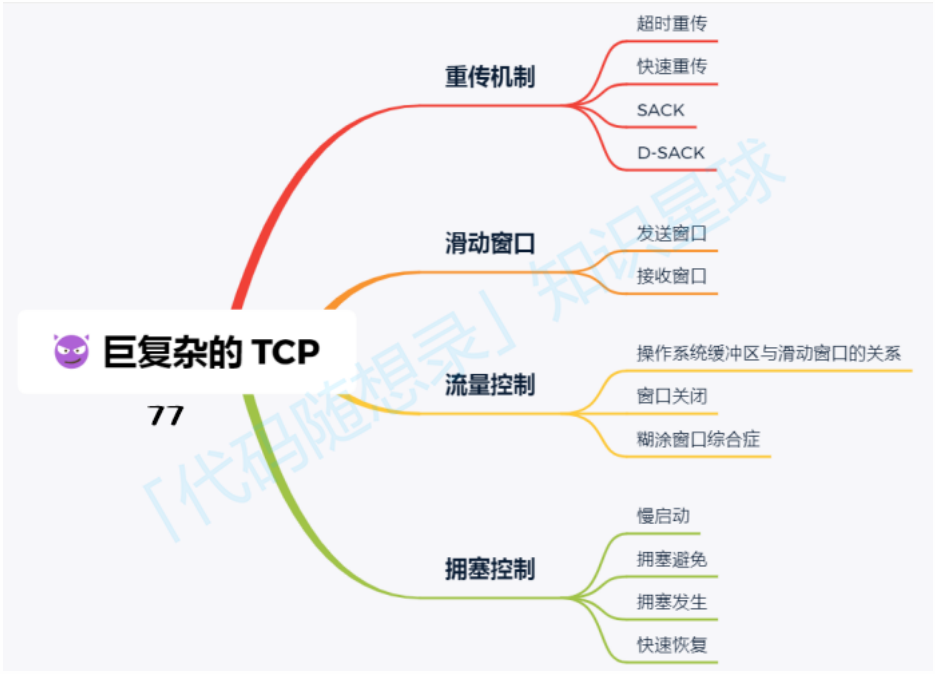
5、TIME_WAIT 过多有什么危害?

过多的TIME-WAIT 状态主要的危害有两种：

1. 内存资源占用；
2. 对端口资源的占用，一个 TCP 连接至少消耗一个本地端口；

如果发起连接一方的 TIME_WAIT 状态过多，占满了所有端口资源，则会导致无法创建新连接。

TCP相关知识点：

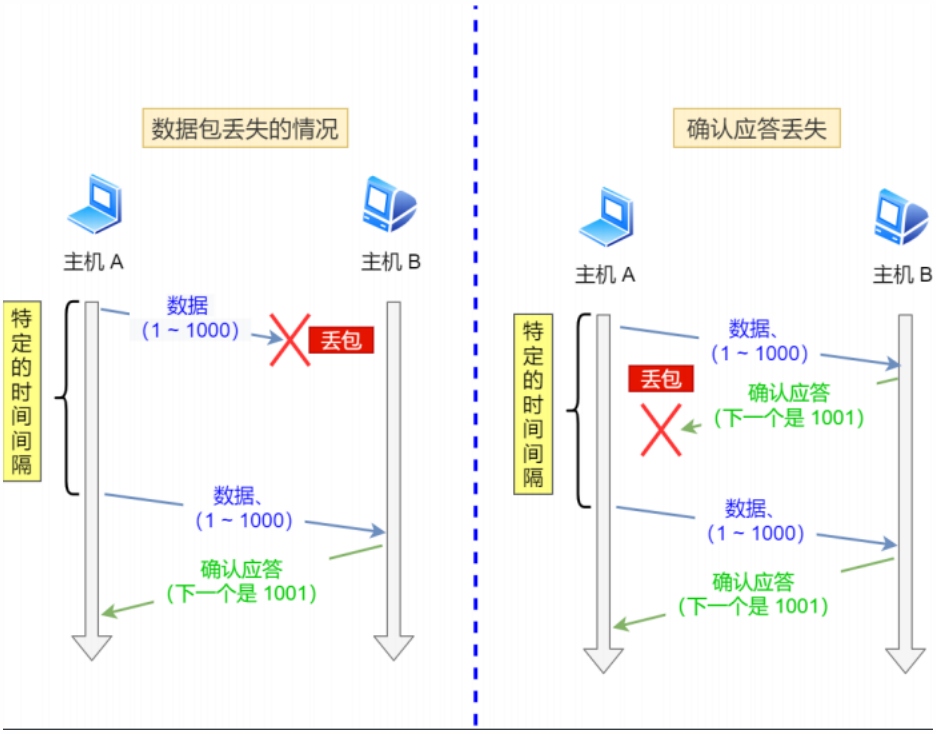


重传机制（TCP 传输层）

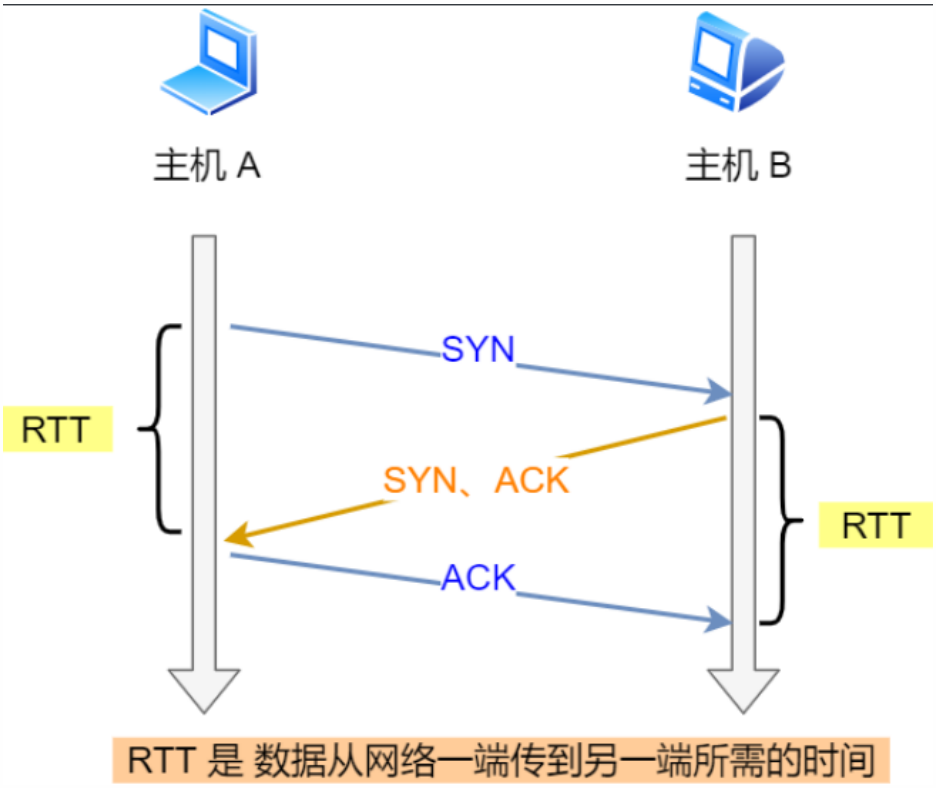
1、超时重传：

设定一个计时器，当超过指定的时间后，没有收到对方的确认ACK应答报文，就会重发该数据。

超时重传的两种情况如下：

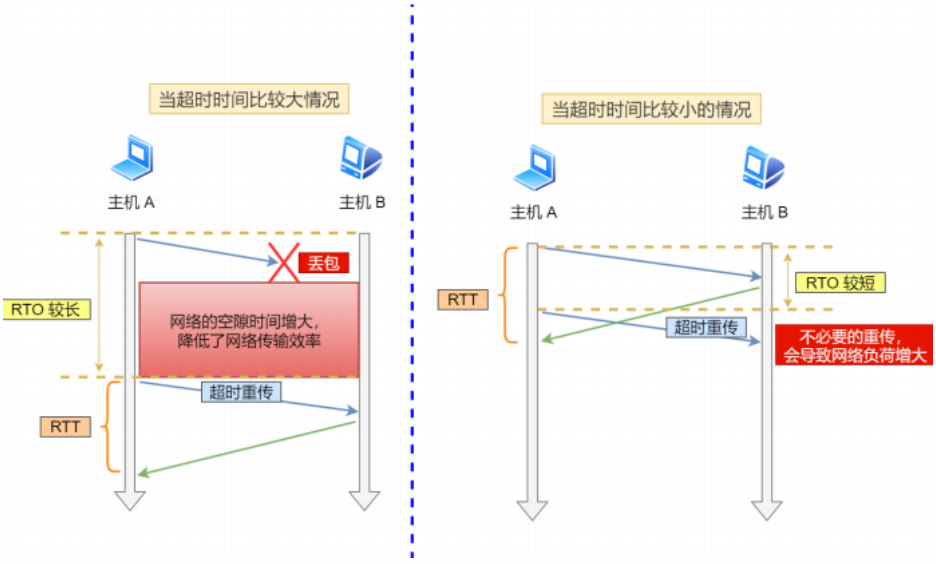


RTT(round-trip time):往返时延

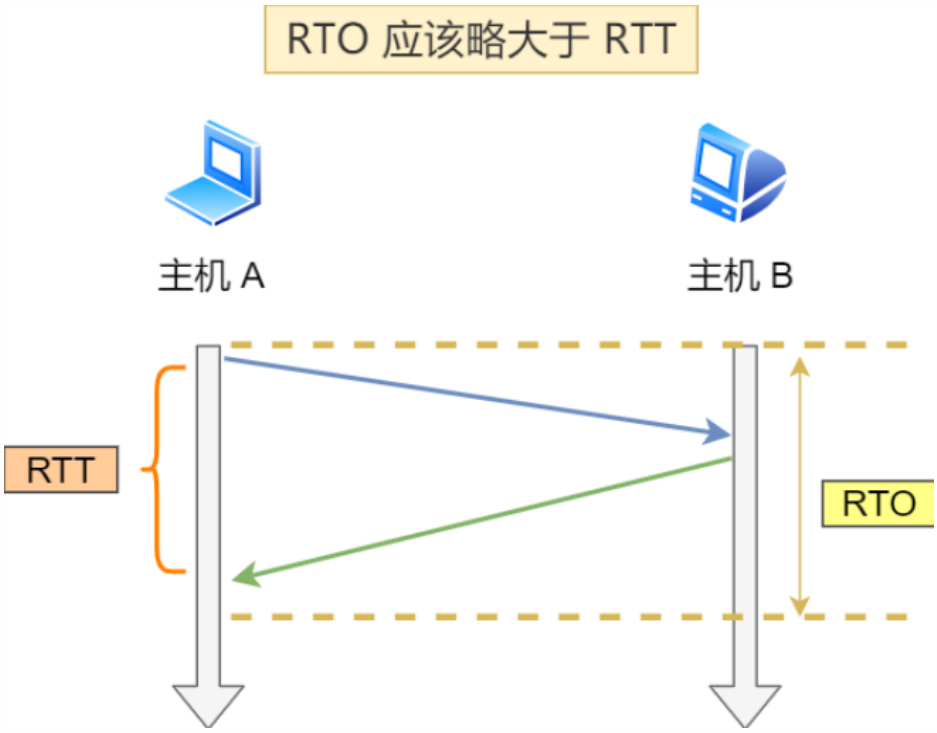


RTO（Retransmission Timeout）：超时重传时间

RTO较长或较短可能发生的情况如下：



由上图可知：超时重传时间RTO的值 应该略大于 报文往返RTT的值，且是动态变化的。



估计往返时间，通常需要采样以下两个：

- 需要 TCP 通过采样 RTT 的时间，然后进行加权平均，算出一个平滑 RTT 的值，而且这个值还是要不断变化的，因为网络状况不断地变化。
- 除了采样 RTT，还要采样 RTT 的波动范围，这样就避免如果 RTT 有一个大的波动的话，很难被发现的情况。

RTO的计算方式：

① 首次计算 RTO，其中 R1 为第一次测量的 RTT

$SRTT = R1$

$DevRTT = R1/2$

$RTO = \mu * SRTT + \partial * DevRT = \mu * R1 + \partial * (R1/2)$

② 后续计算 RTO，其中 R2 为最新测量的 RTT

$SRTT = SRTT + \alpha (RTT - SRTT) = R1 + \alpha * (R2 - R1)$

$DevRTT = (1-\beta)*DevRTT + \beta *(|RTT-SRTT|) =(1-\beta) * (R1/2) + \beta * (|R2 - R1|)$

$RTO = \mu * SRTT + \partial * DevRTT$

其中 SRTT 是计算平滑的RTT， DevRTR 是计算平滑的RTT 与 最新 RTT 的差距。

在 Linux 下，$\alpha = 0.125$，$\beta = 0.25$， $\mu = 1$，$\partial = 4$。别问怎么来的，问就是大量实验中调出来的。

如果超时重发的数据，再次超时又要重传的时候，TCP的策略是将超时间隔加倍。

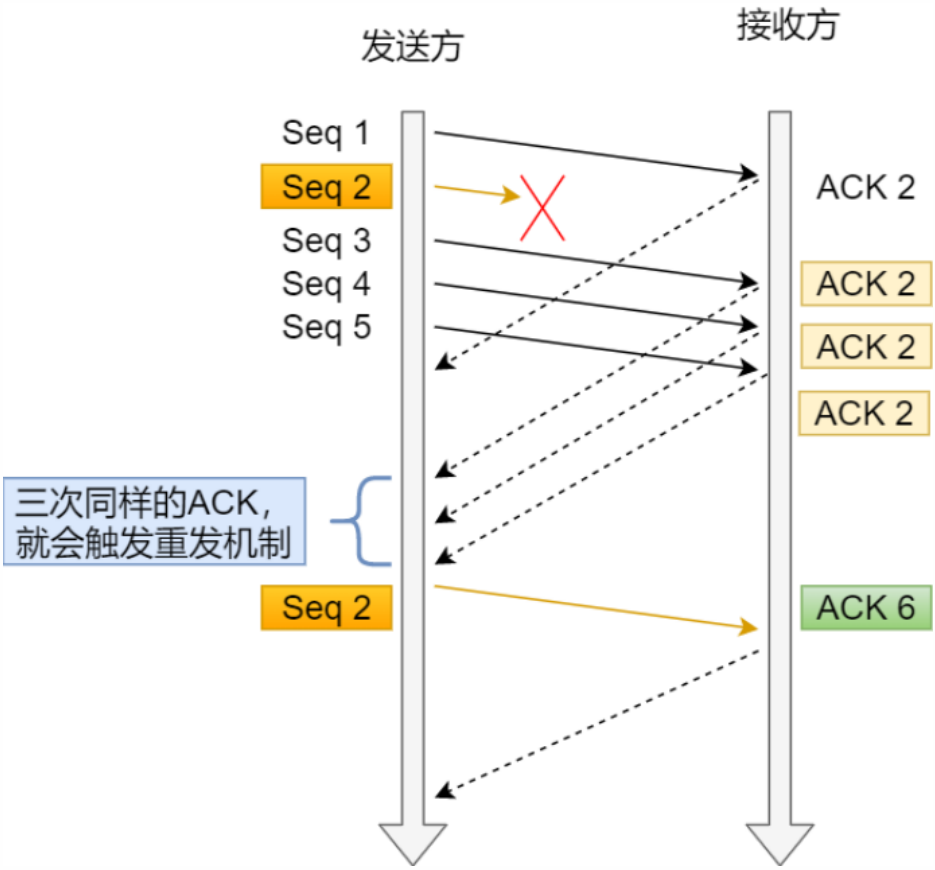
2、快速重传：

以数据驱动重传。

工作原理：

当收到三个相同的ACK报文时，会在定时器过期之前，重传丢失的报文段。

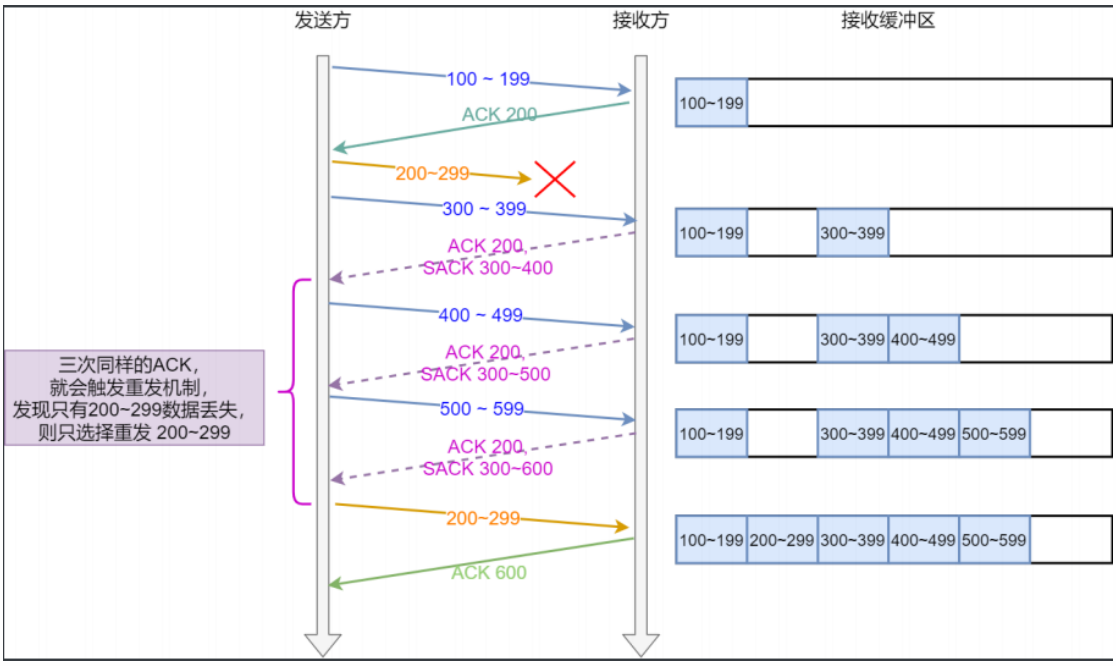
快速重传解决了超时时间的问题，但还面临另外一个问题：重传的时候，是重传之前的一个，还是重传所有的问题。



3、SACK（Selective Acknowledgment,选择性确认）：

为了解决 重传哪些报文的问题而提出。

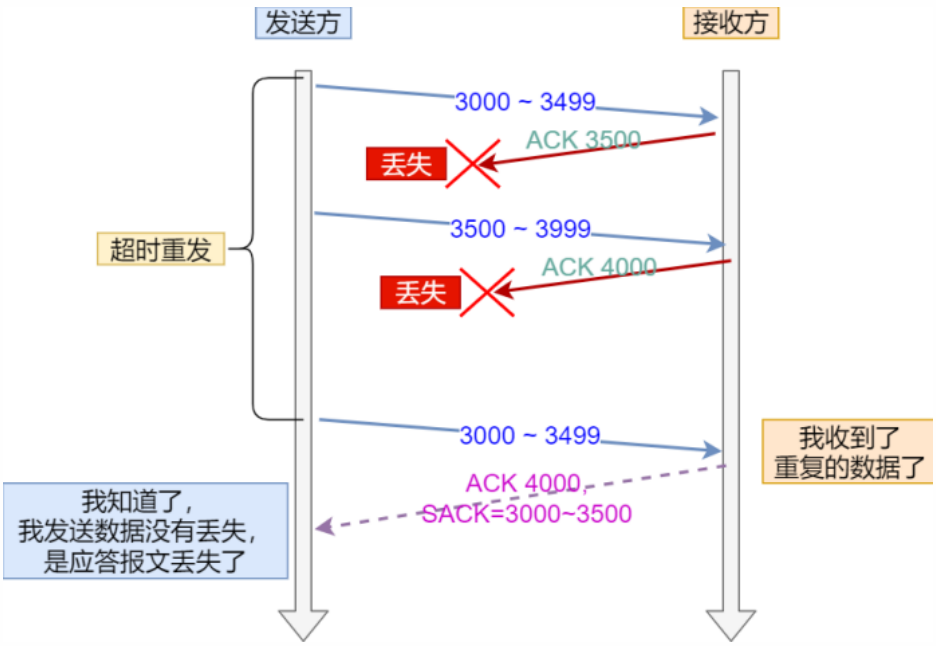
这种方式需要在 TCP 头部「选项」字段里加一个 SACK 的东西，它可以将缓存的地图发送给发送方，这样发送方就可以知道哪些数据收到了，哪些数据没收到，知道了这些信息，就可以只重传丢失的数据。



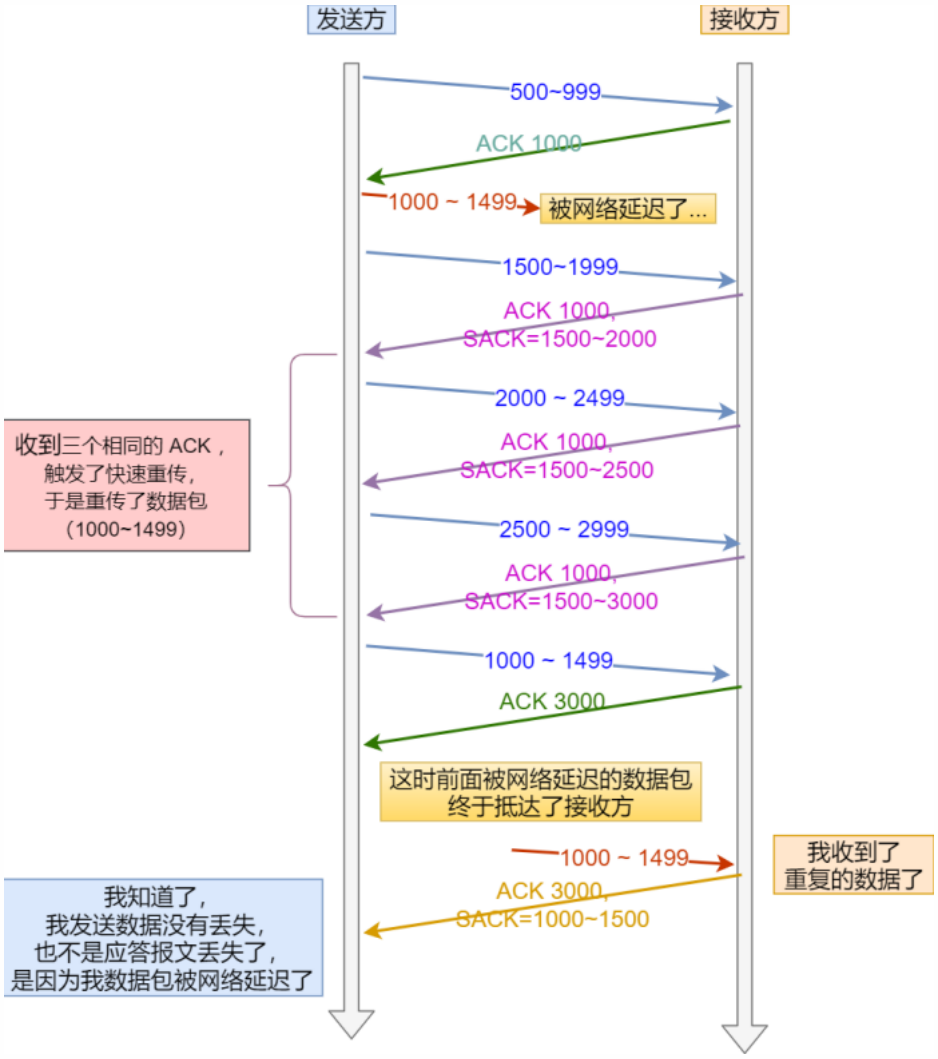
4、D-SACK（Duplicate SACK）：

主要使用了SACK来高罗素【发送方】有哪些数据被重复接收了。下面举例来说明D-SACK的作用：

ACK丢包：



网络延时：

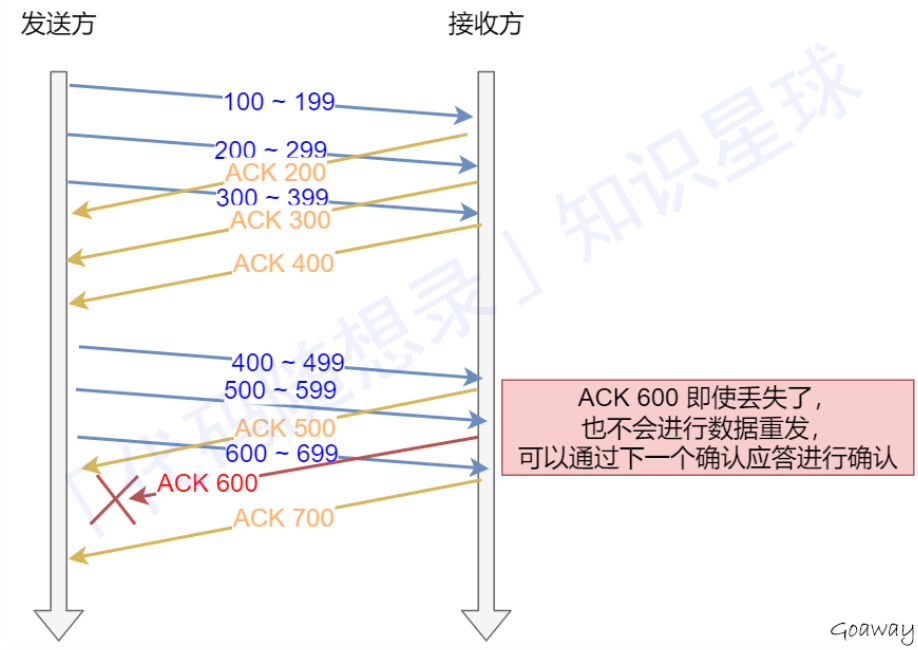


使用D-SACK的好处：

（1）可以让【发送方】知道，是发出去的包丢了，还是接收方回应的ACK包丢了；
（2）可以知道是不是【发送方】的数据包被网络演示了；
（3）可以知道网络中是不是把【发送方】的数据包给复制了。

滑动窗口（TCP 传输层）

1. 什么是窗口



TCP每发送一个数据，都需要一次应答，然后继续发送，这样为每个数据包都进行确认应答，**缺点是：数据往返时间越长，网络吞吐量越低。**

引入窗口即使往返时间较长，也不会降低网络通信效率。

可以指定窗口大小，窗口大小就是无需等待确认应答，继续发送数据的最大值。

窗口实现就是操作系统开辟的一个缓存空间，发送方主机在等到确认应答返回之前，必须在缓冲区中保留已发送的数据。如果按期收到确认应答，此时数据就可以从缓存区清除。

图中的 ACK 600 确认应答报文丢失，也没关系，因为可以通过下一个确认应答进行确认，只要发送方收到了 ACK 700 确认应答，就意味着 700 之前的所有数据「接收方」都收到了。这个模式就叫**累计确认或者累计应答。**

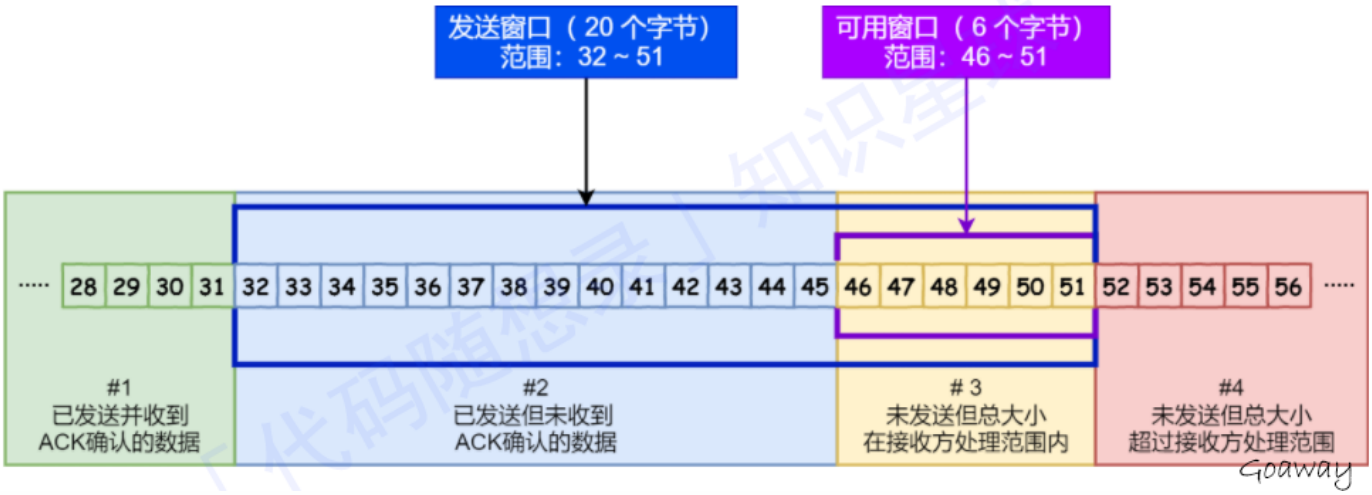
2. 什么决定窗口大小

TCP头部有一个字段叫window，窗口大小。

这个字段是接收端告诉发送端自己还有多少缓冲区可以接收数据。于是发送端就可以根据这个接收端的处理能力来发送数据，而不会导致接收端处理不过来

通常窗口的大小是由接收方的窗口大小来决定的。

3. 发送方的滑动窗口

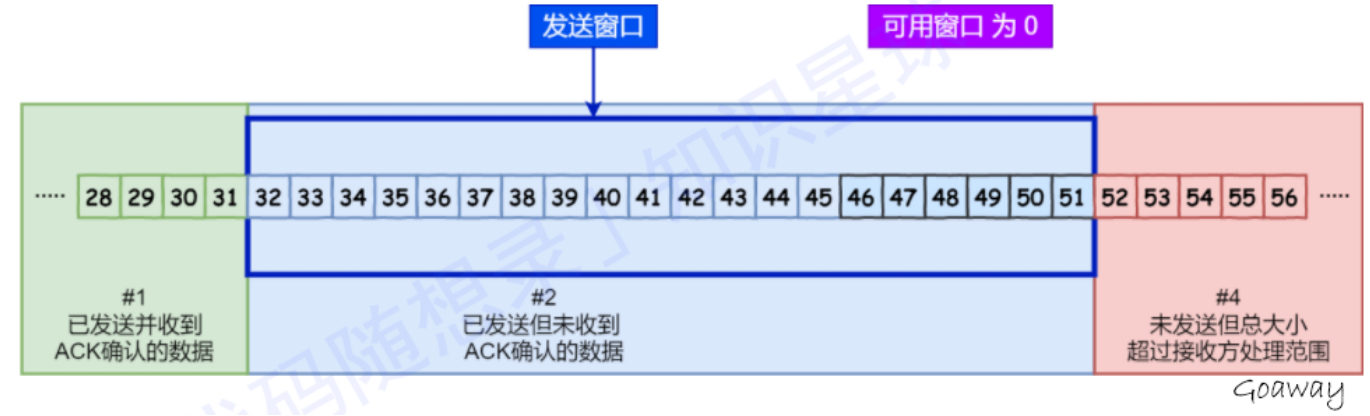


#1 是已发送并收到 ACK确认的数据：1到31 字节

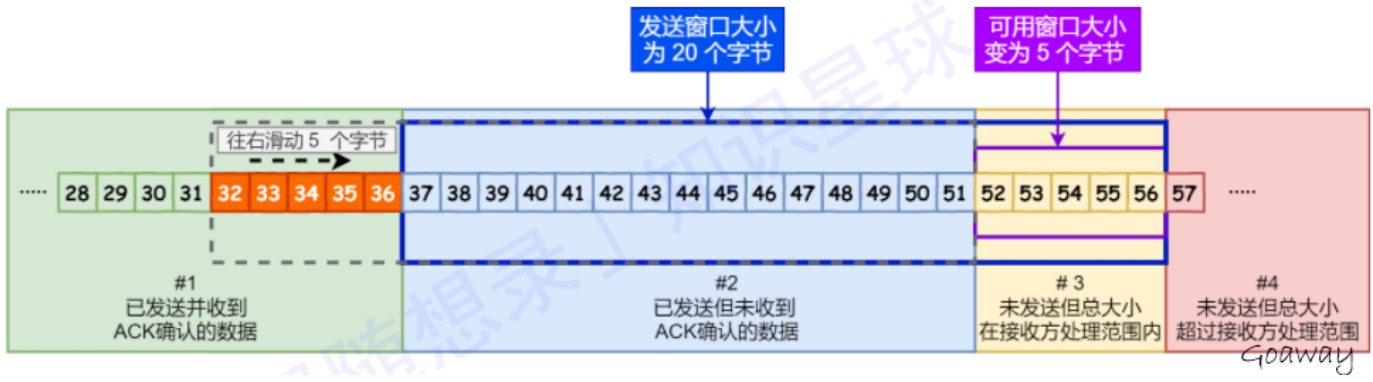
#2 是已发送但未收到 ACK确认的数据：32到45 字节

#3 是未发送但总大小在接收方处理范围内（接收方还有空间）：46到51字节

#4 是未发送但总大小超过接收方处理范围（接收方没有空间）：52字节以后

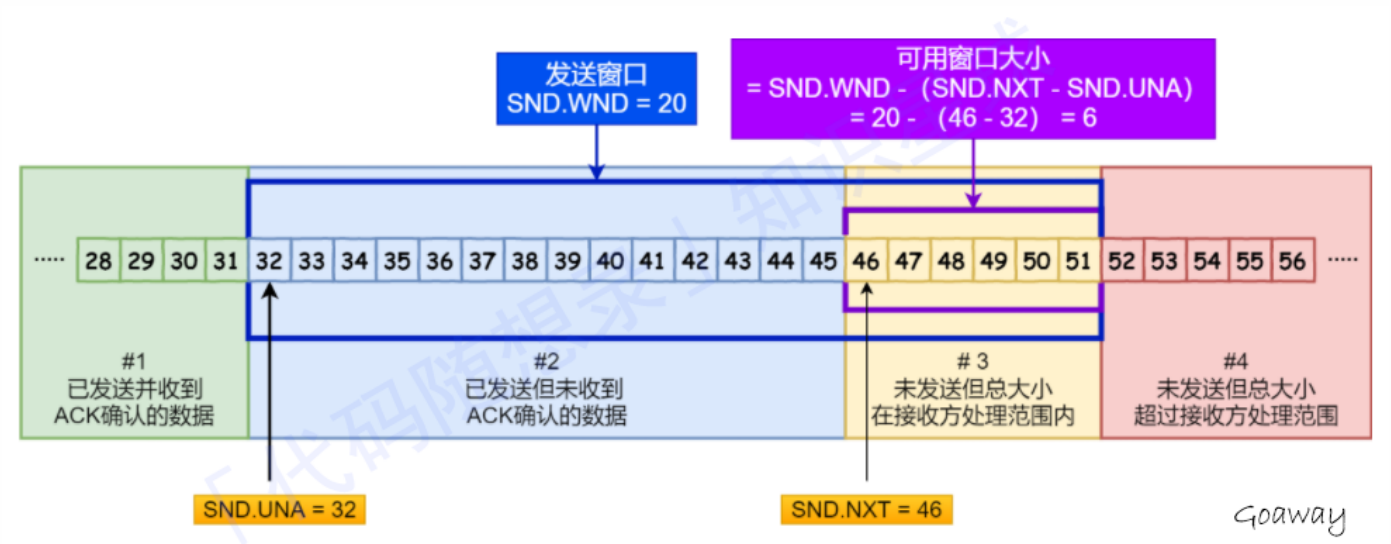


数据全都发送以后，可用窗口大小为0，在没收到ACK确认应答之前无法继续发送数据。



在收到32-36的ACK应答后，如果窗口大小不变，则滑动窗口先后移动5个字节，那么52-56变成可用窗口，可以继续发送数据。

4. 程序是如何表示发送方的四个部分的呢?

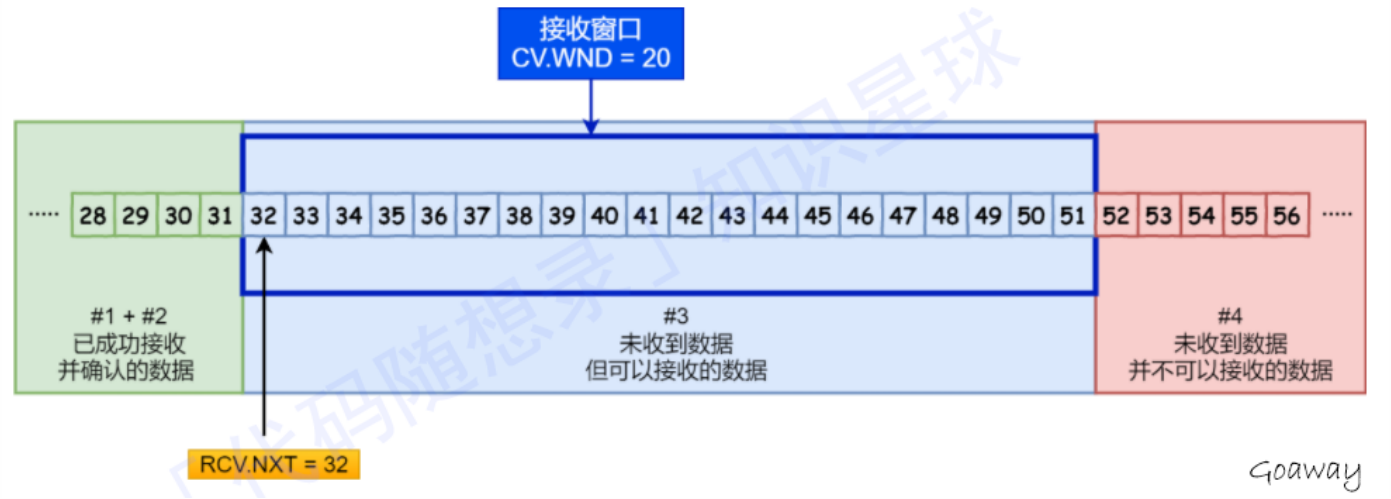


- SND.WND：表示发送窗口的大小（大小是由接收方指定的）；
- SND.UNA：是一个绝对指针，它指向的是已发送但未收到确认的第一个字节的序列号，也就是 #2 的第一个字节。
- SND.NXT：也是一个绝对指针，它指向未发送但可发送范围的第一个字节的序列号，也就是 #3 的第一个字节。
- 指向 #4 的第一个字节是个相对指针，它需要 SND.UNA 指针加上 SND.WND 大小的偏移量，就可以指向 #4 的第一个字节了。

可用窗口大小 = 发送窗口 - 已发送未确定(SND.WND -（SND.NXT - SND.UNA))

已发送未确认 =（SND.NXT - SND.UNA)

5. 接收方滑动窗口



#1 + #2 是已成功接收并确认的数据（等待应用进程读取）

#3 是未收到数据但可以接收的数据

#4 未收到数据并不可以接收的数据

三部分两个指针：

- RCV.WND：表示接收窗口的大小，它会通告给发送方。

- RCV.NXT：是一个指针，它指向期望从发送方发送来的下一个数据字节的序列号，也就是 #3 的第一个字节。
- 指向 #4 的第一个字节是个相对指针，它需要 RCV.NXT 指针加上 RCV.WND 大小的偏移量，就可以指向 #4 的第一个字节了。

拥塞控制

拥塞控制通过拥塞窗口来防止过多的数据注入网络，使得网络中的路由器或者链路过载。

- 拥塞窗口cwnd是发送方维护的一个状态变量，根据网络拥塞程度而变化。
- 发送窗口的值是swnd = min(cwnd, rwnd)，也就是拥塞窗口和接收窗口中的最小值。
- 网络中没有出现拥塞，cwnd增大，出现拥塞，cwnd减小。

其实只要发送方没有在规定时间内接收到 ACK 应答报文，也就是发生了超时重传，就会认为网络出现了拥塞。

拥塞控制的常见算法：

1. 慢启动
2. 拥塞避免
3. 拥塞发生
4. 快速恢复

1. 慢启动

慢启动的算法记住一个规则就行：**当发送方每收到一个 ACK，拥塞窗口 cwnd 的大小就会加 1**。

慢启动算法，发包的个数是指数性的增长。1->2->4->8->2的次方。

有一个慢启动门限ssthresh状态变量：

1. 当 cwnd < ssthresh 时，使用慢启动算法。
2. 当 cwnd >= ssthresh 时，就会使用拥塞避免算法。

2. 拥塞避免

一般来说 ssthresh 的大小是65535字节。超过后会进入拥塞避免算法。

那么进入拥塞避免算法后，它的规则是：**每当收到一个 ACK 时，cwnd 增加1/cwnd**。

拥塞避免算法就是将原本慢启动算法的指数增长变成了线性增长，还是增长阶段，但是增长速度缓慢了一些。

网络就会慢慢进入了拥塞的状况了，于是就会出现丢包现象，这时就需要对丢失的数据包进行重传。

当触发了重传机制，也就进入了拥塞发生算法。

3. 快重传

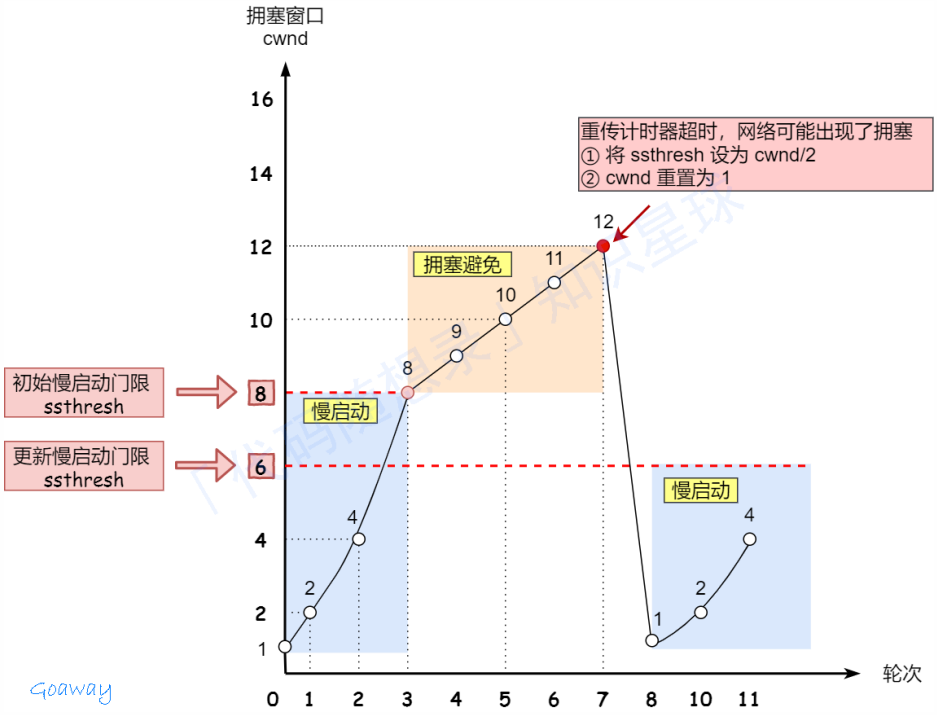
发生拥塞，进行数据重传：

1. 超时重传
2. 快速重传

（1）发生超时重传的发生算法：

这个时候，ssthresh 和 cwnd 的值会发生变化

1. ssthresh 设为 cwnd/2，
2. cwnd 重置为 1



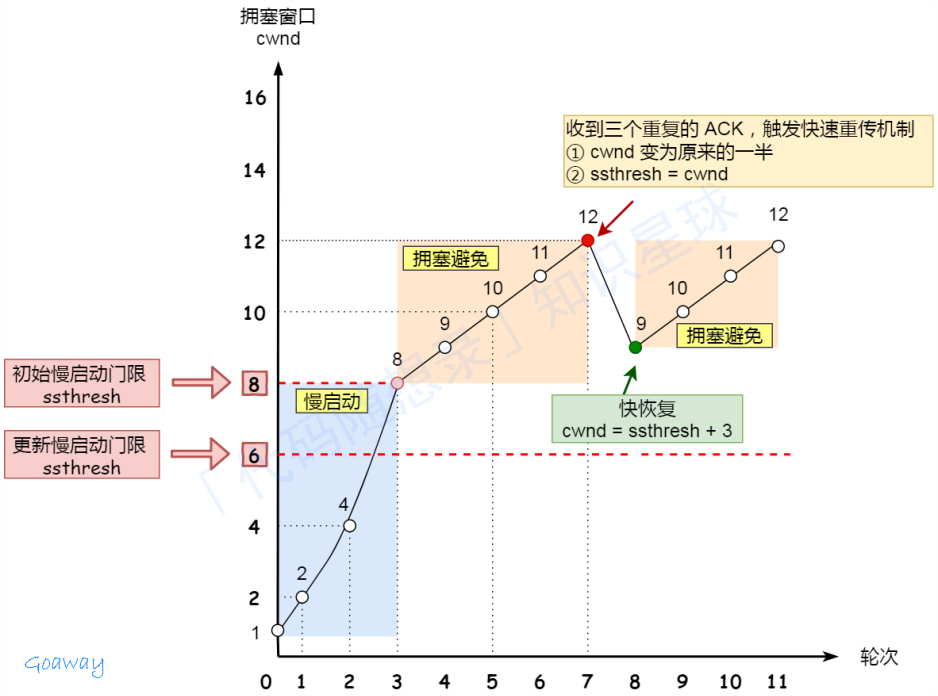
突然回到慢启动导致网络卡顿。

（2）快速重传拥塞算法：

TCP 认为这种情况不严重，因为大部分没丢，只丢了一小部分，则 ssthresh 和 cwnd 变化如下：

1. cwnd = cwnd/2 ，也就是设置为原来的一半
2. ssthresh = cwnd
3. 进入快速恢复算法

4. 快恢复



快速重传和快速恢复算法一般同时使用，快速恢复算法是认为，还能收到 3 个重复 ACK 说明网络也不那么糟糕，所以没有必要像 RTO 超时那么强烈。

进入快速恢复：

1. 拥塞窗口 cwnd = ssthresh + 3 （3 的意思是确认有 3 个数据包被收到了）；
2. 重传丢失的数据包；
3. 如果再收到重复的 ACK，那么 cwnd 增加 1；
4. 如果收到新数据的 ACK 后，把 cwnd 设置为第一步中的 ssthresh 的值，原因是该 ACK 确认了新的数据，说明从 duplicated ACK 时的数据都已收到，该恢复过程已经结束，可以回到恢复之前的状态了，也即再次进入拥塞避免状态；

而是还在比较高的值，后续呈线性增长。

TCP 与 UDP

1. TCP与UDP的区别

1、连接

TCP是面向连接的，在传输前需要三次握手建立连接，UDP不需要连接，即刻传输数据。

2、服务形式

TCP只能一对一，点对点服务，UDP支持一对一、一对多、多对多通信。

3、可靠性

TCP保证数据可靠交付，拥有确认应答和重传机制，无重复、不丢失、按序到达;
UDP尽可能交付，不保证可靠性。

4、连接控制机制

TCP拥有流量控制、拥塞控制，保证传输安全性等，UDP在网络拥堵情况下不会降低发送速率。

5、首部大小

TCP首部长度不适用选项字段是20字节，使用选项字段长度增加(可变)，UDP首部固定8字节。

6、传输方式

TCP基于字节流，没有边界，但是保证传输顺序和可靠性;
UDP继承了IP层特性，基于数据包，有边界可能出现乱序和丢包。

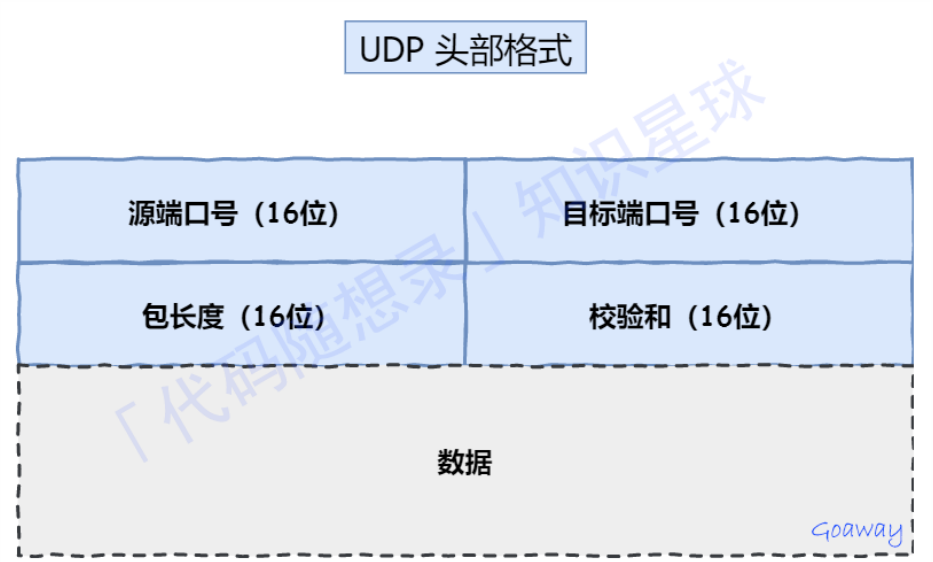
7、分片方式

TCP数据大于MSS时会在TCP层将数据进行分片传输，到达目的地后同样在传输层进行合并，如果有某个片丢失则只需要重传丢失的分片即可;

UDP数据大于MTU时会在IP层分片，同样也在目的IP层合并，如果某个IP分片丢失，则需要将所有分片都进行重传，开销大。

2. TCP与UDP头部格式

2.1 UDP



目标端口和源端口:

主要是告诉 UDP 协议应该把报文发给哪个进程。

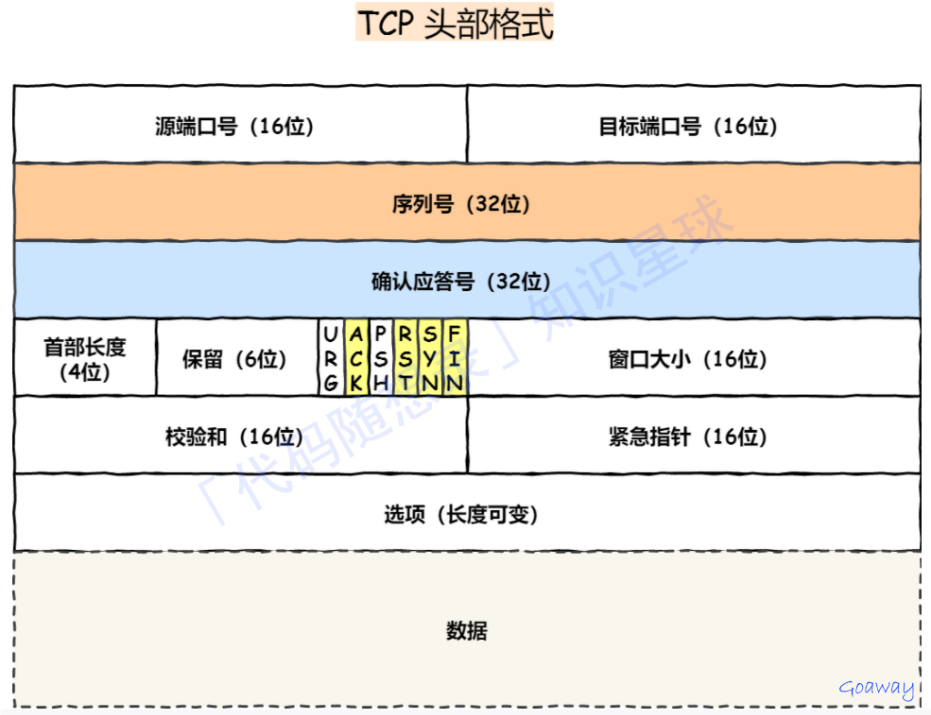
包长度:

该字段保存了 UDP 首部的长度跟数据的长度之和。

校验和:

校验和是为了提供可靠的 UDP 首部和数据而设计，接收方使用检验和来检查该报文段中是否出现差错。

2.2 TCP



源端口号和目的端口号：

用于多路复用/分解来自或送到上层应用的数据。告诉主机报文段来自哪里，传给哪个上层协议或应用程序。

序列号：

该报文段首字节的字节流编号，用来解决网络包乱序问题。确认应答号：对发送来的 TCP 报文段的响应，值是收到的 TCP 报文段的序号值加1，用来解决不丢包的问题。序列号和确认应答号都用于实现可靠数据传输。

首部长度：

标识 TCP 头部有多少字节，最长 60。

窗口大小：

接收窗口，告诉对方本端TCP缓冲区还有多少空间可以接收数据，用来做流量控制。

标志字段：

ACK：用于指示确认应答号值是否有效，置1表示包含一个对已成功接收报文段的确认；

RST：用于重置一个已经混乱的连接，或拒绝一个无效的数据段或者连接请求；

SYN：用于连接建立过程，请求建立一个连接；

FIN：用于断开连接，表示发送方没有数据要传输了。

检验和：

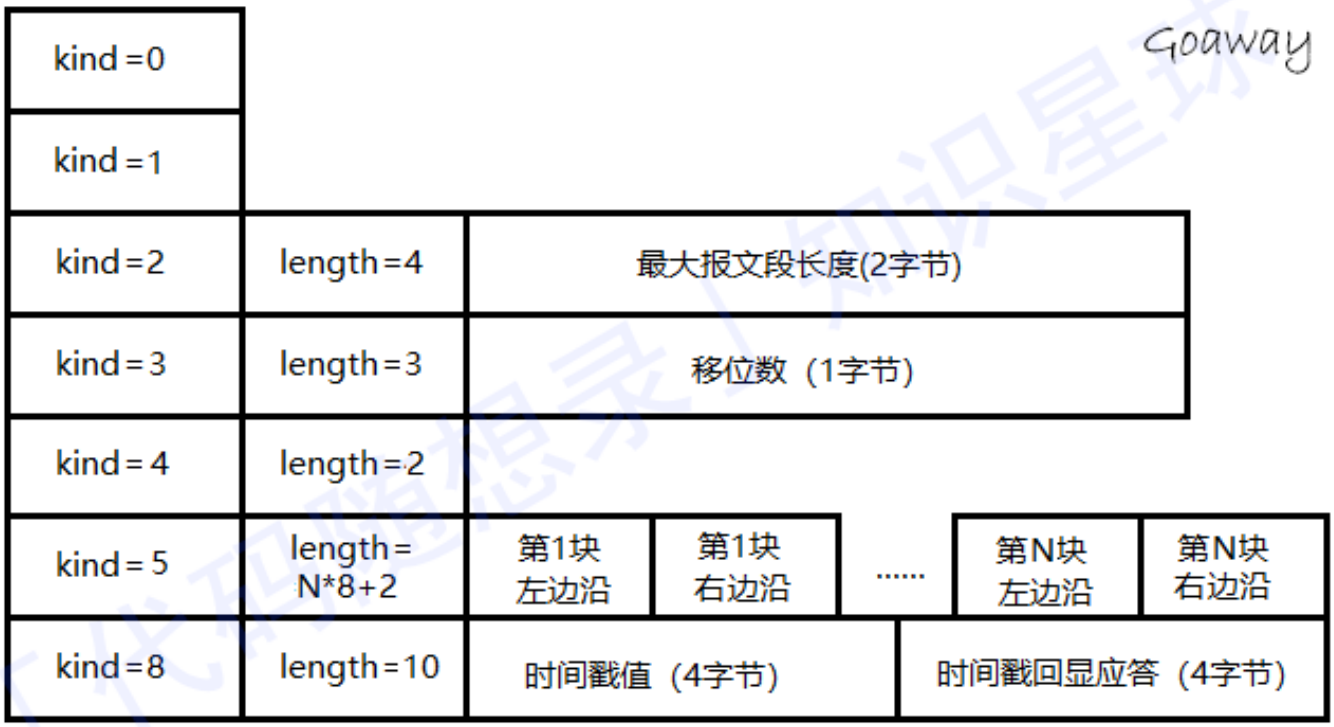
接收方使用检验和来检查该报文段(头部+数据)中是否出现差错（CRC算法），同 UDP。

列举TCP选项

TCP头部的最后一个选项字段（options）是可变长的可选信息。这部分最多包含40字节，因为TCP头部最长是60字节（其中还包含前面讨论的20字节的固定部分）。

- 选项的第一个字段kind说明选项的类型，有的TCP选项没有后面两个字段，仅包含1字节的kind字段。
- 第二个字段length（如果有的话）指定该选项的总长度。该长度包括kind字段和length字段占据的2字节。
- 第三个字段info（如果有的话）是选项的具体信息。

常见的TCP选项有7种：



1、kind=0，选项表结束（EOP）选项。

一个报文段仅用一次。放在末尾用于填充，用途是说明：首部已经没有更多的消息，应用数据在下一个32位字开始处。

2、kind=1，空操作（NOP）选项。

没有特殊含义，一般用于将TCP选项的总长度填充为4字节的整数倍。

3、kind=2，最大报文段长度（MSS）选项。

TCP连接初始化时，通信双方使用该选项来协商最大报文段长度。TCP模块通常将MSS设置为（MTU-40）字节（减掉的这40字节包括20字节的TCP头部和20字节的IP头部）。

这样携带TCP报文段的IP数据报的长度就不会超过MTU（假设TCP头部和IP头部都不包含选项字段，并且这也是一般情况），从而避免本机发生IP分片。对以太网而言，MSS值是1460（1500-40）字节。

4、kind=3，窗口扩大因子选项。

TCP连接初始化时，**通信双方使用该选项来协商接收窗口的扩大因子。**在TCP的头部中，接收窗口大小是用16位表示的，故最大为65535字节，但实际上TCP模块允许的接收窗口大小远不止这个数（为了提高TCP通信的吞吐量）。窗口扩大因子解决了这个问题。

假设 TCP 头部中的接收通告窗口大小是 N，窗口扩大因子（移位数）是 M，那么 TCP 报文段的实际接收通告窗口大小是 N*2M，或者说 N 左移 M 位。注意，M的取值范围是 0～14。

和 MSS 选项一样，窗口扩大因子选项只能出现在同步报文段中，否则将被忽略。但同步报文段本身不执行窗口扩大操作，即同步报文段头部的接收窗口大小就是该 TCP 报文段的实际接收窗口大小。当连接建立好之后，每个数据传输方向的窗口扩大因子就固定不变了。

5、kind=4，选择性确认（Selective Acknowledgment，SACK）选项。

TCP 通信时，如果某个 TCP 报文段丢失，则 TCP 会重传最后被确认的 TCP 报文段后续的所有报文段，这样原先已经正确传输的 TCP 报文段也可能重复发送，从而降低了 TCP 性能。

SACK 技术使 TCP 只重新发送丢失的 TCP 报文段，而不用发送所有未被确认的 TCP 报文段。选择性确认选项用在连接初始化时，表示是否支持 SACK 技术。

6、kind=5，SACK实际工作的选项。

该选项的参数告诉发送方本端已经收到并缓存的不连续的数据块，从而让发送端可以据此检查并重发丢失的数据块。

每个块边沿（edge of block）参数包含一个4字节的序号。其中块左边沿表示不连续块的第一个数据的序号，而块右边沿则表示不连续块的最后一个数据的序号的下一个序号。这样一对参数（块左边沿和块右边沿）之间的数据是没有收到的。因为一个块信息占用8字节，所以 TCP 头部选项中实际上最多可以包含4个这样的不连续数据块（考虑选项类型和长度占用的2字节）。

7、kind=8，时间戳选项。

该选项提供了较为准确的计算通信双方之间的回路时间（Round Trip Time，RTT）的方法，为TCP流量控制提供信息。

3. SYN攻击

1、原理

攻击者伪造不同IP地址的SYN报文请求连接，服务端收到连接请求后分配资源，回复ACK+SYN包，但是由于IP地址是伪造的，无法收到回应，久而久之造成服务端半连接队列被占满，无法正常工作。

2、避免方式

（1）修改半连接队列大小

使服务端能够容纳更多半连接。此外还可以修改服务端超时重传次数，使服务端尽早丢弃无用连接

（2）正常服务端行为是收到客户端SYN报文后

将其加入到内核半连接队列，接着发送ACK+SYN报文给客户端，当收到客户端ACK报文后把连接从半连接队列移动到accept队列。

当半连接队列满时，启动syn cookie,后续连接不进入半连接队列，而是计算一个cookie值，作为请求报文序列号发送给客户端，如果服务端收到客户端确认报文，会检查ack包合法性，如果合法直接加入到accept队列

4. TCP保活机制

1、概念

在一个定义的时间段内TCP连接无任何活动时，会启动TCP保活机制，每隔一定时间间隔发送一个探测报文，等待响应。

2、机制

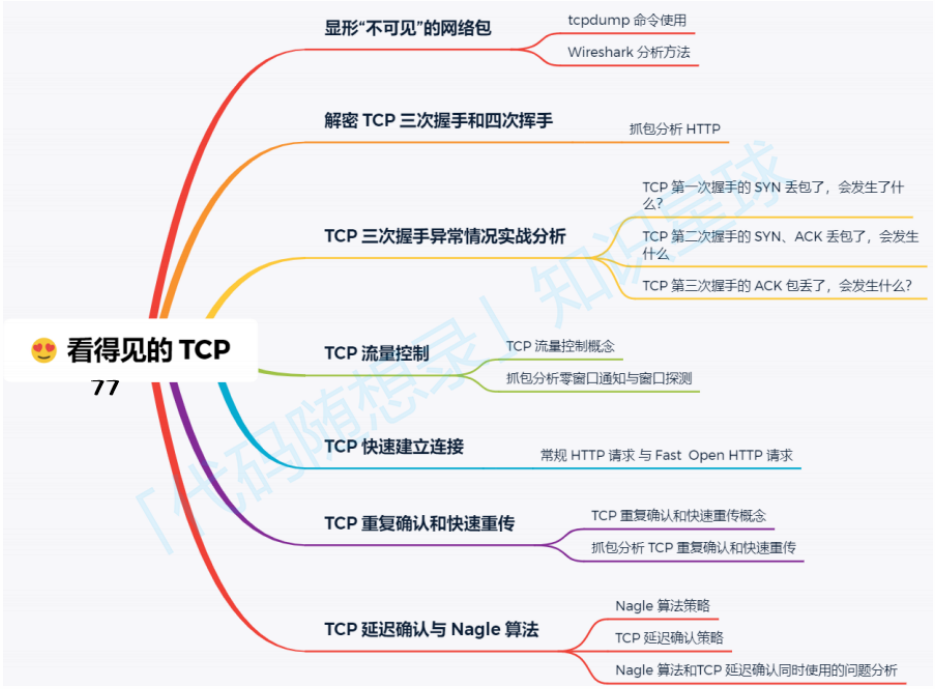
1. 对端正常响应，重置保活时间;
2. 对端程序崩溃，响应一个RTS报文，将TCP连接重置;
3. 保活报文不可达，等待达到保活探测次数后关闭连接。

3、TCP为啥需要流量控制

1. 由于通讯双方网速不同，通讯方任意一方发送过快都会导致对方详细处理不过来，所以就需要把数据放到缓冲区中
2. 如果缓冲区满了，发送方还在疯狂发送，那接收方只能把数据包丢弃。因此我们需要控制发送速率
3. 我们缓冲区剩余大小称之为接收窗口，用变量win表示。如果win=0，则发送方停止发送

TCP 抓包实践

思维导图:



1、显形"不可见"的网络包

（1）最常用的网络抓包和分析工具:

tcpdump和Wireshark，那这两者有什么区别呢?

- tcpdump 仅支持命令行格式使用，常用在 Linux 服务器中抓取和分析网络包。
- Wireshark 除了可以抓包外，还提供了可视化分析网络包的图形页面。

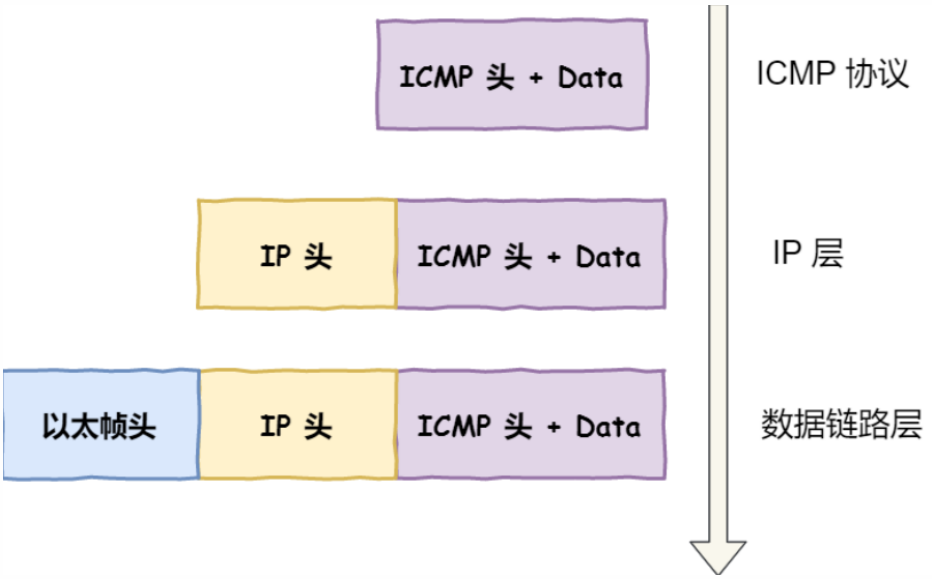
两者经常搭配使用，先用 tcpdump 命令在 Linux 服务器上抓包（保存成.pcap后缀格式的文件），接着把抓包的文件拖出到Windows 电脑后，用 Wireshark 可视化分析。

（2）tcpdump在Linux下如何抓包？

tcpdump 使用 —— 选项类		
选项	**示例**	**说明**
-i	tcpdump -i eth0	指定网络接口，默认是 0 浩接口（如 eth0 ），any 表示所有接口
-nn	tcpdump -nn	不解析 IP 地址和端口号的名称
-c	tcpdump -c 5	限制要抓取的网络包的个数
-w	tcpdump -w file.pcap	保持到文件中，文件名通常以 .pcap 为后缀

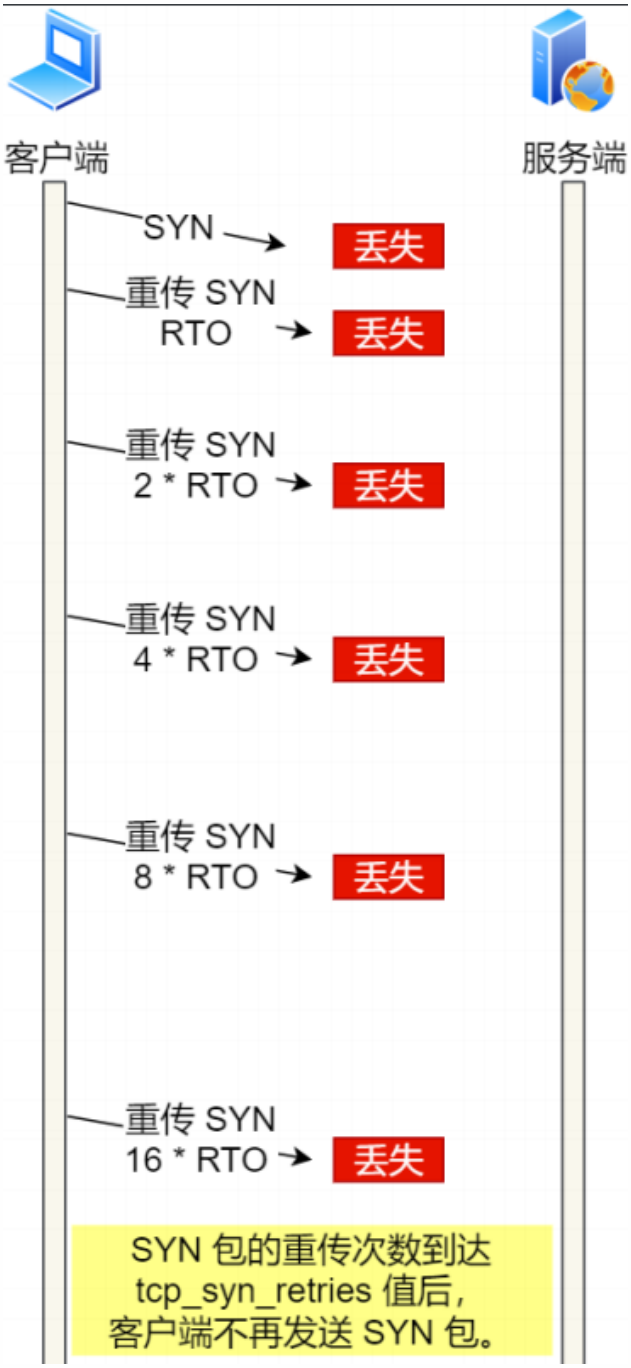
tcpdump 使用 —— 过滤表达式类		
选项	**示例**	**说明**
host、src host、dst host	tcpdump -nn host 192.168.1.100	主机过滤
port、src port、dst port	tcpdump -nn port 80	端口过滤
ip、ip6、arp、tcp、udp、icmp	tcpdump -nn tcp	协议过滤
and、or、not	tcpdump -nn host 192.168.1.100 and port 80	逻辑表达式
tcp[tcoflages]	tcpdump -nn "tcp[tcpflags] & tcp-syn != 0"	特定状态的 TCP 包

（3）Wireshark工具如何分析数据包？



2、TCP三次握手异常情况实战分析

（1）实验一：TCP第一次握手SYN 丢包



总结：

通过实验一的实验结果，我们可以得知，当客户端发起的 TCP 第一次握手 SYN 包，在超时时间内没收到服务端的 ACK，就会在超时重传 SYN 数据包，每次超时重传的 RTO 是翻倍上涨的，直到 SYN 包的重传次数到达 `tcp_syn_retries` 值后，客户端不再发送 SYN 包。

（2）实验二：TCP第二次握手SYN、ACK丢包

客户端设置了防火墙，粗暴地将来自服务端的数据都丢弃

通过实验二的实验结果，我们可以得知，当 TCP 第二次握手 SYN、ACK 包丢了后，客户端 SYN 包会发生超时重传，服务端 SYN、ACK 也会发生超时重传。

客户端 SYN 包超时重传的最大次数，是由 tcp_syn_retries 决定的，默认值是 5 次；服务端 SYN、ACK 包时重传的最大次数，是由 tcp_synack_retries 决定的，默认值是 5 次。

网络包进入主机后的顺序如下：

- 进来的顺序 Wire -> NIC -> **tcpdump -> netfilter/iptables**

- 出去的顺序 **iptables -> tcpdump** -> NIC -> Wire

（3）实验三：TCP第三次握手ACK丢包

在服务端配置防火墙，屏蔽客户端TCP报文中标志位是ACK的包。

如果客户端不发送数据，什么时候才会断开处于ESTABLISHED状态的连接？这就需要提到TCP的保活机制了：

定义一个时间段，在这个时间段内，如果没有任何连接相关的活动，TCP 保活机制会开始作用，每隔一个时间间隔，发送一个「探测报文」，该探测报文包含的数据非常少，如果连续几个探测报文都没有得到响应，则认为当前的 TCP 连接已经死亡，系统内核将错误信息通知给上层应用程序。

总结：

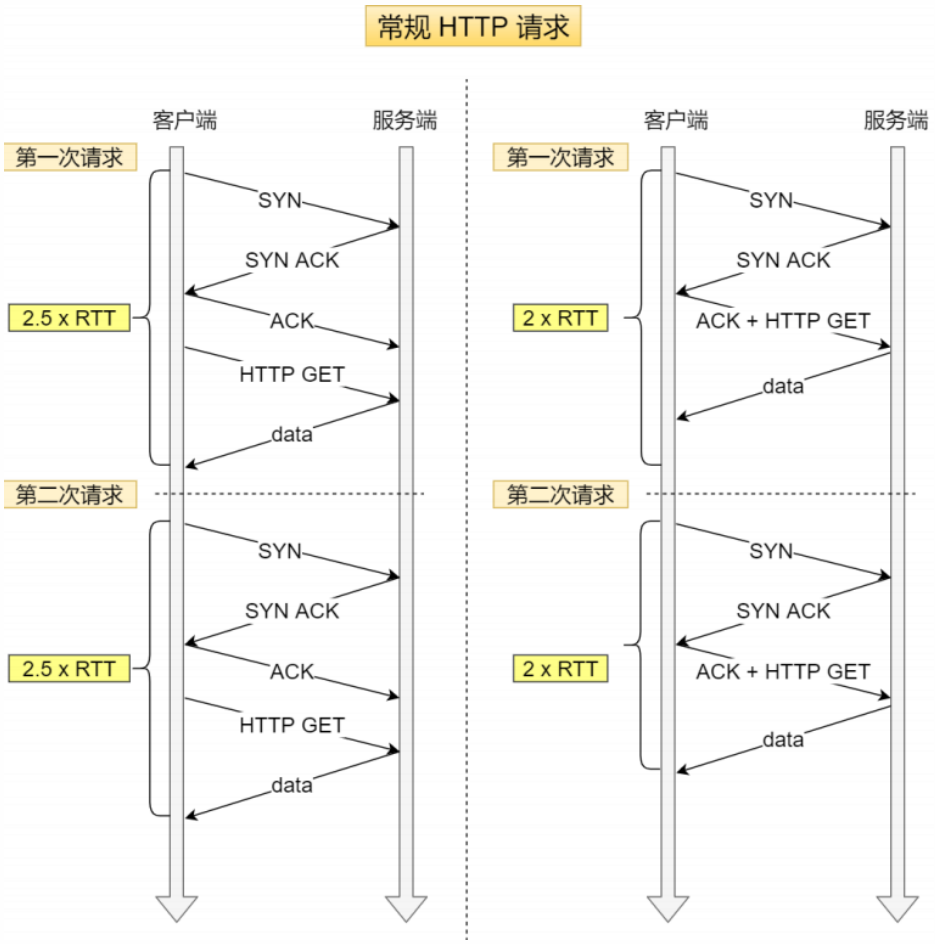
在建立 TCP 连接时，如果第三次握手的 ACK，服务端无法收到，则服务端就会短暂处于 `SYN_RECV` 状态，而客户端会处于 `ESTABLISHED` 状态。

由于服务端一直收不到 TCP 第三次握手的 ACK，则会一直重传 SYN、ACK 包，直到重传次数超过 `tcp_synack_retries` 值（默认值 5 次）后，服务端就会断开 TCP 连接。

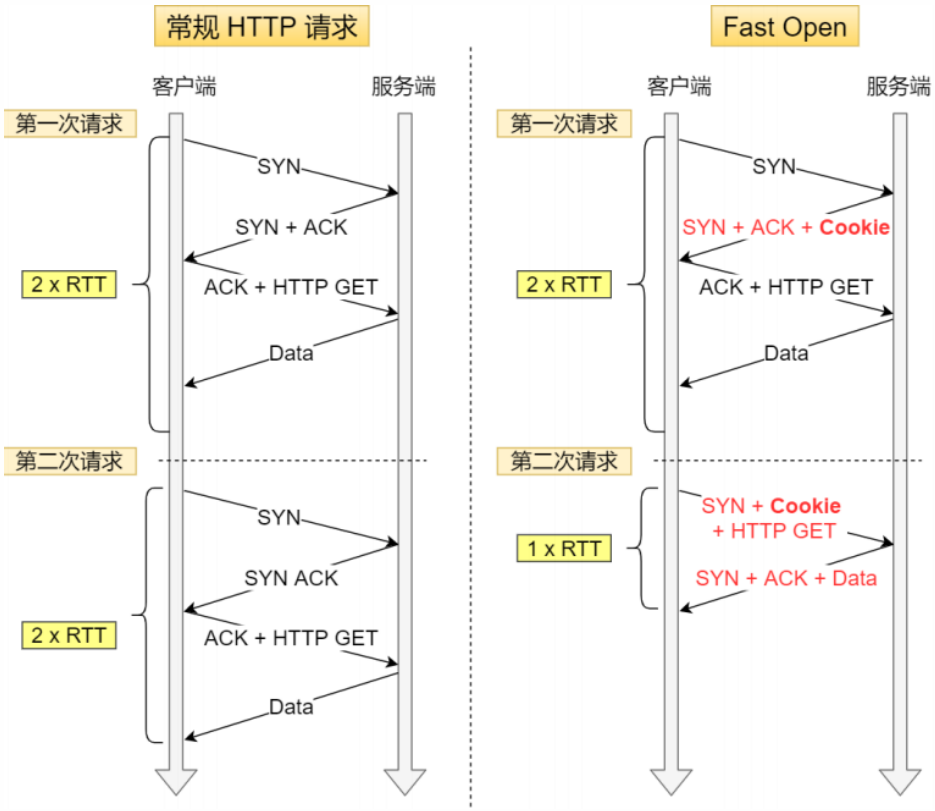
而客户端则会有两种情况：

- 如果客户端没发送数据包，一直处于 `ESTABLISHED` 状态，然后经过 2 小时 11 分 15 秒才可以发现一个「死亡」连接，于是客户端连接就会断开连接。
- 如果客户端发送了数据包，一直没有收到服务端对该数据包的确认报文，则会一直重传该数据包，直到重传次数超过 `tcp_retries2` 值（默认值 15 次）后，客户端就会断开 TCP 连接。

3、TCP快速建立连接



使用TCP Fast Open功能，可以减少TCP连接建立的时延。



- 在第一次建立连接的时候，服务端在第二次握手产生一个 `Cookie` （已加密）并通过 SYN、ACK 包一起发给客户端，于是客户端就会缓存这个 `Cookie` ，所以第一次发起 HTTP Get 请求的时候，还是需要 2 个 RTT 的时延；
- 在下次请求的时候，客户端在 SYN 包带上 `Cookie` 发给服务端，就提前可以跳过三次握手的过程，因为 `Cookie` 中维护了一些信息，服务端可以从 `Cookie` 获取 TCP 相关的信息，这时发起的 HTTP GET 请求就只需要 1 个 RTT 的时延；

注：客户端在请求并存储了 Fast Open Cookie 之后，可以不断重复 TCP Fast Open 直至服务器认为 Cookie 无效（通常为过期）

4、TCP流量控制

发送窗口决定了一次能发多少字节的数据，而MSS决定了这些字节要分多少包才能发送完。

5、TCP延迟确认与Nagle算法

当TCP报文承载的数据非常小的时候，整个网络的效率很低。就好像快递员开着大货车送一个小包裹一样浪费。所以就出现了常见的两种策略，来减少小报文的传输。

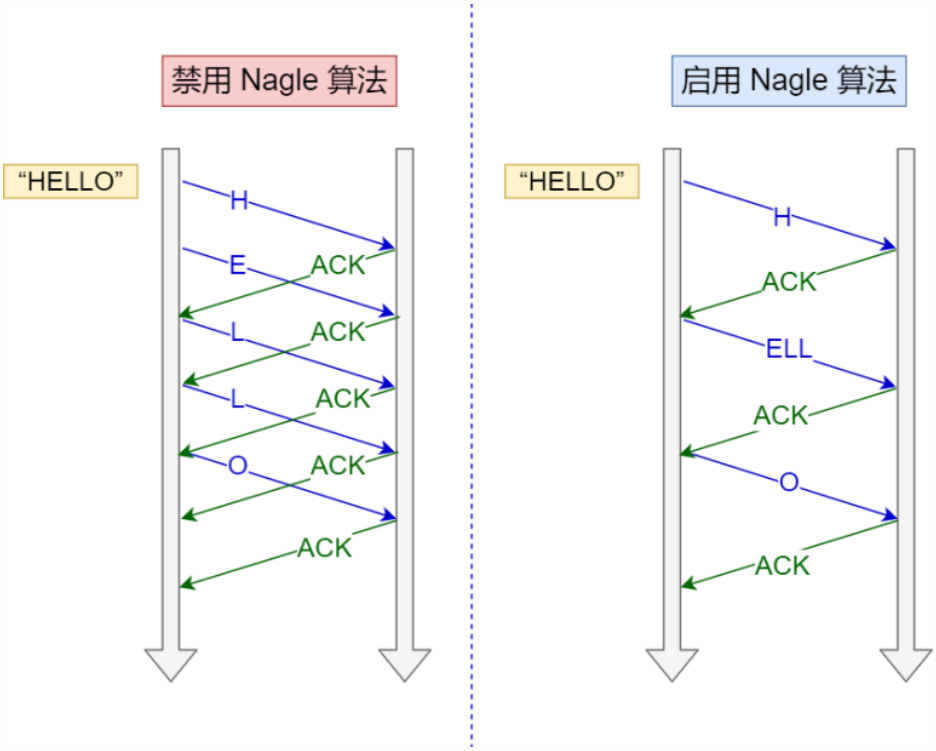
1. Nagle算法
2. 延迟确认

（1）Nagle算法是如何避免大量TCP小数据报文的传输的？

Nagle 算法的策略：

- 没有已发送未确认报文时，立刻发送数据。
- 存在未确认报文时，直到「没有已发送未确认报文」或「数据长度达到 MSS 大小」时，再发送数据。

只要没满足上面条件中的一条，发送方一直在囤积数据，直到满足上面的发送条件。

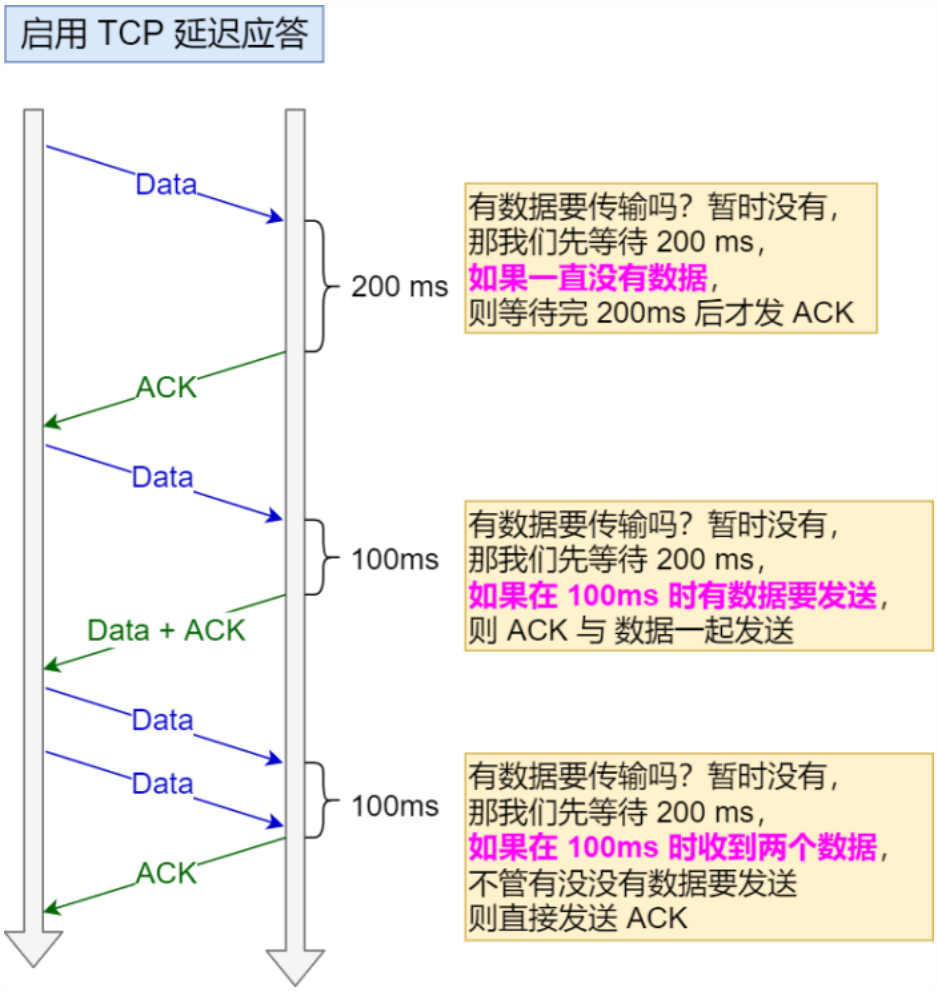


从上图可以看出：Nagle算法一定会有一个小报文，在最开始的时候。

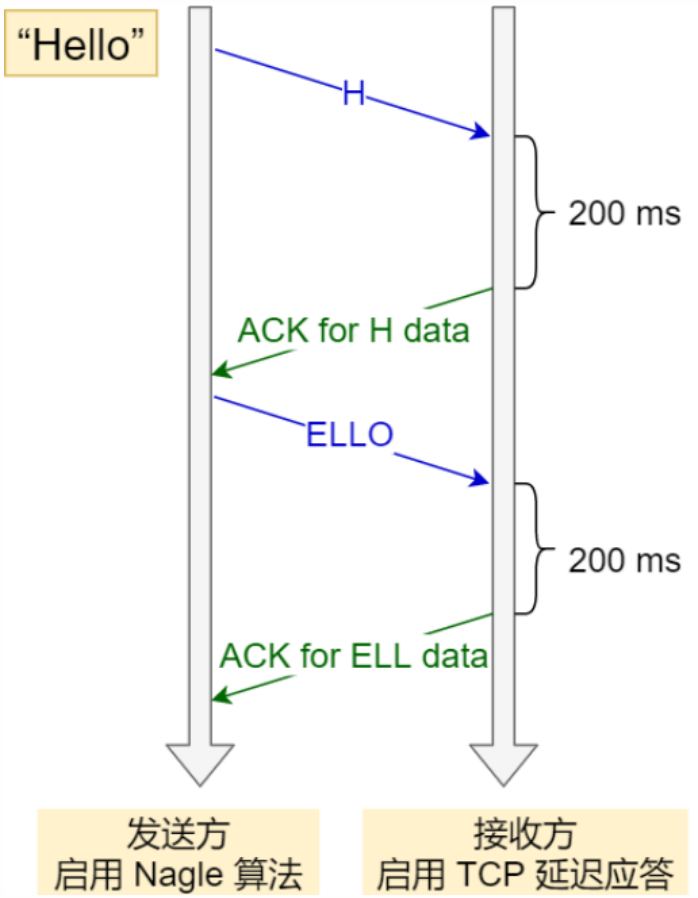
（2）延迟确认：为解决ACK传输效率低的问题，而提出的。

TCP 延迟确认的策略：

- 当有响应数据要发送时，ACK 会随着响应数据一起立刻发送给对方
- 当没有响应数据要发送时，ACK 将会延迟一段时间，以等待是否有响应数据可以一起发送
- 如果在延迟等待发送 ACK 期间，对方的第二个数据报文又到达了，这时就会立刻发送 ACK



当延迟确认和Nagle混合使用时，会出现新的问题（导致时耗增长）

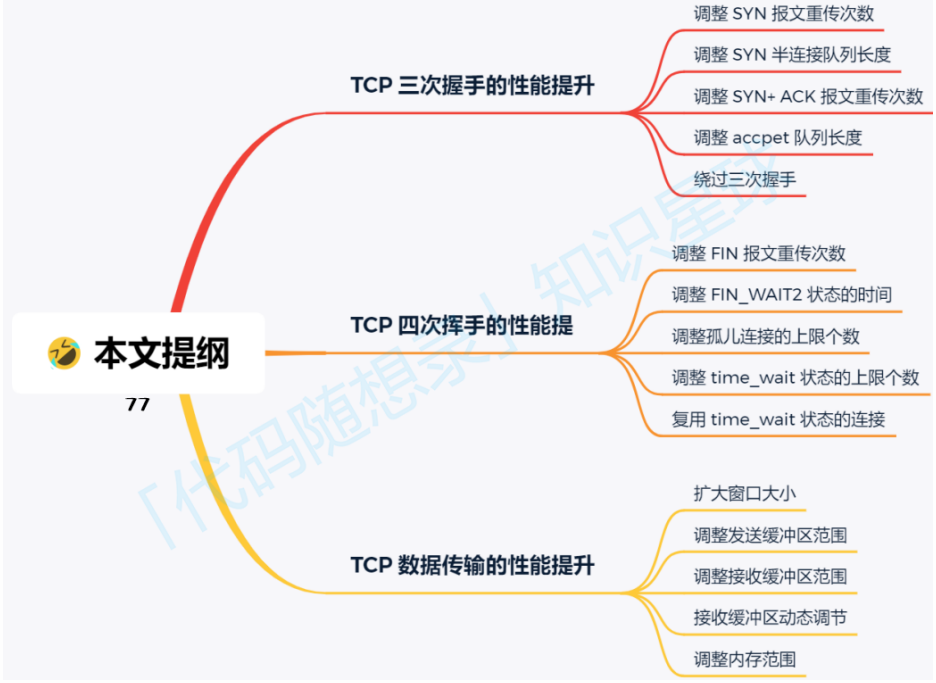


要解决上述问题，只有两个办法：

1. 发送方关闭Nagle算法
2. 接收方关闭TCP延迟确认

TCP连接与断开优化

思维导图：



关于优化TCP握手的几个TCP参数：

优化三次握手的策略	
策略	TCP 内核参数
调整 SYN 报文的重传次数	tcp_syn_retries
调整 SYN 半连接队列长度	tcp_max_syn_backlog、somaxconn、backlog
调整 SYN+ACK 报文的重传次数	tcp_synack_retries
调整 accpet 队列长度	min(backlog, somaxconn)
绕过三次握手	tcp_fastopen

1、客户端的优化

当客户端发起SYN包时，可以通过tcp_synb_retries控制其重传的次数。

2、服务端的优化

当服务端 SYN 半连接队列溢出后，会导致后续连接被丢弃，可以通过 `netstat -s` 观察半连接队列溢出的情况，如果 SYN 半连接队列溢出情况比较严重，可以通过 `tcp_max_syn_backlog、somaxconn、backlog` 参数来调整 SYN 半连接队列的大小。

服务端回复 SYN+ACK 的重传次数由 `tcp_synack_retries` 参数控制。如果遭受 SYN 攻击，应把 `tcp_syncookies` 参数设置为 1，表示仅在 SYN 队列满后开启 syncookie 功能，可以保证正常的连接成功建立。

服务端收到客户端返回的 ACK，会把连接移入 accpet 队列，等待进行调用 accpet() 函数取出连接。

可以通过 `ss -lnt` 查看服务端进程的 accept 队列长度，如果 accept 队列溢出，系统默认丢弃 ACK，如果可以把 `tcp_abort_on_overflow` 设置为 1 ，表示用 RST 通知客户端连接建立失败。

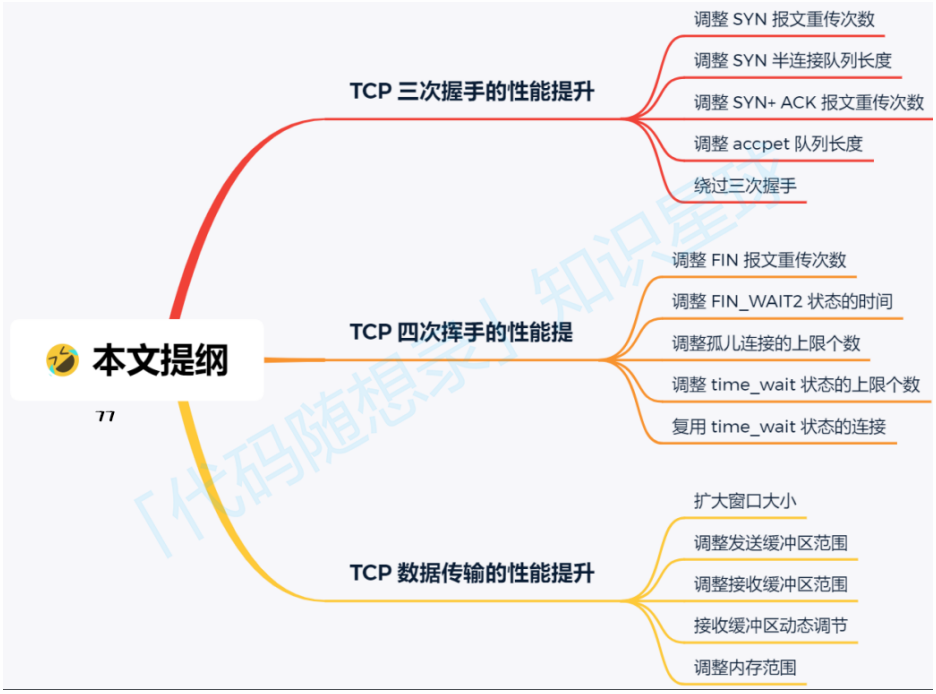
如果 accpet 队列溢出严重，可以通过 listen 函数的 `backlog` 参数和 `somaxconn` 系统参数提高队列大小，accept 队列长度取决于 min(backlog, somaxconn)。

3、绕过三次握手

TCP Fast Open 功能可以绕过三次握手，使得 HTTP 请求减少了 1 个 RTT 的时间，Linux 下可以通过 `tcp_fastopen` 开启该功能，同时必须保证服务端和客户端同时支持。

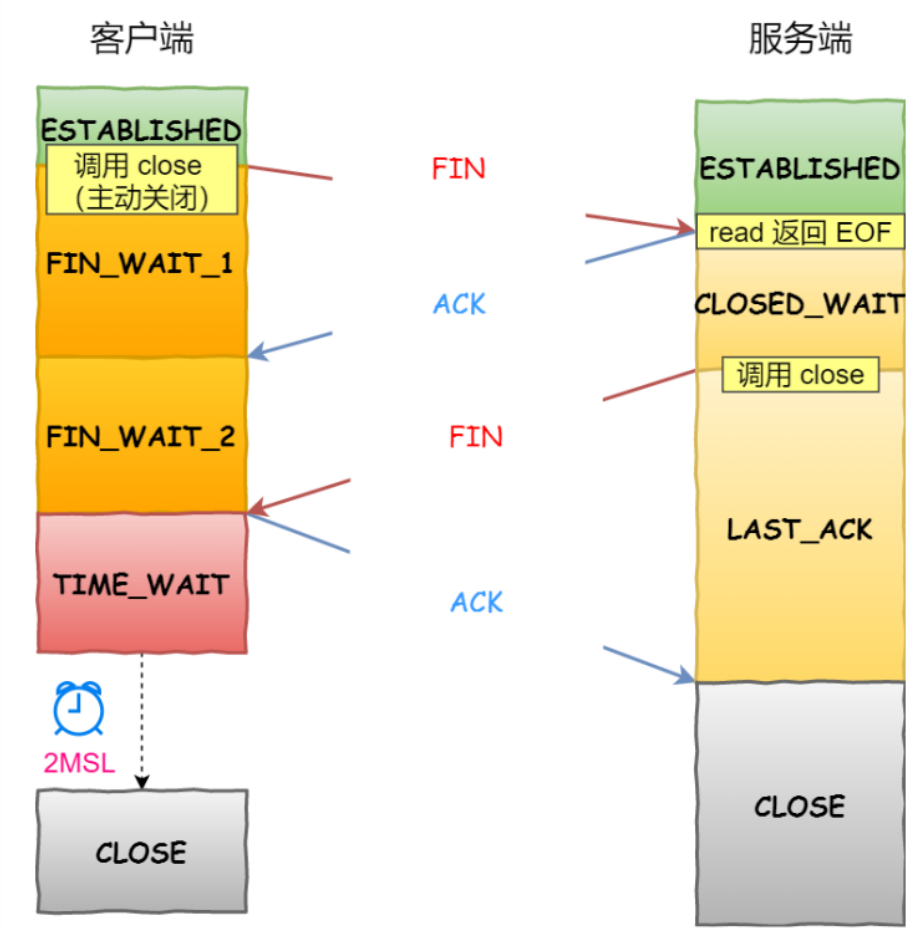
TCP内核参数

思维导图：



1、TCP四次挥手的性能提升

TCP四次挥手的状态变迁如下：



注：主动关闭连接的一方才有TIME_WAIT状态。

主动关闭方和被动关闭方的优化思路也不同，下面将进行分别讨论：

主动方的优化

关闭连接的方式通常有两种，分别是 RST 报文关闭和 FIN 报文关闭。

如果进程异常退出了，内核就会发送 RST 报文来关闭，它可以不走四次挥手流程，是一个暴力关闭连接的方式。

安全关闭连接的方式必须通过四次挥手，它由进程调用 close 和 shutdown 函数发起 FIN 报文（shutdown 参数须传入 SHUT_WR 或者 SHUT_RDWR 才会发送 FIN）。

1、close 和 shutdown 有什么区别

调用了 close 函数意味着完全断开连接，完全断开不仅指无法接收数据，而且也不能发送数据。

此时，调用了 close 函数的一方的连接叫做「孤儿连接」，如果你用 netstat -p 命令，会发现连接对应的进程名为空。

使用 close 函数关闭连接是不优雅的。于是，就出现了一种优雅关闭连接的 shutdown 函数，它可以控制只关闭一个方向的连接。

```
int shutdown(int sock, int howto);
```

其中第二个参数决定断开连接的方式，主要有三种：

（1）SHUT_RD(0)

关闭连接的「读」这个方向，如果接收缓冲区有已接收的数据，则将会被丢弃，并且后续再收到新的数据，会对数据进行 ACK，然后悄悄地丢弃。也就是说，对端还是会接收到 ACK，在这种情况下根本不知道数据已经被丢弃了。

（2）SHUT_WR(1)

关闭连接的「写」这个方向，这就是常被称为「半关闭」的连接。如果发送缓冲区还有未发送的数据，将被立即发送出去，并发送一个 FIN 报文给对端。

（3）SHUT_RDWR(2)

相当于 SHUT_RD 和 SHUT_WR 操作各一次，关闭套接字的读和写两个方向。

2、FIN_WAIT1状态的优化

如果 FIN_WAIT1 状态连接很多，我们就需要考虑降低 tcp_orphan_retries 的值。当重传次数超过 tcp_orphan_retries 时，连接就会直接关闭掉（即：新增的孤儿连接将不再走四次挥手，而是直接发送 RST 复位报文强制关闭）。

注：tcp_max_orphans 参数，定义了「孤儿连接」的最大数量。

3、FIN_WAIT2状态的优化

当主动方收到 ACK 报文后，会处于 FIN_WAIT2 状态，就表示主动方的发送通道已经关闭，接下来将等待对方发送 FIN 报文，关闭对方的发送通道。

这时，如果连接是用 shutdown 函数关闭的，主动方连接可以一直处于 FIN_WAIT2 状态，因为它可能还可以发送或接收数据。

但对于 close 函数关闭的孤儿连接，由于无法再发送和接收数据，所以这个状态不可以持续太久，而 tcp_fin_timeout 控制了这个状态下连接的持续时长，默认值是 60 秒（与 TIME_WAIT 状态持续的时间是相同的）。它意味着对于孤儿连接（调用 close 关闭的连接），如果在 60 秒后还没有收到 FIN 报文，连接就会直接关闭。

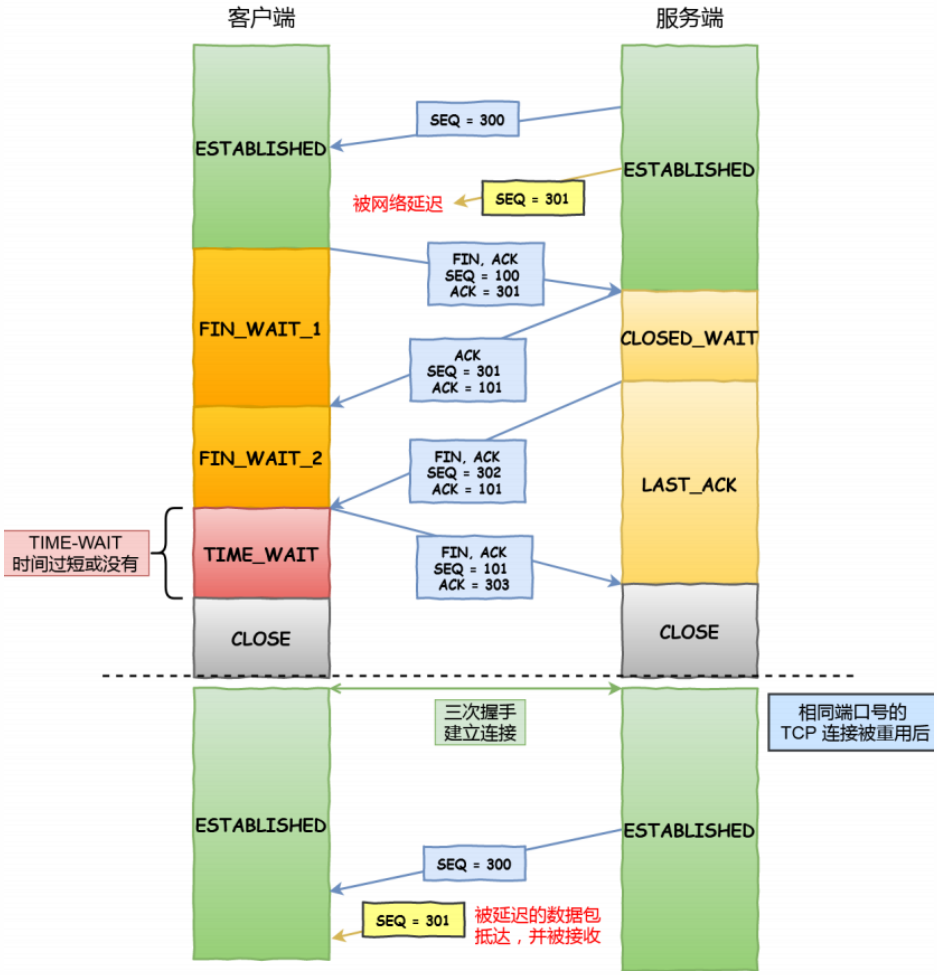
4、TIME_WAIT状态的优化

TIME_WAIT的状态尤其重要，主要是两个原因

- 防止具有相同「四元组」的「旧」数据包被收到；
- 保证「被动关闭连接」的一方能被正确的关闭，即保证最后的 ACK 能让被动关闭方接收，从而帮助其正常关闭；

（1）防止收到历史数据，从而导致数据错乱的问题

若TIME_WAIT等待时间过短，被延迟的数据包抵达后会发生什么呢？



TIME_WAIT设计为2MSL，足以让两个方向上的数据包都被丢弃，使得原来连接的数据包在网络中都自然消失，再出现的数据包一定都是新建立连接所产生的。

注：

MSL 全称是 Maximum Segment Lifetime，它定义了一个报文在网络中的最长生存时间（报文每经过一次路由器的转发，IP 头部的 TTL 字段就会减 1，减到 0 时报文 就被丢弃，这就限制了报文的最长存活时间）。

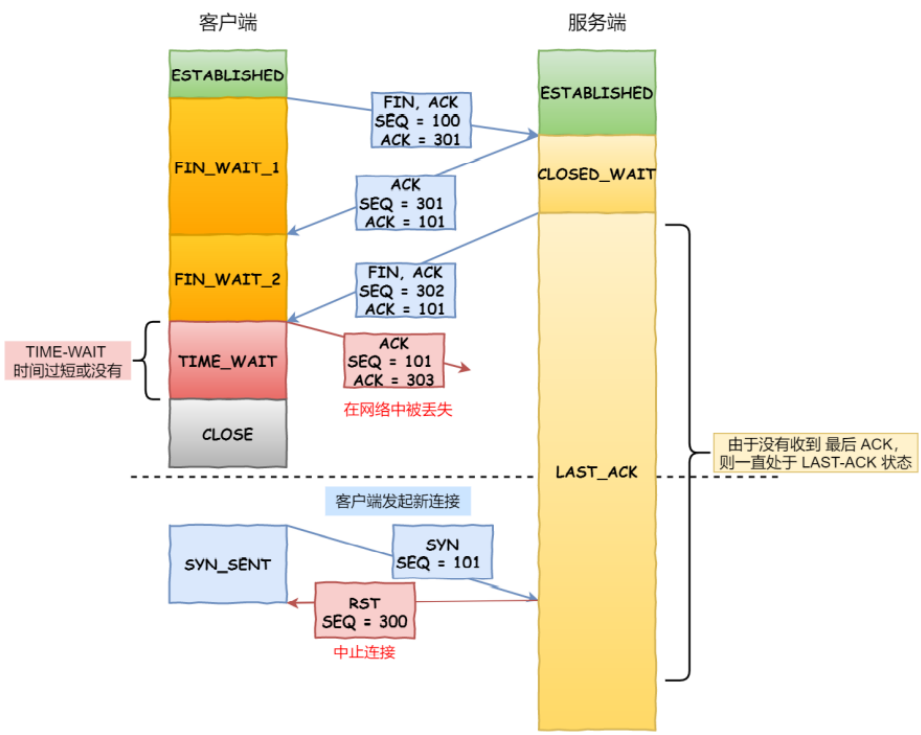
（2）为什么是 2 MSL 的时长呢?

这其实是相当于至少允许报文丢失一次。比如，若 ACK 在一个 MSL 内丢失，这样被 动方）发的 FIN 会在第 2 个 MSL 内到达，TIME_WAIT 状态的连接可以应对。

在Linux系统中，MSL的值固定为30秒。

等待足够的时间以确保最后的ACK能让被动关闭方接收，从而帮助其正确关闭

（3）假设TIME_WAIT没有等待或等待的时间过短，断开连接会造成什么?



如上图红色框框客户端四次挥手的最后一个 ACK 报文如果在网络中被丢失了，此时如果客户端 TIMEWAIT 过短或没有，则就直接进入了 CLOSE 状态了，那么服务端则会一直处在 LAST-ACK 状态。

当客户端发起建立连接的 SYN 请求报文后，服务端会发送 RST 报文给客户端，连接建立的过程就会被终止。

注：

Linux 提供了 tcp_max_tw_buckets 参数，当 TIME_WAIT 的连接数量超过该参数时，新关闭的连接就不再经历 TIME_WAIT 而直接关闭。查看系统的TIME_WAIT的连接数量：

```
(base) [redacted]~$ cat /proc/sys/net/ipv4/tcp_max_tw_buckets
262144
```

但tcp_max_tw_buckets 也不是越大越好，毕竟内存和端口都是有限的。

还有一种方式是复用处于TIME_WAIT状态的连接，那就是打开 tcp_tw_reuse 参数。

但是需要注意，该参数是只用于客户端（建立连接的发起方），因为是在调用 connect() 时起作用的，而对于服务端（被动 连接方）是没有用的。

tcp_tw_reuse 从协议角度理解是安全可控的，可以复用处于 TIME_WAIT 的端口为新的连接所用。

什么是协议角度理解的安全可控呢?

主要有两点：

（1）只适用于连接发起方，也就是 C/S 模型中的客户端；

（2）对应的 TIME_WAIT 状态的连接创建时间超过 1 秒才可以被复用。

使用这个选项，还有一个前提，需要打开对 TCP 时间戳的支持（对方也要打开 ）。

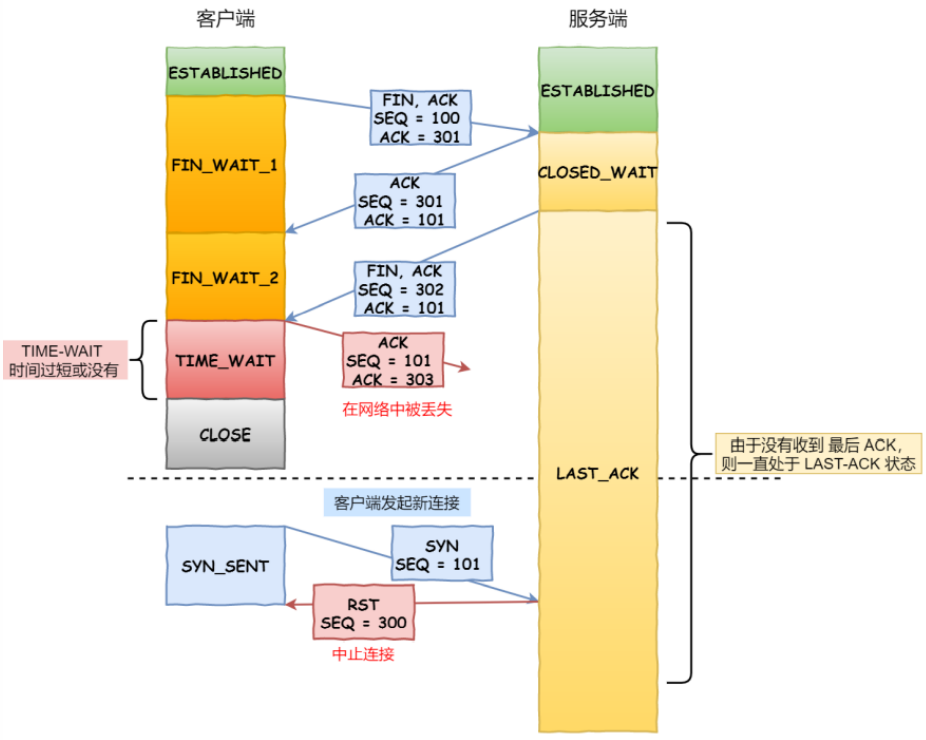
由于引入了时间戳，带来了一些好处：

1. 我们在前面提到的 2MSL 问题就不复存在了，因为）复的数据包会因为时间戳过期被自然丢弃；
2. 同时，它还可以防止序列号绕回，也是因为）复的数据包会由于时间戳过期被自然丢弃。

时间戳是在 TCP 的选项字段里定义的，开启了时间戳功能，在 TCP 报文传输的时候会带上发送报文的时间戳。

TCP option 字段			
类型	总长度（字节）	数据	描述
0	-	-	选项列表末尾标识
1	-	-	没有意义，用于 32 位对齐使用
2	4	MSS 值	三次握手时，发送端告知可以接收的最大报文段大小
3	3	窗口移位	指明最大窗口扩展后的大小
4	2	-	表明支持 SACK 选择性确认功能
5	可变	确认报文段	选择性确认窗口中间的报文段
8	10	Timestamps 时间戳	用于更精准的计算 RTT，以及解决序列号绕回的问题
14	3	校验和算法	双方认可后，可使用新的校验和算法
15	可变	校验和	当 16 位标准校验和放不下时，放置在这里
34	可变	FOC	TFO中Cookie

开启tcp_tw_reuse功能后，如果四次挥手中的最后一次ACK在网络中丢失了，会发生什么？

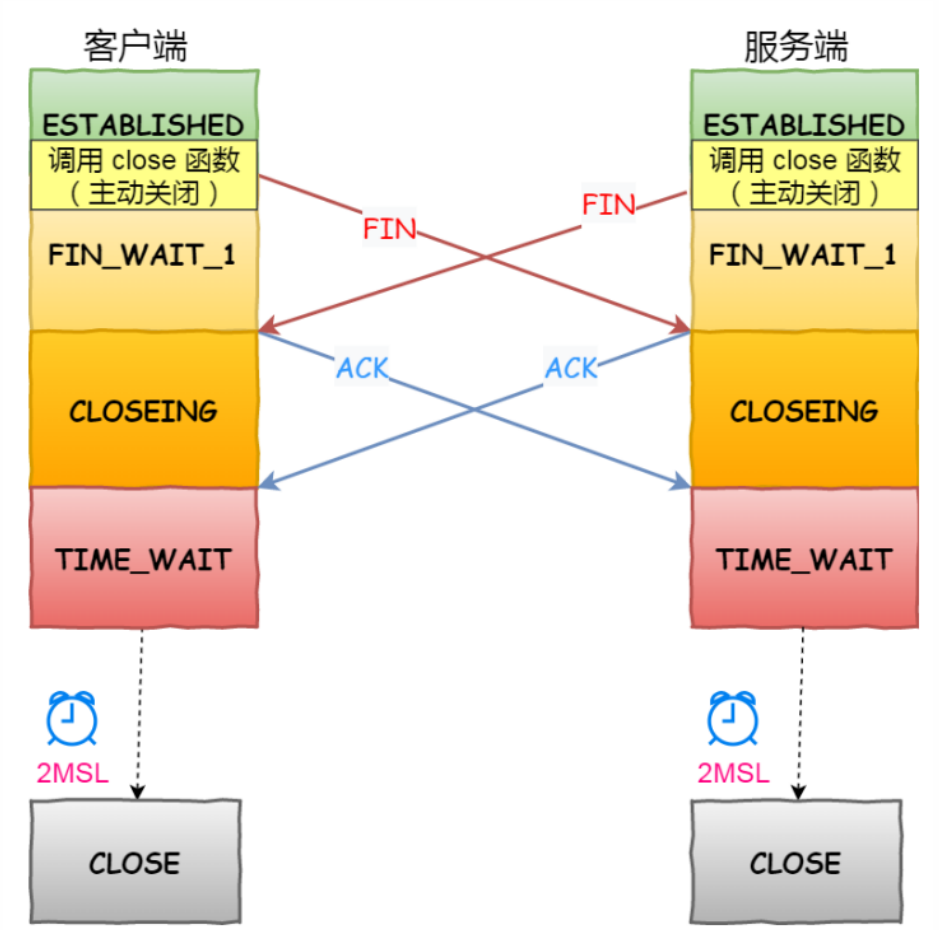


被动方的优化

1、如果连接双方同时关闭连接，会怎么样?

由于 TCP 是双全工的协议，所以是会出现两方同时关闭连接的现象，也就是同时发送了 FIN 报文。

此时，上面介绍的优化策略仍然适用。两方发送 FIN 报文时，都认为自己是主动方，所以都进入了 FIN_WAIT1 状态，FIN 报文的重发次数仍由 tcp_orphan_retries 参数控制。



接下来，双方在等待 ACK 报文的过程中，都等来了 FIN 报文。这是一种新情况，所以连接会进入一种叫做 CLOSING 的新状态，它替代了 FIN_WAIT2 状态。接着，双方内核回复 ACK 确认对方发送通道的关闭后，进入 TIME_WAIT 状态，等待 2MSL 的时间后，连接自动关闭。

小结：

针对 TCP 四次挥手的优化，我们需要根据主动方和被动方四次挥手状态变化来调整系统 TCP 内核参数。

优化四次挥手的策略	
策略	TCP 内核参数
调整 FIN 报文重传次数	tcp_orphan_retries
调整 FIN_WAIT2 状态的时间 (只适用 close 函数关闭的连接)	tcp_fin_timeout
调整孤儿连接的上限个数 (只适用 close 函数关闭的连接)	tcp_max_orphans
调整 time_wait 状态的上限个数	tcp_max_tw_buckets
复用 time_wait 状态的连接 (只适用客户端)	tcp_tw_reuse 、tcp_timestamps

2、TCP数据传输性能的提升

TCP 连接是由内核维护的，内核会为每个连接建立内存缓冲区：

如果连接的内存配置过小，就无法充分使用网络带宽，TCP 传输效率就会降低；
如果连接的内存配置过大，很容易把服务器资源耗尽，这样就会导致新连接无法建立；

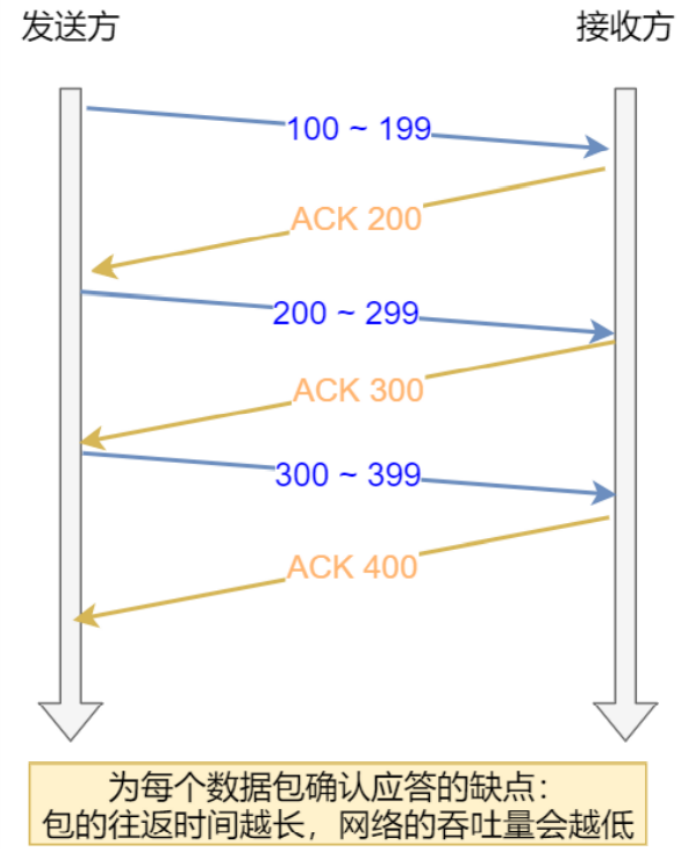
因此，我们必须理解 Linux 下 TCP 内存的用途，才能正确地配置内存大小。

（1）滑动窗口是如何影响传输速度的?

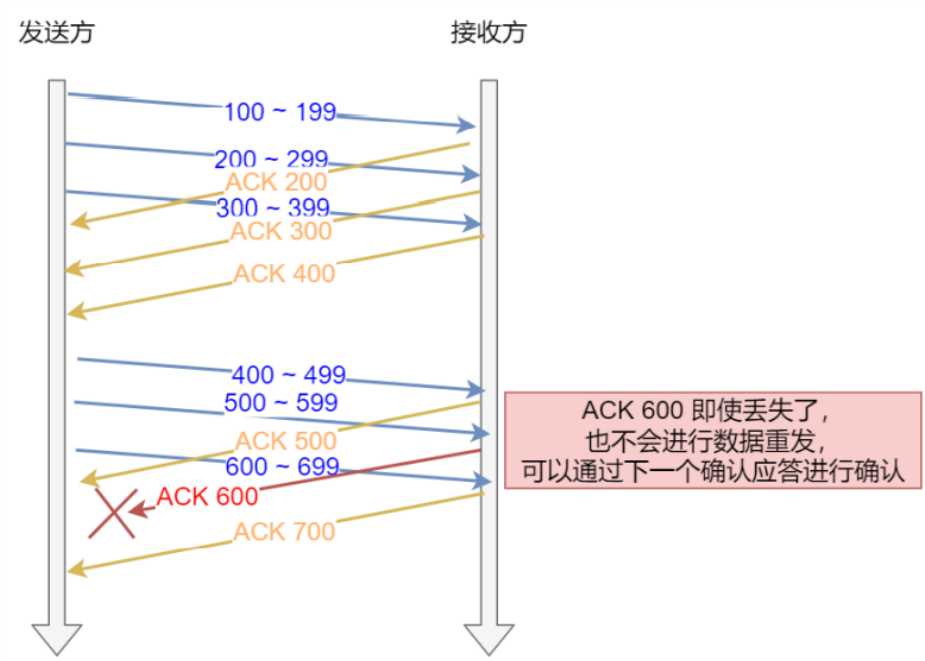
TCP 会保证每一个报文都能够抵达对方，机制如下：报文发出去后，必须接收到对方返回的确认报文 ACK，如果迟迟未收到，就会超时）发该报文，直到收到对方的 ACK 为止。

因此TCP报文发出去后，并不会立马从内存中删除，因为重传时还需要用到。由于TCP是由内核维护的，所以报文存放在内核缓冲区。

应答：



批量发送，批量应答：



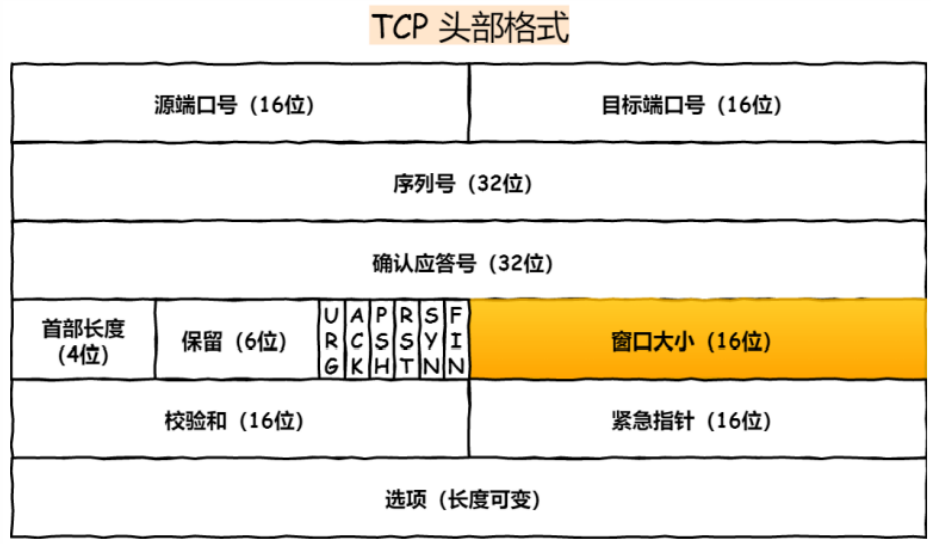
引发的问题：

当接收方硬件不如发送方，或者系统繁忙、资源紧张时，是无法瞬间处理这么多报文的。于是，这些报文只能被丢掉，使得网络效率非常低。

为了解决这种现象发生，TCP 提供一种机制可以让「发送方」根据「接收方」的实际接收能力控制发送的数据量，这就是滑动窗口的由来。

接收方根据它的缓冲区，可以计算出后续能够接收多少字节的报文，这个数字叫做接收窗口。当内核接收到报文时，必须用缓冲区存放它们，这样剩余缓冲区空间变小，接收窗口也就变小了；当进程调用 read 函数后，数据被读入了用户空间，内核缓冲区就被清空，这意味着主机可以接收更多的报文，接收窗口就会变大。

因此，接收窗口并不是恒定不变的，接收方会把当前可接收的大小放在 TCP 报文头部中的窗口字段，这样就可以起到窗口大小通知的作用。



从上图中可以看到，窗口字段只有 2 个字节，因此它最多能表达 65535 字节大小的窗口，也就是 64KB 大小。

这个窗口大小最大值，在当今高速网络下，很明显是不够用的。

所以后续有了扩充窗口的方法：

在 TCP 选项字段定义了窗口扩大因子，用于扩大 TCP 通告窗口，其值大小是 2^14，这样就使 TCP 的窗口大小从 16 位扩大为 30 位（2^16 * 2^ 14 = 2^30），所以此时窗口的最大值可以达到 1GB。

TCP option 字段			
类型	总长度（字节）	数据	描述
0	-	-	选项列表末尾标识
1	-	-	没有意义，用于 32 位对齐使用
2	4	MSS 值	三次握手时，发送端告知可以接收的最大报文段大小
3	3	窗口移位	指明最大窗口扩展后的大小
4	2	-	表明支持 SACK 选择性确认功能
5	可变	确认报文段	选择性确认窗口中间的报文段
8	10	Timestamps 时间戳	用于更精准的计算 RTT，以及解决序列号绕回的问题
14	3	校验和算法	双方认可后，可使用新的校验和算法
15	可变	校验和	当 16 位标准校验和放不下时，放置在这里
34	可变	FOC	TFO中Cookie

网络的传输能力是有限的，当发送方依据发送窗口，发送超过网络处理能力的报文时，路由器会直接丢弃这些报文。因此，缓冲区的内存并不是越大越好。

（2）如何确定最大传输速度？

TCP的传输速度，受限于发送窗口，接收窗口以及网络设备传输能力。

其中，窗口大小由内核缓冲区大小决定。如果缓冲区与网络传输能力匹配，那么缓冲区的利用率就达到了最大化。

（3）如何计算网络传输能力呢?

网络是有「带宽」限制的，带宽描述的是网络传输能力

它与内核缓冲区的计量单位不同:

带宽是单位时间内的流量，表达是「速度」，比如常见的带宽 100 MB/s;
缓冲区单位是字节，当网络速度乘以时间才能得到字节数。

带宽时延积BDP决定网络中飞行报文的大小。所谓【飞行】，是指它们就在网络线路、路由器等网络设备上。

带宽时延积 BDP = RTT * 带宽

其中：RTT为网络时延。

由于发送缓冲区大小决定了发送窗口的上限，而发送窗口口决定了「已发送未确认」的飞行报文的上限。因此，发送缓冲区不能超过「带宽时延积」。

发送缓冲区与带宽时延积的关系如下:

如果发送缓冲区「超过」带宽时延积，超出的部分就没办法有效的网络传输，同时导致网络过载，容易丢包;

如果发送缓冲区「小于」带宽时延积，就不能很好的发挥出网络的传输效率。

（4）怎样调整缓冲区大小?

发送缓冲区大小可自行调节，三个数值分别为：动态范围的最小值，初始默认值，动态范围的最大值。

```
(base) [redacted]~$ cat /proc/sys/net/ipv4/tcp_wmem
4096    16384    4194304
```

接收缓冲区范围调节

```
[redacted]~$ cat /proc/sys/net/ipv4/tcp_rmem
4096    87380    6291456
```

接收缓冲区可以根据系统空闲内存的大小来调节接收窗口:

如果系统的空闲内存很多，就可以自动把缓冲区增大一些，这样传给对方的接收窗口也会变大，因而提升发 送方发送的传输数据数

反之，如果系统的内存很紧张，就会减少缓冲区，这虽然会降低传输效率，可以保证更多的并发连接正常工作

发送缓冲区的调节功能是自动开启的，而接收缓冲区则需要配置 tcp_moderate_rcvbuf 为 1 来开启调节功能

调节TCP内存范围

上面三个数字单位不是字节，而是「页面大小」，1 页表示 4KB，它们分别表示：

- 当 TCP 内存小于第 1 个值时，不需要进行自动调节；
- 在第 1 和第 2 个值之间时，内核开始调节接收缓冲区的大小；
- 大于第 3 个值时，内核不再为 TCP 分配新内存，此时新连接是无法建立的；

根据实际场景调节的策略

在高并发服务器中，为了兼顾网速与大量的并发连接，**我们应当保证缓冲区的动态调整的最大值达到带宽时延积，而最小值保持默认的 4K 不变即可。而对于内存紧张的服务而言，调低默认值是提高并发的有效手段。**

同时，如果这是网络 IO 型服务器，那么，**调大 tcp_mem 的上限可以让 TCP 连接使用更多的系统内存，这有利于提升并发能力**。需要注意的是，tcp_wmem 和 tcp_rmem 的单位是字节，而 tcp_mem 的单位是页面大小。而且，**千万不要在 socket 上直接设置 SO_SNDBUF 或者 SO_RCVBUF，这样会关闭缓冲区的动态调整功能。**

小结：

数据传输的优化策略	
策略	**TCP 内核参数**
扩大窗口大小	tcp_window_scaling
调整发送缓冲区范围	tcp_wmem
调整接收缓冲区范围	tcp_rmem
打开接收缓冲区动态调节	tcp_moderate_rcvbuf
调整内存范围	tcp_mem

HTTP/HTTPS

1. HTTP特性与简述

特性：简单、灵活、易于扩展、应用广泛和跨平台。

简述：

Web 上的通信都是建立在 HTTP 协议上的

1. 客户端发起 HTTP 请求；
2. 服务器做出响应处理后，返回 HTTP 响应报文

最初设想的基本理念是：借助多文档之间相互关联形成的超文本（HyperText），连成可相互参阅的 WWW（World Wide Web，万维网）

WWW 构建技术（3 项）：

1. HTML (HyperText Markup Language)：作为页面的文本标记语言
2. HTTP (HyperText Transfer Protocol)：文档传递协议；
3. URL (Uniform Resource Locator)：指定文档所在地址

HTTP 版本：

1. 1996 年的 5 月，HTTP/1.0，并记载于 RFC1945。现在 HTTP/1.0，仍然被使用在服务器端；
2. 1997 年 1 月，发布 HTTP/1.1 ，是目前主流的 HTTP 协议版本
3. 现今，HTTP/2.0 正在制订中，但还未得到广泛的使用

1.1 简单

基本报文格式为header+body，头部信息也是key-value简单文本的形式，易于理解。

1.2 灵活和易于扩展

1. HTTP协议里的各种请求方法、URI/URL、状态码、头字段等每个组成要求都没有被固定死，都允许开发人员自定义和扩充;
2. HTTP工作在应用层（OSI第七层），下层可以随意变化;
3. HTTPS就是在HTTP与TCP之间增加了SSL/TSL安全传输层，HTTP/3把TCP换成了基于UDP的QUIC。

1.3 无状态、明文传输、不安全

1、无状态：

服务器不会去记忆HTTP的状态，所以不需要额外的资源来记录状态信息，这能减轻服务器的负担。但它在完成有关联性的操作时会非常麻烦。

2、明文传输：

传输过程中的信息，是可方便阅读的，通过浏览器的F12控制台或Wireshark抓包都可以直接肉眼查看

为我们调试工作带来了极大的便利性，但信息透明，容易被窃取。

3、不安全：

1. 通信使用明文（不加密），内容可能被窃听
2. 不验证通信方的身份，因此有可能遭遇伪装
3. 无法证明报文的完整性，所以有可能已遭篡改

可以用 HTTPS 的方式解决，也就是通过引入 SSL/TLS 层，使得在安全上达到了极致。

2. HTTP/HTTPS的区别

协议名词释义

协议（protocol）：通信双方需要遵守的 统一规则。

HTTP：Hyper Text Transfer Protocol：超文本传输协议

HTTPS：Hyper Text Transfer Protocol Secure：超文本安全传输协议

SSL：Secure Socket Layer 安全套接字

TSL：Transport Layer Security 安全传输层协议

HTTPS = HTTP+SSL/TSL

3. HTTPS

HTTP： 以明文的方式在网络中传输数据

HTTPS 解决HTTP 不安全的缺陷，在 TCP 和 HTTP 网络层之间加入了 SSL/TLS 安全协议，使得报文能够加密传输。

HTTPS 在 TCP 三次握手之后，还需进行 SSL/TLS 的握手过程，才可进入加密报文传输。

HTTP 的端口号是 80，HTTPS 的端口号是 443。

HTTPS 协议需要向 CA（证书权威机构）申请数字证书，来保证服务器的身份是可信的。

3.1 特点

1. 信息加密：交互信息无法被窃取
2. 校验机制：无法篡改通信内容，篡改了就不能正常显示
3. 身份证书：证明报文的完整

3.2 优点

1. 在数据传输过程中，使用秘钥加密，安全性更高
2. 可认证用户和服务器，确保数据发送到正确的用户和服务器

3.3 缺点

1. 握手阶段延时较高：在会话前还需进行SSL握手
2. 部署成本高：需要购买CA证书；需要加解密计算，占用CPU资源，需要服务器配置或数目高

3.4 加密方式

1、对称加密：

只使用一个密钥，运算速度快，密钥必须保密，无法做到安全的密钥交换；

2、非对称加密：

使用两个密钥，公钥可以任意分发而私钥保密，解决密钥交换问题，但速度慢。

3、混合加密：

实现信息的机密性，解决窃听风险；

HTTPS采用对称加密和非对称加密结合的[混合加密]方式。

通信建立前：

采用非对称加密的方式交换[会话密钥]，后续不再使用非对称加密；

通信过程中：

全部使用对称加密的[会话密钥]方式，加密明文数据。

3.5 验证流程

1、client发起HTTP请求，连接到server端口

2、Server将自己的信息以数字证书的形式返回给client

（证书包含私钥公钥、网站地址、证书颁发机构、失效日期等）

3、验证证书合法性

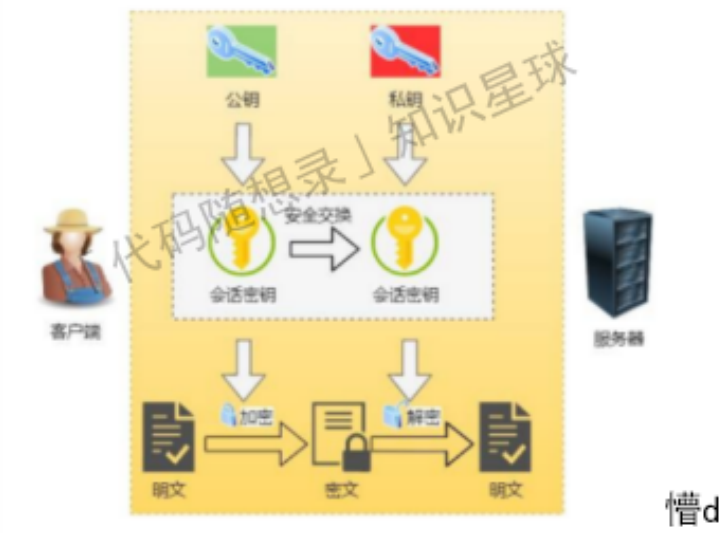
Server收到client响应后会先验证证书合法性（地址是否一致、是否过期）。

4、生成随机密码（RSA签名）

验证通过或用户接受了不受信任的证书，浏览器会生成一个随机的对称密钥（session key），并用公钥加密，让server用私钥解密，解密后用这个对称密钥进行传输；

5、生成对称加密算法

验证server身份后，client生成一个对称加密的算法和对应密钥，以公钥加密后发送给server。（只有server的私钥才能对其进行解密）



代码随想录」知识星球

槽d

3.6 数字证书的相关信息

1、摘要算法用来实现完整性

2、数字签名是私钥对摘要的加密

实现了身份认证和不可抵赖性

3、公钥的分发需要使用数字证书

必须由CA的信任链来认证，否则就是不可信的

4、证书体系中，Root CA最大，也叫根证书

相当于是自己证明自己的，必须相信，如果不相信，这整个信任链就走不下去了。上网的时候只要有证书，就可以顺着一直找到根证书来确定是否可信;

5、如果遇到CA伪造

就会加这个CA加入CRL证书吊销列表中，终止信任。

3.7 数字证书的获取过程

1. 根据服务器公钥、证书颁发者、证书用途、过期时间等经哈希计算得到值H1;
2. 将H1由CA私钥加密生成签名数字证书;
3. 客户端收到数字证书后经同样哈希计算得到值H2;
4. 经嵌入在浏览器或者操作系统的CA公钥解密数字证书，对比H1、H2相等验证服务器真实性，取出服务器公钥。

3.8 输入URL到显示的过程

1、相关概念

（1）URL

uniform resolution locator，统一资源定位符，定位互联网上资源（俗称网址）

（2）DNS

domain name system/server，域名解析服务器

（3）IP

internet protocol address，互联网上的每个网络和每台主机分配一个逻辑地址。

2、URL

格式： Scheme：//host.domain:port/path/filename

1. Scheme：定义因特网服务类型 http/https/ftp/file
2. Host：定义域主机（http默认主机为www）
3. Domain：因特网域名。Baidu.com
4. Port：主机上端口号（http：80，https：443）
5. Path：服务器上的路径（若省略，则文档必位于网站根目录）
6. Filename：文档/资源名称。

3、DNS

（1）概念

1. DNS协议用来将域名转换为IP地址，也可将IP地址转换为相应的域名地址
2. DNS：面向用户 IP：面向主机
3. 域名服务主要是基于UDP实现的，服务器端口号为53

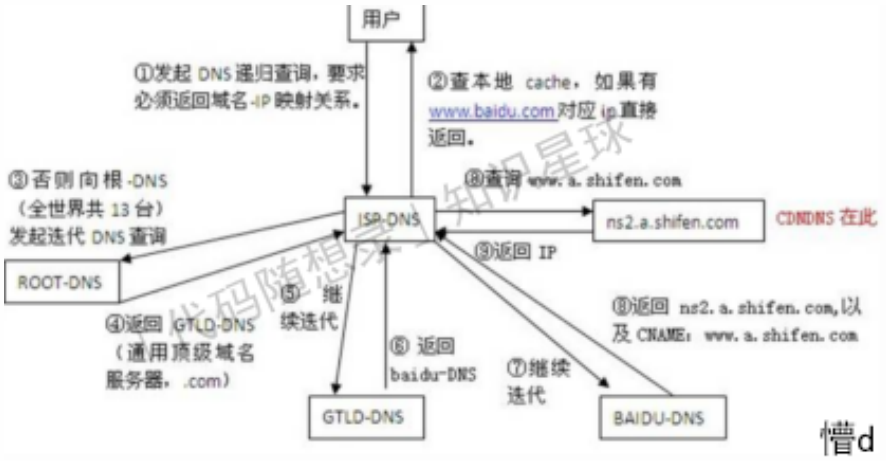
（2）解析过程

浏览器查询URL对应IP：浏览器缓存→操作系统缓存→路由器缓存;
三种类型的DNS服务器：根DNS服务器、顶级域DNS服务器、权威DNS服务器;

1. 浏览器检查浏览器自身的DNS缓存中，是否有域名对应的DNS缓存
2. 查看系统的hosts文件（C:\Windows\System32\drivers\etc）是否有域名对应的IP地址
3. 浏览器发起DNS系统调用，向本地配置的首选DNS服务器发起域名解析请求（通过UDP协议，向DNS的53端口发起请求）
4. 首先，请求会在运营商DNS服务器（本地服务器）上进行请求，若找到对应的条目，且没有过期，则解析成功；否则，进入5
5. 运营商DNS服务器，根据解析请求，迭代查询，首先找到根域名服务器IP地址，然后找到根域的DNS地址，发送请求
6. 根域服务器收到请求后，根据域名，返回对应的顶级域服务器IP地址，并返回给运营商DNS服务器
7. 运营商DNS服务器收到根域名服务器传回来的顶级域名服务器IP地址后，向顶级域名服务器发送请求
8. 顶级域名服务器收到请求后，返回该域名对应的权威域名服务器IP地址，并返回给运营商DNS服务器
9. 运营商DNS服务器获得权威域名服务器的响应信息后，返回给请求的主机，DNS解析完成

DNS主要通过UDP通信，报文结构分为头部header、查询部分qurestion、应答部分answer/authority/addition

例如：查询百度对应IP



4. HTTP请求

http请求为DNS解析获得IP地址后，通过TCP三次握手、HTTP请求响应信息、关闭TCP连接。

4.1 当三次握手之后，发送HTTP请求报文

其主要由请求行、请求头、请求体构成

1、请求行 -> (请求方法；URL；协议版本号)

请求方法： GET、POST、PUT、DELETE、PATCH、HEAD、OPTIONS、TRACE

URL： <协议>：//<主机>：<端口>/<路径>?<参数>

协议版本号： HTTP版本号

例如：POST /chapter17/user.html HTTP/1.1

2、请求头

包含请求的附加信息，有key：value组成

3、请求体

承载多个请求参数的数据。含回车、换行和请求数据（并非都有）

4.2 服务器处理请求并返回HTTP报文

响应报文主要由响应行、响应头和响应主体构成。

1、相应行 -> (协议版本；状态码；状态码描述)

2、状态码规则

3、常见状态码以及描述

200：客户端请求成功

206：partial content 服务器已经正确处理部分GET请求，实现断点续传或同时分片下载，该请求必须包含Range请求头来指示客户端期望得到的范围

301（永久重定向）：该资源已被永久移动到新位置，将来任何对该资源的访问都要使用本响应返回的若干个URL之一

302（临时重定向）：请求的资源现在临时从不同的URI中获得

304：如果客户端发送一个待条件的GET请求并且该请求以经被允许，而文档内容未被改变，则返回304,该响应不包含包体（即可直接使用缓存）

400：请求报文语法有误，服务器无法识别

401：请求需要认证

403：请求的对应资源禁止被访问

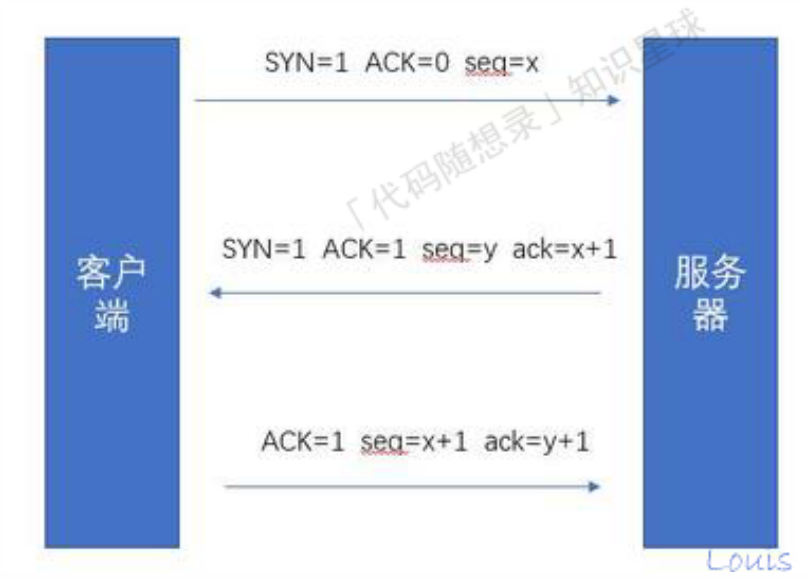
404：服务器无法找到对应资源

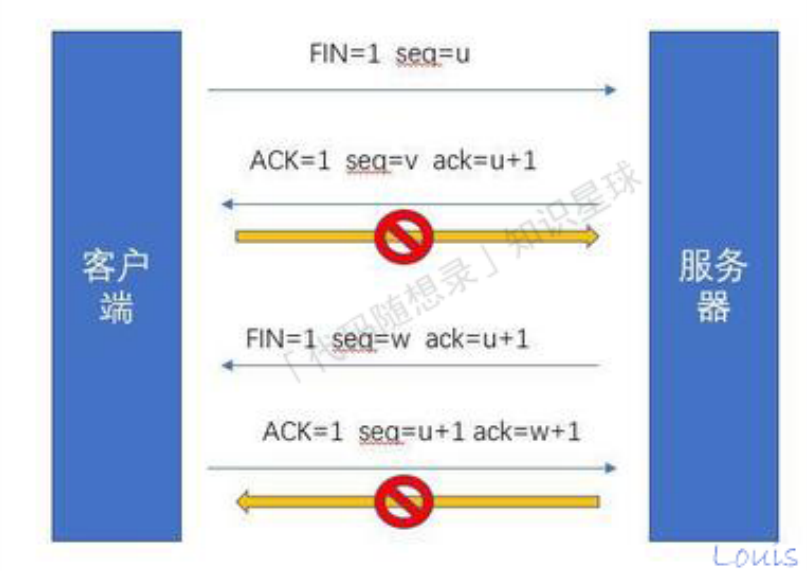
500：服务器内部错误

503：服务器正忙

5. HTTP请求过程

1. 首先，我们在浏览器地址栏中，输入要查找页面的URL，按下Enter
2. 浏览器依次在 浏览器缓存 -->>>系统缓存 -->>>路由器缓存中去寻找匹配的URL，若有，就会直接在屏幕中显示出页面内容。若没有，则跳到第三步操作
3. 发送HTTP请求前，浏览器需要先进行域名解析(即DNS解析)，以获取相应的IP地址;（浏览器DNS缓存、路由器缓存、DNS缓存）
4. 获取到IP地址之后，浏览器向服务器发起TCP连接，与浏览器建立TCP三次握手
5. 握手成功之后，浏览器就会向服务器发送HTTP请求，来请求服务器端的数据包
6. 服务器处理从浏览器端收到的请求，接着将数据返回给浏览器
7. 浏览器收到HTTP响应
8. 查询状态，状态成功则进行下一步，不成功则弹出相应指示
9. 再读取页面内容、进行浏览器渲染、解析HTML源码;（生成DOM树、解析CCS样式、处理JS交互，客户端和服务器交互）进行展示
0. 关闭TCP连接（四次挥手）





6. HTTP请求方法

1. GET：申请获取资源，不对服务器产生影响
2. POST：客户端向服务器提交数据。会影响服务器，服务器可能动态创建新的资源或更新原有资源
3. HEAD：类似GET，仅要求服务器返回头部信息
4. PUT：上传某个资源
5. DELETE：删除某个资源
6. TRACE：用于测试。要求目标服务器返回原始的HTTP请求内容
7. CONNECT：用于代理服务器
8. OPTION：查询服务器对特定URL支持的请求方法

7. GET和POST的区别

7.1 概念

GET和POST都是HTTP请求方法

1、GET

申请获取资源，不对服务器产生影响

2、POST

客户端向服务器提交数据，会影响服务器，服务器可能动态创建新的资源或更新原有资源

请求数据：若为GET，则请求数据为空；其主要是在POST中进行使用，使用于需要填表单(FORM)场景

7.2 请求报文

1、GET请求报文：

2、POST请求报文：

请求行中请求方法为POST，URL为空，协议版本为HTTP1.1

与GET不同在于它的请求参数是位于请求数据中，最后一行name...为请求数据，且，请求数据和请求头之间必须空出一行

3、安全+幂等：

安全：HTTP协议中，安全是指请求方法不会破坏服务器上的资源

幂等：多次执行相同的操作，结果都相同

GET为安全幂等的，因为它为只读操作，无论操作多少次，服务器上的数据都是安全的，且每次的结果都是相同的

7.3 特点

1、GET

1. GET会被浏览器主动缓存的，如果下一次传输的数据相同，那么就返回缓存中的内容，以求更快的展示数据
2. GET的URL一般都有长度限制，但需注意HTTP协议中并未规定GET请求的长度。这个长度限制主要是由浏览器和Web服务器所决定的，并且各个浏览器对长度的限制也各不相同
3. GET方法只产生一个TCP数据包，浏览器会把请求头和请求数据一并发送出去，服务器响应200 ok（返回数据）。

2、POST

1. POST不安全且不幂等：因为是新增或者提交数据的操作，会修改服务器上的资源；且多次提交数据就会创建多个资源
2. POST方法的请求信息放置在请求数据中，所以其请求信息没有长度限制
3. POST会产生两个TCP数据包，浏览器会先将请求头发送给服务器，待服务器响应100 continue，浏览器再发送请求数据，服务器响应200 ok（返回数据）

7.4 区别

实际上本质都是一样的，并无区别

1. 都是HTTP请求协议的请求方法，而HTTP又是基于TCP/IP的关于数据如何在万维网中如何通信的协议，所以GET/POST实际上都是TCP链接
2. 由于HTTP的规定以及浏览器/服务器的限制，导致它们在应用过程中可能会有所不同

8. HTTP keep-alive

1、重用连接的机制

2、使用方法

在请求头中加入Connecttion：

keep-alive，通知对应在该请求响应完成之后不要关闭，下一次继续用

问：HTTP能不能一次连接多次请求，不等后端返回？

答：可以，HTTP本质是使用socket连接，写入TCP缓冲是可以连接多次的

HTTP无状态：

即使第一次和服务器连接并且登录成功之后，第二次请求服务器仍然不知道当前请求的是哪个用户

9. cookie和session

9.1 cookie

1、作用

1. 在第一次登录服务器之后，返回一些数据（cookie）给浏览器
2. 浏览器将数据保存在本地
3. 两次发送请求时，自动把上一次请求存储的cookie发送给服务器
4. 服务器通过该数据判断用户

2、特点

可存储的数据量有限，一般不会超过4KB，Cookie的保存形式分为会话Cookie和持久Cookie

9.2 session

session的作用与cookie类似，都是为了存储用户相关的信息

1、区别

1. cookie存储在本地浏览器的数据
2. session存储在服务器的数据
3. Cookie存储数据的大小有限制，而Session一般无限制
4. Cookie对用户信息的生命周期的控制方式为累计方式，而Session使用间隔方式

2、优势

数据存储在服务器更加的安全

3、缺陷

会占用服务器资源

9.3 SSO

普通的登录认证机制

普通登录认证机制在登录认证成功后，服务器把用户的登录信息写入 session，并为该用户生成一个 cookie，返回并写入浏览器；当用户再次访问这个系统的时候，请求中会带上这个 cookie，服务端会根据这个 cookie 找到对应的 session，通过session来判断这个用户是否已经登录。

普通的登录认证机制在多系统的环境下，在操作不同的系统时，需要多次登录，会变得很不方便。

单点登录（英语：Single sign-on，缩写为 SSO），在一个多系统的环境中，用户只需要登录一次，就可以同时登陆访问其他互相信任的系统。

SSO的优点

- 降低访问第三方网站风险（用户密码不存储或外部管理）；
- 从不同的用户名和密码的组合减少密码疲劳；
- 减少花费的时间重新输入密码相同的身份；
- 降低IT成本适当降低一些IT帮助台调用有关密码；
- SSO集中的所有其他应用程序和系统，用于身份验证服务器的身份验证，并与技术相结合是为了确保用户不必主动输入凭据一次以上。

10. http版本特性

10.1 HTTP1.1版本新特性

1、默认持久连接

只要客户端任意一端没有明确提出断开TCP连接，就一直保持连接，可以发送多次HTTP请求
HTTP1.0默认使用短连接，而HTTP1.1默认使用长链接

2、管线化

客户端可以同时发送多个HTTP请求，不用等待响应

3、断点续传

利用HTTP消息头使用分块传输编码，将实体主体进行分块传输

10.2 HTTP2.0版本新特性

1、传输格式变化，采用了新的二进制格式

HTTP1.X的解析都是基于文本，文本的表现形式多样，不利于健壮性考虑
HTTP2.0采用二进制，只认0/1组合，实现更加快的方法，健壮性更加完善

2、多路复用

连接共享
一个请求对应一个ID，每个连接都可以有多个请求
接收方可以根据请求的ID将请求再归属到各自不同的服务端请求中
可以极大的提高效率

3、header压缩

在HTTP1.X中，header带有大量信息，而且每次都要重复发送

HTTP2.0通过encoder减少header大小，通讯双方会各自缓存一份header字段表

既可以避免重复header传输，又减小了需要传输的大小

4、服务端推送

把客户端所需要的资源伴随着index.html一起发送到客户端，省去了客户端重复请求的步骤

因为没有发起请求，建立连接等操作，所以静态资源通过服务器推送，可以极大的提升速度

11. HTTP优化方案

1、TCP复用

将多个客户端的HTTP请求复用到一个服务端的TCP连接上

2、HTTP复用

将一个客户端的多个HTTP请求复用到一个TCP连接进行处理

3、内容缓存

将经常用到的内容进行缓存，在下一次获取的时候，就可以直接在内存中获取相应的数据了

4、压缩

文本数据压缩，减少带宽

5、SSL加速

6、TCP缓冲

通过采用TCP缓冲技术，提高服务端响应时间和处理效率

HTTP2的优点

思维导图

HTTP1.1协议的性能问题

1、HTTP1.1常见性能问题

- *延迟难以下降*，虽然现在网络的「带宽」相比以前变多了，但是延迟降到一定幅度后，就很难再下降了，说白了就是到达了延迟的下限；
- *并发连接有限*，谷歌浏览器最大并发连接数是 6 个，而且每一个连接都要经过 TCP 和 TLS 握手耗时，以及 TCP 慢启动过程给流量带来的影响；
- *队头阻塞问题*，同一连接只能在完成一个 HTTP 事务（请求和响应）后，才能处理下一个事务；
- *HTTP 头部巨大且重复*，由于 HTTP 协议是无状态的，每一个请求都得携带 HTTP 头部，特别是对于有携带 cookie 的头部，而 cookie 的大小通常很大；
- *不支持服务器推送消息*，因此当客户端需要获取通知时，只能通过定时器不断地拉取消息，这无疑浪费大量了带宽和服务器资源。

2、为解决HTTP1.1性能问题而提出的常见优化手段：

- 将多张小图合并成一张大图供浏览器 JavaScript 来切割使用，这样可以将多个请求合并成一个请求，但是带来了新的问题，当某张小图片更新了，那么需要重新请求大图片，浪费了大量的网络带宽；
- 将图片的二进制数据通过 base64 编码后，把编码数据嵌入到 HTML 或 CSS 文件中，以此来减少网络请求次数；
- 将多个体积较小的 JavaScript 文件使用 webpack 等工具打包成一个体积更大的 JavaScript 文件，以一个请求替代了很多个请求，但是带来的问题，当某个 js 文件变化了，需要重新请求同一个包里的所有 js 文件；
- 将同一个页面的资源分散到不同域名，提升并发连接上限，因为浏览器通常对同一域名的 HTTP 连接最大只能是 6 个；

上述优化手段都只是“外部”优化，效果不够好

兼容HTTP1.1

HTTP2，为改善HTTP1.1而提出，同时兼容了HTTP1.1，具体做法如下：

第一点，HTTP/2 没有在 URI 里引入新的协议名，仍然用「http://」表示明文协议，用「https://」表示加密协议，于是只需要浏览器和服务器在背后自动升级协议，这样可以让用户意识不到协议的升级，很好的实现了协议的平滑升级。

第二点，只在应用层做了改变，还是基于 TCP 协议传输，应用层方面为了保持功能上的兼容，HTTP/2 把 HTTP 分解成了「语义」和「语法」两个部分，「语义」层不做改动，与 HTTP/1.1 完全一致，比如请求方法、状态码、头字段等规则保留不变。

但是，HTTP/2 在「语法」层面做了很多改造，基本改变了 HTTP 报文的传输格式。

头部压缩

1、HTTP1.1报文中Header部分存在的问题：

- 含很多固定的字段，比如Cookie、User Agent、Accept 等，这些字段加起来也高达几百字节甚至上千字节，所以有必要**压缩**；
- 大量的请求和响应的报文里有很多字段值都是重复的，这样会使得大量带宽被这些冗余的数据占用了，所以有必须要**避免重复性**；
- 字段是 ASCII 编码的，虽然易于人类观察，但效率低，所以有必要改成**二进制编码**；

2、HTTP2的改造：

使用HPACK算法压缩头部。HPACK算法包含三个组成部分：

（i）静态字典
（ii）动态字典
（iii）Huffman编码（压缩算法）

客户端和服务器都会建立和维护【字典】，用长度较小的索引号表示重复的字符串，再用Huffman编码压缩数据。

（1）静态表编码：

HTTP2 为高频出现在头部的字符串和字段建立了一张静态表，它是写入到 HTTP2 框架里的，不会变化的，静态表里共有 61 组：

Index	Header Name	Header Value
1	:authority	
2	:method	GET
3	:method	POST
4	:path	/
5	:path	/index.html
6	:scheme	http
7	:scheme	https
8	:status	200
...	...	...
54	server	
55	set-cookie	
56	strict-transport-security	
57	transfer-encoding	
58	user-agent	
59	vary	
60	via	
61	www-authenticate	

「代码随想录」知识星球

（2）动态表编码：

使得动态表生效的一个前提：必须同一个连接上，重复传输完全相同的HTTP头部。

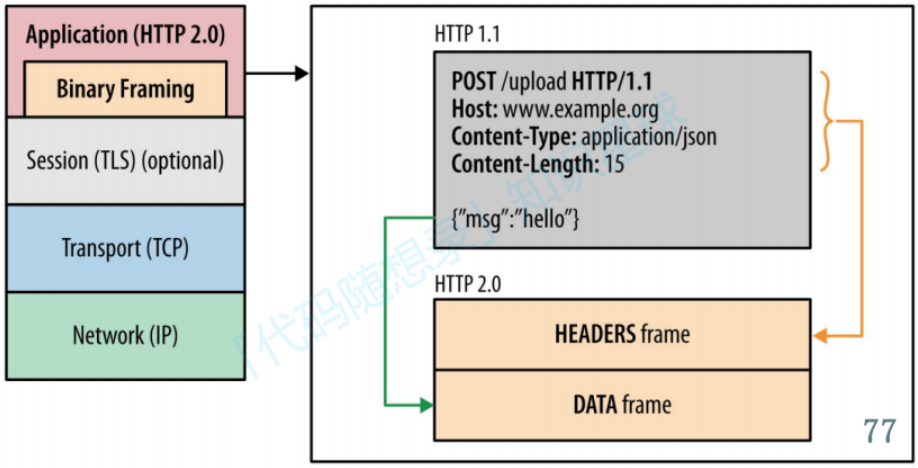
动态表越大，占用的内存也就越大，如果占用了太多内存，是会影响服务器性能的，因此 Web 服务器都会提供类似 http2_max_requests 的配置，用于限制一个连接上能够传输的请求数，避免动态表无限增大，请求数到达上限后，就会关闭 HTTP/2 连接来释放内存。

HTTP2头部的编码通过【静态表、动态表、Huffman编码】共同完成。

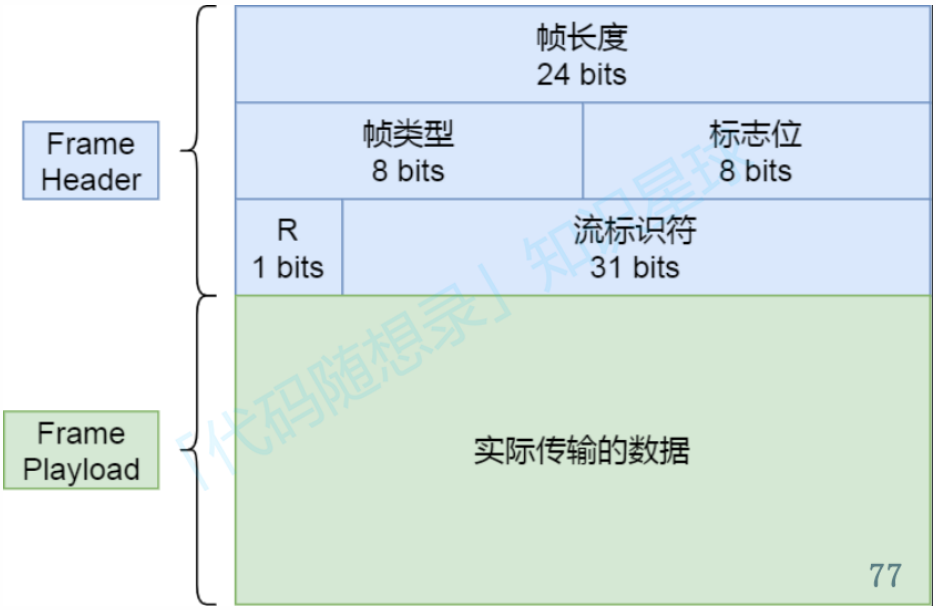
二进制帧

HTTP2更厉害的地方在于：将HTTP1的文本格式改成二进制格式传输数据，极大提高了HTTP传输效率，而且二进制数据使用位运算能高效解析。

两者响应区别（如下图）：



HTTP2二进制帧的结构：



HTTP2总共定义了10种类型的帧，分为两类，即数据帧和控制帧：

	帧类型	类型编码	用途
数据帧	DATA	0x0	传递HTTP包体
	HEADERS	0x1	传递HTTP头部
	PRIORITY	0x2	指定Stream流的优先级
控制帧	RST_STREAM	0x3	终止Stream流
	SETTINGS	0x4	修改连接或者Stream流的配置
	PUSH_PROMISE	0x5	服务端推送资源时描述请求的帧
	PING	0x6	心跳检测，兼具计算RTT往返时间的功能
	GOAWAY	0x7	优雅的终止连接或者通知错误
	WINDOW_UPDATE	0x8	实现流量控制
	CONTINUATION	0x9	传递较大HTTP头部时的持续帧

77

标志位用于携带简单的控制信息，比如：

- **END_HEADERS** 表示头数据结束标志，相当于 HTTP/1 里头后的空行（“\r\n”）；
- **END_STREAM** 表示单方向数据发送结束，后续不会再有数据帧。
- **PRIORITY** 表示流的优先级；

流标识符（Stream ID)：

用来标识该Fream属于哪个Stream。接收方可以根据这个信息从乱序的帧里找到相同Stream ID的帧，从而有序组装信息。

帧数据：

用于存放 通过HPACK算法压缩过的HTTP头部和包体。

并发传输

1、HTTP1.1与HTTP2的比较

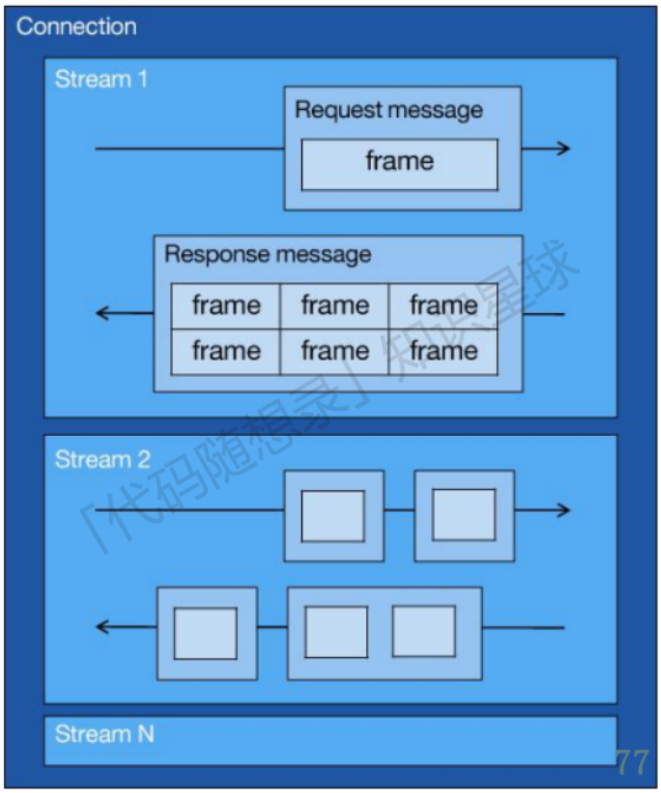
HTTP1.1 基于请求-响应模型。同一个连接种，HTTP完成一个事务（请求与响应），才能处理下一个事务。即：再发出请求等待响应的过程种是没办法做其他事情的，会造成【队头阻塞】问题。

HTTP2通过Stream这个设计（多个Stream复用一条TCP连接，达到并发的效果），解决了【队头阻塞】的问题，提高了HTTP传输的吞吐量。

2、HTTPS并发的实现

HTTP2并发是如何实现的呢?

先来理解三个概念，即：Stream、Message、frame。



从上图可以看到：

- 1 个 TCP 连接包含一个或者多个 Stream，Stream 是 HTTP/2 并发的关键技术；
- Stream 里可以包含 1 个或多个 Message，Message 对应 HTTP/1 中的请求或响应，由 HTTP 头部和包体构成；
- Message 里包含一条或者多个 Frame，Frame 是 HTTP/2 最小单位，以二进制压缩格式存放 HTTP/1 中的内容（头部和包体）；

1. HTTP消息可以由多个Frame构成
2. 一个Frame可以由多个TCP报文构成

在HTTP2连接上，不同Stream的帧可以乱序发送（因此可以并发不同的Stream），接收端可以通过Stream ID 有序组装HTTP消息。

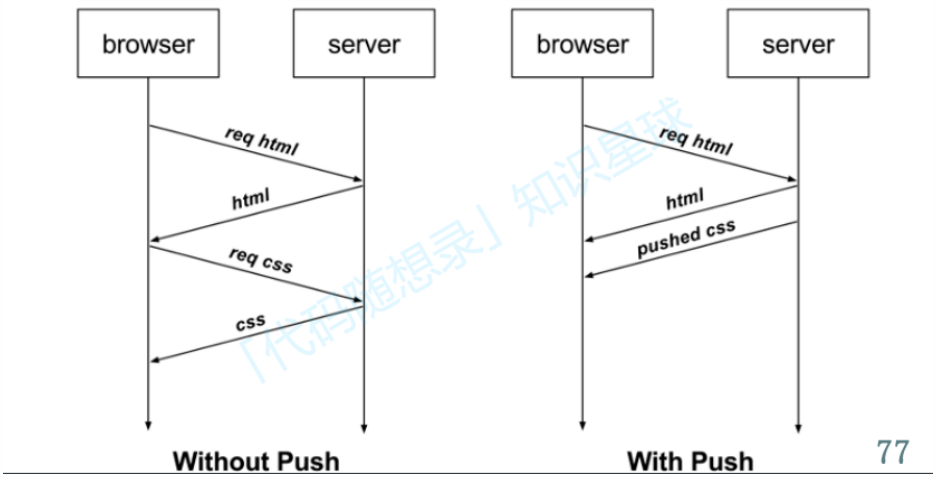
HTTP/2 通过 Stream 实现的并发，比 HTTP/1.1 通过 TCP 连接实现并发要牛逼的多，**因为当 HTTP/2 实现 100 个并发 Stream 时，只需要建立一次 TCP 连接，而 HTTP/1.1 需要建立 100 个 TCP 连接，每个 TCP 连接都要经过 TCP 握手、慢启动以及 TLS 握手过程，这些都是很耗时的。**

HTTP/2 还可以对每个 Stream 设置不同**优先级**，帧头中的「标志位」可以设置优先级，比如客户端访问 HTML/CSS 和图片资源时，希望服务器先传递 HTML/CSS，再传图片，那么就可以通过设置 Stream 的优先级来实现，以此提高用户体验。

服务器主动推送资源

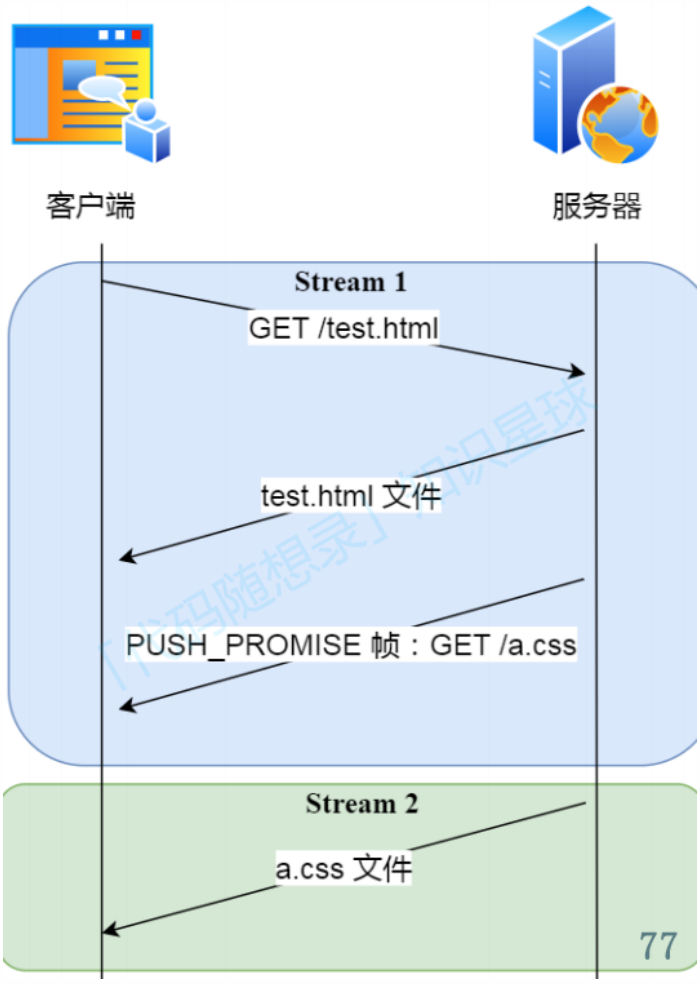
1、HTTP1.1与HTTP2的比较：

HTTP1.1不支持服务器主动推送资源给客户端，都是由客户端向服务器发起请求后，才能获取到服务器响应的资源。在HTTP2中，客户端在访问HTML时，服务器可以直接主动推送CSS文件，减少了消息传递的次数。



HTTP2推送的具体实现方式：

客户端发起的请求，必须使用的是奇数号 Stream，服务器主动的推送，使用的是偶数号 Stream。服务器在推送资源时，会通过 PUSH_PROMISE 帧传输 HTTP 头部，并通过帧中的 Promised Stream ID 字段告知客户端，接下来会在哪个偶数号 Stream 中发送包体。



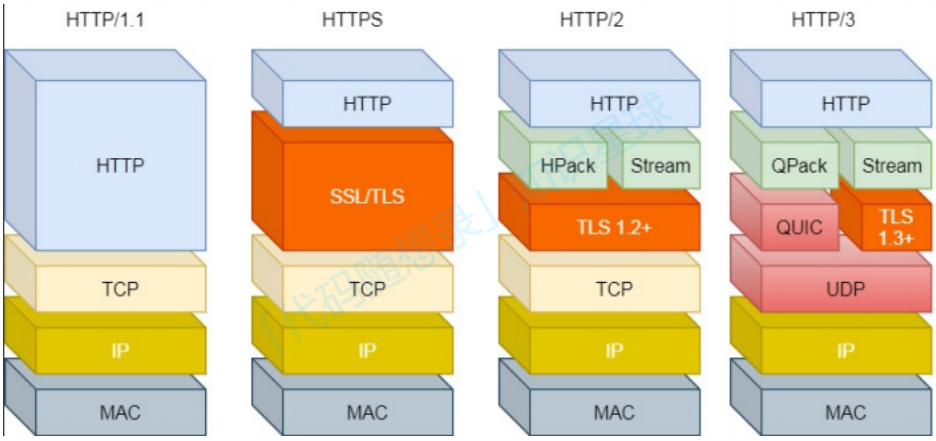
总结

HTTP2协议还包括：流控制、流状态、依赖关系等。

本次主要介绍了关于HTTP如何提升性能：

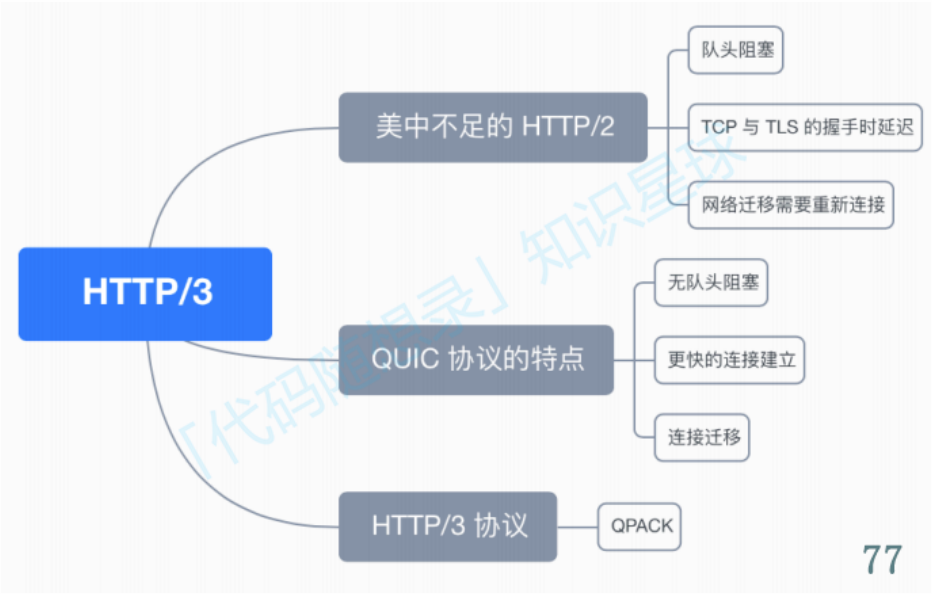
> 第一点，对于常见的 HTTP 头部通过**静态表和 Huffman 编码**的方式，将体积压缩了近一半，而且针对后续的请求头部，还可以建立**动态表**，将体积压缩近 90%，大大提高了编码效率，同时节约了带宽资源。
>
> 不过，动态表并非可以无限增大， 因为动态表是会占用内存的，动态表越大，内存也越大，容易影响服务器总体的并发能力，因此服务器需要限制 HTTP/2 连接时长或者请求次数。
>
> 第二点，**HTTP/2 实现了 Stream 并发**，多个 Stream 只需复用 1 个 TCP 连接，节约了 TCP 和 TLS 握手时间，以及减少了 TCP 慢启动阶段对流量的影响。不同的 Stream ID 才可以并发，即时乱序发送帧也没问题，但是同一个 Stream 里的帧必须严格有序。
>
> 另外，可以根据资源的渲染顺序来设置 Stream 的**优先级**，从而提高用户体验。
>
> 第三点，**服务器支持主动推送资源**，大大提升了消息的传输性能，服务器推送资源时，会先发送 PUSH_PROMISE 帧，告诉客户端接下来在哪个 Stream 发送资源，然后用偶数号 Stream 发送资源给客户端。
>
> HTTP/2 通过 Stream 的并发能力，解决了 HTTP/1 队头阻塞的问题，看似很完美了，但是 HTTP/2 还是存在“队头阻塞”的问题，只不过问题不是在 HTTP 这一层面，而是在 TCP 这一层。
>
> **HTTP/2 是基于 TCP 协议来传输数据的，TCP 是字节流协议，TCP 层必须保证收到的字节数据是完整且连续的，这样内核才会将缓冲区里的数据返回给 HTTP 应用，那么当「前 1 个字节数据」没有到达时，后收到的字节数据只能存放在内核缓冲区里，只有等到这 1 个字节数据到达时，HTTP/2 应用层才能从内核中拿到数据，这就是 HTTP/2 队头阻塞问题。**
>
> 有没有什么解决方案呢？既然是 TCP 协议自身的问题，那干脆放弃 TCP 协议，转而使用 UDP 协议作为传输层协议，这个大胆的决定， HTTP/3 协议做了！

如下图所示：



HTTP3的优点

思维导图



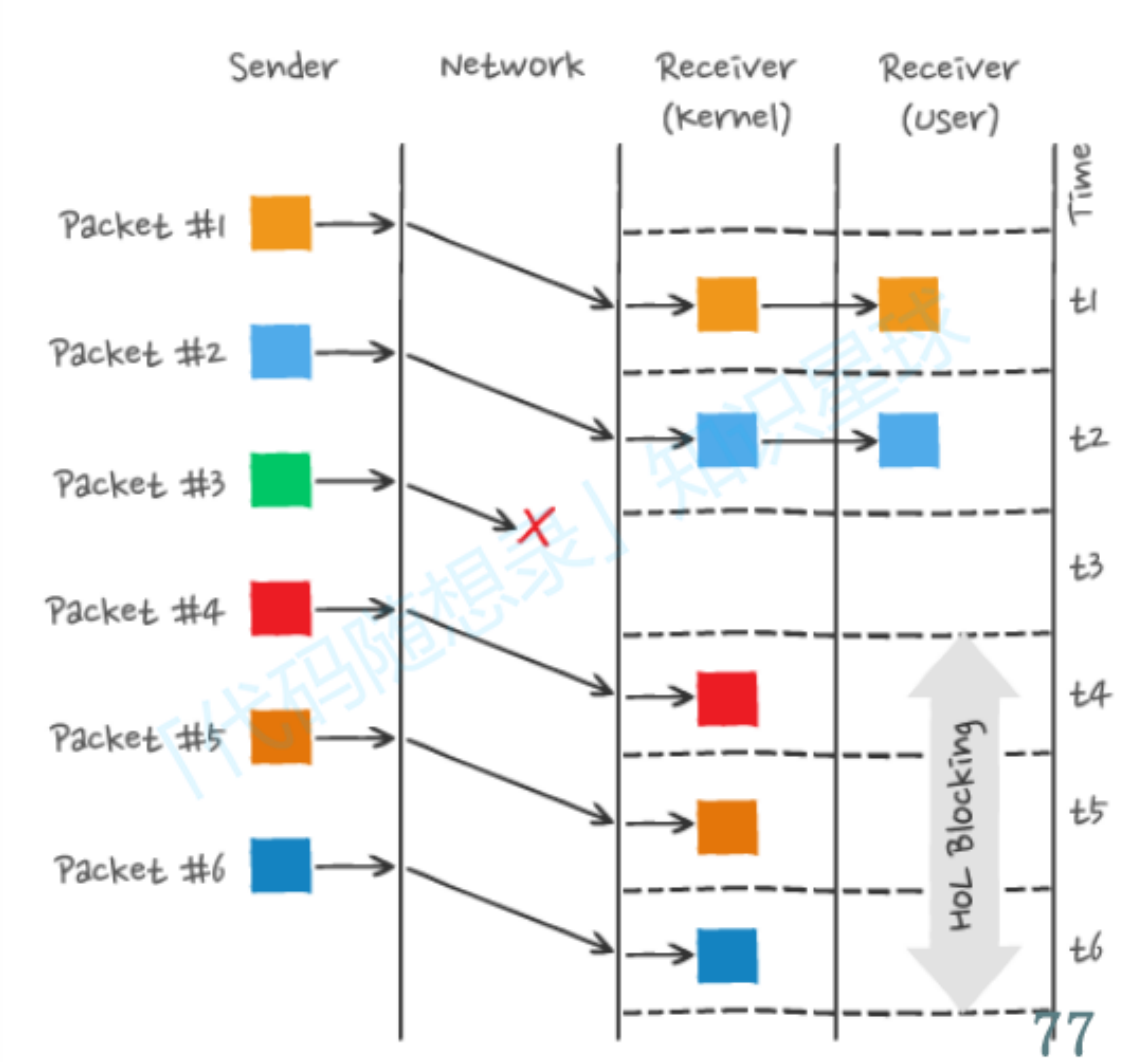
Http2的缺点

HTTP2协议是基于TCP实现的，所以存在三个缺陷：

（i）队头阻塞
（ii）TCP与TLS的握手时延迟
（iii)网络迁移需要重新连接

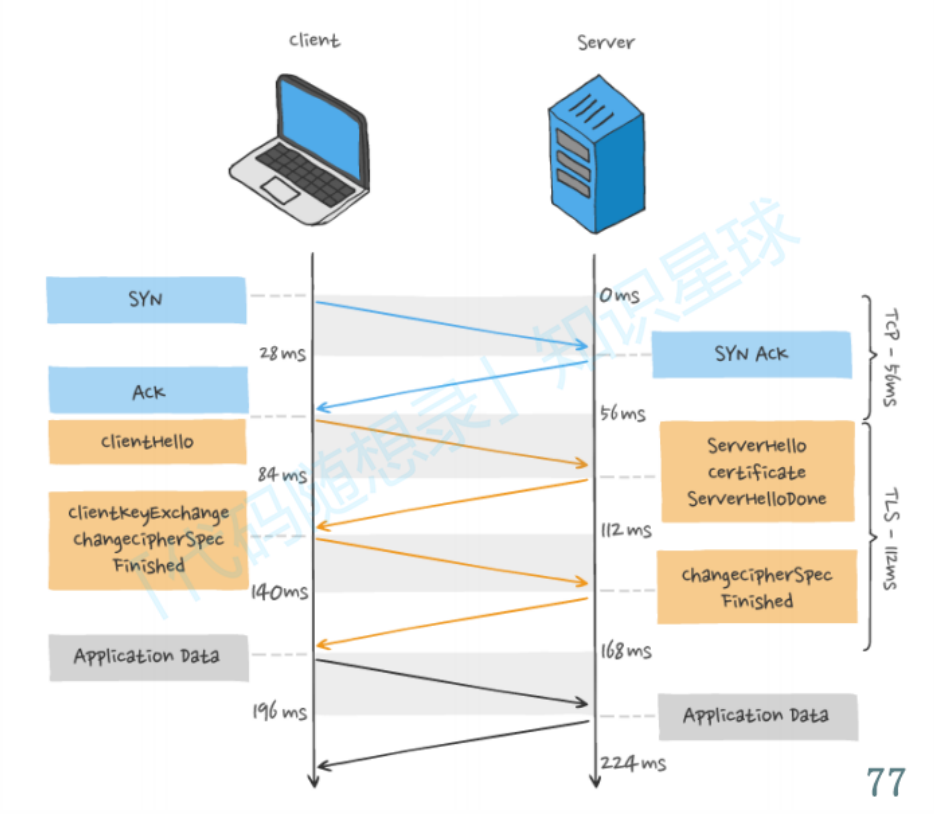
1、队头阻塞

TCP 是字节流协议，TCP 层必须保证收到的字节数据是完整且有序的，如果序列号较低的 TCP 段在网络传输中丢失了，即使序列号较高的 TCP 段已经被接收了，应用层也无法从内核中读取到这部分数据，从 HTTP 视角看，就是请求被阻塞了。



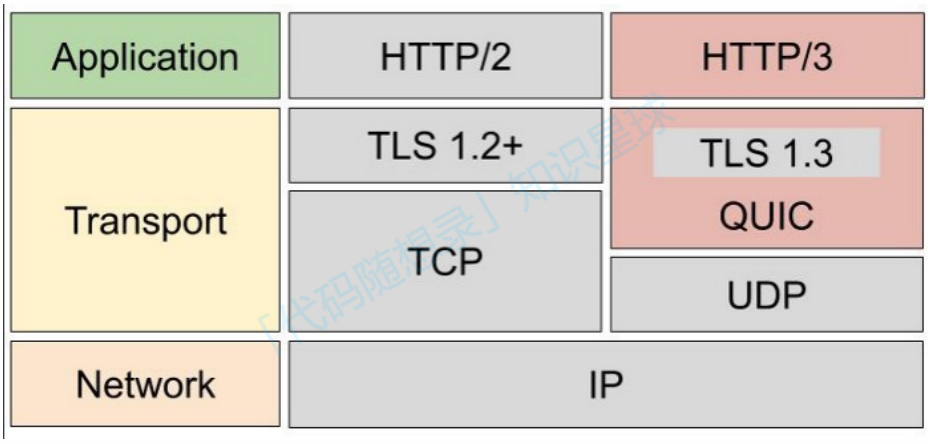
2、TCP与TLS的握手时延迟

发出HTTP请求时，需要经过TCP三次握手和TLS四次握手，共计3RTT的时延才能发出请求数据。



3、网络迁移需要重新连接

一个TCP连接由【源IP地址，源端口，目标IP地址，目标端口】确定。若IP地址或端口发生变暖，这需要重新进行连接。这不利于移动设备切换网络的场景。要解决该问题，就要修改传输层协议。在HTTP3中传输层协议修改为了UDP。



QUIC协议的特点

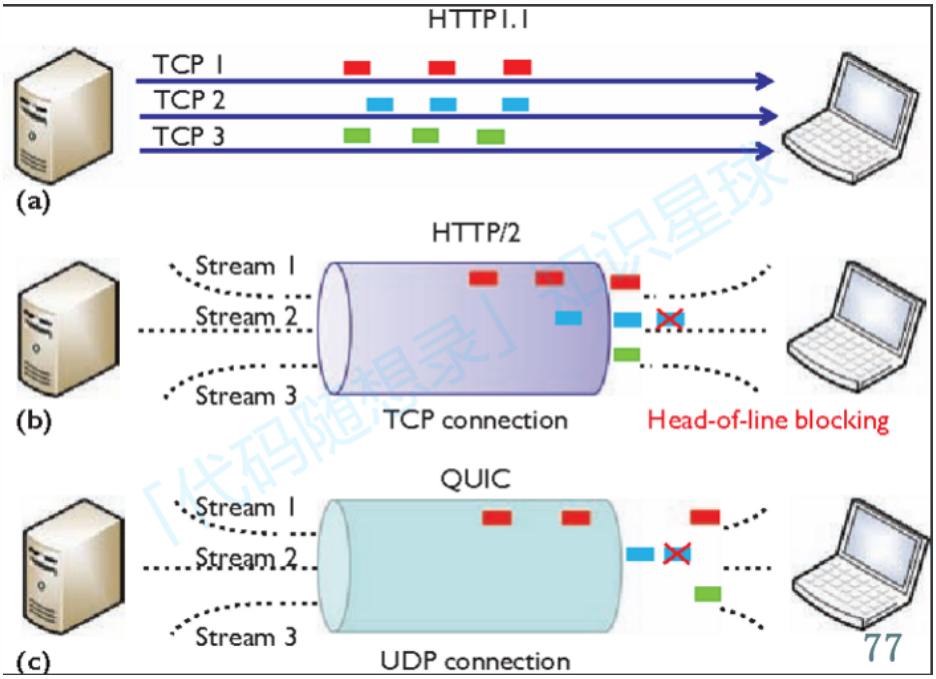
UDP是一个简单的、不可靠的传输协议，而且UDP包之间是无序的，也没有依赖关系。UDP也不需要连接。

HTTP3基于UDP协议在 应用层 实现了QUIC协议，它有类似TCP的连接管理、拥塞窗口、流量控制的网络特性，相当于将不可靠的UDP协议变成可靠的了，无需担心数据包丢包的问题。

1、无队头阻塞

HTTP2:只要某个流中的数据包丢失了，其他流也会因此受影响。

HTTP3:流与流（Stream)之间不影响。

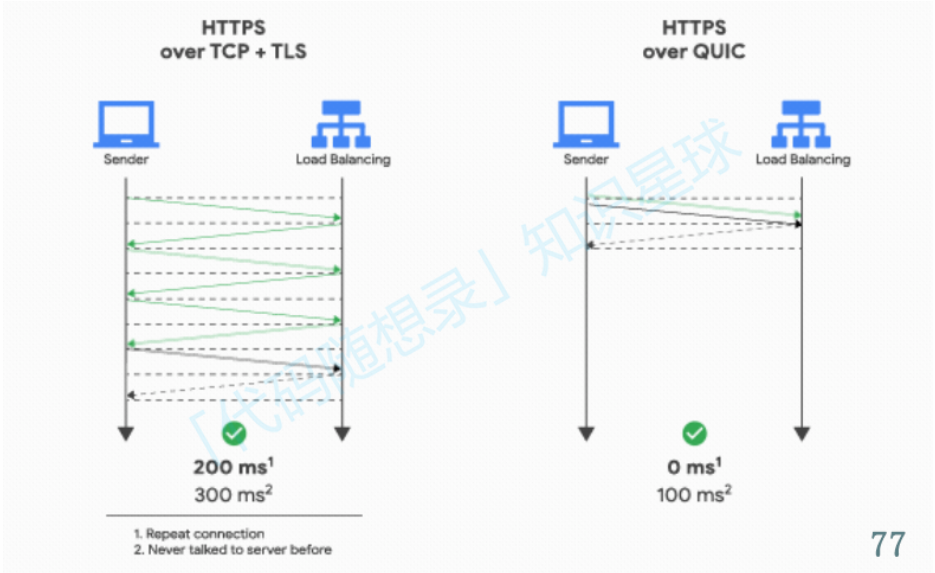


2、更快的连接建立

对于 HTTP/1 和 HTTP/2 协议，TCP 和 TLS 是分层的，分别属于内核实现的传输层、openssl 库实现的表示层，因此它们难以合并在一起，需要分批次来握手，先 TCP 握手，再 TLS 握手。

HTTP/3 在传输数据前虽然需要 QUIC 协议握手，这个握手过程只需要 1 RTT，握手的目的是为确认双方的「连接 ID」，连接迁移就是基于连接 ID 实现的。

但是 HTTP/3 的 QUIC 协议并不是与 TLS 分层，而是QUIC 内部包含了 TLS，它在自己的帧会携带 TLS 里的“记录”，再加上 QUIC 使用的是 TLS1.3，因此仅需 1 个 RTT 就可以「同时」完成建立连接与密钥协商，甚至在第二次连接的时候，应用数据包可以和 QUIC 握手信息（连接信息 + TLS 信息）一起发送，达到 0-RTT 的效果。



3、连接迁移

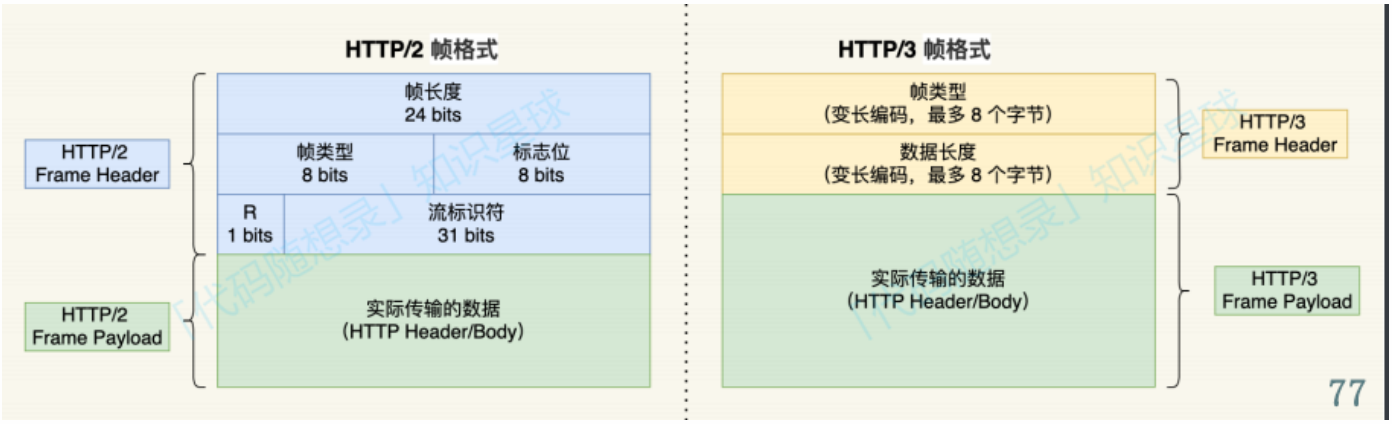
在前面我们提到，基于 TCP 传输协议的 HTTP 协议，由于是通过四元组（源 IP、源端口、目的 IP、目的端口）确定一条 TCP 连接，那么当移动设备的网络从 4G 切换到 WIFI 时，意味着 IP 地址变化了，那么就必须要断开连接，然后重新建立连接，而建立连接的过程包含 TCP 三次握手和 TLS 四次握手的时延，以及 TCP 慢启动的减速过程，给用户的感觉就是网络突然卡顿了一下，因此连接的迁移成本是很高的。

而 QUIC 协议没有用四元组的方式来“绑定”连接，而是通过**连接 ID**来标记通信的两个端点，客户端和服务器可以各自选择一组 ID 来标记自己，因此即使移动设备的网络变化后，导致 IP 地址变化了，只要仍保有上下文信息（比如连接 ID、TLS 密钥等），就可以“无缝”地复用原连接，消除重连的成本，没有丝毫卡顿感，达到了**连接迁移**的功能。

Http3协议

HTTP/3 同 HTTP/2 一样采用二进制帧的结构，不同的地方在于 HTTP/2 的二进制帧里需要定义 Stream，而 HTTP/3 自身不需要再定义 Stream，直接使用 QUIC 里的 Stream，于是 HTTP/3 的帧的结构也变简单了。

1、对比图如下：



2、头部压缩算法：

HTTP2:HPACK
HTTP3:QPACK

3、静态表：

HTTP2:静态表61项
HTTP3:静态表91项

4、动态表：

HTTP2：动态表是具有时序性的，如果首次出现的请求发生了丢包，后续的收到请求，对方就无法解码出HPACK头部，因为对方还没建立好动态表，因此后续的请求解码会阻塞到首次请求中丢失的数据包重传过来。

HTTP3的QPACK解决了上述问题，具体方案如下：

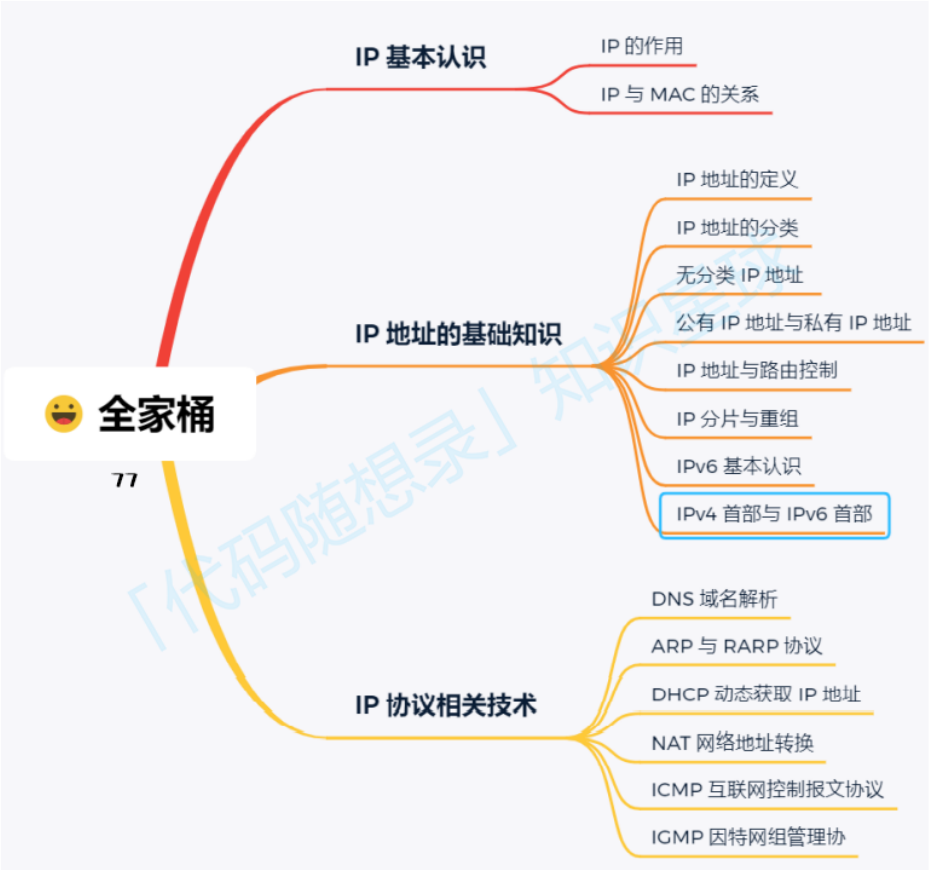
QUIC 会有两个特殊的单向流，所谓的单项流只有一端可以发送消息，双向则指两端都可以发送消息，传输 HTTP 消息时用的是双向流，这两个单向流的用法：

- 一个叫 QPACK Encoder Stream， 用于将一个字典（key-value）传递给对方，比如面对不属于静态表的 HTTP 请求头部，客户端可以通过这个 Stream 发送字典；
- 一个叫 QPACK Decoder Stream，用于响应对方，告诉它刚发的字典已经更新到自己的本地动态表了，后续就可以使用这个字典来编码了。

这两个特殊的单向流是用来**同步双方的动态表**，编码方收到解码方更新确认的通知后，才使用动态表编码 HTTP 头部。

IP基础

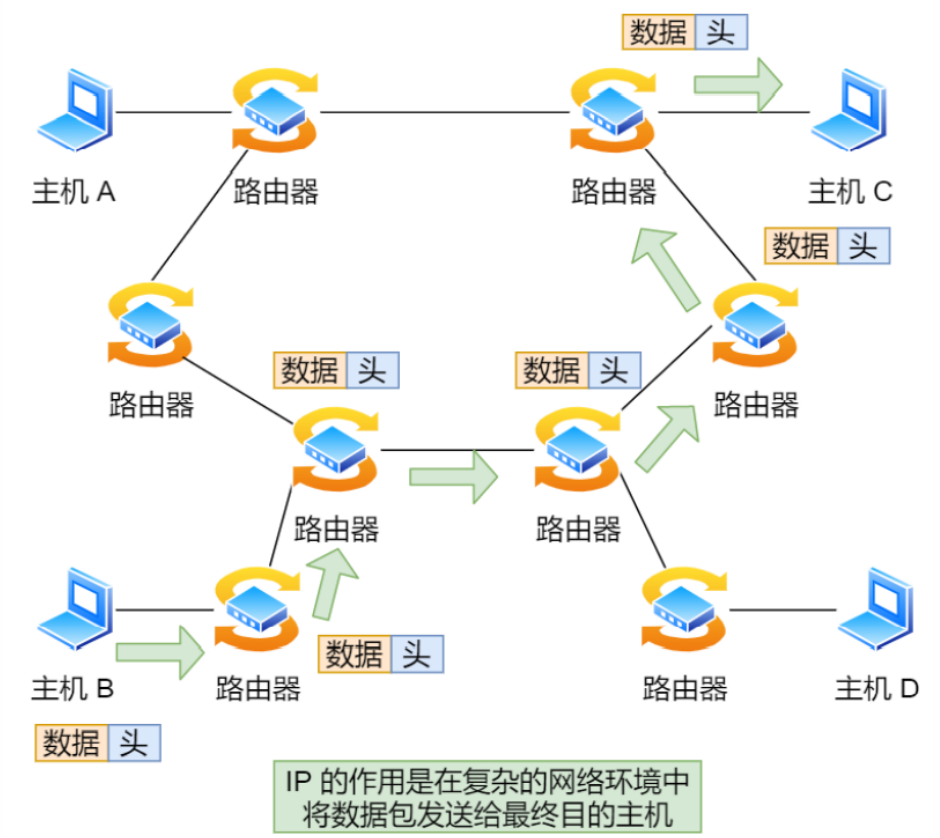
思维导图：



IP基础认识

IP位于TCP/IP参考模型的第三层，也就是网络层。

网络层的主要作用是：实现主机与主机之间的通信，也叫点对点通信。



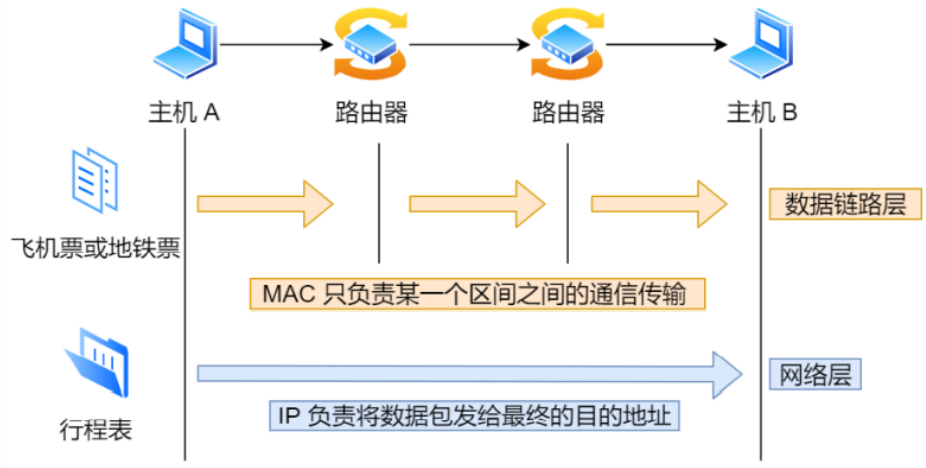
1、网络层（IP)与数据链路层(MAC)有什么关系呢?

MAC的作用:

实现【直连】的两个设备之间通信。

IP的作用:

负责在【没有直连】的两个网络之间进行通信传输。

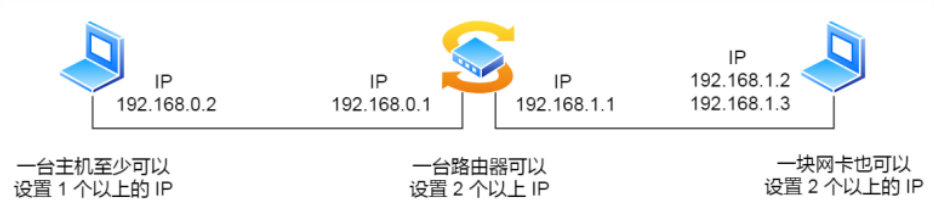


在网络数据包传输中，源IP地址和目标IP地址在传输过程中是不会变的，只有源MAC地址和目标MAC一直在变化。

IP地址基础知识

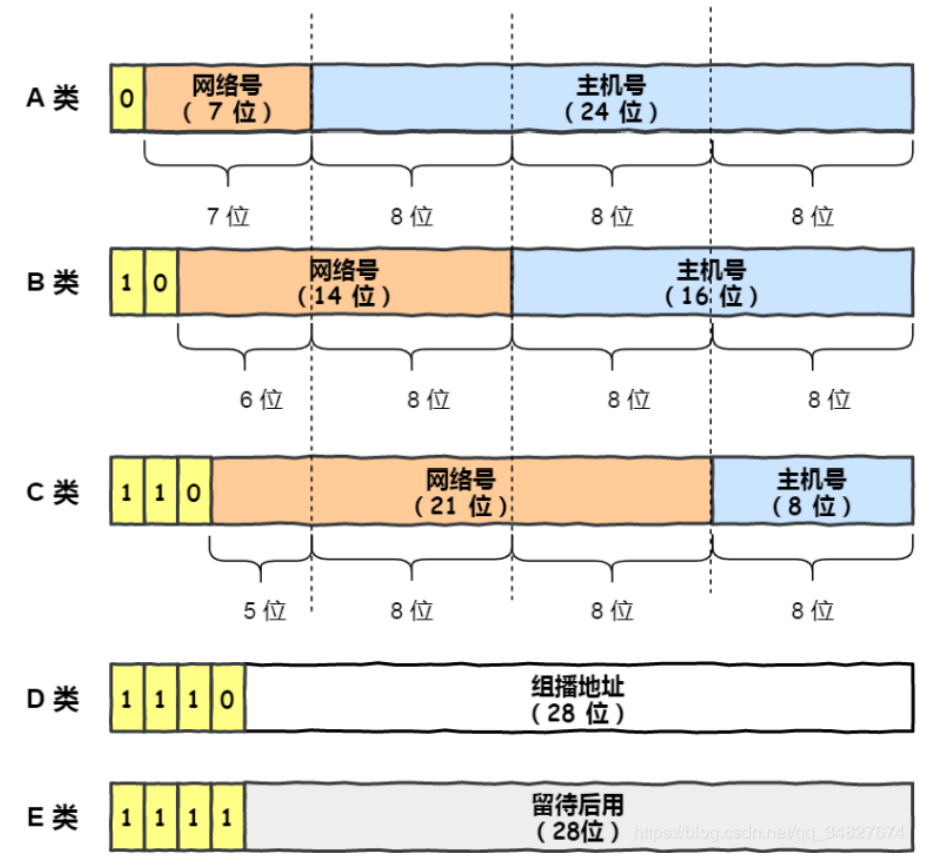
实际上让 43 亿台计算机全部连网其实是不可能的，更何况 IP 地址是由「网络标识」和「主机标识」这两个部分组成的，所以实际能够连接到网络的计算机个数更是少了很多。

但可以根据一种可以更换IP地址的技术NAT，使得可连接计算机数超过43亿台。



1、IP地址分类

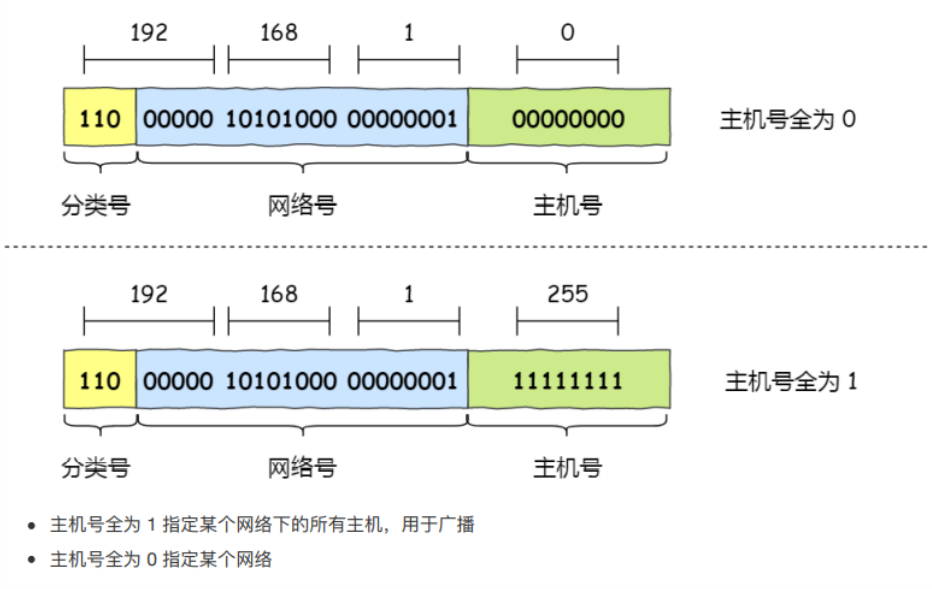
IP地址被分为了五种类型：



（1）A、B、C类地址

类别	IP 地址范围	最大主机数
A	0.0.0.0 ~ 127.255.255.255	16777214
B	128.0.0.0 ~ 191.255.255.255	65534
C	192.0.0.0 ~ 223.255.255.255	254

最大主机数=2^主机号位数-2，其中有两个地址，全0和全1是比较特殊的：

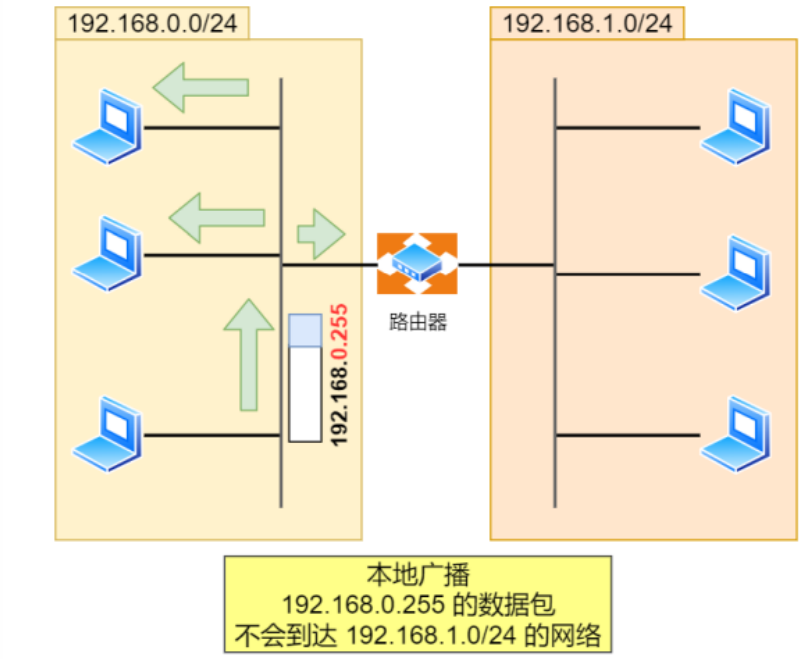


- 主机号全为 1 指定某个网络下的所有主机，用于广播
- 主机号全为 0 指定某个网络

广播地址用于在同一个链路中相互连接的主机之间发送数据包。广播地址可以分为本地广播和直接广播两种。

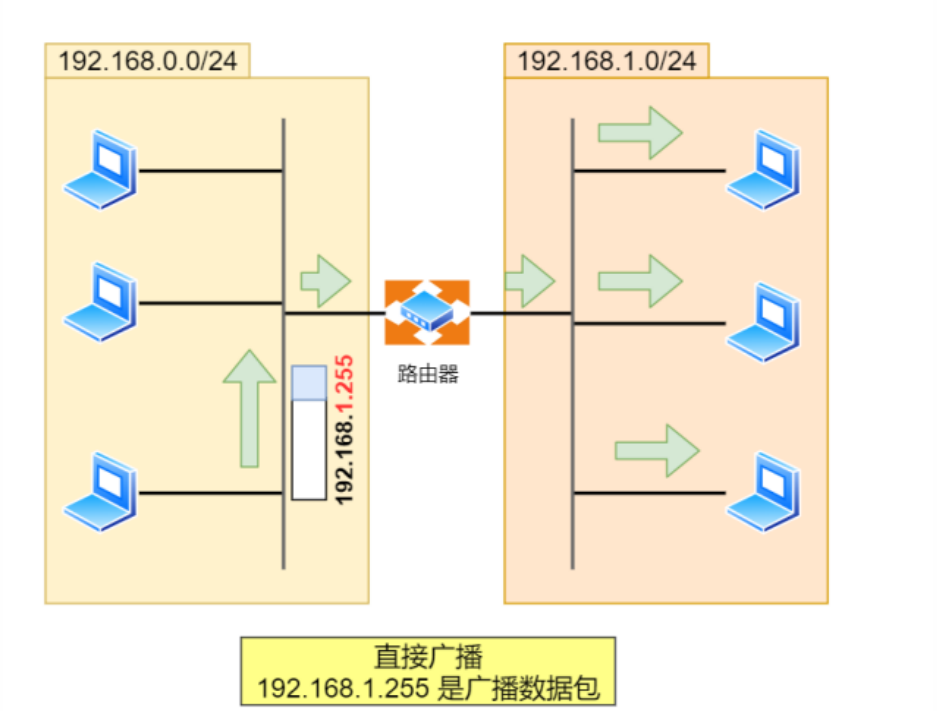
在本网络内广播的叫做本地广播。例如网络地址为 192.168.0.0/24 的情况下，广播地址是 192.168.0.255 。

因为这个广播地址的 IP 包会被路由器屏蔽，所以不会到达 192.168.0.0/24 以外的其他链路上。



在不同网络之间的广播叫做直接广播。

例如网络地址为 192.168.0.0/24 的主机向 192.168.1.255/24 的目标地址发送 IP 包。收到这个包的路由器，将数据转发给 192.168.1.0/24，从而使得所有 192.168.1.1~192.168.1.254 的主机都能收到这个包（由于直接广播有一定的安全问题，多数情况下会在路由 器上设置为不转发。）

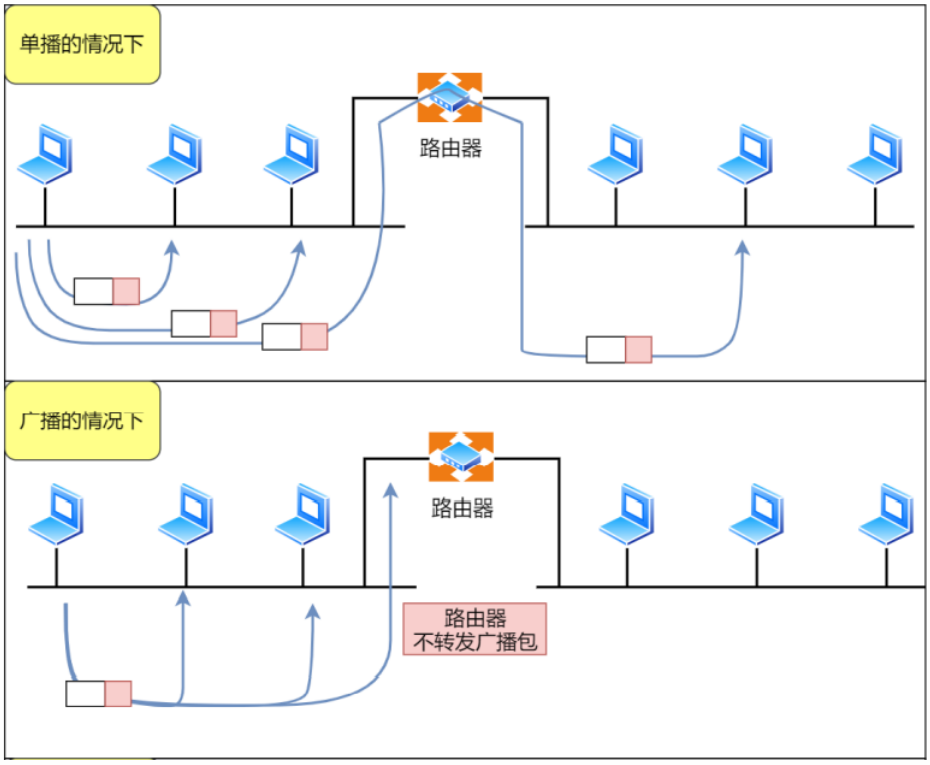


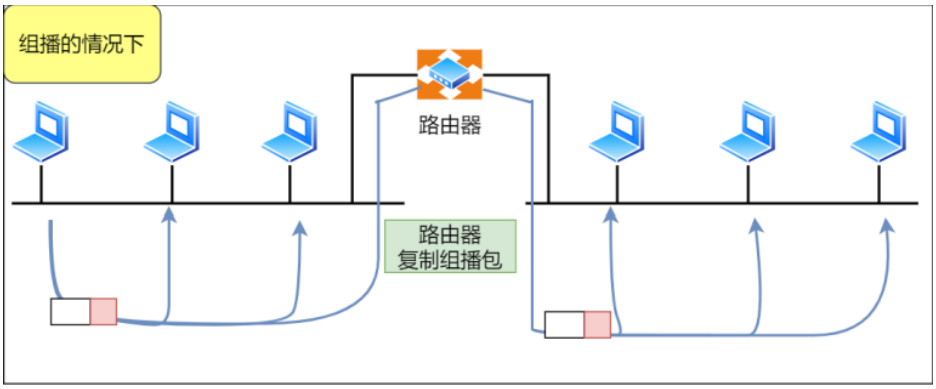
（2）什么是D、E类地址?

D 类和 E 类地址是没有主机号的，所以不可用于主机 IP，D 类常被用于多播，E 类是预留的分类，暂时未使用。

类别	IP 地址范围	用途
D	224.0.0.0 ~ 239.255.255.255	IP 多播
E	240.0.0.0 ~ 255.255.255.255	预留使用

多播地址用于将包发送给特定组内的所有主机。由于广播无法穿透路由，若想给其他网段发送同样的包，就可以使用可以穿透路由的多播。





多播使用的 D 类地址，其前四位是 1110 就表示是多播地址，而剩下的 28 位是多播的组编号。

从 224.0.0.0 ~ 239.255.255.255 都是多播的可用范围，其划分为以下三类：

224.0.0.0 ~ 224.0.0.255 为预留的组播地址，只能在局域网中，路由器是不会进行转发的。
224.0.1.0 ~ 238.255.255.255 为用户可用的组播地址，可以用于 Internet 上。
239.0.0.0 ~ 239.255.255.255 为本地管理组播地址，可供内部网在内部使用，仅在特定的本地范围内有效。

（3）IP分类的优缺点

优点：

可以根据IP地址的前四位来判别IP地址属于哪个类别。简单明了，选路简单。

缺点：

同一网络下没有地址层次，比如一个公司里用了 B 类地址，但是可能需要根据生产环境、测试环境、开发环境来划分地址层次，而这种 IP 分类是没有地址层次划分的功能，所以这就缺少地址的灵活性。

A、B、C类有个尴尬处境，就是不能很好的与现实网络匹配。

C 类地址能包含的最大主机数量实在太少了，只有 254 个，估计一个网吧都不够用。

而 B 类地址能包含的最大主机数量口太多了，6 万多台机器放在一个网络下面，一般的企业基本达不到这个规模，闲着的地址就是浪费。这两个缺点，都可以在 CIDR 无分类地址解决。

（4）CIDR无分类地址

这种方式不再有分类地址的概念，32 比特的 IP 地址被划分为两部分，前面是网络号，后面是主机号。

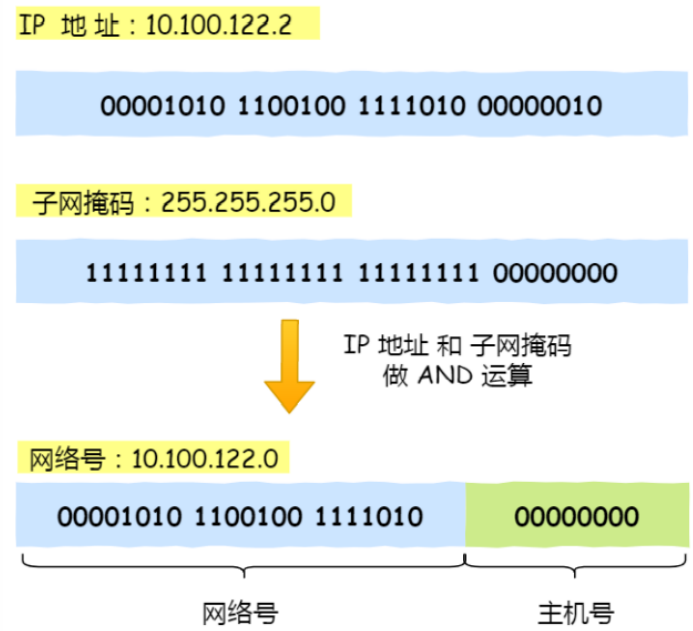
(i)如何划分网络号和主机号呢?

表示形式 a.b.c.d/x，其中 /x 表示前 x 位属于网络号， x 的范围是 0 ~ 32 ，这就使得 IP 地址更加具有灵活性。

可用地址个数	254
子网掩码	255.255.255.0
网络号	10.100.122.0
第一个可用地址	10.100.122.1
最后可用地址	10.100.122.254
广播地址	10.100.122.255

还有另一种划分网络号与主机号形式，那就是子网掩码，掩码的意思就是掩盖掉主机号，剩余的就是网络号。

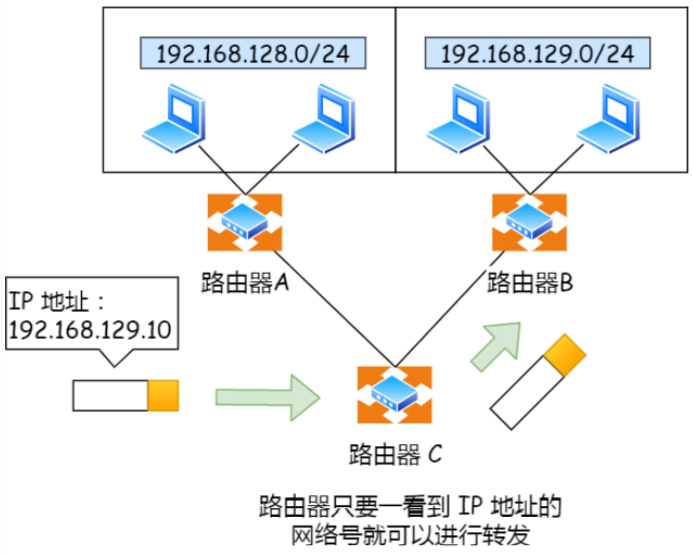
将子网掩码和 IP 地址按位计算 AND，就可得到网络号。



（ii）为什么要分离网络号和主机号?

因为两台计算机要通讯，首先要判断是否处于同一个广播域内，即网络地址是否相同。如果网络地址相同，表明接受方在本网络上，那么可以把数据包直接发送到目标主机。

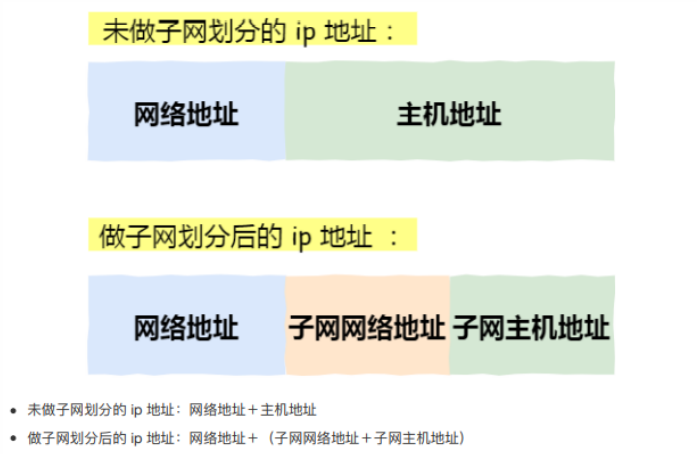
路由器寻址工作中，也就是通过这样的方式来找到对应的网络号的，进而把数据包转发给对应的网络内。



（iii）如何进行子网划分?

通过子网掩码划分出网络号和主机号，那实际上子网掩码还有一个作用，那就是划分子网。

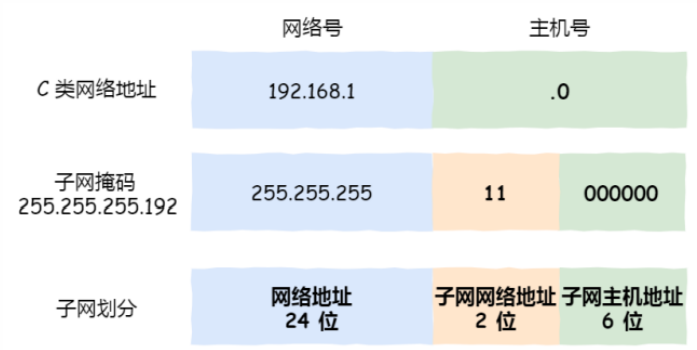
子网划分实际上是将主机地址分为两个部分：子网网络地址和子网主机地址。形式如下：



- 未做子网划分的 ip 地址：网络地址＋主机地址
- 做子网划分后的 ip 地址：网络地址＋（子网网络地址＋子网主机地址）

假设对 C 类地址进行子网划分，网络地址 192.168.1.0，使用子网掩码 255.255.255.192 对其进行子网划分。

C 类地址中前 24 位是网络号，最后 8 位是主机号，根据子网掩码可知从 8 位主机号中借用 2 位作为子网号（对应4个子网）。



划分后的4个子网如下：

子网号	网络地址	主机地址范围	广播地址
0	192.168.1.0	192.168.1.1 ~ 192.168.1.62	192.168.1.63
1	192.168.1.64	192.168.1.65 ~ 192.168.1.126	192.168.1.127
2	192.168.1.128	192.168.1.129 ~ 192.168.1.190	192.168.1.191
3	192.168.1.192	192.168.1.193 ~ 192.168.1.254	192.168.1.255

2、公有IP地址与私有IP地址

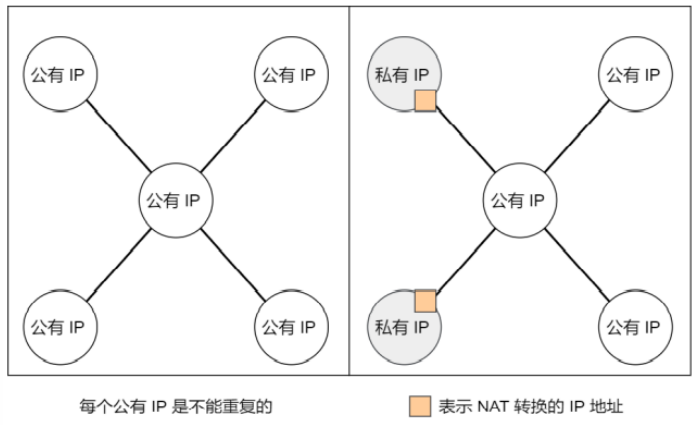
在A、B、C分类地址，实际又分公有IP地址和私有IP地址。

类别	IP 地址范围	最大主机数	私有 IP 地址范围
A	0.0.0.0 ~ 127.255.255.255	16777214	10.0.0.0 ~ 10.255.255.255
B	128.0.0.0 ~ 191.255.255.255	65534	172.16.0.0 ~ 172.31.255.255
C	192.0.0.0 ~ 223.255.255.255	254	192.168.0.0 ~ 192.168.255.255

平时我们办公室、家里、学校用的 IP 地址，一般都是私有 IP 地址。因为这些地址允许组织内部的 IT 人员自己管理、自己分配，而且可以重复。

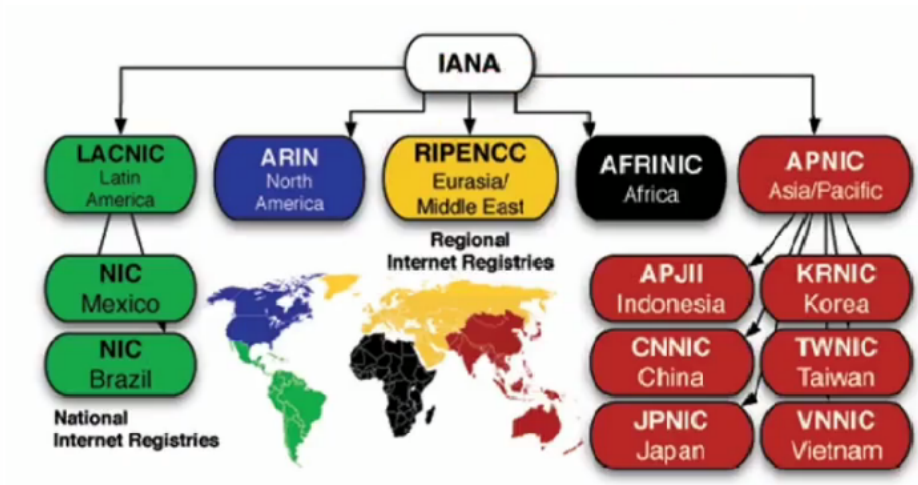
因此，你学校的某个私有 IP 地址和我学校的可以是一样的。 就像每个小区都有自己的楼编号和门牌号，你小区家可以叫 1 栋 101 号，我小区家也可以叫 1 栋 101，没有任何问题。但一旦出了小区，就需要带上中山路 666 号（公网 IP 地址），是国家统一分配的，不能两个小区都叫中山路 666。

所以，公有 IP 地址是有个组织统一分配的，假设你要开一个博客网站，那么你就需要去申请购买一个公有 IP，这样全世界的人才能访问。并且公有 IP 地址基本上要在整个互联网范围内保持唯一



那么公有IP地址由谁管理呢?

私有 IP 地址通常是内部的 IT 人员管理，公有 IP 地址是由 ICANN 组织管理，中文叫「互联网名称与数字地址分配机构」。 IANA 是 ICANN 的其中一个机构，它负责分配互联网 IP 地址，是按州的方式层层分配。

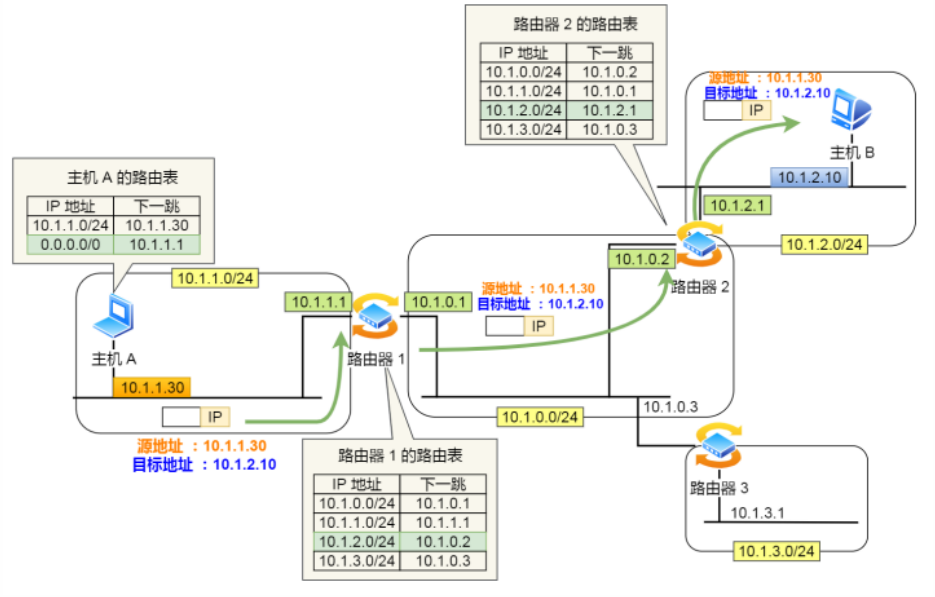


3、IP地址与路由控制

IP地址的网络地址这一部分是用于进行路由控制。路由控制表中记录着网络地址与下一步应该发送至路由器的地址。在主机和路由器上都会有各自的路由器控制表。

在发送 IP 包时，首先要确定 IP 包首部中的目标地址，再从路由控制表中找到与该地址具有相同网络地址的记录，根据该记录将 IP 包转发给相应的下一个路由器。如果路由控制表中存在多条相同网络地址的记录，就选择相同位数最多的网络地址，也就是最长匹配。

举例说明:



环回地址是不会流向网络：

环回地址是在同一台计算机上的程序之间进行网络通信时所使用的一个默认地址。 计算机使用一个特殊的 IP 地址 127.0.0.1 作为环回地址。

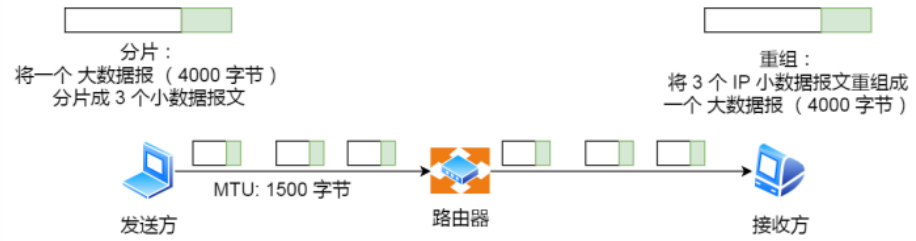
与该地址具有相同意义的是一个叫做 localhost 的主机名。使用这个 IP 或主机名时，数据包不会流向网络

4、IP分片与重组

每种数据链路的最大传输单元 MTU 都是不相同的，如 FDDI 数据链路 MTU 4352、以太网的 MTU 是 1500 字节等。

每种数据链路的 MTU 之所以不同，是因为每个不同类型的数据链路的使用目的不同。使用目的不同，可承载的 MTU 也就不同。

其中，我们最常见数据链路是以太网，它的 MTU 是 1500 字节。 那么当 IP 数据包大小大于 MTU 时， IP 数据包就会被分片。 经过分片之后的 IP 数据报在被重组的时候，只能由目标主机进行，路由器是不会进行重组的。



在分片传输中，一旦某个分片丢失，则会造成整个 IP 数据报作废，所以 TCP 引入了 MSS 也就是在 TCP 层进行分片不由 IP 层分片，那么对于 UDP 我们尽量不要发送一个大于 MTU 的数据报文。

5、IPV6基本认识

IPV6:更多的地址，更好的安全性和扩展性。

但IPV4和IPV6不能兼容。所以不但要我们电脑、手机之类的设备支持，还需要网络运营商对现有的设备进行升级，所以这可能是 IPv6 普及率比较慢的一个原因。

（1）IPV6的亮点

可分配地址变多

IPv6 可自动配置，即使没有 DHCP 服务器也可以实现自动分配IP地址，真是便捷到即插即用啊。

IPv6 包头包首部长度采用固定的值 40 字节，去掉了包头校验和，简化了首部结构，减轻了路由器负荷，大 大提高了传输的性能。

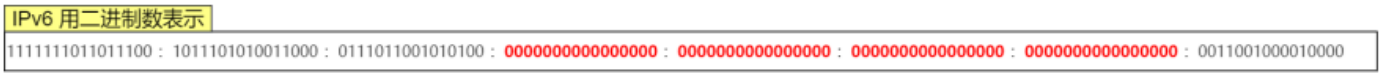
IPv6 有应对伪造 IP 地址的网络安全功能以及防止线路窃听的功能，大大提升了安全性。

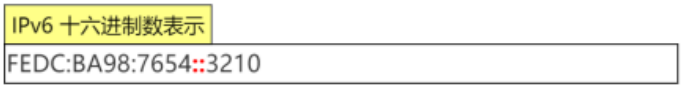
（2）IPV6地址的标识方法

IPv4 地址长度共 32 位，是以每 8 位作为一组，并用点分十进制的表示方式。

IPv6 地址长度是 128 位，是以每 16 位作为一组，每组用冒号 「:」 隔开。

如果出现连续的 0 时还可以将这些 0 省略，并用两个冒号 「::」隔开。但是，一个 IP 地址中只允许出现一次两个连续的冒号。





（3）IPV6地址的结构

IPv6 类似 IPv4，也是通过 IP 地址的前几位标识 IP 地址的种类。 IPv6 的地址主要有以下类型地址：

单播地址：用于一对一的通信；
组播地址：用于一对多的通信；
任播地址：用于通信最近的节点，最近的节点是由路由协议决定；
没有广播地址。

未定义	0000 ... 0000 (128 位)	::/128
环回地址	0000 ... 0001 (128 位)	::1/128
唯一本地地址	1111 110 ...	FC00::/7
链路本地单播地址	1111 1110 10 ...	FE80::/10
多播地址	1111 1111	FF00::/8
全局单播地址	其他	

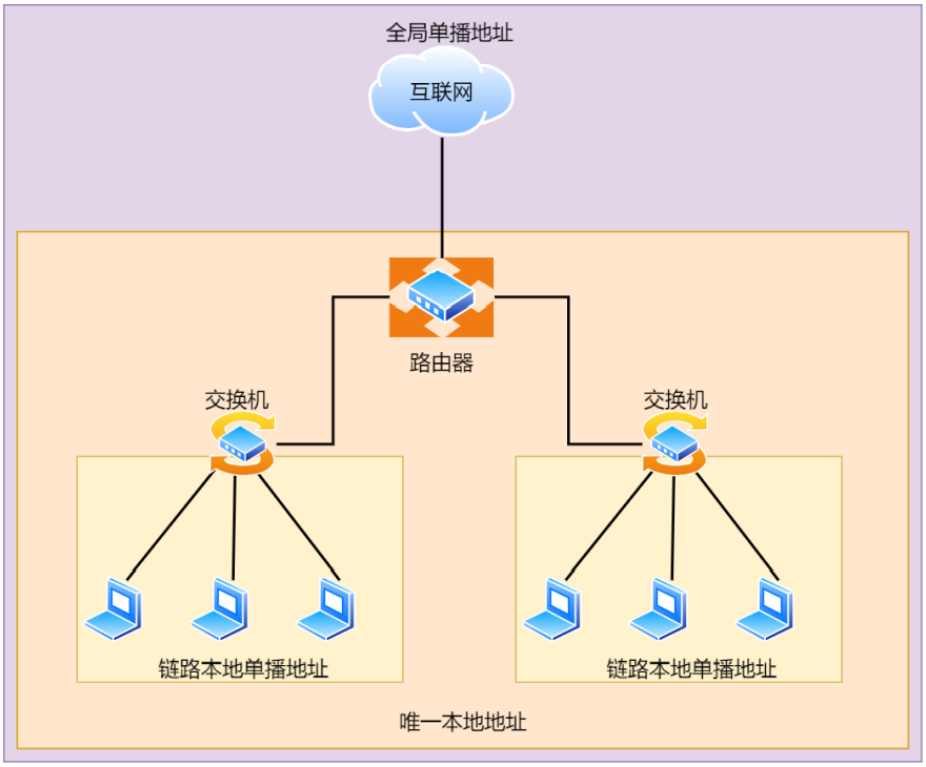
IPV6单播地址类型：

对于一对一通信的 IPv6 地址，主要划分了三类单播地址，每类地址的有效范围都不同。

在同一链路单播通信，不经过路由器，可以使用链路本地单播地址，IPv4 没有此类型

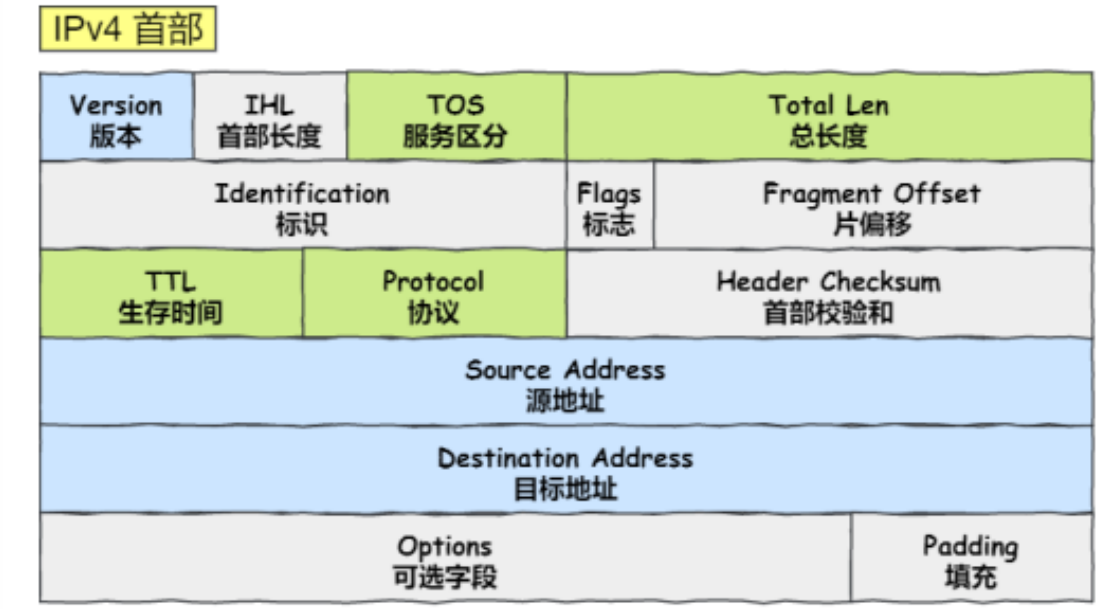
在内网里单播通信，可以使用唯一本地地址，相当于 IPv4 的私有 IP

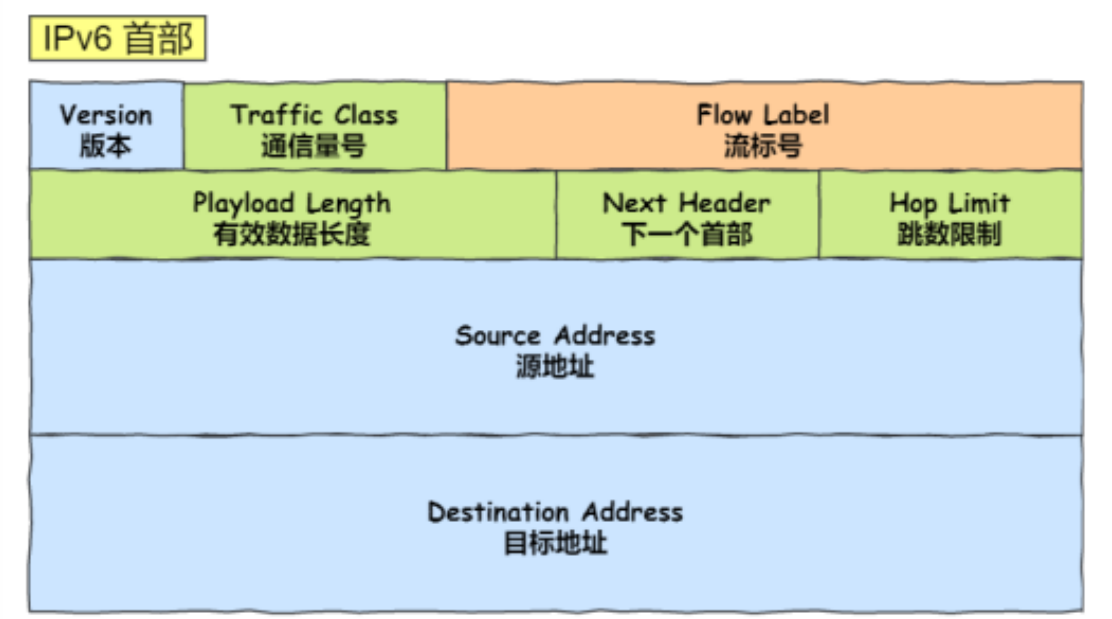
在互联网通信，可以使用全局单播地址，相当于 IPv4 的公有 IP



6、IPV4首部与IPV6首部

IPv4 首部与 IPv6 首部 IPv4 首部与 IPv6 首部的差异如下图：





其中：

IPV6相比IPV4的首部改进

取消了首部校验和字段：

因为在数据链路层和传输层都会校验，因此 IPv6 直接取消了 IP 的校验。

取消了分片/重新组装相关字段：

分片与重组是耗时的过程，IPv6 不允许在中间路由器进行分片与重组，这 种操作只能在源与目标主机，这将大大提高了路由器转发的速度。

取消选项字段：

选项字段不再是标准 IP 首部的一部分了，但它并没有消失，而是可能出现在 IPv6 首部中的 「下一个首部」指出的位置上。删除该选项字段使的 IPv6 的首部成为固定长度的 40 字节。

DNS基础

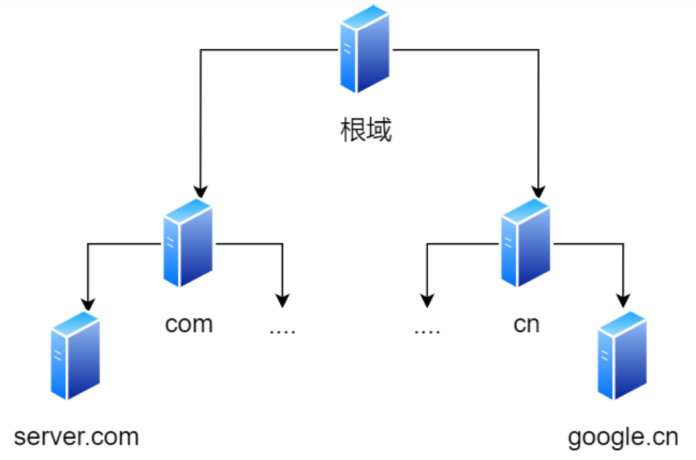
1、DNS（Domain Name Server）域名解析

我们在上网的时候，通常使用的方式是域名，而不是 IP 地址，因为域名方便人类记忆。那么实现这一技术的就是 DNS 域名解析，DNS 可以将域名网址自动转换为具体的 IP 地址。

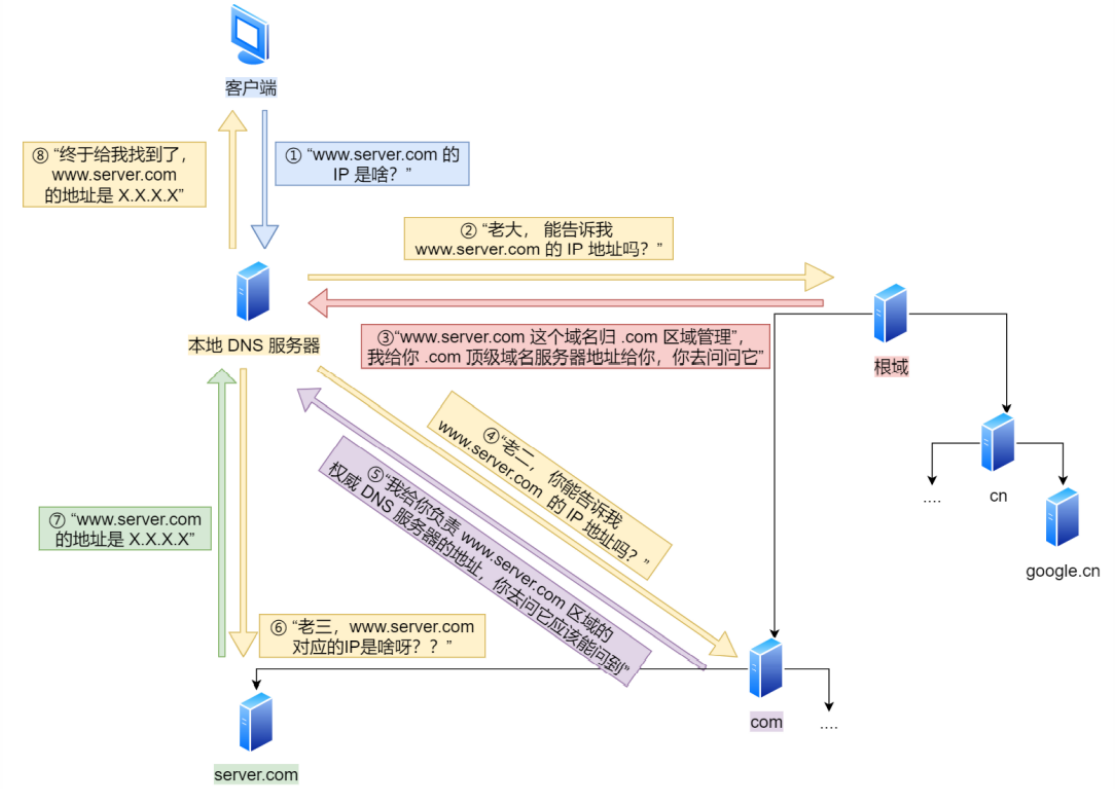
（1）域名的层级关系

DNS 中的域名都是用句点来分隔的，比如 www.server.com ，这里的句点代表了不同层次之间的界限。

在域名中，越靠右的位置表示其层级越高。



（2）域名解析的工作流程：只指路不带路



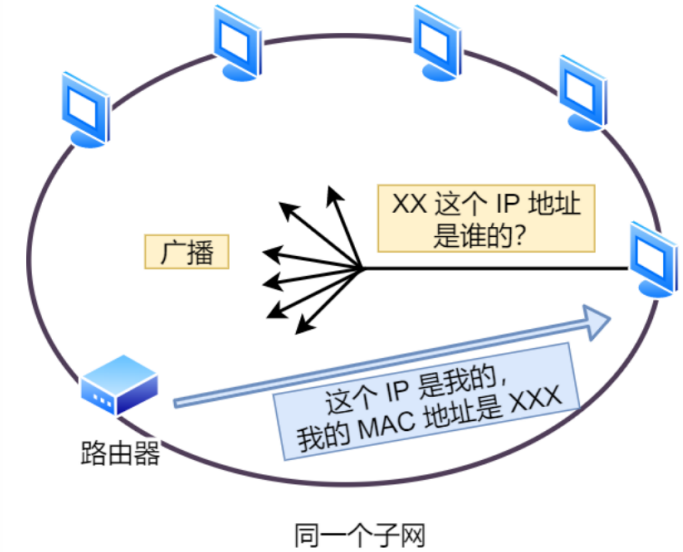
2、ARP与RARP协议

（1）ARP协议

在传输一个 IP 数据报的时候，确定了源 IP 地址和目标 IP 地址后，就会通过主机「路由表」确定 IP 数据包下一跳。然而，网络层的下一层是数据链路层，所以我们还要知道「下一跳」的 MAC 地址。

由于主机的路由表中可以找到下一跳的 IP 地址，所以可以通过 ARP 协议（Address Resolution Protocol，地址解析协议），求得下一跳的 MAC 地址。

ARP是如何知道对方的MAC地址的呢？ARP 是借助 ARP 请求与 ARP 响应两种类型的包确定 MAC 地址的。



主机会通过广播发送 ARP 请求，这个包中包含了想要知道的 MAC 地址的主机 IP 地址。

当同个链路中的所有设备收到 ARP 请求时，会去拆开 ARP 请求包里的内容，如果 ARP 请求包中的目标 IP 地址与自己的 IP 地址一致，那么这个设备就将自己的 MAC 地址塞入 ARP 响应包返回给主机。

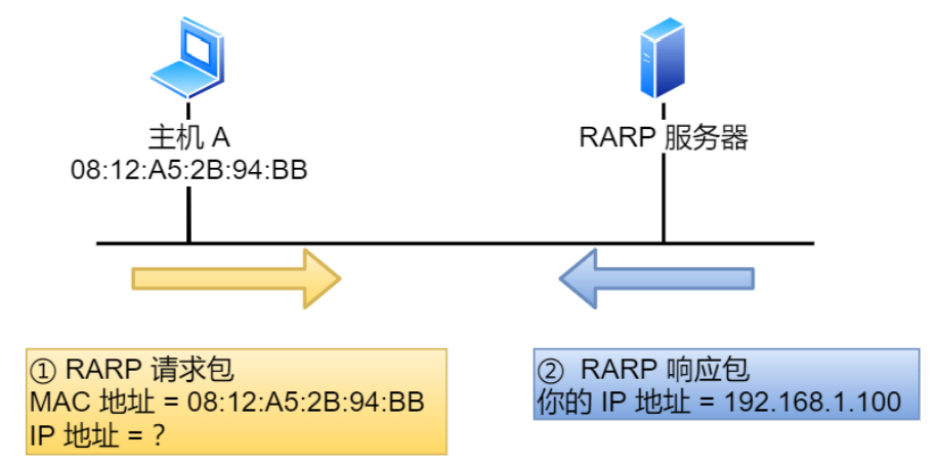
操作系统通常会把第一次通过 ARP 获取的 MAC 地址缓存起来，以便下次直接从缓存中找到对应 IP 地址的 MAC 地址。 不过，MAC 地址的缓存是有一定期限的，超过这个期限，缓存的内容将被清除。

（2）RARP协议

ARP 协议是已知 IP 地址求 MAC 地址，那 RARP 协议正好相反，它是已知 MAC 地址求 IP 地址。

例如将打印机服务器等小型嵌入式设备接入到网络时就经常会用得到。通常这需要架设一台 RARP 服务器，在这个服务器上注册设备的 MAC 地址及其 IP 地址。然后再将这个设备接入到网络

接着：



3、DHCP动态获取IP地址

DHCP(Dynamic Host Configuration Protocol,动态主机配置协议)。我们的电脑通常都是通过 DHCP 动态获取 IP 地址，大大省去了配 IP 信息繁琐的过程。

接下来，我们来看看我们的电脑是如何通过 4 个步骤的过程，获取到 IP 的：

- 客户端首先发起 **DHCP 发现报文（DHCP DISCOVER）** 的 IP 数据报，由于客户端没有 IP 地址，也不知道 DHCP 服务器的地址，所以使用的是 UDP **广播**通信，其使用的广播目的地址是 255.255.255.255（端口 67） 并且使用 0.0.0.0（端口 68） 作为源 IP 地址。DHCP 客户端将该 IP 数据报传递给链路层，链路层然后将帧广播到所有的网络中设备。
- DHCP 服务器收到 DHCP 发现报文时，用 **DHCP 提供报文（DHCP OFFER）** 向客户端做出响应。该报文仍然使用 IP 广播地址 255.255.255.255，该报文信息携带服务器提供可租约的 IP 地址、子网掩码、默认网关、DNS 服务器以及 **IP 地址租用期**。
- 客户端收到一个或多个服务器的 DHCP 提供报文后，从中选择一个服务器，并向选中的服务器发送 **DHCP 请求报文（DHCP REQUEST**进行响应，回显配置的参数。
- 最后，服务端用 **DHCP ACK 报文**对 DHCP 请求报文进行响应，应答所要求的参数。

一旦客户端收到 DHCP ACK 后，交互便完成了，并且客户端能够在租用期内使用 DHCP 服务器分配的 IP 地址。

说明： DHCP 客户端进程监听的是 68 端口号，DHCP 服务端进程监听的是 67 端口号。

如果租约的 DHCP IP 地址快期后，客户端会向服务器发送 DHCP 请求报文：

服务器如果同意继续租用：

则用 DHCP ACK 报文进行应答，客户端就会延长租期。

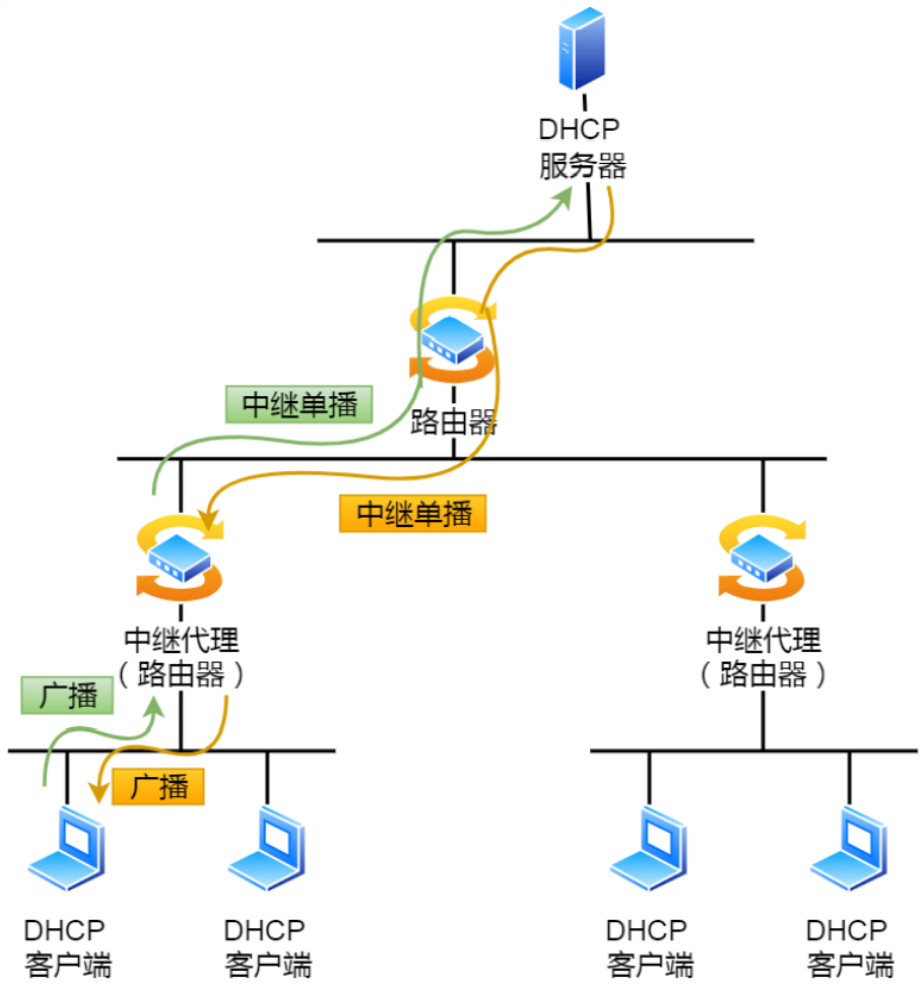
服务器如果不同意继续租用：

则用 DHCP NACK 报文，客户端就要停止使用租约的 IP 地址。 可以发现，DHCP 交互中，全程都是使用 UDP 广播通信。

疑问：

咦，用的是广播，那如果 DHCP 服务器和客户端不是在同一个局域网内，路由器口不会转发广播包，那不是每个网络 都要配一个 DHCP 服务器?

为了解决这一问题，就出现了 DHCP 中继代理。有了 DHCP 中继代理以后，对不同网段的 IP 地址分配也 可以由一个 DHCP 服务器统一进行管理。



DHCP 客户端会向 DHCP 中继代理发送 DHCP 请求包，而 DHCP 中继代理在收到这个广播包以后，再以单播的形式发给 DHCP 服务器。

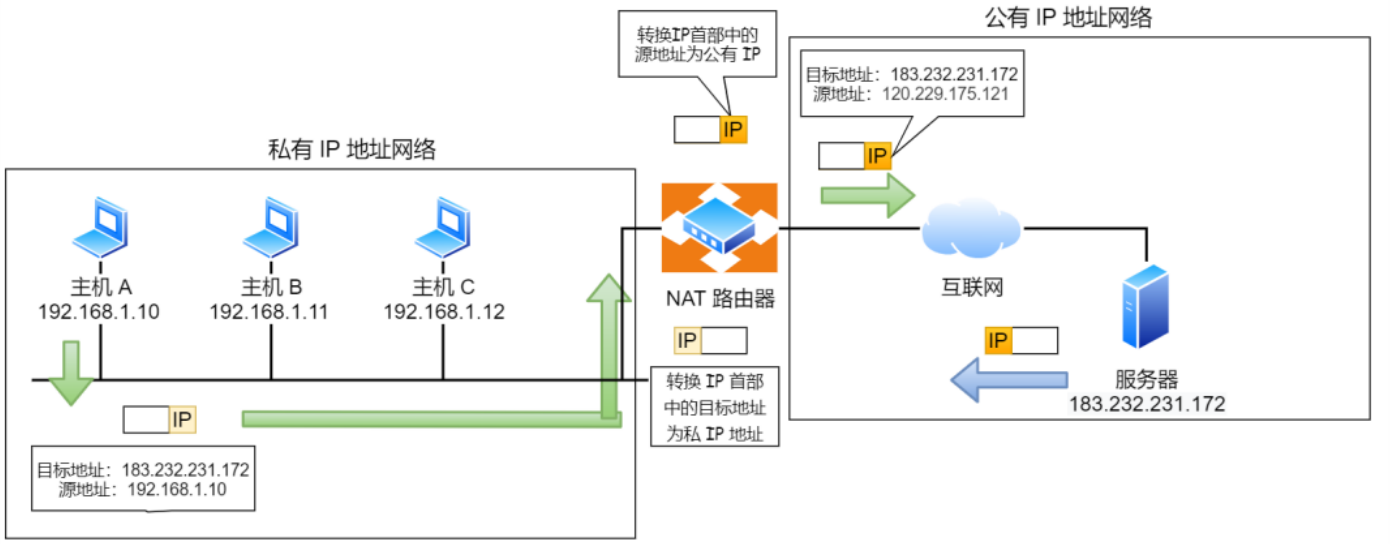
服务器端收到该包以后再向 DHCP 中继代理返回应答，并由 DHCP 中继代理将此包广播给 DHCP 客户端 。

因此，DHCP 服务器即使不在同一个链路上也可以实现统一分配和管理IP地址。

4、NAT(Network Address Translation, 网络地址转换)

IPv4 的地址是非常紧缺的，在前面我们也提到可以通过无分类地址来减缓 IPv4 地址耗尽的速度，但是互联网的用户增速是非常惊人的，所以 IPv4 地址依然有被耗尽的危险。

于是，提出了一种网络地址转换 NAT 的方法，再次缓解了 IPv4 地址耗尽的问题。 简单的来说 NAT 就是同个公司、家庭、教室内的主机对外部通信时，把私有 IP 地址转换成公有 IP 地址。

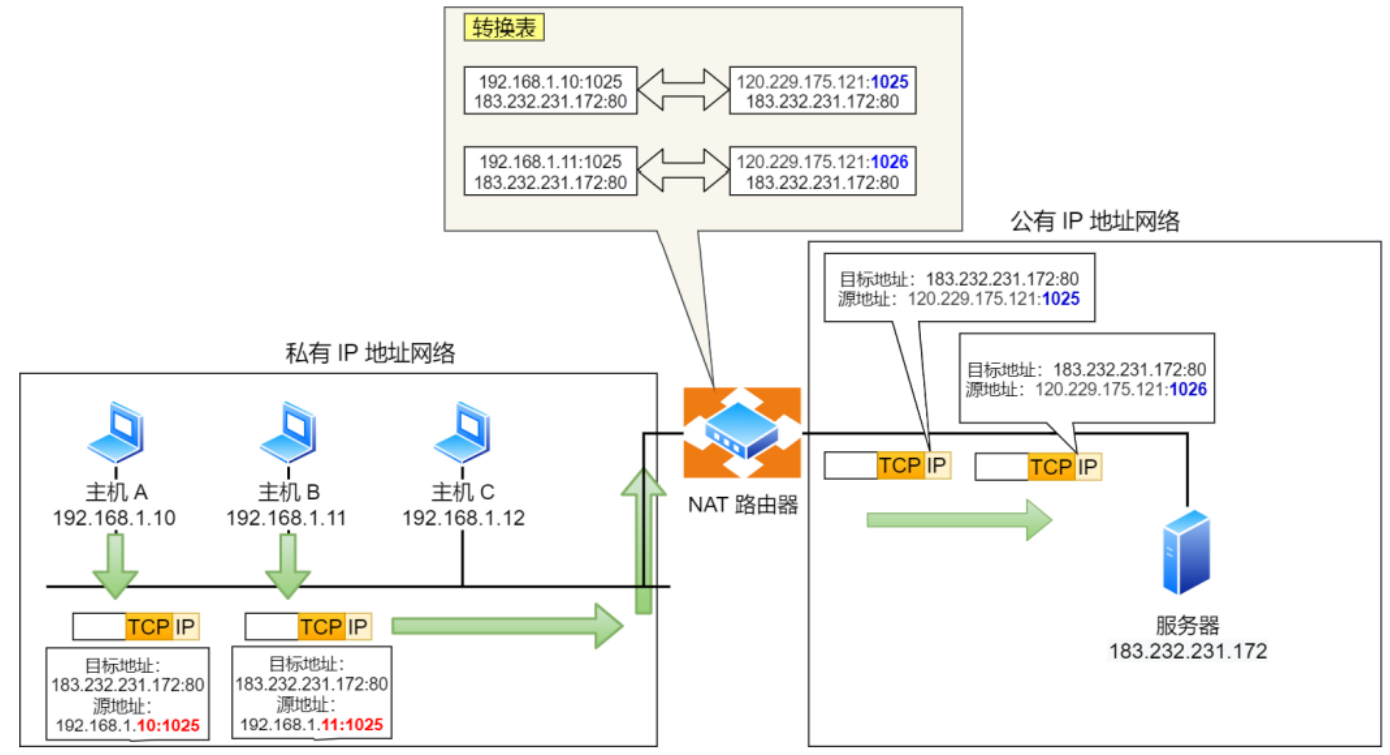


疑问：

那不是 N 个私有 IP 地址，你就要 N 个公有 IP 地址？这怎么就缓解了 IPv4 地址耗尽的问题？这不瞎扯吗？

确实是，普通的 NAT 转换没什么意义。 由于绝大多数的网络应用都是使用传输层协议 TCP 或 UDP 来传输数据的。 因此，可以把 IP 地址 + 端口号一起进行转换。 这样，就用一个全球 IP 地址就可以了，这种转换技术就叫网络地址与端口转换 NAPT。

来看看下面的图解：



图中有两个客户端 192.168.1.10 和 192.168.1.11 同时与服务器 183.232.231.172 进行通信，并且这两个客户端的本地端口都是 1025。 此时，两个私有 IP 地址都转换 IP 地址为公有地址 120.229.175.121，但是以不同的端口号作为区分。

于是，生成一个 NAPT 路由器的转换表，就可以正确地转换地址跟端口的组合，令客户端 A、B 能同时与服务器之间进行通信。 这种转换表在 NAT 路由器上自动生成。例如，在 TCP 的情况下，建立 TCP 连接首次握手时的 SYN 包一经发出， 就会生成这个表。而后口随着收到关闭连接时发出 FIN 包的确认应答从表中被删除。

（1）NAT有缺点吗?

由于 NAT/NAPT 都依赖于自己的转换表，因此会有以下的问题：

外部无法主动与 NAT 内部服务器建立连接，因为 NAPT 转换表没有转换记录。

转换表的生成与转换操作都会产生性能开销。

通信过程中，如果 NAT 路由器重启了，所有的 TCP 连接都将被重置。

（2）如何解决NAT潜在的问题呢?

改用IPV6：

IPv6 可用范围非常大，以至于每台设备都可以配置一个公有 IP 地址，就不搞那么多花里胡哨的地址转换了，但是 IPv6 普及速度还需要一些时间。

NAT 穿透技术：

NAT 穿越技术能够让网络应用程序主动发现自己位于 NAT 设备之后，并且会主动获得 NAT 设备的公有 IP，并为自己建立端口映射条目，注意这些都是 NAT设备后的应用程序自动完成的。 也就是说，客户端主动从 NAT 设备获取公有 IP 地址，然后自己建立端口映射条目，然后用这个条目对外通信， 就不需要 NAT 设备来进行转换了。

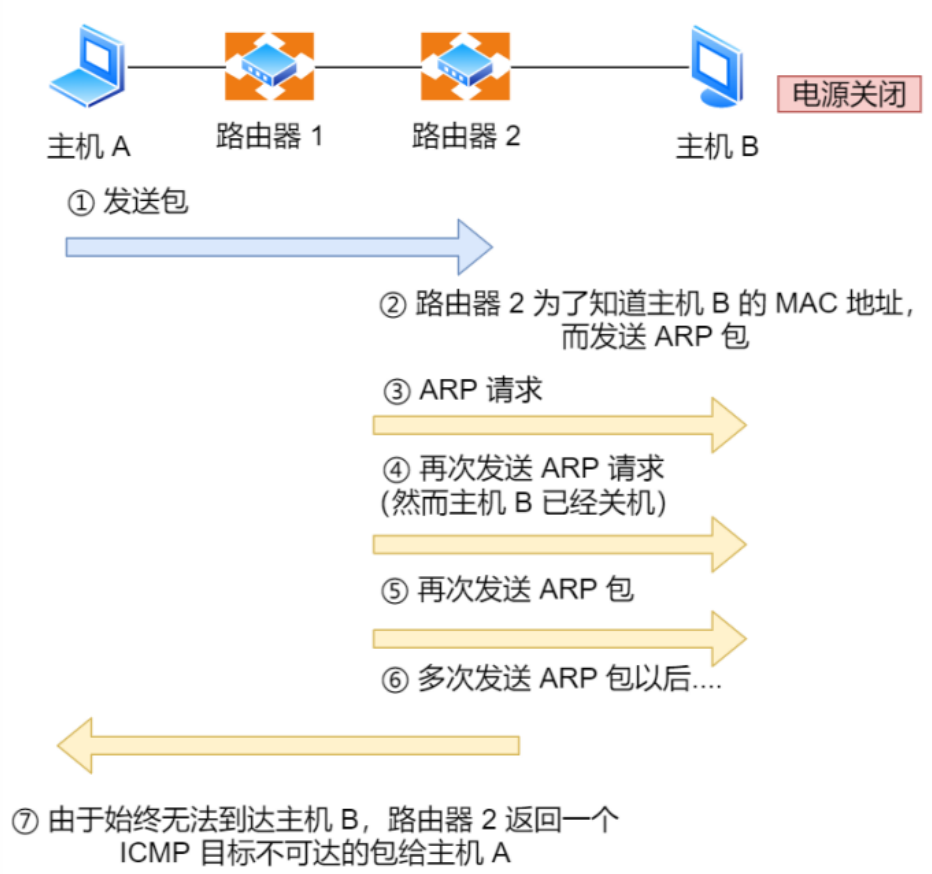
5、ICMP互联网控制报文协议

ICMP（ Internet Control Message Protocol，也就是互联网控制报文协议）。

网络包在复杂的网络传输环境里，常常会遇到各种问题。 当遇到问题的时候，总不能死个不明不白，没头没脑的作风不是计算机网络的风格。所以需要传出消息，报告遇到 了什么问题，这样才可以调整传输策略，以此来控制整个局面。

（1）ICMP功能都有啥

ICMP的主要功能包括：确认 IP 包是否成功送达目标地址、报告发送过程中 IP 包被废弃的原因和改善网络设置等。在 IP 通信中如果某个 IP 包因为某种原因未能达到目标地址，那么这个具体的原因将由 ICMP 负责通知。



ICMP 的这种通知消息会使用IP进行发送 。 因此，从路由器 2 返回的 ICMP 包会按照往常的路由控制先经过路由器 1 再转发给主机A 。收到该 ICMP 包的主机 A 则分解 ICMP 的首部和数据域以后得知具体发生问题的原因。

（2）ICMP的类型

ICMP大致可以分为两大类：

- 一类是用于诊断的查询消息，也就是「查询报文类型」
- 一类是通知出错原因的错误消息，也就是「差错报文类型」

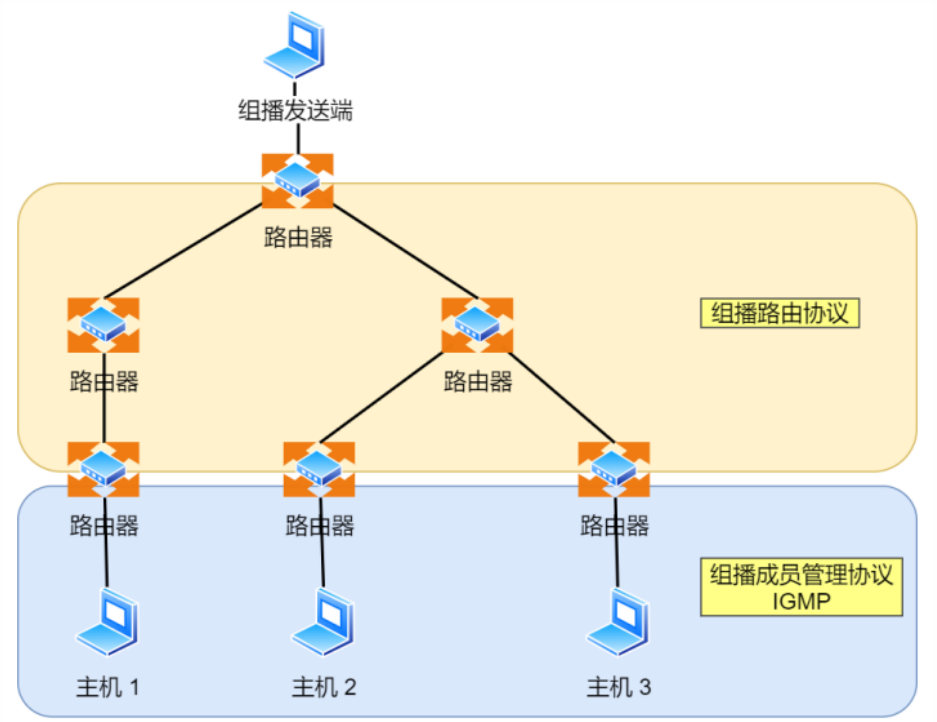
ICMP 类型

	内容	种类
0	回送应答（Echo Reply）	查询报文类型
3	目标不可达（Destination Unreachable）	差错报文类型
4	原点抑制（Source Quench）	差错报文类型
5	重定向或改变路由（Redirect）	差错报文类型
8	回送请求（Echo Request）	查询报文类型
11	超时（Time Exceeded）	差错报文类型

6、IGMP因特网组管理协议

在前面我们知道了组播地址，也就是 D 类地址，既然是组播，那就说明是只有一组的主机能收到数据包，不在一组的主机不能收到数组包，怎么管理是否是在一组呢?

那么，就需要 IGMP 协议了。



IGMP 是因特网组管理协议，工作在主机（组播成员）和最后一跳路由之间，如上图中的蓝色部分。

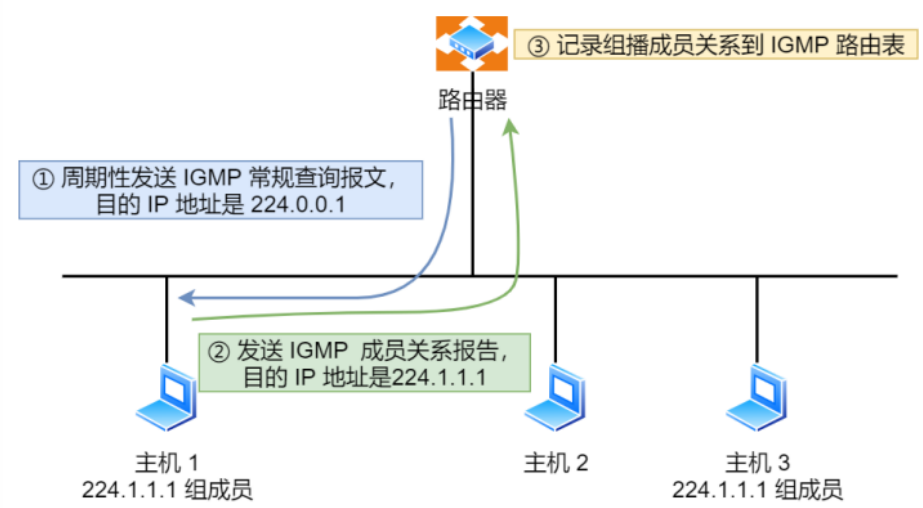
IGMP 报文向路由器申请加入和退出组播组，默认情况下路由器是不会转发组播包到连接中的主机，除非主机通过 IGMP 加入到组播组，主机申请加入到组播组时，路由器就会记录 IGMP 路由器表，路由器后续就会转 发组播包到对应的主机了。

IGMP 报文采用 IP 封装，IP 头部的协议号为 2，而且 TTL 字段值通常为 1，因为 IGMP 是工作在主机与连接的路由器之间。

（1）IGMP的工作机制

IGMP 分为了三个版本分别是，IGMPv1、IGMPv2、IGMPv3。 接下来，以 IGMPv2 作为例子，说说常规查询与响应和离开组播组这两个工作机制。

常规查询与响应机制:



（i）路由器会周期性发送目的地址为 224.0.0.1 （表示同一网段内所有主机和路由器） IGMP 常规查询报文。

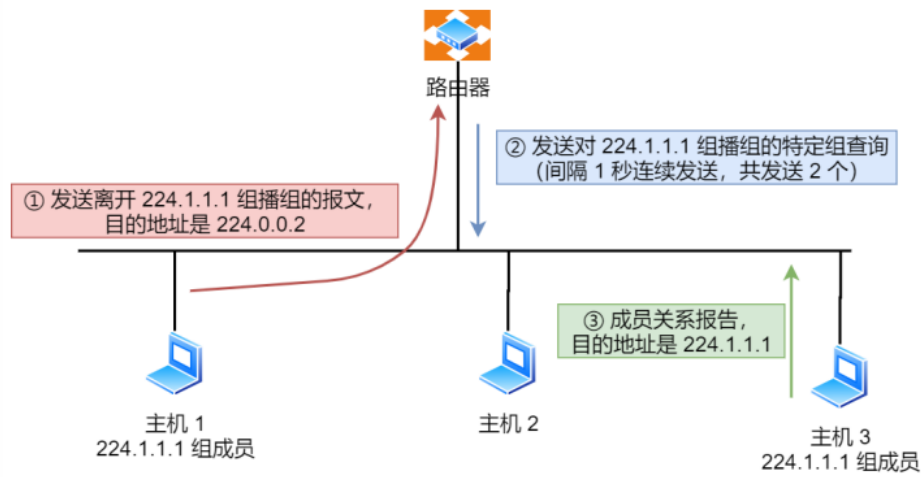
（ii）主机1 和 主机 3 收到这个查询，随后会启动「报告延迟计时器」，计时器的时间是随机的，通常是 0~10 秒，计时器超时后主机就会发送 IGMP 成员关系报告报文（源 IP 地址为自己主机的 IP 地址，目的 IP 地址为组播地址）。如果在定时器超时之前，收到同一个组内的其他主机发送的成员关系报告报文，则自己不再发 送，这样可以减少网络中多余的 IGMP 报文数。

（iii）路由器收到主机的成员关系报文后，就会在 IGMP 路由表中加入该组播组，后续网络中一旦该组播地址的数据到达路由器，它会把数据包转发出去

离开组播组工作机制：

情况1：

网段中仍有该组播组



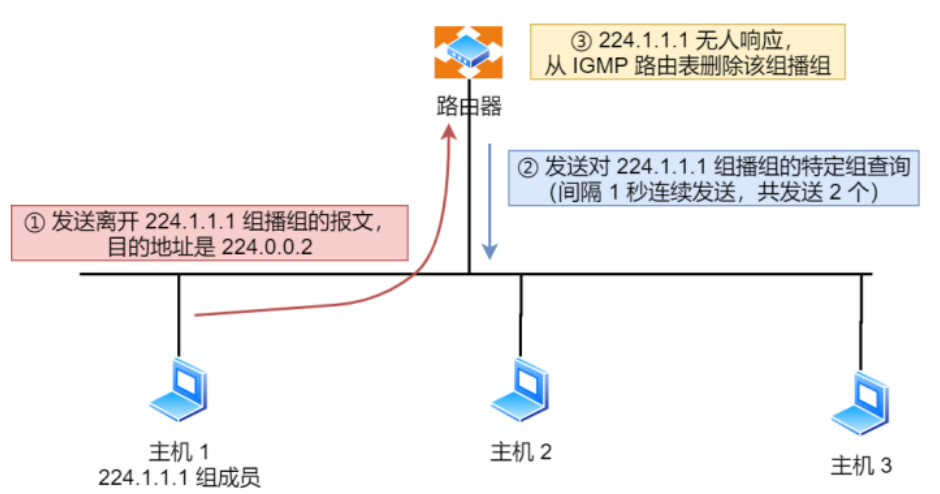
（i）主机 1 要离开组 224.1.1.1，发送 IGMPv2 离组报文，报文的目的地址是 224.0.0.2（表示发向网段内的所有路由器）

（ii）路由器 收到该报文后，以 1 秒为间隔连续发送 IGMP 特定组查询报文（共计发送 2 个），以便确认该网络是否还有 224.1.1.1 组的其他成员。

（iii）主机 3 仍然是组 224.1.1.1 的成员，因此它立即响应这个特定组查询。路由器知道该网络中仍然存在该组播组的成员，于是继续向该网络转发 224.1.1.1 的组播数据包。

情况2：

网段中没有该组播组



（i）主机 1 要离开组播组 224.1.1.1，发送 IGMP 离组报文。

（ii）路由器收到该报文后，以 1 秒为间隔连续发送 IGMP 特定组查询报文（共计发送 2 个）。此时在该网段内，组 224.1.1.1 已经没有其他成员了，因此没有主机响应这个查询。

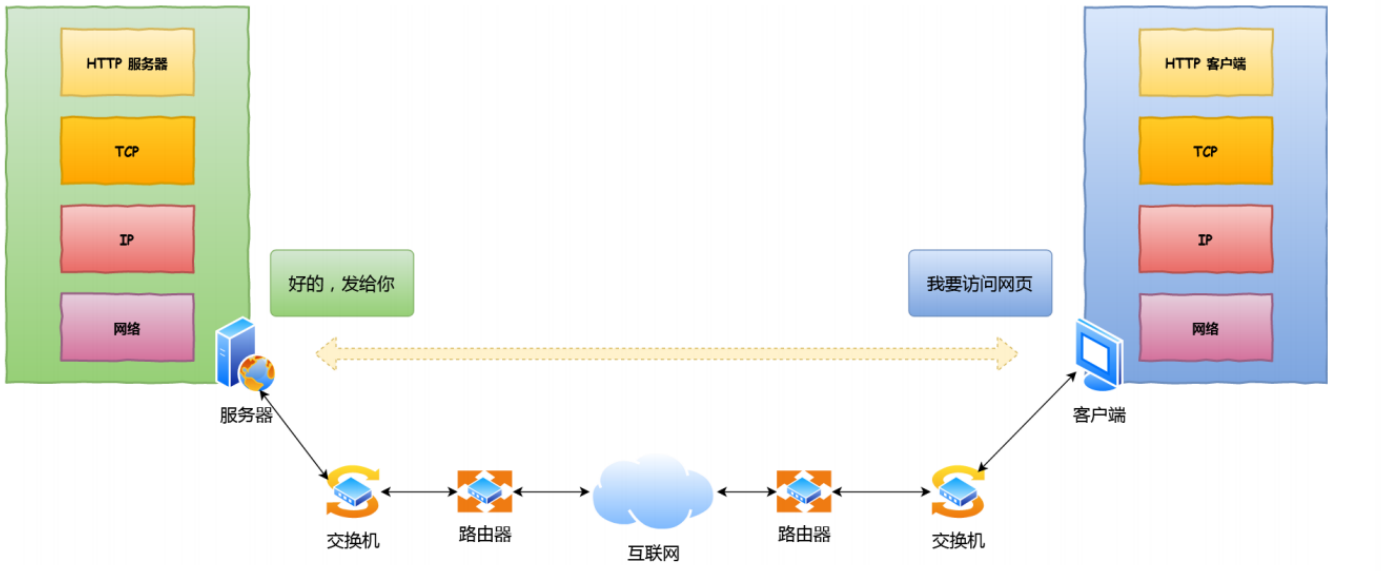
（iii）一定时间后，路由器认为该网段中已经没有 224.1.1.1 组播组成员了，将不会再向这个网段转发该组播地址的数据包。

注：组播地址不是用于机器ip地址的，因为组播地址没有网络号和主机号，所以跟dhcp没关系。组播地址一般是用于 udp协议，机器发送UDP组播数据时，目标地址填的是组播地址，那么在组播组内的机器都能收到数据包。 是否加入组播组和离开组播组，是由socket一个接口实现的，主机ip是不用改变的。

网络综合

问题

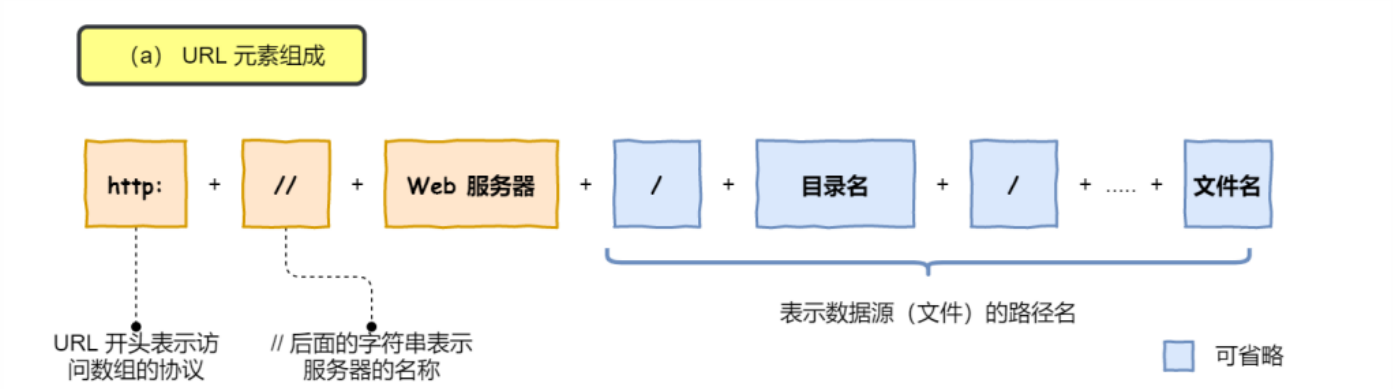
键入网址到网页显示，期间发生了什么?

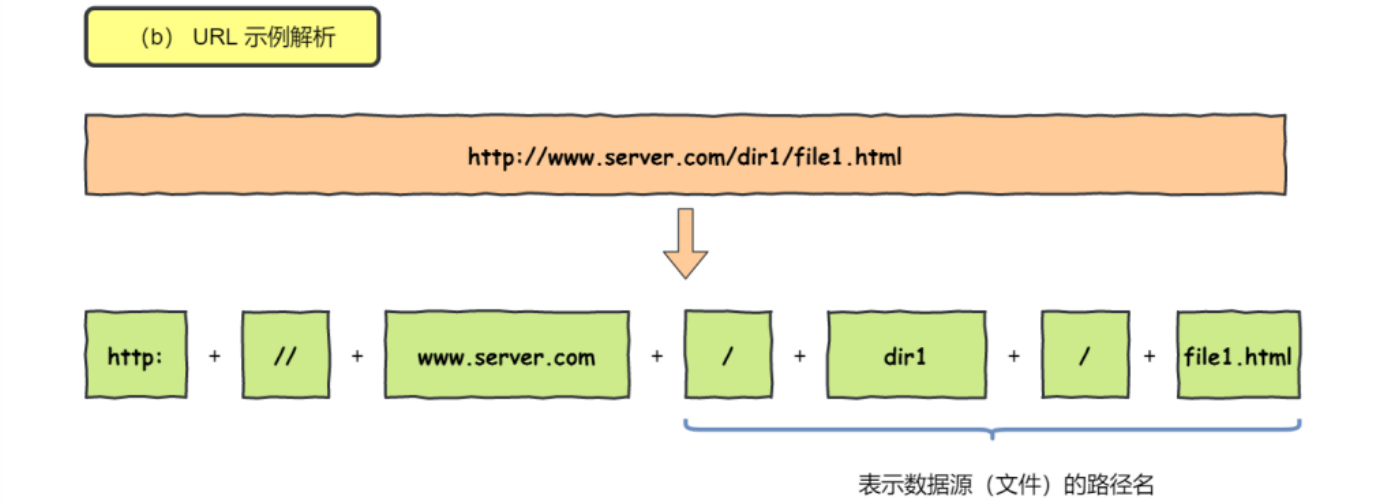


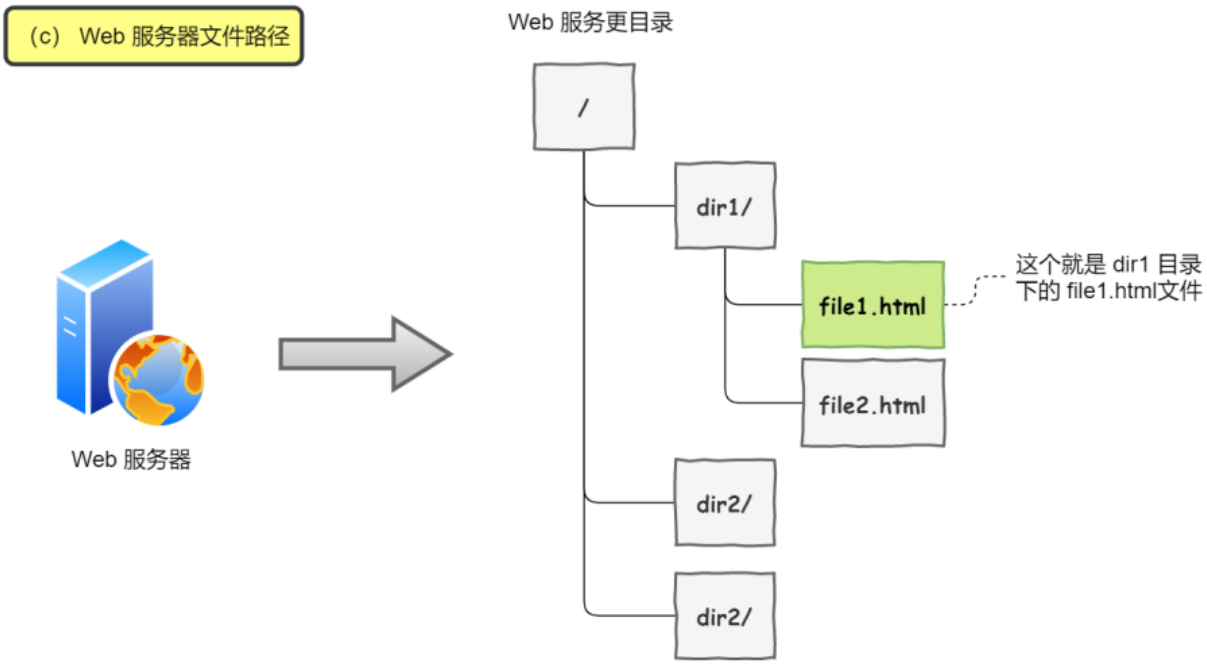
STEP1

解析URL，生成发送给WEB服务器的请求信息。

1、解析URL





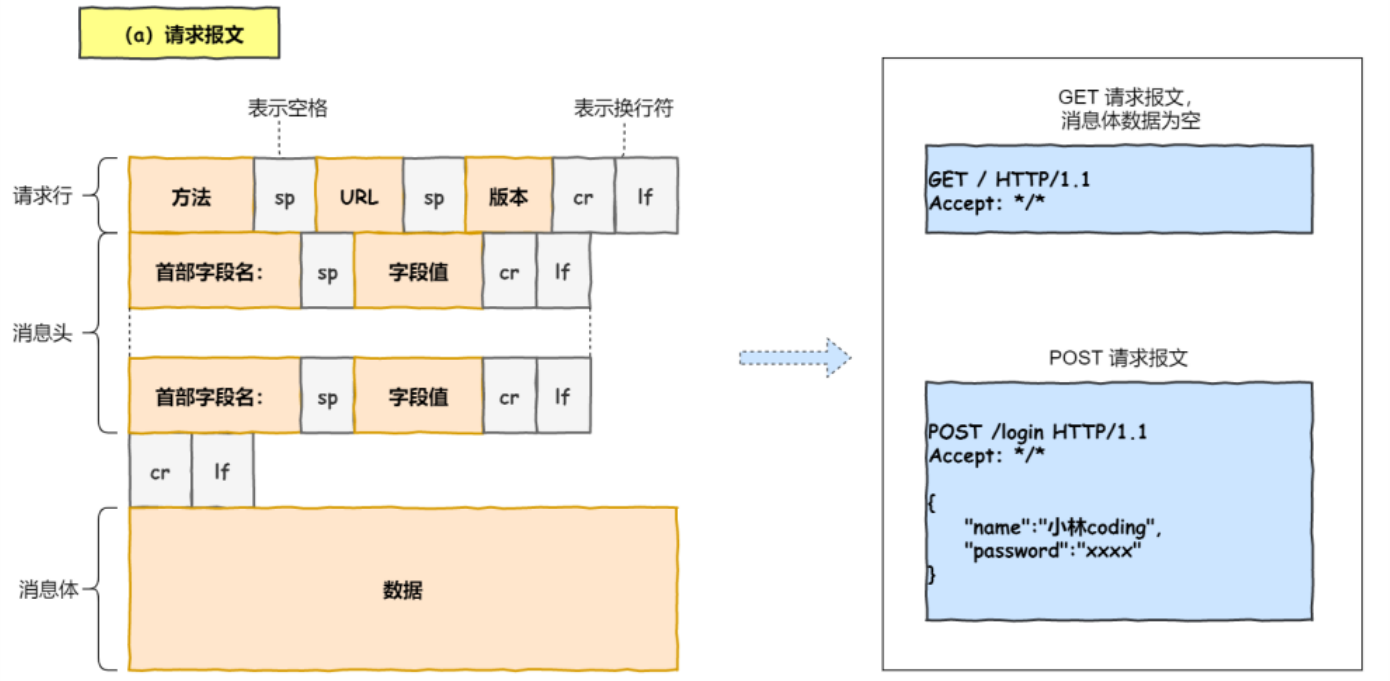


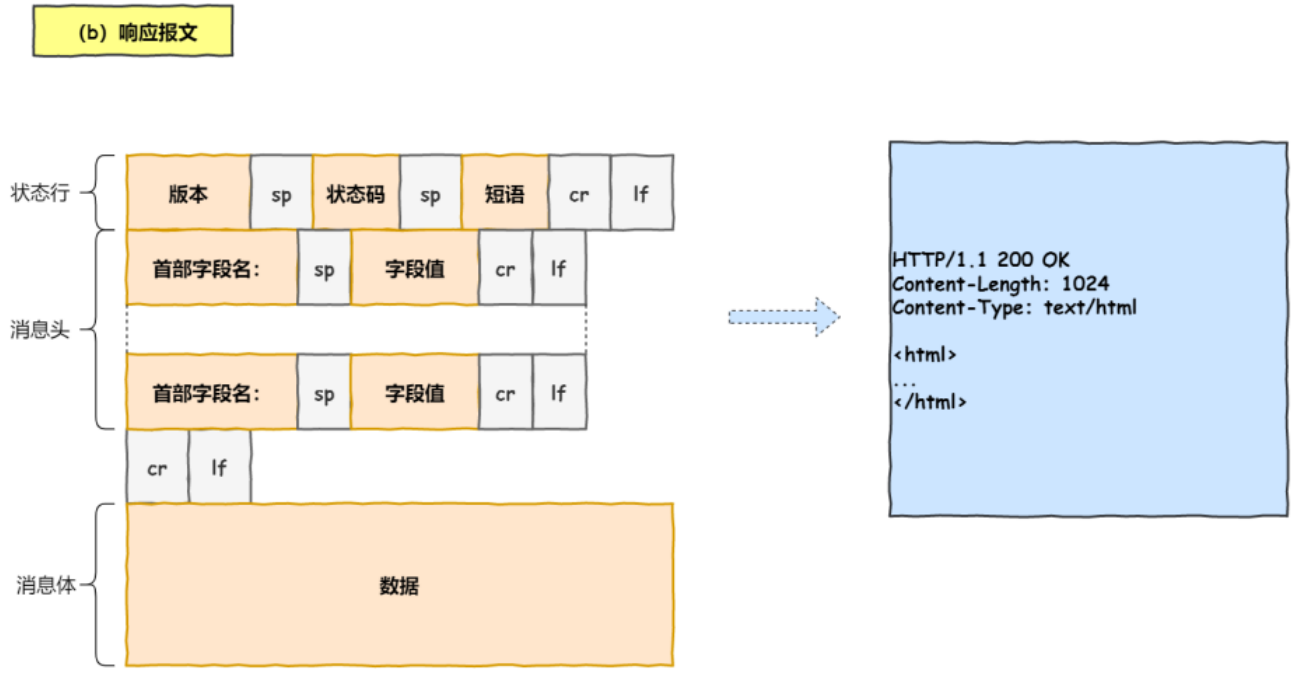
所以kk的 URL 实际上是请求服务器里的文件资源。

当图（a）示意图中的蓝色部分元素省略后（即：没有路径名），代表访问根目录下事先设置的默认文件，也就是 /index.html 或者 /default.html 这些文件，这样就不会发生混乱了。

2、生成HTTP请求信息

对 URL 进行解析之后，浏览器确定了 Web 服务器和文件名，接下来就是根据这些信息来生成 HTTP 请求消息了。





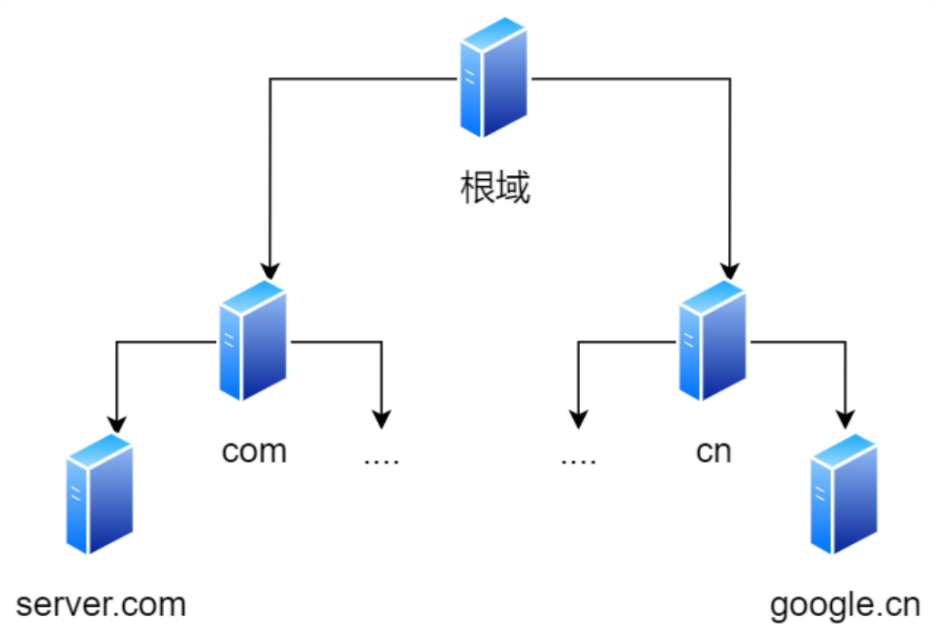
STEP2

通过浏览器解析 URL 并生成 HTTP 消息后，需要委托操作系统将消息发送给 Web 服务器。

但在发送之前，还有一项工作需要完成，那就是查询服务器域名对应的 IP 地址，因为委托操作系统发送消息时，必须提供通信对象的 IP 地址。

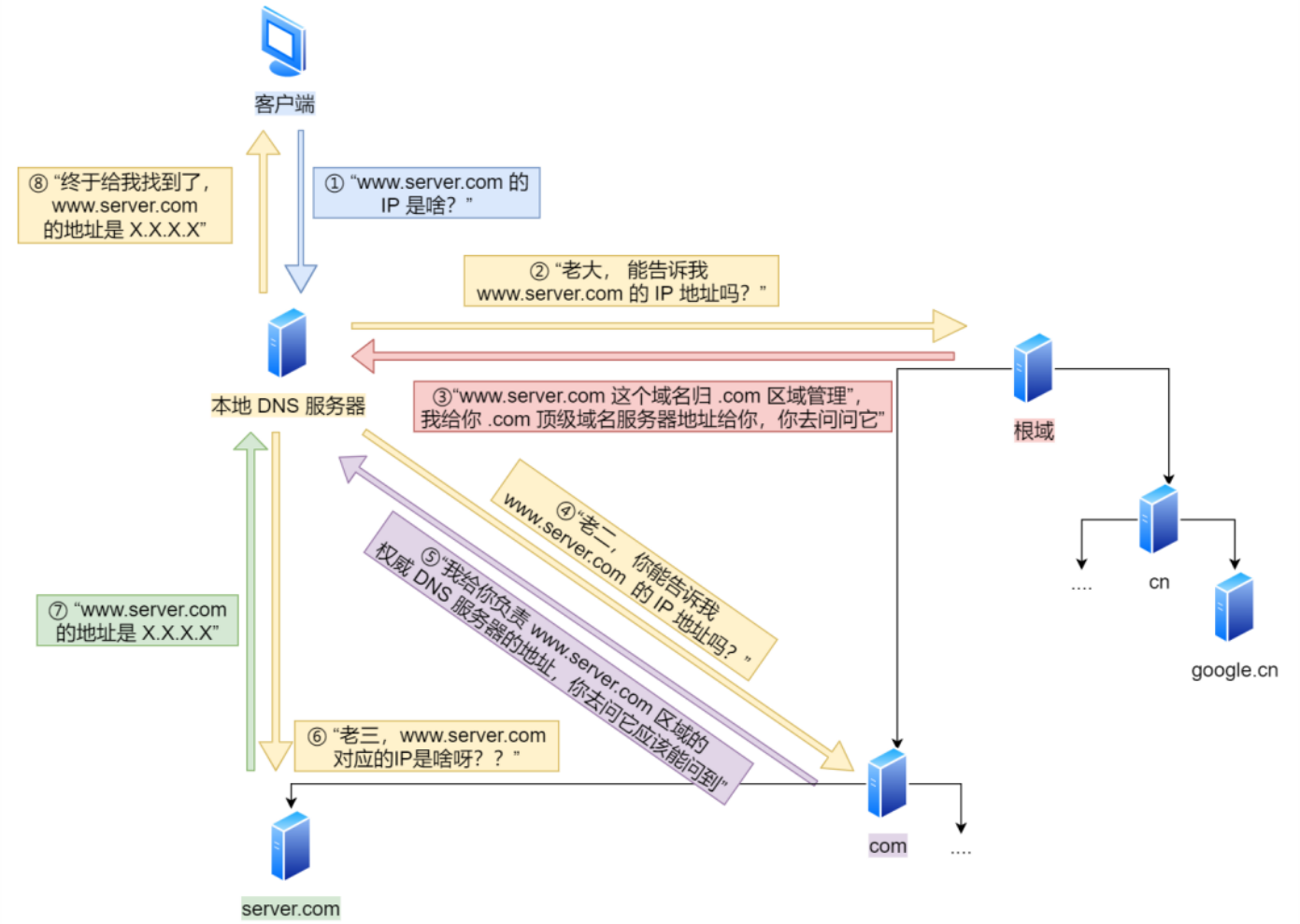
有一种服务器就专门保存了 Web 服务器域名与 IP 的对应关系，它就是 DNS 服务器。

1、域名的层级关系



根域的 DNS 服务器信息保存在互联网中所有的 DNS 服务器中。这样一来，任何 DNS 服务器就都可以找到并访问根域 DNS 服务器了。 因此，客户端只要能够找到任意一台 DNS 服务器，就可以通过它找到根域 DNS 服务器，然后再一路顺藤摸瓜找到位于下层的某台目标 DNS 服务器。

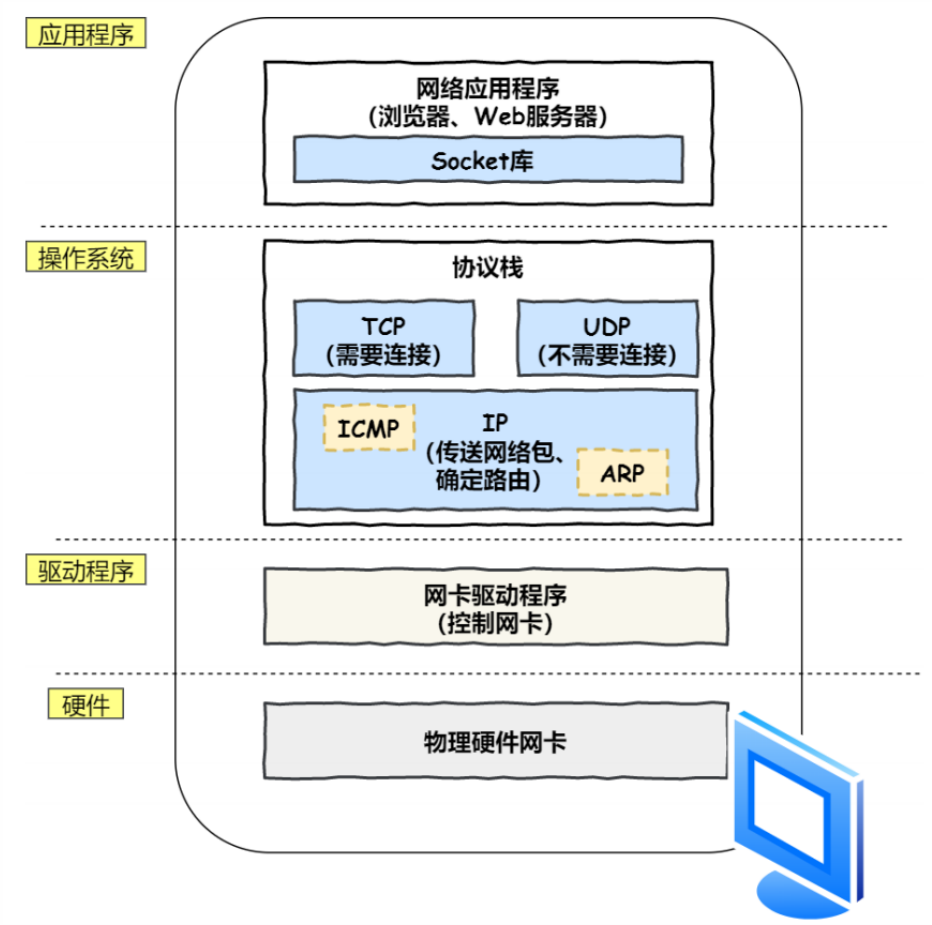
如下图所示：



2、如何发送数据包

数据包表示：“DNS 老大哥厉害呀，找到了目的地了！我还是很迷茫呀，我要发出去，接下来我需要谁的帮助呢?”

通过 DNS 获取到 IP 后，就可以把 HTTP 的传输工作交给操作系统中的协议栈。协议栈的内部分为几个部分，分别承担不同的工作。上下关系是有一定的规则的，上面的部分会向下面的部分委托工作，下面的部分收到委托的工作并执行。



应用程序（浏览器）通过调用 Socket 库，来委托协议栈工作。协议栈的上半部分有两块，分别是负责收发数据的 TCP 和 UDP 协议，它们两会接受应用层的委托执行收发数据的操作。

协议栈的下面一半是用 IP 协议控制网络包收发操作，在互联网上传数据时，数据会被切分成一块块的网络包，而将网络包发送给对方的操作就是由 IP 负责的。 此外 IP 中还包括 ICMP 协议和 ARP 协议。

- ICMP 用于告知网络包传送过程中产生的错误以及各种控制信息
- ARP 用于根据 IP 地址查询相应的以太网 MAC 地址

IP 下面的网卡驱动程序负责控制网卡硬件，而最下面的网卡则负责完成实际的收发操作，也就是对网线中的信号执行发送和接收操作。

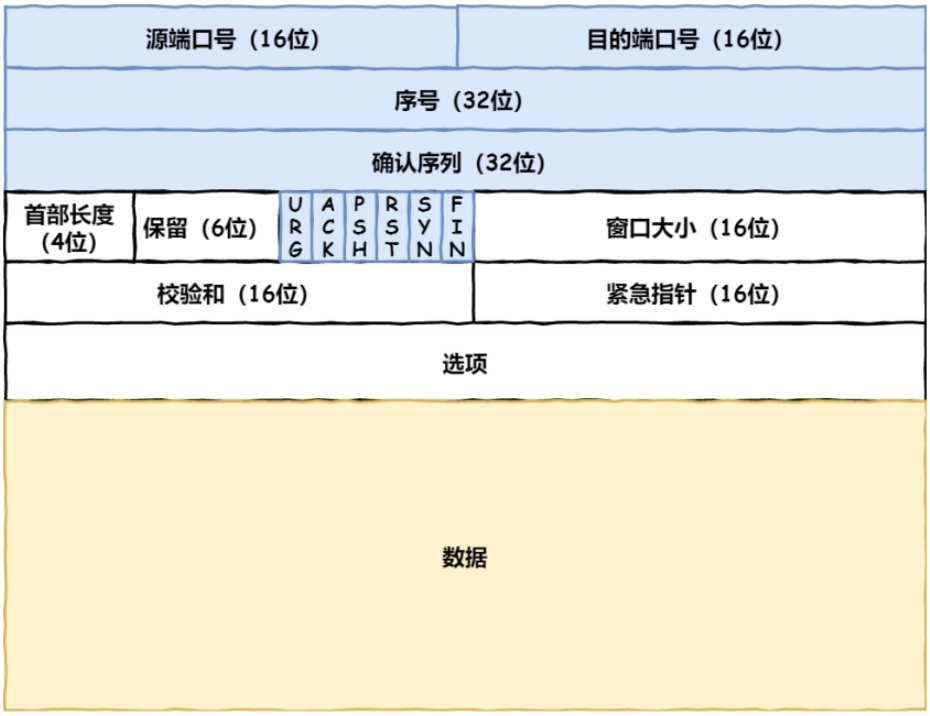
3、可靠传输—TCP

数据包看了这份指南表示：“原来我需要那么多大佬的协助啊，那我先去找找 TCP 大佬！”

（1）TCP包头格式

HTTP 是基于 TCP 协议传输的，所以在这我们先了解下 TCP 协议。

TCP包头格式：



首先，源端口号 和 目标端口号是不可少的，如果没有这两个端口号，数据就不知道应该发给哪个应用。

接下来有包的序号，这个是为了解决包乱序的问题。

还有应该有的是确认号，目的是确认发出去对方是否有收到。如果没有收到就应该重新发送，直到送达，这个是为了解决不丢包的问题。

接下来还有一些状态位。例如 SYN 是发起一个连接， ACK 是回复， RST 是重新连接， FIN 是结束连接等。TCP 是面向连接的，因而双方要维护连接的状态，这些带状态位的包的发送，会引起双方的状态变更。

还有一个重要的就是窗口大小。TCP 要做流量控制，通信双方各声明一个窗口（缓存大小），标识自己当前能够的处理能力，别发送的太快，撑死我，也别发的太慢，饿死我。

除了做流量控制以外，TCP还会做拥塞控制，对于真正的通路堵车不堵车，它无能为力，唯一能做的就是控制自己，也即控制发送的速度。不能改变世界，就改变自己嘛。

（2）TCP 传输数据之前，要先三次握手建立连接

这个所谓的「连接」，只是双方计算机里维护一个状态机，在连接建立的过程中，双方的状态变化时序图就像这样。

一开始，客户端和服务端都处于 CLOSED 状态。先是服务端主动监听某个端口，处于 LISTEN 状态。

然后客户端主动发起连接 SYN ，之后处于 SYN-SENT 状态。

服务端收到发起的连接，返回 SYN ，并且 ACK 客户端的 SYN ，之后处于 SYN-RCVD 状态。

客户端收到服务端发送的 SYN 和 ACK 之后，发送 ACK 的 ACK ，之后处于 ESTABLISHED 状态， 因为它一发一收成功了。

服务端收到 ACK 的 ACK 之后，处于 ESTABLISHED 状态，因为它也一发一收了。

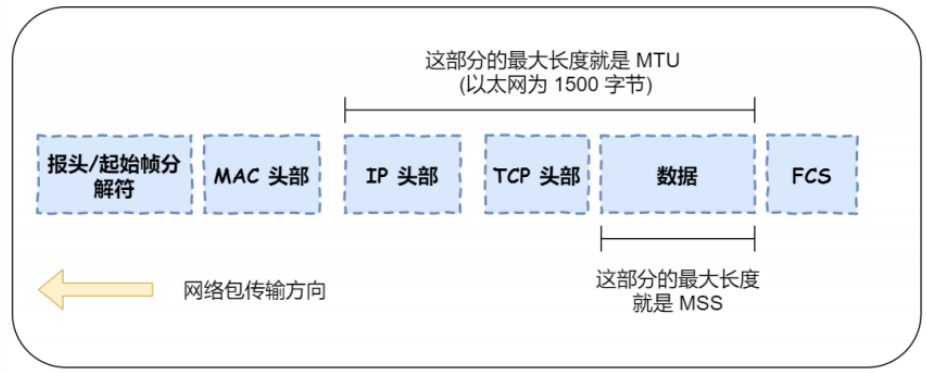
所以三次握手目的是保证双方都有发送和接收的能力。

（3）如何查看TCP的连接状态?

TCP 的连接状态查看，在 Linux 可以通过 netstat -napt 命令查看。

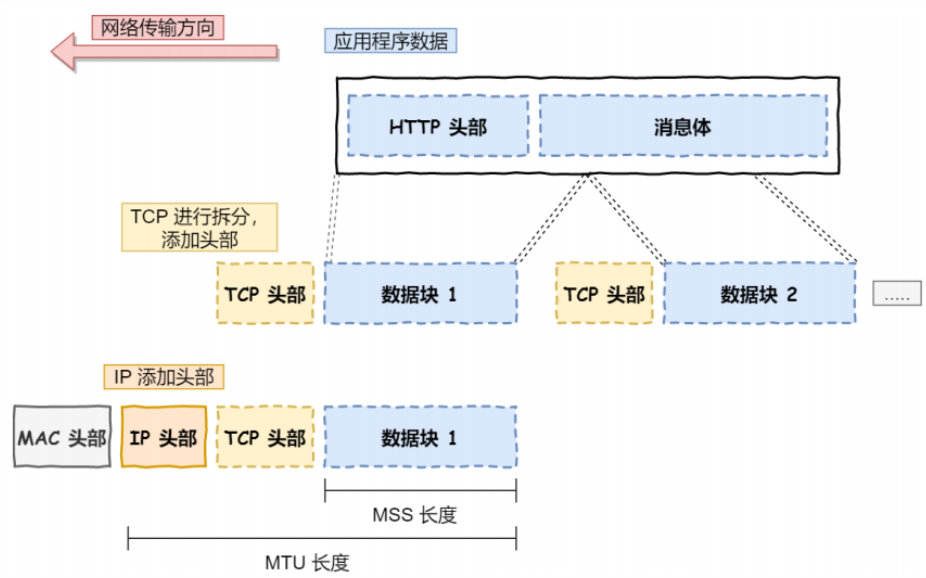
（4）TCP分割数据

如果 HTTP 请求消息比较长，超过了 MSS 的长度，这时 TCP 就需要把 HTTP 的数据拆解成一块块的数据发送，而不是一次性发送所有数据。



- MTU ：一个网络包的最大长度，以太网中一般为 1500 字节。
- MSS ：除去 IP 和 TCP 头部之后，一个网络包所能容纳的 TCP 数据的最大长度。

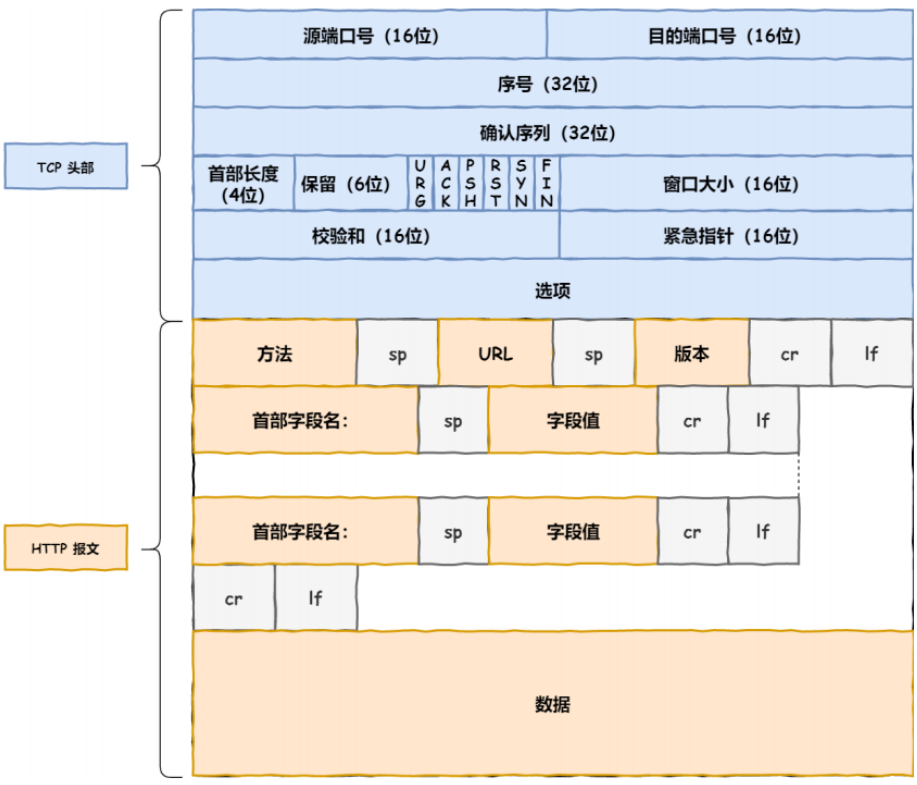
数据会被以 MSS 的长度为单位进行拆分，拆分出来的每一块数据都会被放进单独的网络包中。也就是在每个被拆分的数据加上 TCP 头信息，然后交给 IP 模块来发送数据。



（5）TCP报文生成

TCP 协议里面会有两个端口，一个是浏览器监听的端口（通常是随机生成的），一个是 Web 服务器监听的端口（HTTP 默认端口号是 80 ， HTTPS 默认端口号是 443 ）。

在双方建立了连接后，TCP 报文中的数据部分就是存放 HTTP 头部 + 数据，组装好 TCP 报文之后，就需交给下面的网络层处理。网络包的报文如下图：

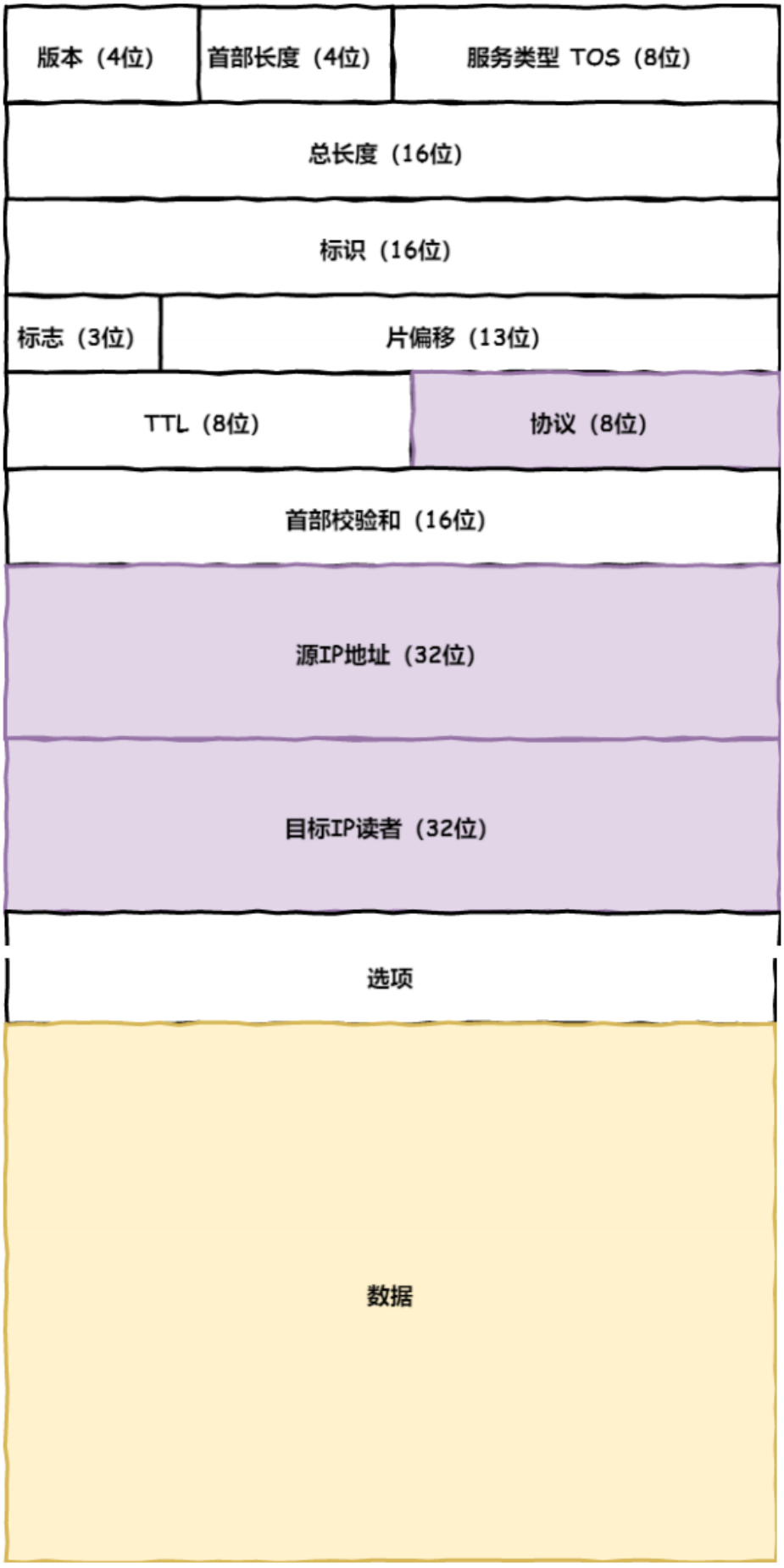


此时，遇上了 TCP 的数据包激动表示：“太好了，碰到了可靠传输的 TCP 传输，它给我加上 TCP 头部，我不再孤单了，安全感十足啊！有大佬可以保护我的可靠送达！但我应该往哪走呢？”（ 远程定位—IP ）

4、IP包头格式

TCP 模块在执行连接、收发、断开等各阶段操作时，都需要委托 IP 模块将数据封装成网络包发送给通信对象。

（1）IP包头格式如下：



在 IP 协议里面需要有源地址 IP 和 目标地址 IP：

- 源地址IP：即是客户端输出的 IP 地址

- 目标地址：即通过 DNS 域名解析得到的 Web 服务器 IP

因为 HTTP 是经过 TCP 传输的，所以在 IP 包头的协议号，要填写为 06 （十六进制），表示协议为 TCP。

（2）假设客户端有多个网卡

就会有多个 IP 地址，那 IP 头部的源地址应该选择哪个 IP 呢？

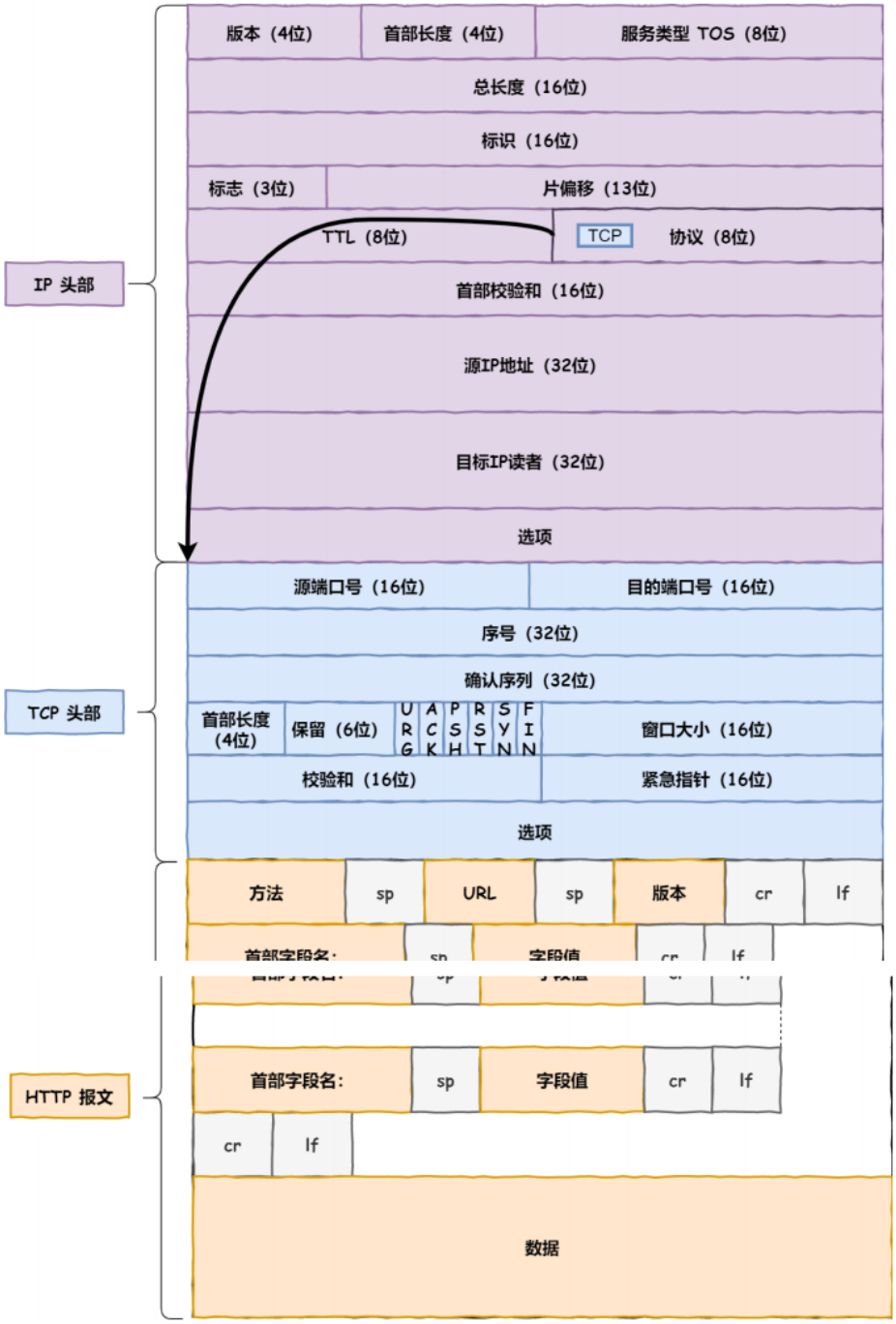
当存在多个网卡时，在填写源地址 IP 时，就需要判断到底应该填写哪个地址。这个判断相当于在多块网卡中判断应该使用哪个一块网卡来发送包。这个时候就需要根据路由表规则，来判断哪一个网卡作为源地址 IP。

在 Linux 操作系统，我们可以使用 route -n 命令查看当前系统的路由表。

```
[root@lincoding ~]# route -n

Kernel IP routing table
Destination     Gateway         Genmask         Flags Metric Ref    Use Iface
192.168.3.0     0.0.0.0         255.255.255.0   U     0      0        0 eth0
192.168.10.0    0.0.0.0         255.255.255.0   U     0      0        0 eth1
0.0.0.0         192.168.3.1     0.0.0.0         UG    0      0        0 eth0
```

第三条目比较特殊，它目标地址和子网掩码都是 0.0.0.0 ，这表示默认网关，如果其他所有条目都无法匹配，就会自动匹配这一行。并且后续就把包发给路由器，Gateway 即是路由器的 IP 地址。

（3）IP报文生成



5、两点传输-MAC

此时，加上了 IP 头部的数据包表示：“有 IP 大佬给我指路了，感谢 IP 层给我加上了 IP 包头，让我有了远程定位的能力！不会害怕在浩瀚的互联网迷茫了！可是目的地好远啊，我下一站应该去哪呢？”

生成了 IP 头部之后，接下来网络包还需要在 IP 头部的前面加上 MAC 头部。

（1）MAC包头格式

MAC 头部是以太网使用的头部，它包含了接收方和发送方的 MAC 地址等信息。

一般在 TCP/IP 通信里，MAC 包头的协议类型只使用：

- 0800：IP 协议
- 0806：ARP 协议

（2）MAC发送方和接收方如何确认

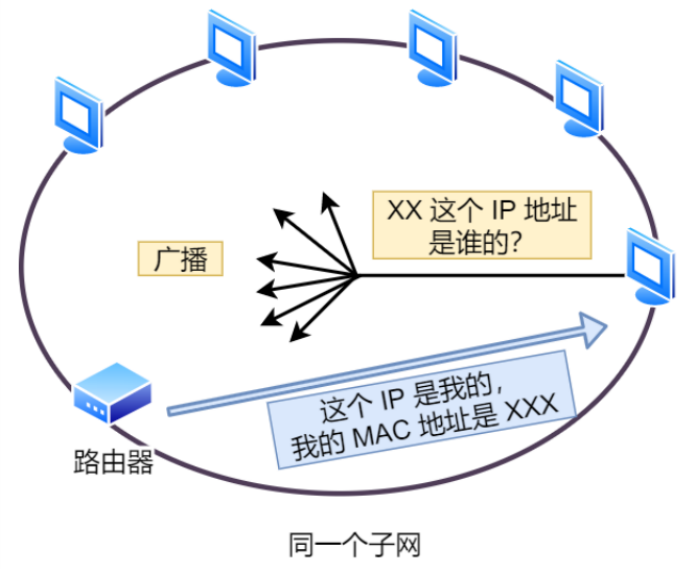
发送方的 MAC 地址获取就比较简单了，MAC 地址是在网卡生产时写入到 ROM 里的，只要将这个值读取出来写入到 MAC 头部就可以了。

接收方的 MAC 地址就有点复杂了，只要告诉以太网对方的 MAC 的地址，以太网就会帮我们把包发送过去，那么很显然这里应该填写对方的 MAC 地址。

所以先得搞清楚应该把包发给谁，这个只要查一下路由表就知道了。在路由表中找到相匹配的条目，然后把包发给 Gateway 列中的 IP 地址就可以了。

（3）既然知道要发给谁，按如何获取对方的 MAC 地址

不知道对方 MAC 地址？不知道就喊呗。此时就需要 ARP 协议帮我们找到路由器的 MAC 地址。



（4）如果每次都要广播获取，那不是很麻烦吗?

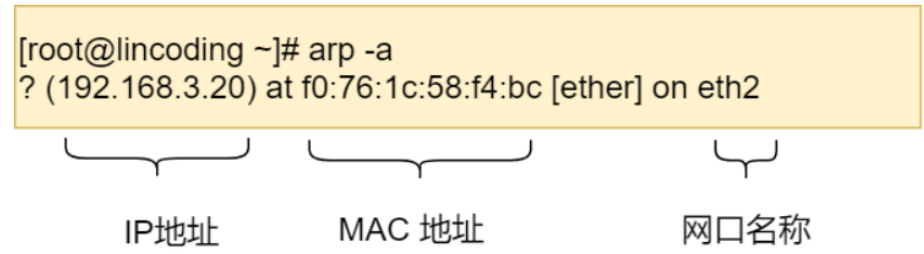
后续操作系统会把本次查询结果放到一块叫做 ARP 缓存的内存空间留着以后用，不过缓存的时间就几分钟。也就是说，在发包时：

先查询 ARP 缓存，如果其中已经保存了对方的 MAC 地址，就不需要发送 ARP 查询，直接使用 ARP 缓存中的地址。

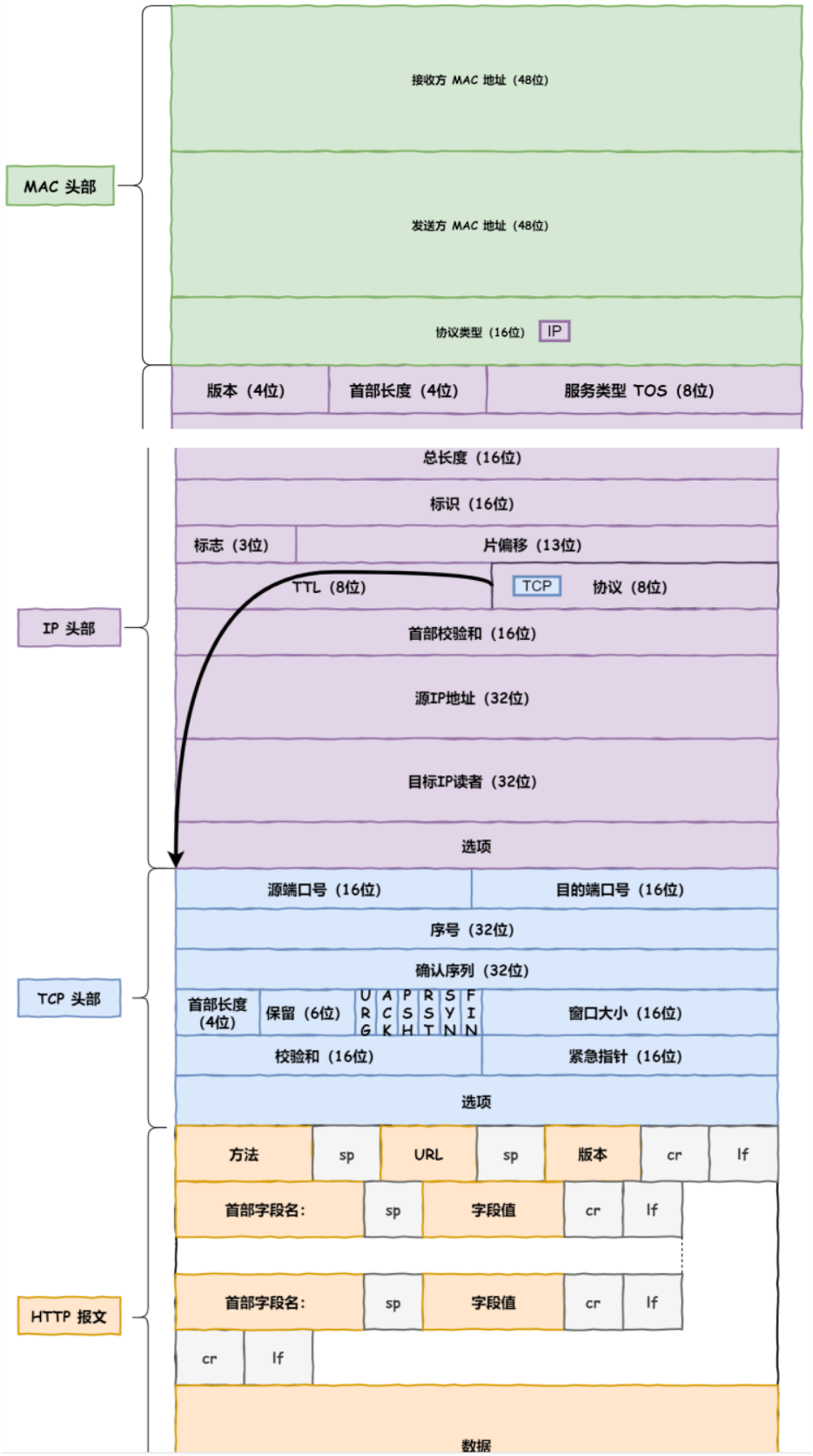
而当 ARP 缓存中不存在对方 MAC 地址时，则发送 ARP 广播查询。

（5）查看ARP缓存内容

在 Linux 系统中，我们可以使用 arp -a 命令来查看 ARP 缓存的内容。



（6）MAC报文生成

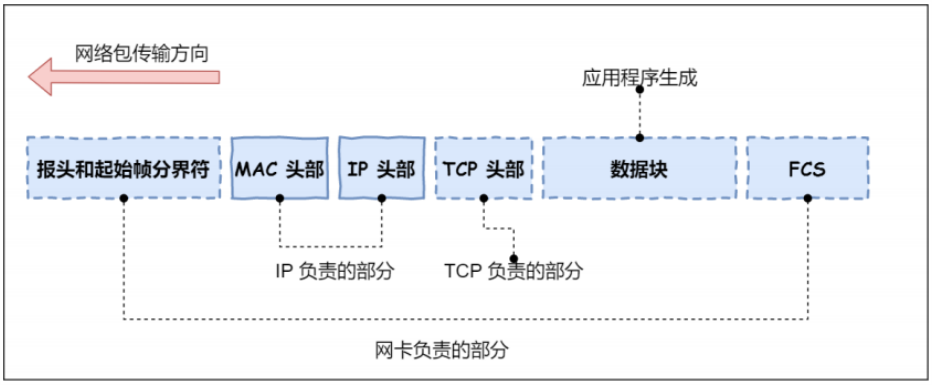


6、出口-网卡

此时，加上了 MAC 头部的数据包万分感谢，说道：“感谢 MAC 大佬，我知道我下一步要去哪了！我现在有很多头部兄弟，相信我可以到达最终的目的地！”。带着众多头部兄弟的数据包，终于准备要出门了。

网络包只是存放在内存中的一串二进制数字信息，没有办法直接发送给对方。因此，我们需要将数字信息转换为电信号，才能在网线上传输，也就是说，这才是真正的数据发送过程。负责执行这一操作的是网卡，要控制网卡还需要靠网卡驱动程序。

网卡驱动从 IP 模块获取到包之后，会将其复制到网卡内的缓存区中，接着会在其开头加上报头和起始帧分界符，在末尾加上用于检测错误的帧校验序列。



- 起始帧分界符是一个用来表示包起始位置的标记
- 末尾的 FCS （帧校验序列）用来检查包传输过程是否有损坏

最后网卡会将包转为电信号，通过网线发送出去。

7、送别者—交换机

唉，真是不容易，发一个包，真是历经千辛万苦。致此，一个带有许多头部的数据终于踏上寻找目的地的征途了！

下面来看一下包是如何通过交换机的？交换机的设计是将网络包原样转发到目的地。交换机工作在 MAC 层，也称为二层网络设备。

（1）交换机的包接收操作

首先，电信号到达网线接口，交换机里的模块进行接收，接下来交换机里的模块将电信号转换为数字信号。

然后通过包末尾的 FCS 校验错误，如果没问题则放到缓冲区。这部分操作基本和计算机的网卡相同，但交换机的工作方式和网卡不同。

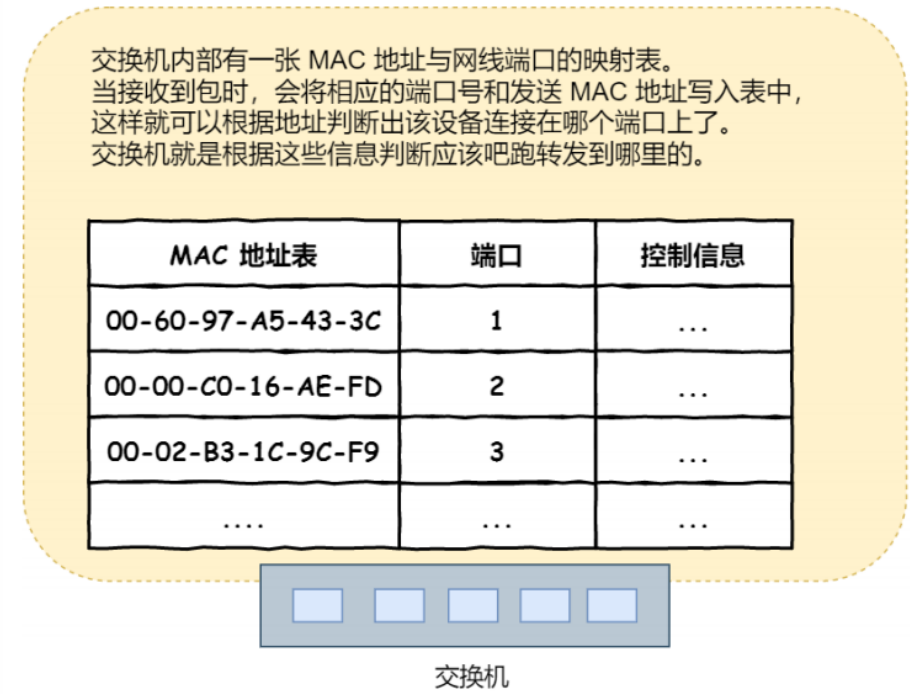
计算机的网卡本身具有 MAC 地址，并通过核对收到的包的接收方 MAC 地址判断是不是发给自己的，如果不是发给自己的则丢弃；相对地，交换机的端口不核对接收方 MAC 地址，而是直接接收所有的包并存放到缓冲区中。

因此，和网卡不同，交换机的端口不具有 MAC 地址。

将包存入缓冲区后，接下来需要查询一下这个包的接收方 MAC 地址是否已经在 MAC 地址表中有记录了。

交换机的 MAC 地址表主要包含两个信息：

- 一个是设备的 MAC 地址
- 一个是该设备连接在交换机的哪个端口上



交换机根据 MAC 地址表查找 MAC 地址，然后将信号发送到相应的端口。

（2）当 MAC 地址表找不到指定的 MAC 地址会怎么样？、

地址表中找不到指定的 MAC 地址。这可能是因为具有该地址的设备还没有向交换机发送过包，或者这个设备一段时间没有工作导致地址被从地址表中删除了。

这种情况下，交换机无法判断应该把包转发到哪个端口，只能将包转发到除了源端口之外的所有端口上，无论该设备连接在哪个端口上都能收到这个包。

这样做不会产生什么问题，因为以太网的设计本来就是将包发送到整个网络的，然后只有相应的接收者才接收包，而其他设备则会忽略这个包。

发送了包之后目标设备会作出响应，只要返回了响应包，交换机就可以将它的地址写入 MAC 地址表，下次也就不需要把包发到所有端口了。

此外，如果接收方 MAC 地址是一个广播地址，那么交换机会将包发送到除源端口之外的所有端口。

以下两个属于广播地址：

- MAC 地址中的 FF:FF:FF:FF:FF:FF
- IP 地址中的 255.255.255.255

8、出境大门—路由器

数据包通过交换机转发抵达了路由器，准备要离开土生土长的子网了。此时，数据包和交换机离别时说道：“感谢交换机兄弟，帮我转发到出境的大门，我要出远门啦！”

（1）路由器与交换机的区别

网络包经过交换机之后，现在到达了路由器，并在此被转发到下一个路由器或目标设备。

这一步转发的工作原理和交换机类似，也是通过查表判断包转发的目标。 不过在具体的操作过程上，路由器和交换机是有区别的。

- 因为路由器是基于 IP 设计的，俗称三层网络设备，路由器的各个端口都具有 MAC 地址和 IP 地址；

- 而交换机是基于以太网设计的，俗称二层网络设备，交换机的端口不具有 MAC 地址。

（2）路由器的基本原理

路由器的端口具有 MAC 地址，因此它就能够成为以太网的发送方和接收方；同时还具有 IP 地址，从这个意义上来说，它和计算机的网卡是一样的。

当转发包时，首先路由器端口会接收发给自己的以太网包，然后路由表查询转发目标，再由相应的端口作为发送方将以太网包发送出去。

（3）路由器的包接收操作

首先，电信号到达网线接口部分，路由器中的模块会将电信号转成数字信号，然后通过包末尾的 FCS 进行错误校验。如果没问题则检查 MAC 头部中的接收方 MAC 地址，看看是不是发给自己的包，如果是就放到接收缓冲区中，否则就否则就丢弃这个包。

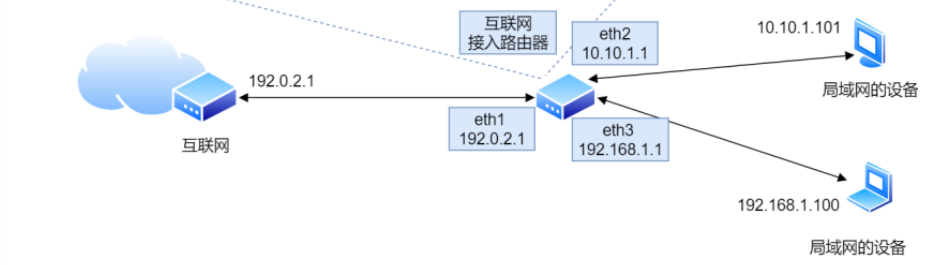
总的来说，路由器的端口都具有 MAC 地址，只接收与自身地址匹配的包，遇到不匹配的包则直接丢弃。

（4）查询路由表确定输出端口

完成包接收操作之后，路由器就会去掉包开头的 MAC 头部。 MAC 头部的作用就是将包送达路由器，其中的接收方 MAC 地址就是路由器端口的 MAC 地址。因此，当包到达路由器之后，MAC 头部的任务就完成了，于是 MAC 头部就会被丢弃。

接下来，路由器会根据 MAC 头部后方的 IP 头部中的内容进行包的转发操作。转发操作分为几个阶段，首先是查询路由表判断转发目标。

目标地址 (Destination)	子网掩码 (Netmask)	网关 (Gateway)	接口 (Interface)	跃点数 (Metric)
10.10.1.0	255.255.255.0	——	eth2	1
192.168.1.0	255.255.255.0	——	eth3	1
0.0.0.0	0.0.0.0	192.0.2.1	eth1	1



（5）路由器的发送操作

首先，我们需要根据路由表的网关列判断对方的地址。

如果网关是一个 IP 地址：

则这个IP 地址就是我们要转发到的目标地址，还未抵达终点，还需继续需要路由器转发。

如果网关为空：

则 IP 头部中的接收方 IP 地址就是要转发到的目标地址，也是就终于找到 IP 包头里的目标地址了，说明已抵达终点。

知道对方的 IP 地址之后，接下来需要通过 ARP 协议根据 IP 地址查询 MAC 地址，并将查询的结果作为接收方 MAC 地址。

路由器也有 ARP 缓存，因此首先会在 ARP 缓存中查询，如果找不到则发送 ARP 查询请求。

接下来是发送方 MAC 地址字段，这里填写输出端口的 MAC 地址。还有一个以太类型字段，填写 0800 （十六进制）表示 IP 协议。

网络包完成后，接下来会将其转换成电信号并通过端口发送出去。这一步的工作过程和计算机也是相同的。

发送出去的网络包会通过交换机到达下一个路由器。由于接收方 MAC 地址就是下一个路由器的地址，所以交换机会根据这一地址将包传输到下一个路由器。

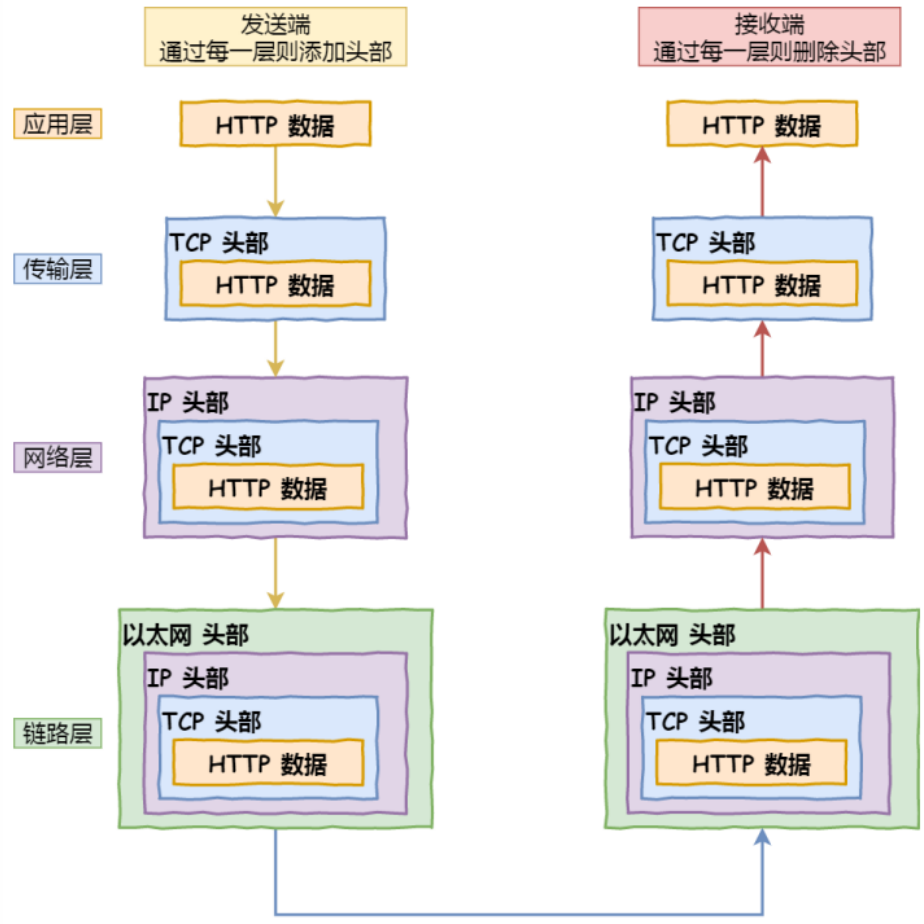
接下来，下一个路由器会将包转发给再下一个路由器，经过层层转发之后，网络包就到达了最终的目的地。

不知你发现了没有，在网络包传输的过程中，源 IP 和目标 IP 始终是不会变的，一直变化的是 MAC 地址，因为需要 MAC 地址在以太网内进行两个设备之间的包传输。

9、交换机转发

数据包通过多个路由器道友的帮助，在网络世界途经了很多路程，最终抵达了目的地的城门！城门值守的路由器，发现了这个小兄弟数据包原来是找城内的人，于是它就将数据包送进了城内，再经由城内的交换机帮助下，最终转发到了目的地了。数据包感慨万千的说道：“多谢这一路上，各路大侠的相助！”

数据包抵达了服务器，服务器肯定高兴呀，开始扒数据包的皮!



一个数据包臭不要脸的感受：

我一开始我虽然孤单、不知所措，但没有停滞不前。我依然满怀信心和勇气开始了征途。

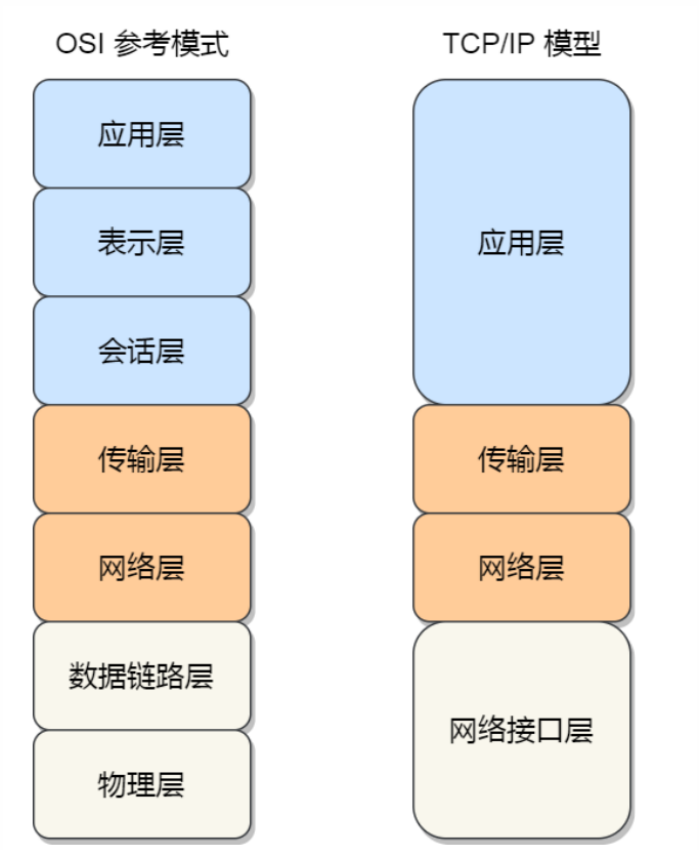
我很庆幸遇到了各路神通广大的大佬，有可靠传输的 TCP、有远程定位功能的 IP、有指明下一站位置的 MAC 等这些大佬都给我前面加上了头部，使得我能在交换机和路由器的转发下，抵达到了目的地！

这一路上的经历，让我认识到了网络世界中各路大侠协作的）要性，是他们维护了网络世界的秩序，感谢他们！

Linux收发网络包

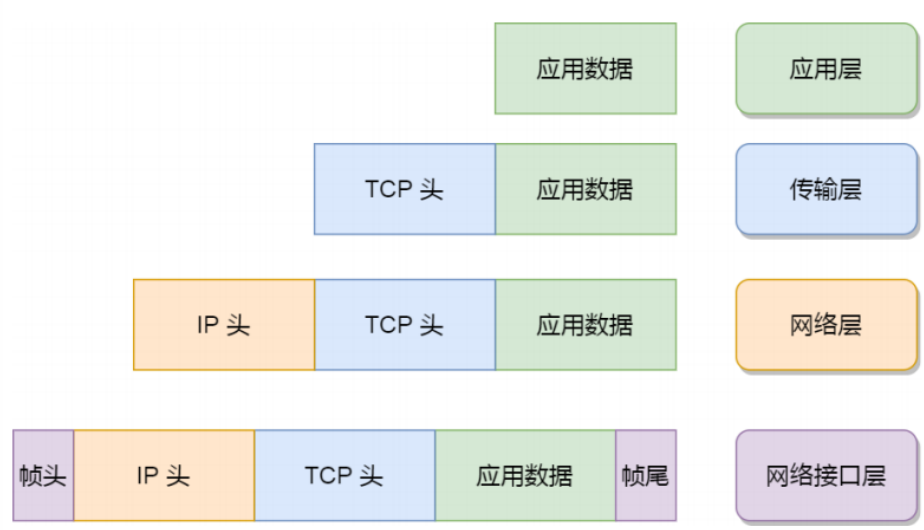
网络模型

两种网络模型：

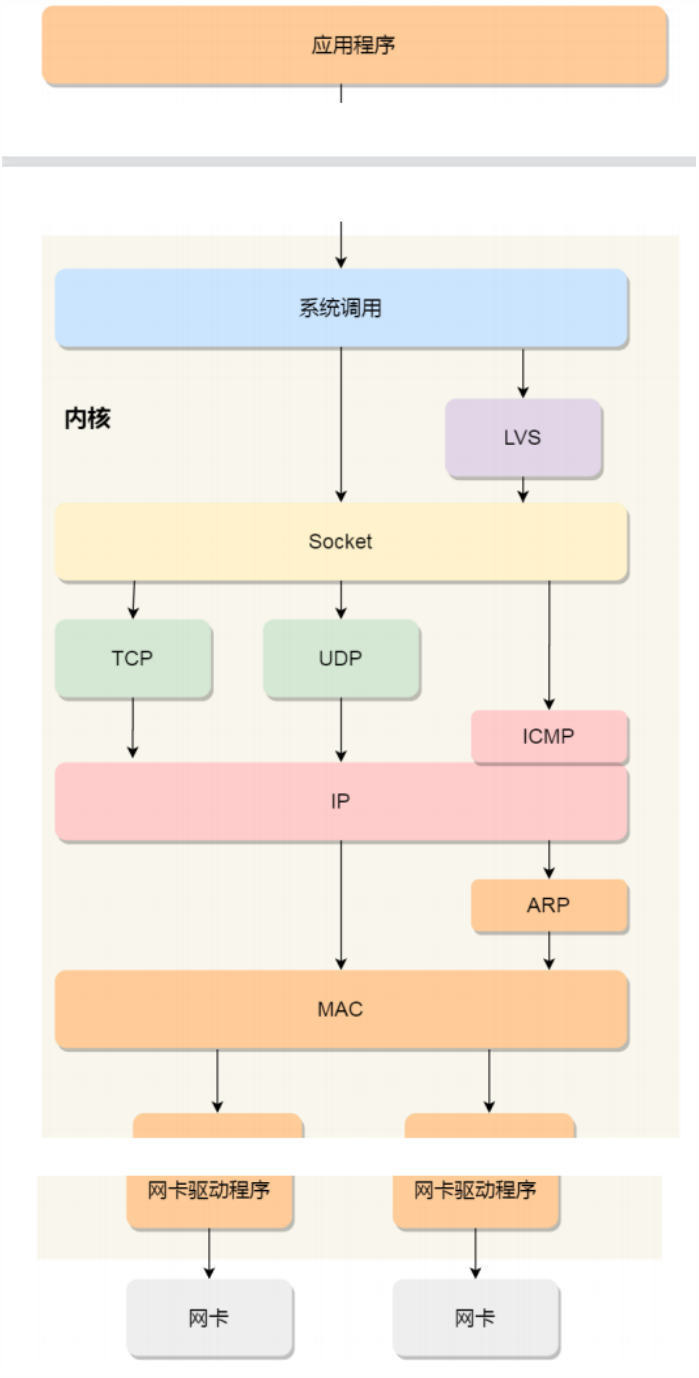


- 应用层，负责向用户提供一组应用程序，比如 HTTP、DNS、FTP 等；
- 传输层，负责端到端的通信，比如 TCP、UDP 等；
- 网络层，负责网络包的封装、分片、路由、转发，比如 IP、ICMP 等；
- 网络接口层，负责网络包在物理网络中的传输，比如网络包的封帧、 MAC 寻址、差错检测，以及通过网卡传输网络帧等；

对应的数据封装：


应用数据
应用层
TCP 头
应用数据
传输层
IP 头
TCP 头
应用数据
网络层
帧头
IP 头
TCP 头
应用数据
帧尾
网络接口层

Linux网络协议栈



从上述网络协议栈，可以看出：

- 应用程序需要通过系统调用，来跟 Socket 层进行数据交互；
- Socket 层的下面就是传输层、网络层和网络接口层；
- 最下面的一层，则是网卡驱动程序和硬件网卡设备；

收发流程

网卡是计算机里的一个硬件，专门负责接收和发送网络包，当网卡接收到一个网络包后，会通过 DMA 技术，将网络包放入到 Ring Buffer，这个是一个环形缓冲区。

1、发送

首先，应用程序会调用 Socket 发送数据包的接口，由于这个是系统调用，所以会从用户态陷入到内核态中的 Socket 层，Socket 层会将应用层数据拷贝到 Socket 发送缓冲区中。

接下来，网络协议栈从 Socket 发送缓冲区中取出数据包，并按照 TCP/IP 协议栈从上到下逐层处理。

如果使用的是 TCP 传输协议发送数据，那么会在传输层增加 TCP 包头，然后交给网络层；

网络层会给数据包增加 IP 包，然后通过查询路由表确认下一跳的 IP，并按照 MTU 大小进行分片。 分片后的网络包，就会被送到网络接口层，在这里会通过 ARP 协议获得下一跳的 MAC 地址，然后增加帧头和帧尾，放到发包队列中。

这一些准备好后，会触发软中断告诉网卡驱动程序，这里有新的网络包需要发送，最后驱动程序通过 DMA，从发包队列中读取网络包，将其放入到硬件网卡的队列中，随后物理网卡再将它发送出去。

2、接收

为了解决频繁中断带来的性能开销，Linux 内核在 2.6 版本中引入了 NAPI 机制，它是混合「中断和轮询」的方式来接收网络包，它的核心概念就是不采用中断的方式读取数据，而是首先采用中断唤醒数据接收的服务程序，然后 poll 的方法来轮询数据。

比如，当有网络包到达时，网卡发起硬件中断，于是会执行网卡硬件中断处理函数，中断处理函数处理完需要「暂时屏蔽中断」，然后唤醒「软中断」来轮询处理数据，直到没有新数据时才恢复中断，这样一次中断处理多个网络包，于是就可以降低网卡中断带来的性能开销。

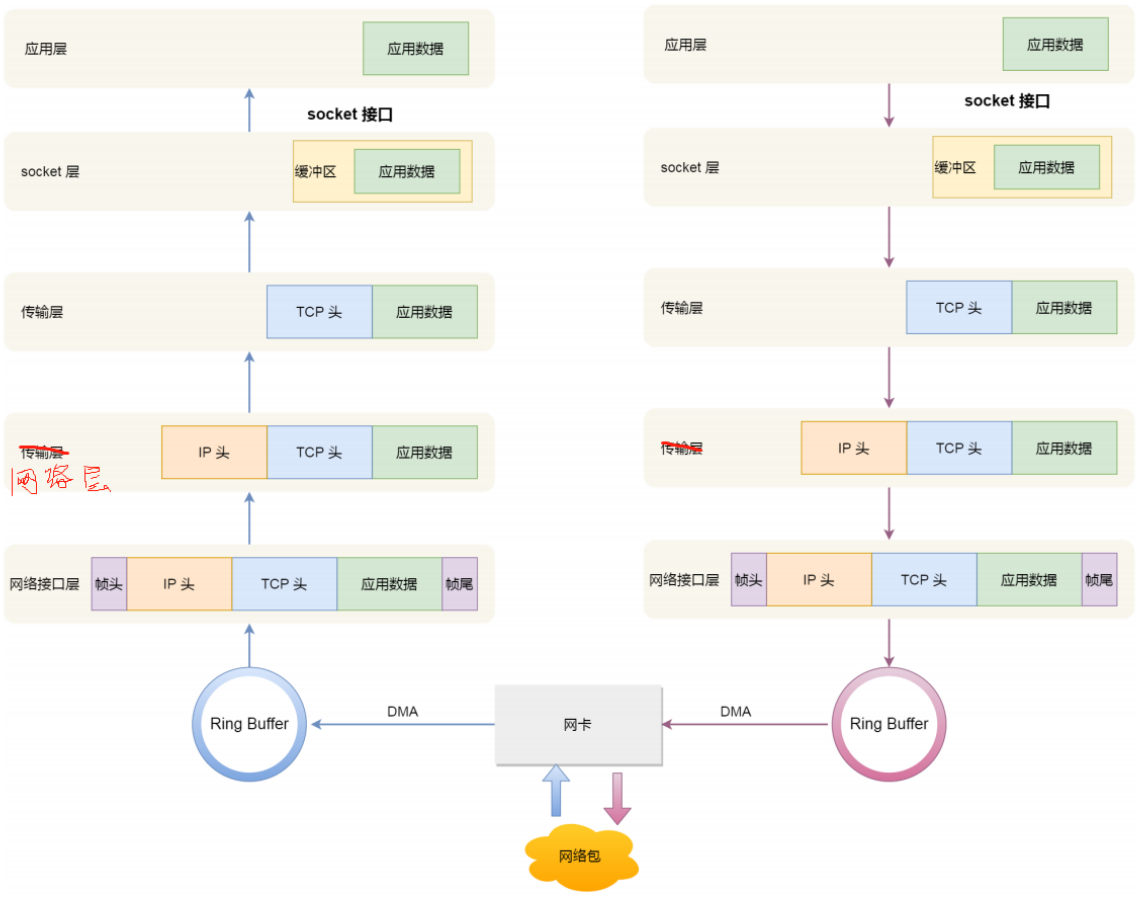
软中断会从 Ring Buffer 中拷贝数据到内核 struct sk_buff 缓冲区中，从而可以作为一个网络包交给网络协议栈进行逐层处理。

首先，会先进入到网络接口层，在这一层会检查报文的合法性，如果不合法则丢弃，合法则会找出该网络包的上层协议的类型，比如是 IPv4，还是 IPv6，接着再去掉帧头和帧尾，然后交给网络层。到了网络层，则取出 IP 包，判断网络包下一步的走向，比如是交给上层处理还是转发出去。

当确认这个网络包要发送给本机后，就会从 IP 头里看看上一层协议的类型是 TCP 还是 UDP，接着去掉 IP 头，然后交给传输层。

传输层取出 TCP 头或 UDP 头，根据四元组「源 IP、源端口、目的 IP、目的端口」 作为标识，找出对应的 Socket，并把数据拷贝到 Socket 的接收缓冲区。

最后，应用层程序调用 Socket 接口，从内核的 Socket 接收缓冲区读取新到来的数据到应用层。



Web页面请求

1. Web的页面请求流程以及涉及到技术

【浏览器地址栏输入URL回车后涉及到的流程】

1.1 查找DNS缓存

1. 先查找浏览器DNS缓存，看是否存放目标网络的IP地址;
2. 如果不在浏览器缓存，则浏览器将对操纵系统发起系统调用，查询操作系统本地缓存;
3. 如果不在操作系统本地缓存，则浏览器会查询与之相连的路由器缓存;
4. 如果不在路由器缓存，则浏览器会检查ISP【本地通信服务商】缓存;

若以上四步均没有查询到目标网络的IP地址，则发起DNS查询。

1.2 发起DNS查询

判断DNS服务器和我们的主机是否在同一子网内

1. 在同一子网，则采用 ARP 地址解析协议对 DNS 服务器进行 ARP 查询
2. 不在同一子网，则采用 ARP 地址解析协议对默认网关进行查询

若此时还是查询不到 IP 地址，则根据拿到 DNS 服务器或者默认网关的 IP 地址，继续进行 DNS 请求
使用53端口先向本地 DNS 服务器发送 UDP 请求包，此处一般使用 UDP 协议（如果响应包太大，则使用 TCP 协议）

没有查询到 IP 地址：

则它会发送一个递归查询请求，一层一层向高层DNS服务器查询，直到查询到 IP 地址，则将结果返回

【解释：DNS 是分布式域名服务器，每台服务器只维护一部分 IP 地址到网络地址的映射，没有任何一台服务器能够维持全部的映射关系】。

1.3 封装TCP数据包

拿到 IP 地址后，根据 URL 中的端口可知端口号【HTTP：80；HTTPS：443】，一般先会先尝试建立 HTTP 连接;

准备 TCP 数据包：

步骤：

1. 将应用层传递下来的实际数据，在传输层添加TCP首部;
2. 将传输层传下来的数据在网络层添加IP首部;
3. 将网络层传输下来的数据，在数据链路层添加以太网首部，并在传输介质中进行传输。

1.4 浏览器与目标服务器建立TCP连接

经过上述DNS和ARP查询流程后，浏览器会收到目标服务器的IP和MAC地址，然后经过三次握手后建立TCP连接;

1、使用HTTP协议

浏览器发送请求到服务器，如果使用的是HTTP协议，则服务器直接返回结果;

2、使用HTTPS协议

如果不是 HTTP 协议，则服务器会返回一个以 3 开头的重定向消息，告诉浏览器使用的 HTTPS，IP 没变，只是端口号变成 443；完成四次挥手；
重新建立 TCP 连接，将端口号修改为 443，同时沟通好双方的使用的认证算法、加密和解密算法，在次过程中也会检查对方的 CA 安全证书，采用 SSL 加密技术进行传输数据。

1.5 浏览器发送HTTP/HTTPS请求到web服务器

主要使用两种请求方式：

1. 浏览器发送get请求，要求目标服务器提供输入的网页;
2. 浏览器发送post请求，表示填写的是表单。

1.6 服务器处理请求并发挥一个响应

服务器会从浏览器接受请求并将其传递给请求处理程序并响应;

1.7 服务器发回一个HTTP响应

一般响应包包含：请求的网页以及状态码，压缩类型，如何缓存的页面，设置的cookie;

1.8 浏览器显示HTML页面

1. 渲染HTML骨架；涉及到Ajax技术;
2. 检查HTML标记并发送GET请求以获取网页上的其他元素【图像、CSS样式、JS文件等】，该静态文件一般由浏览器缓存，再次访问，不用重新请求;
3. 最后会看到请求色彩斑斓的网页。

2. web页面请求涉及到的问题

2.1 DNS服务器

对于DNS服务器若采用集中式的设计有以下问题

1、单点故障

如果 DNS 服务器崩溃，那么整个网络随之瘫痪。通信容量(traaffic volume)，单个 DNS 服务器不得不处理所有的 DNS 查询，这种查询级别可能是上百万上千万级，一台服务器很难满足;

2、远距离集中式数据库

单个 DNS 服务器不可能 邻近 所有的用户，假设在美国的 DNS 服务器不可能临近让澳大利亚的查询使用，其中查询请求势必会经过低速和拥堵的链路，造成严重的时延;

3、维护

维护成本巨大，而且还需要频繁更新。

2.2 DNS查询的三种方式

1、递归查询

DNS 客户端要求 DNS 服务器（一般为 DNS 递归解析器）将使用所请求的资源记录响应客户端，或者如果解析器无法找到该记录，则返回错误消息;

2、迭代查询

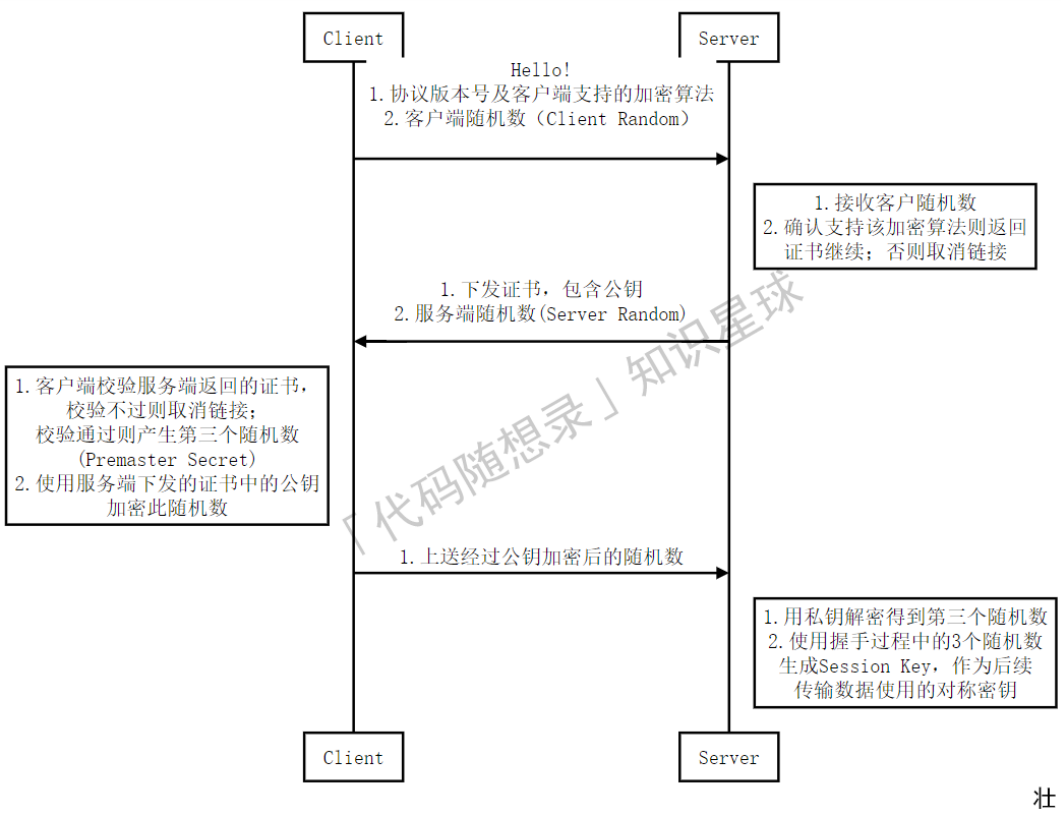
如果所查询的 DNS 服务器与查询名称不匹配，则其将返回对较低级别域名空间具有权威性的 DNS 服务器的引用。然后，DNS 客户端将对引用地址进行查询。此过程继续使用查询链中的其他 DNS 服务器，直至发生错误或超时为止;

3、非递归查询

当 DNS 解析器客户端查询 DNS 服务器以获取其有权访问的记录时通常会进行此查询，因为其对该记录具有权威性，或者该记录存在于其缓存内。

DNS 服务器通常会缓存 DNS 记录，查询到来后能够直接返回缓存结果，以防止更多带宽消耗和上游服务器上的负载。

2.3 HTTPS三次握手流程



壮

RPC远端过程调用

功能：客户端可以像调用本地函数一样的调用服务端的函数

RPC的本质是提供了一种轻量无感知的跨进程通信的方式。

RPC调用的流程：

1. 客户端调用函数（函数3个参数：待调用的接口名+函数名，待调用函数的形参和接收结果的指针）
2. 客户端存根收到调用之后将要待调用的接口名+函数名，待调用函数中需要传的参数和执行函数后接收执行结果的指针类型打包序列化成二进制数据
3. 将序列化后的数据通过网络传输到服务端
4. 服务端进行反序列化从客户端传来的数据
5. 根据反序列化后的数据去找到并执行客户端需要调用的函数
6. 将函数执行的结果序列化成二进制数据
7. 将执行结果通过网络传输给客户端
8. 客户端反序列化从服务端传来的二进制数据
9. 客户端得到执行结果

代码：

序列化：Go默认Gob编码，Gob支持HTTP，TCP

Json可以跨语言，Json只支持TCP

Protobuf的语法：

message相当于结构体，service像一个接口，里面的数字是按顺序写的，并不是赋值，范围是1-15

```
message Test{ // 命名是大驼峰
 string aaa = 1;
 int32 bbb = 2;
 repeated string ccc = 3; // 这个repeated修饰符代表是可变数组
}
```

RPC 的架构：

1. 传输层（TCP传输插件，HTTP传输插件）
2. 协议层（协议插件，序列化插件，解压缩插件）
3. 集群层（服务发现插件，连接管理插件，负载均衡插件，路由插件，容错插件，配置管理插件）
4. 入口层（动态代理插件，链路追踪插件，过滤链插件）

把每个功能都抽象成一个接口，这样便于以后的扩展

服务发现分为 CP 和 AP

CP：强一致性

1. 用Zookeeper作为服务注册中心
2. 服务平台管理端创建一个服务根目录
3. 服务提供方发起注册，然后会在提供方目录创建一个临时节点
4. 服务请求方发起订阅，然后会在调用方目录创建一个临时节点，然后会 watch 提供方目录中的所有节点，只要提供方目录有任何变化，zookeeper就会通知请求方

CP 存在一个问题，当超大规模集中上线的时候，大量的服务注册，就会导致 zookeeper 集群 CPU 飙升，最后不能工作

对于上面的问题可以**牺牲强一致性**，选择AP最终一致，采用消息总线机制，可以用消息队列来缓存

1. 注册中心节点收到注册请求的时候，服务列表发生变化，会发送一个消息到消息总线中，每个消息都有一个递增的版本号
2. 消息总线会主动推送消息到各个注册中心中，注册中心也会定时拉取消息，获取到的消息放到消息回放中，相当于一个接口，用来存可以调用的方法，只存大于本地版本号的消息，从而实现一致性
3. 消费者订阅从注册中心内存获取到所有接口信息，然后存到消费者缓存里
4. 采用推拉模式，消费者可以及时的拿到服务端中变化的情况和内存缓存的数据合并

为了性能，采用了注册中心和消费者的内存缓存，通过**异步推拉**来实现一致性

如何选用AP和CP主要看业务场景，并不是所有情况下AP就是最好的。

健康检测：

调用方定时向提供方发送心跳包来判断状态，比如说在一定时间内连续发送的包，失败的次数大于设置的最大失败上限，就得把健康转换亚健康，如果一个时间段内就没成功过，从亚健康转换到死亡，具体的失败次数看个人的设置

在有些场景下根据失败数来定义一个连接是否健康不是特别的准确，可以计算可用率来判断是否健康，在一个时间段内，成功个数/总发包的个数。

状态有三种：

健康，亚健康，死亡，除了死完不能转换成亚健康，其他都可以双向转换

路由策略：

主要的应用是灰度发布，平滑切换

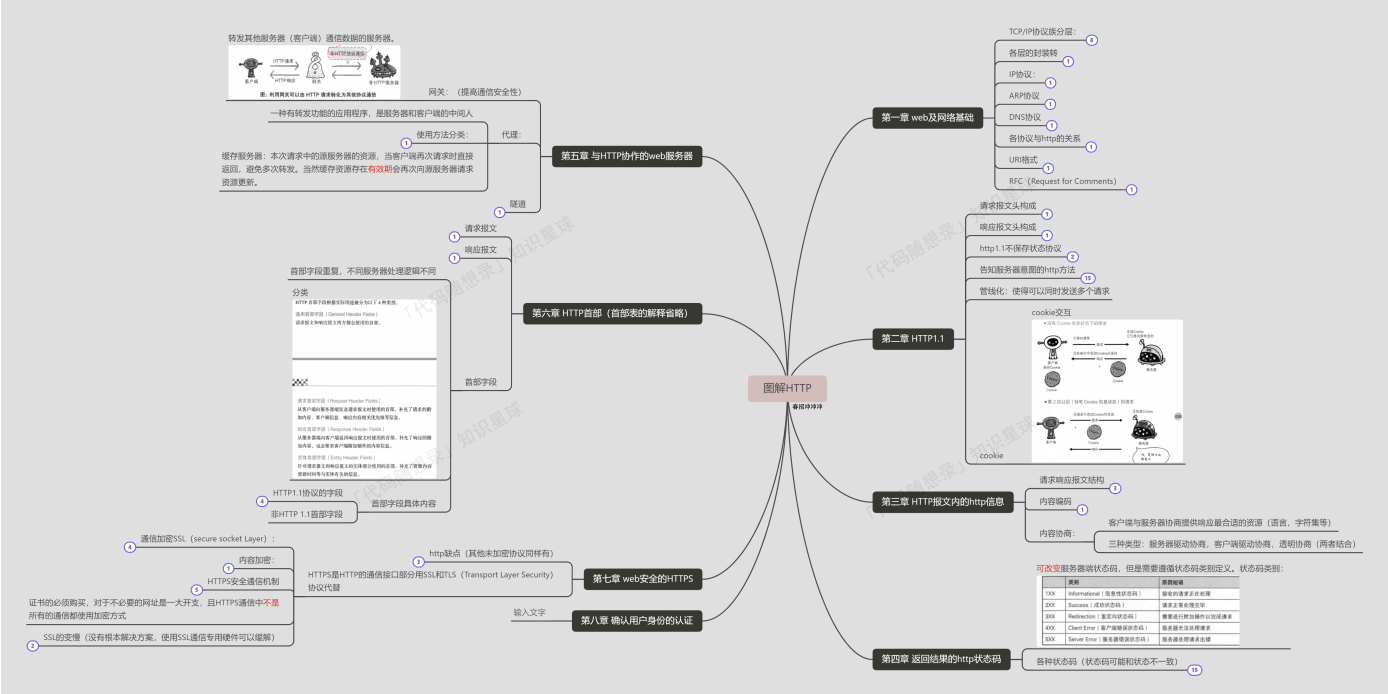
负载均衡：

RPC中实现智能负载，根据不同的场景智能的调整每个节点的权值

《图解HTTP》

思维导图

原图链接：「代码随想录」知识星球



chapter 3

1. 以太网变迁

1、总线型以太网

1. 使用总线、基于 CSMA/CD 协议
2. 以太网中所有主机都可以收到 数据包
3. 一次只能有一台主机通信

2、使用集线器的以太网

1. 集线器 本质仍然是使用总线、基于 CSMA/CD 协议
2. 以太网中所有主机都可以收到 数据包
3. 一次只能有一台主机通信

3、使用交换机的以太网

1. 交换机具有“缓存”功能，不再基于 CSMA/CD
2. 交换机会利用“帧交换表”根据 目的 MAC 地址，将数据包从 **特定的“接口”**转发出去，而不是向 以太网中所有主机 广播数据包
3. 可以通信有多台主机通信

现在的以太网中，大多都采用交换机了。

以太网经过太多版本变迁，现在认为只要满足以下 3 个性质的就是以太网：

1. 将包发送到 MAC 头部的接收方 MAC 地址代表的目的地
2. 用发送方 MAC 地址识别发送方
3. 用以太类型识别包的内容

2. 添加MAC头部

IP 地址相当于“远程定位”，但是在 茫茫的网络世界中，**如何将 “数据包” 传输到目的地呢**？即，如何每次判断 数据包 的 下一站是什么呢?

所以，需要给 IP 数据，添加一个 MAC 头部，实现数据包在**以太网中的**传输

源 MAC 地址：

从网卡的 ROM 中直接读取；

目的 MAC 地址：

- 首先，查询 ARP缓存中是否有 目的IP地址。如果命中缓存，则直接返回，否则需要执行 APR 协议；
- 然后，在 以太网中 广播 ARP协议，获取 目的IP地址 对应的 MAC 地址

Note： ARP 缓存是有 有效期的，一段时间后会自动删除！（因为 IP地址和 MAC地址的对应关系不是永久的。

3. 网卡-发挥作用

将 MAC 头部添加在 IP 头部前面后，整个 数据包就构建完成了。

但是，IP 模块生成的网络包只是存放在内存中的一串数字信息（即，010101....），没有办法直接发送给对方。

因此，我们需要**将数字信息转换为电或光信号**，才能在网线/光纤 上传输

而负责将 将数字信息转换为电或光信号的便是：**网卡 + 网卡驱动程序**

网卡 ROM 中固化的 MAC 地址会由网卡驱动程序读取并分配给 MAC 模块

网卡驱动从 IP 模块获取包之后，会将其复制到网卡内的缓冲区中，并在开头加上 报头和起始帧分界符（用于 帧定界），在末尾加上用于检测错误的帧校验序列 FCS （用于 差错检测）

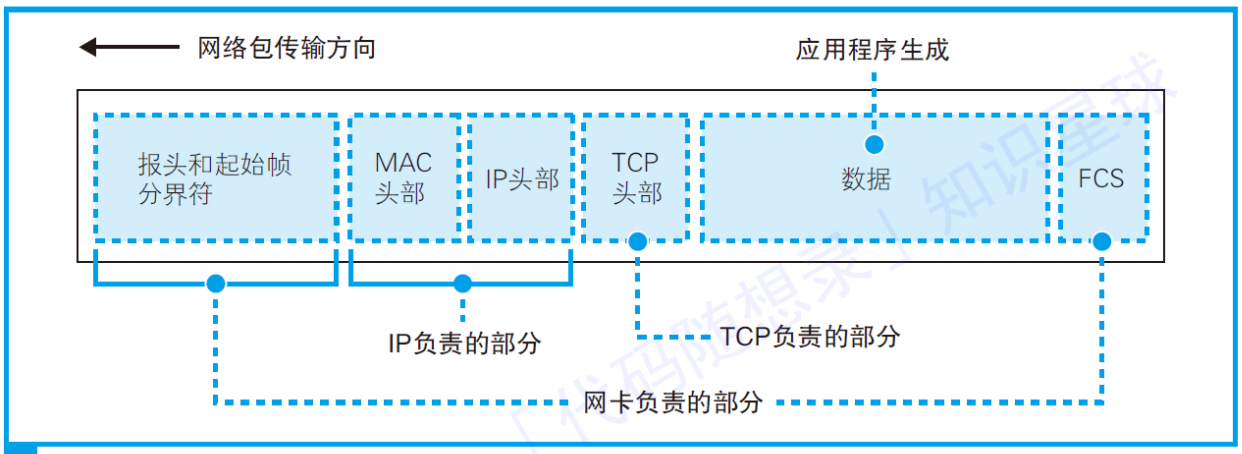


图 2.24 网卡发送出去的包
图中显示了协议栈和网卡对包的处理过程。MAC 头部很容易被误解为是由网卡来处理的，实际上它是由 TCP/IP 软件来负责的。

posper

最终，网卡 将此时的数据包转换为 电/光信号，利用网线转发出去

到这为止，数据包添加无数个 头部后，并由网卡 转换为 电/光信号后，才能真正离开计算机，踏上 寻找目的地的征途

4. 网卡-接受数据包

将电/光信号转换为 数字信息，并放入 缓存区中；

检查 FCS，检测包在传输过程中是否发成错误：

发生错误： 丢弃

无错误：

检测 目的MAC地址 和 自己的 MAC地址是否一致：

如果一致，则将数据包放入 缓存区中；
否则（不是自己的包），则 丢弃之。

此时，MAC 模块的工作就完成了。接下来网卡会发出 “中断” 通知计算机，让 网卡驱动程序工作

5. 交换机

交换机并不只是简单地让信号流过，而是先接收信号并将其还原为数字信息，然后再重新转换成信号并发送出去的过程

交换机内部有一张MAC地址与网线端口的对应表。当接收到包时，会将相应的端口号码和发送方MAC地址写入表中，这样就可以根据地址判断出该设备连接在哪个端口上了。交换机就是根据这些信息判断应该把包转发到哪里的。

MAC地址	端口	控制信息
00-60-97-A5-43-3C	2	…
00-00-C0-16-AE-FD	7	…
00-02-B3-1C-9C-F9	8	…
…	…	…

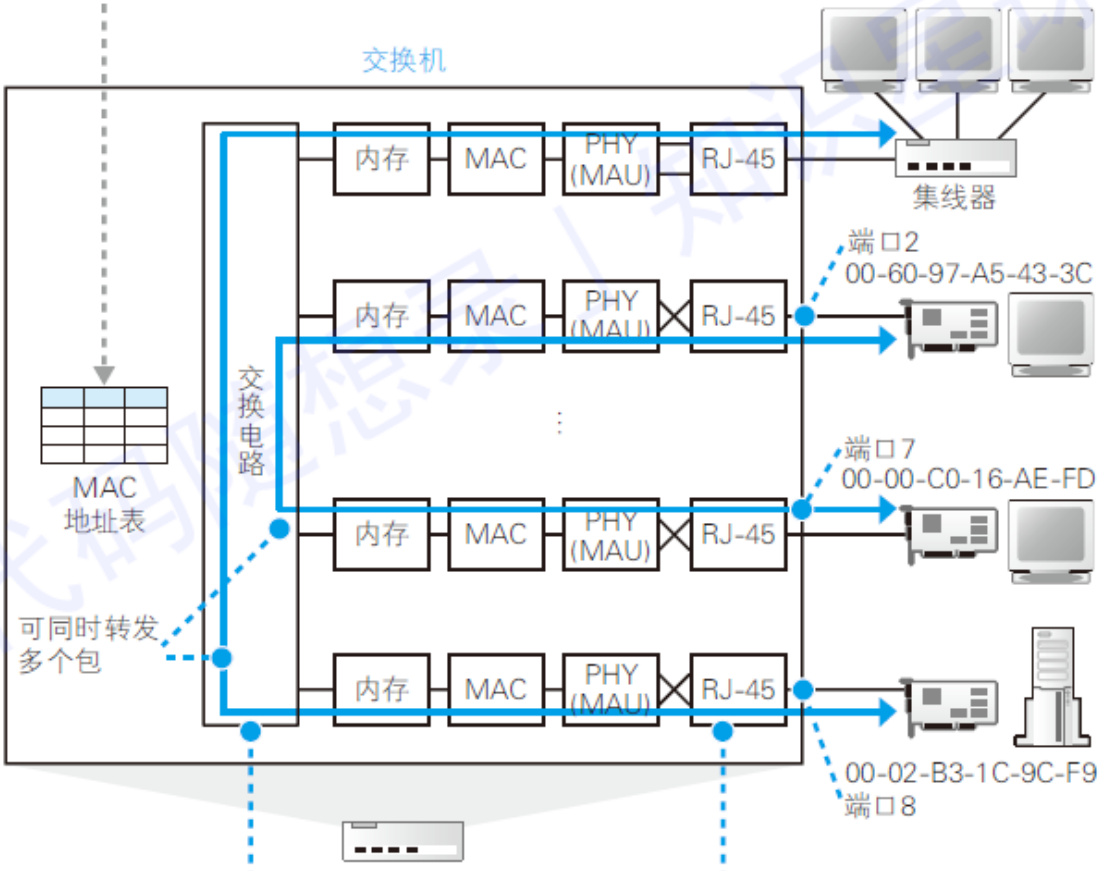


图 3.7　交换机的结构

首先，信号到达网线接口，并由 PHY（MAU）模块进行接收；

PHY（MAU）模块会将网线中的信号转换为通用格式，然后传递给 MAC 模块；

MAC 模块将信号转换为数字信息，然后通过包末尾的 FCS 校验错误，如果没有问题则存放到缓冲区中（这个过程基本和 网卡接受数据包 相同）

交换机端口的 MAC 模块 “类似于” 网卡。但仅仅是“类似”，不同点在于 交换机的端口不具有**MAC地址**

1、如何转发?

答：根据“帧交换表” + 目的 MAC 地址，判断从 哪个“端口”转发 数据包

2、如何维护 “帧交换表”? （即，自学习算法）

收发包时，将发送方 MAC 地址 和 输入端口号写入 帧交换表；
一段时间后，自动删除“帧交换表”中 过期的条目（因为 MAC 地址 和 端口的对应关系不是永久的，比如 换网卡了）

3、特殊操作：

当 转发的目的端口 和输入端口相同时，则直接丢弃该包（相当于兜圈了，丢之）
当 帧交换表中查询不到 目的MAC地址时，采用 “洪泛”（将该包从除了输入端口的所有端口转发出去）

有人会说：“这样做会发送多余的包，会不会造成网络拥塞呢？”

其实完全不用过于担心，因为发送了包之后目标设备会作出响应，只要返回了响应包，交换机就可以将它的地址写入地址表，下次也就不需要把包发到所有端口了。

局域网中每秒可以传输上千个包，多出一两个包并无大碍。（差不多就行了）

广播地址：

MAC 地址中的 FF:FF:FF:FF:FF:FF；
IP 地址中的 255.255.255.255 都是广播地址

6. 交换器 vs 集线器

交换机只将包转发到具有特定MAC 地址的设备连接的端口，可以**同时转发多个包**；

集线器会将输入的信号广播到所有的端口，如果同时输入多个信号就会发生碰撞，**无法同时传输多路信号**

7. 路由器 vs 交换机

路由器是基于 IP 设计，是 三层网络设备，路由器的各个端口都具有 MAC 地址和 IP 地址；

交换机是基于以太网（MAC层）设计，是 二层网络设备，交换机的端口不具有 MAC 地址；

路由器的结构：

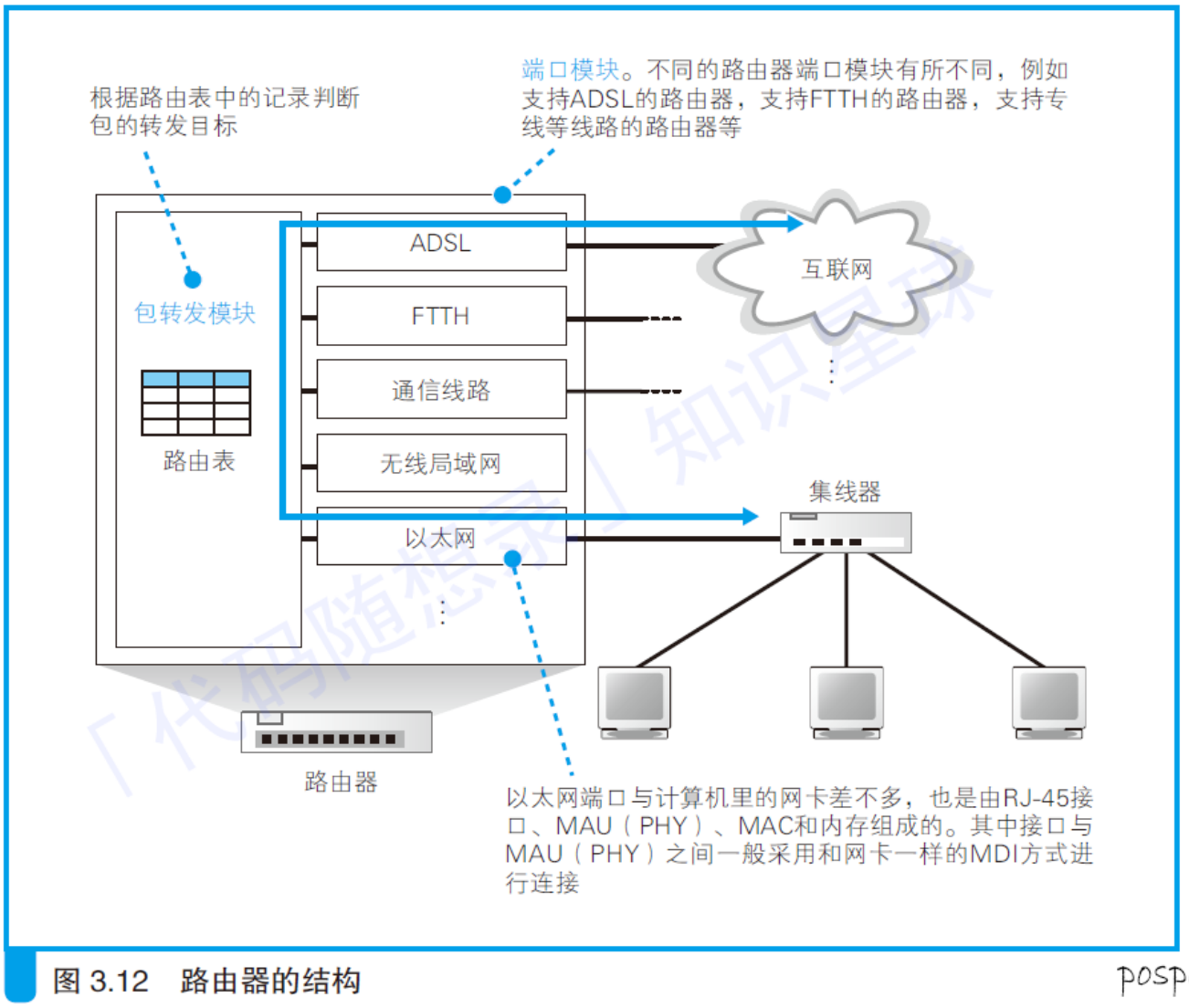


图 3.12　路由器的结构

posper

路由器分为 2 个主要模块：

1. **转发模块：** 负责判断包的转发目的地
2. **端口模块：** 负责包的收发操作

可以将路由器的转发模块想象成 IP 模块，将端口模块想象成网卡

通过更换网卡，计算机不仅可以支持以太网，也可以支持无线局域网，路由器也是一样。

如果路由器的端口模块安装了支持无线局域网的硬件，就可以支持无线局域网了。

此外，计算机的网卡除了以太网和无线局域网之外很少见到支持其他通信技术的品种，而路由器的端口模块则支持除局域网之外的多种通信技术，如ADSL、FTTH，以及各种宽带专线等，只要端口模块安装了支持这些技术的硬件即可

8. 路由器工作原理

首先，会通过端口将发过来的包接收进来，这一步的工作过程取决于端口对应的通信技术。

对于以太网端口来说： 就是按照以太网规范进行工作

而无线局域网端口： 则按照无线局域网的规范工作

转发模块会根据接收到的数据包中的 目的IP 地址，在路由表中进行查询，以此判断转发目标；

转发模块将包转移到转发目标对应的端口，端口再按照硬件的规则将包发送出去，也就是转发模块委托端口模块将包发送出去的意思

路由器的路由表

目标地址（Destination）	子网掩码（Netmask）	网关（Gateway）	接口（Interface）	跃点数（Metric）
10.10.1.0	255.255.255.0	——	e2	1
10.10.1.101	255.255.255.255	——	e2	1
192.168.1.0	255.255.255.0	——	e3	1
192.168.1.10	255.255.255.255	——	e3	1
0.0.0.0	0.0.0.0	192.0.2.1	e1	1

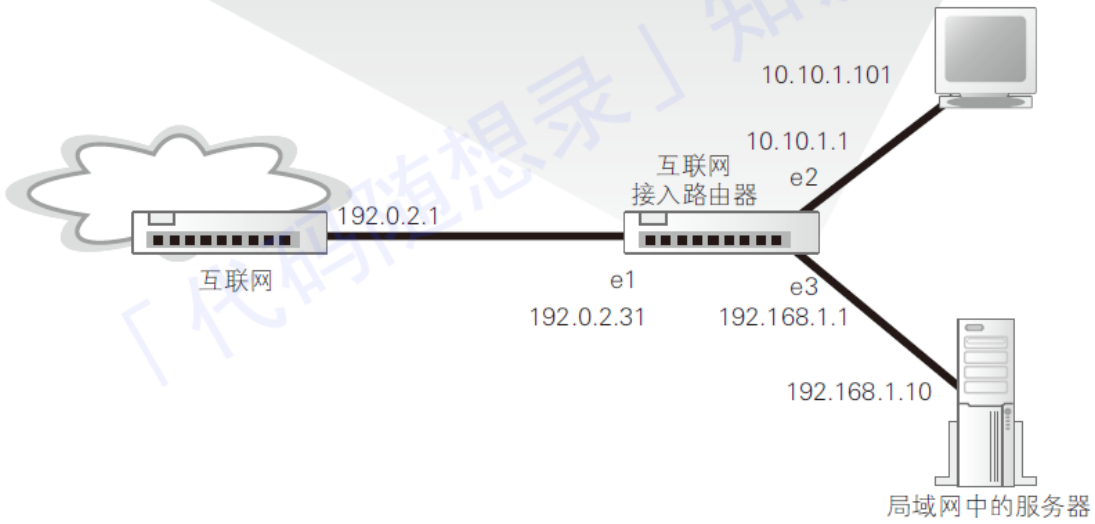


图 3.13　路由器根据路由表对包进行转发

posper

路由聚合：

即，将多个子网合并成一个子网，只将 合并后的子网存入 路由表中；
好处：减少路由表的条目数，提高查询效率 和 可维护性

路由器会忽略主机号，只匹配网络号。

有时，也可以将某台具体计算机的地址写入路由表中：

这时的子网掩码为 255.255.255.255

路由器 - 接收包

路由器的端口有各种不同的类型，这里我们只介绍以太网端口是如何接收包的

首先，信号到达网线接口部分，其中的PHY（MAU）模块和MAC 模块将信号转换为数字信息

然后，通过包末尾的FCS 进行错误校验，如果没问题则检查MAC 头部中的接收方MAC 地址，看看是不是发给自己的包

- 如果是，就放到接收缓冲区中
- 否则（接收方MAC地址和 目的MAC地址不一致），就丢弃这个包

路由器 - 发送包

完成包接收操作之后，路由器就会丢弃包开头的MAC 头部。

MAC 头部的作用就是将包送达路由器，其中的接收方MAC 地址就是路由器端口 的MAC 地址

接下来，路由器会根据MAC 头部后方的IP 头部中的内容进行包的转发操作。

路由器转发操作分为以下几个阶段，首先 需要查询路由表判断转发目标。

1、判断转发目标

利用 子网掩码 + 目的 IP 地址进行“逻辑与”运算，找到 结果和 路由表中 第一列匹配的行

如果成功匹配，路由器就会将网络包交给接口列中指定的网络接口（即，端口。也即，源 IP 地址），并转发到网关列中指定的 IP 地址（即，目的 IP 地址。【这里可能为空，见第 2 步】）；

如果“每一条都无法匹配”，则会转发给“默认路由/网关”（即，子网掩码为 0.0.0.0 行对应的“网关列”）

2、包发送操作

假设这里路由器让然是基于 以太网规则进行发送 数据包的，则需要在包前面 添加 MAC 头部

3、如何 判断转发目标?

- 如果路由表的网关列内容为 IP 地址，则该地址就是下一个转发目标；
- 如果路由表的网关列内容为空，则 IP 头部中的接收方 IP 地址就是下一个转发目标

4、找到转发目标的 IP 地址后，则需要添加 MAC 头部：

源 MAC 地址：路由器输出端口的 MAC 地址（见 1 中第 2 步）；
目的 MAC 地址：根据 转发目标的 IP 地址，执行 ARP 协议，获取目的 MAC 地址
执行 ARP 之前，先查询路由器的 ARP 缓存

接下来是发送方 MAC 地址字段，这里填写输出端口的 MAC 地址。还有一个以太类型字段，填写 0800 （十六进制）表示 IP 协议。

网络包完成后，接下来会将其转换成电信号并通过端口发送出去。这一步的工作过程和计算机也是相同的。

发送出去的网络包会通过交换机到达下一个路由器。由于接收方 MAC 地址就是下一个路由器的地址，所以交换机会根据这一地址将包传输到下一个路由器。

接下来，下一个路由器会将包转发给再下一个路由器，经过层层转发之后，网络包就到达了最终的目的地。 posper

需要 MAC 地址在 以太网内进行 两个设备之间的包传输。

5、为什么有了IP地址，还需要 MAC 地址?

IP 协议本身没有传输包的功能，因此包的实际传输要委托以太网来进行。

路由器是基于 IP 设计的，而交换机是基于以太网设计的，换句话说，路由器将包的传输工作委托给交换机来进行。

当然，这里讲的内容只适用于原本本实现 IP 和以太网机制的纯粹的路由器和交换机，**实际的路由器有内置交换机功能**的，比如用于连接互联网的家用路由器就属于这一种，对于这种路由器，上面内容可能就不适用了。

但是，如果把这种“不纯粹”的路由器拆分成“纯粹”的路由器和“纯粹”的交换机，则它们各自都适用上面的内容。
IP 并不是委托以太网将包传输到最终目的地，而是传输到下一个路由器。

在创建MAC 头部时，也是从IP 的路由表中查找出下一个路由器的IP 地址；

然后，通过ARP 查询出MAC 地址，然后将MAC 地址写入MAC 头部中的
这表示IP 对以太网的委托只是将包传输到下一个路由器就行了。

当包到达下一个路由器后，下一个路由器又会重新委托以太网将包传输到再下一个路由器。

随着这一过程**反复执行**，包就会最终到达IP 的目的地，也就是通信的对象。

IP 本身不负责包的传输，而是委托各种通信技术将包传输到下一个路由器，这样的设计是有重要意义的:

即，可以根据需要灵活运用各种通信技术
这也是 IP 的最大特点。正是有了这一特点，我们才能够构建出互联网这一规模巨大的网络。

9. 防火墙

如果不管什么网络包都能达到 服务器的话，那服务器也忒不安全了

所以需要在服务器前面加一层 防火墙（类似 海关），屏蔽一些不允许通过的包，降低服务器的风险

可以在防火墙中设置各种规则，当包到达防火墙时，会根据这些规则判断是允许通过还是阻止通过：

1、如果判断结果为阻止

那么这个包会被丢弃并被记录下来（用于分析，更好地防止非法入侵）

2、如果包被判断为允许通过

则该包会被转发出去，这个转发的过程和路由器是相同的

包过滤方式的防火墙可根据接收方IP 地址、发送方IP 地址、接收方端口号、发送方端口号、控制位等信息来判断是否允许某个包通过。

10. 负载均衡

当服务器的访问量上升时，仅仅一台服务器的性能是跟不上的

此时，可以考虑**使用多台服务器来分担负载**

要采用这样的方法，必须有一个机制将客户端发送的请求分配到每台服务器上（即，实现负载均衡）

常见的 实现负载均衡的方法：

1、轮询

DNS 服务器中填写域名对应多个 IP 地址，通过 DNS 服务器按次序返回不同的 IP 地址（挨次用，转一圈又回到第一个 IP 地址）

缺点：

例1：

如果其中一台服务器 宕机了，但是 轮询 的方式不会跳过该台服务器，DNS 服务器仍然会返回这台 宕机的 服务器 IP 地址

例2：

当页面操作需要 跨网页时，轮询不可用（例如在购物网站中，可能会在第一个页面中输入地址和姓名，在第二个页面中输入信用卡号，轮询 可能会将这 2 个操作分配给不同的服务器去执行，这样的执行结果肯定是不对的）

2、负载均衡器（为了解决 轮询的问题）

首先，要用负载均衡器的 IP 地址代替 Web 服务器的实际地址注册到DNS 服务器上

chapter 6

1. 服务器接受数据

1、网卡的 MAC 模块将网络包从 电/光信号 还原为数字信息，校验 FCS 并存入缓冲区

2、协议栈的 IP 模块会检查 IP 头部

（1）判断是不是发给自己的；
（2）判断网络包是否经过分片；
（3）将包转交给 TCP模块或者udo模块

前面的步骤对于任何包都是一样的，但后面的 TCP 模块的操作则根据包的内容有所区别。即，tcp 报文主要分为3类: 连接报文（3报文握手🤝)、数据收发 报文、断开连接 报文（4报文挥手👋）

3、如果收到的是发起连接的包，则 TCP 模块会

（1）确认 TCP 头部的控制位 SYN；
（2）检查接收方端口号；
（3）为相应的等待连接套接字复制一个新的副本；
（4）记录发送方 IP 地址和端口号等信息

4、收到数据包时，TCP 模块会

（1）根据收到的包的发送方 IP 地址、发送方端口号、接收方 IP 地址、接收方端口号找到相对应的套接字；
（2）将数据块拼合起来并保存在接收缓冲区中；
（3）向客户端返回 ACK

5、如果收到的是断开连接的包，则4次挥手👋

操作系统

操作系统基础

什么是操作系统

> 操作系统(Operating System)，介于硬件资源和应用程序之间的一个系统软件

如果将它理解为“掌控计算机的系统”是否更能精确的描述OS所做的事情呢?

如果要更深入的掌握这个问题，可以问一问：如果没有了操作系统，你使用的PC机还能干什么？或者说，你能够使用你的PC机做什么呢?

操作系统的功能

> 操作系统位于硬件资源之上，管理硬件资源; 应用程序之下，为应用程序提供服务，同时管理应用程序

1、资源分配，资源回收

计算机必要重要的硬件资源无非就是 CPU、内存、硬盘、I/O设备。
而这些资源总是有限的，因此需要有效管理，资源管理最终只有两个问题：资源分配、资源回收。

资源分配： 体现在CPU上，比如进程调度，多个进程同时请求CPU下，应该给哪一个进程呢？再比如内存分配，内存不够了怎么办？A进程非法访问了B进程的内存地址怎么办？内存内、外碎片问题等。

资源回收： 考虑内存回收后的合并等等。

2、为应用程序提供服务

操作系统将硬件资源的操作封装起来，提供相对统一的接口（系统调用）供开发者调用。

如果没有操作系统，应用程序将直接面对硬件，除去给开发者带来的编程困难不说，直接访问硬件，使用不当极有可能直接损坏硬件资源。

3、管理应用程序

即控制进程的生命周期：进程开始时的环境配置和资源分配、进程结束后的资源回收、进程调度等。

4、操作系统内核的功能

（1）**进程调度能力：** 管理进程、线程，决定哪个进程、线程使用CPU。

（2）**内存管理能力：** 决定内存的分配和回收。

（3）**硬件通信能力：** 管理硬件，为进程和硬件之间提供通信。

（4）**系统调用能力：** 应用程序进行更高限权运行的服务，需要系统调用，用户程序和操作系统之间的接口。

> 健壮性，又称鲁棒性，即使很粗鲁的对待程序，它还是可以运行得很棒！

操作系统的角色

> 魔术师与管理者

1、管理者

主要分为：CPU管理、内存管理、外存管理、IO管理；以及自己的健壮性和安全性管理。

2、魔术师:

比如操作系统会让每个进程都觉得自己独占CPU、独占整片物理内存，而实际上每个进程都只是在某一时间段内占用CPU，仅仅只是占用实际一点点物理内存。

用户程序与操作系统的关系

> 相互调用

1、操作系统的角度

计算机启动后启动的第一个软件就是操作系统，随后启动的所有进程都运行在操作系统之上，使用操作系统提供的服务，同时被操作系统监控，进程结束后也由操作系统回收。

2、进程角度

调用操作系统提供的服务，实现自己的功能。

进程

进程基础

1、进程的概念

我们编译的代码可执行文件只是储存在硬盘的静态文件，运行时被加载到内存，CPU执行内存中指令，这个运行的程序被称为进程。

进程是对运行时程序的封装，操作系统进行资源调度和分配的基本单位。

2、进程的实现

中断发生后操作系统底层的工作步骤

1. 硬件压入堆栈程序计数器等
2. 硬件从中断向量装入新的程序计数器
3. 汇编语言过程保存寄存器值
4. 汇编语言过程设置新的堆栈
5. C中断服务例程运行（典型地读和缓冲输入）
6. 调度程序决定下一个将运行的进程
7. C过程返回至汇编代码

8. 汇编语言过程开始运行新的当前进程

进程表：

为了实现进程模型，操作系统维护着一张表格(一个结构数组)，即进程表。

每个进程占有一个进程表项。(有些著作称这些为进程控制块)

该表项包含了一个进程状态的重要信息

包括程序计数器、堆栈指针、内存分配状况、所打开文件的状态、账号的调度信息，以及其他在进程由运行态转换到就绪态或阻塞态时必须保存的信息，从而**保证该进程随后能再次启动**，就像从未中断过一样

3、并发与并行

（1）单个核心在很短时间内分别执行多个进程，称为并发

（2）多个核心同时执行多个进程称为并行

（3）对于并发来说，CPU需要从一个进程切换到另一个进程，这个过程需要保存进程的状态信息

4、进程的状态

某个进程在某个时刻所处的状态分为以下几种，运行态、就绪态、阻塞态。对于阻塞状态;

比如read系统调用阻塞，进程会占用内存空间，这是一种浪费行为，于是操作系统会有跟内存管理中物理页置换到磁盘一样的行为，**把阻塞的进程置换到磁盘中**，此时进程**未占用物理内存**，我们称之为**挂起**;

挂起不仅仅可能是物理内存不足，比如sleep系统调用过着用户执行Ctrl+Z也可能导致挂起。

除了创建和结束一般有三个状态：

运行态： 该时刻进程占用CPU

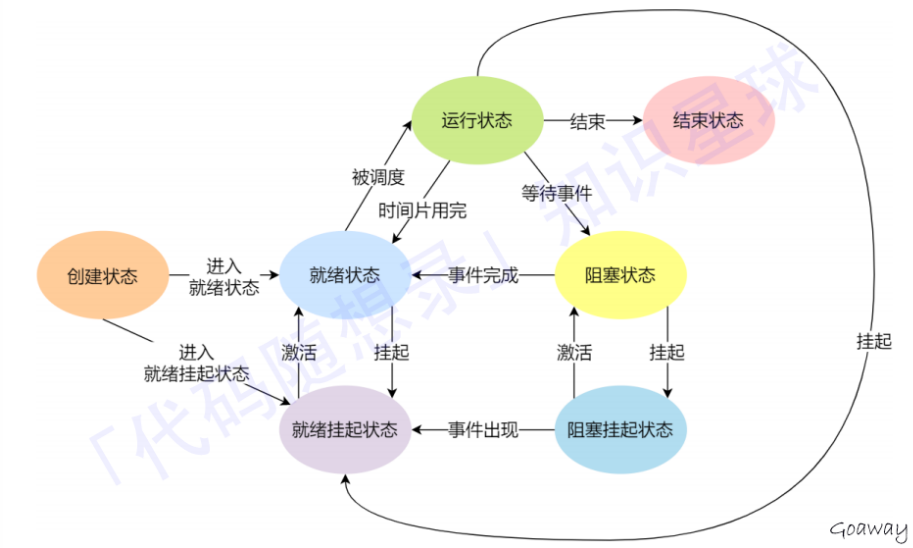
就绪态： 可运行，由于其他进程处于运行状态而暂时停止运行

阻塞态： 该进程正在等待某一事件发生（如等待输入/输出操作的完成）而暂时停止运行

阻塞态的进程**占用着物理内存**，在虚拟内存管理的操作系统中，通常会把阻塞态的进程的物理内存空间换出到硬盘，等需要再次运行的时候，再从硬盘换入到物理内存。

挂起态：新的状态，描述进程没有占用实际的物理内存空间的情况，这个状态就是挂起状态

阻塞挂起状态： 进程在外存（硬盘）并等待某个事件的出现
就绪挂起状态： 进程在外存（硬盘），但只要进入内存，马上运行



特点：

1. 就绪态和运行态可以相互转换，其它的都是单向转换。就绪态的进程通过调度算法从而获得CPU 时间，转为运行状态；
2. 运行态的进程，在分配给它的 CPU 时间片用完之后就会转为就绪状态，等待下一次调度。
3. 阻塞态是缺少需要的资源从而由运行态转换而来，但是该资源不包括 CPU 时间，缺少 CPU 时间会从运行态转换为就绪态。

5、进程控制块（PCB）

操作系统对进程的感知，是通过进程控制块PCB数据结构来描述的。它是进程存在的唯一标识，其包括以下信息：

1. **进程描述信息：** 进程标识符、用户标识符等；
2. **进程控制和管理信息：** 进程状态，进程优先级等；
3. **进程资源分配清单：** 虚拟内存地址空间信息，打开文件列表，IO设备信息等；
4. **CPU相关信息：** 当进程切换时，CPU寄存器的值都被保存在相应PCB中，以便CPU重新执行该进程时能从断点处继续执行；

PCB通过链表形式组织起来，比如有就绪队列、阻塞队列等，方便增删，方便进程管理。

6、进程状态的切换

进程的状态分类：

1. 就绪状态（ready）：等待被调度
2. 运行状态（running）
3. 阻塞状态（waiting）：等待资源

转换关系：

只有就绪态和运行态可以互相转换，其他都是单向转换。

就绪态的进程通过调度算法从而获得CPU时间，转为运行状态；而运行状态的进程，在分配给它的CPU时间片完之后就会转为就绪状态，等待下一次调度。

进程因为等待资源而阻塞，但是该资源不包括 CPU 时间，缺少 CPU 时间会从运行态转换为就绪态。

当进程等待的外部事件发生时（如一些输入到达），则又阻塞态转换为就绪态，如果此时没有其他进程运行，变转换为运行态，否则该进程将处于就绪态，等待CPU空闲轮到它运行。

7、进程切换为何比线程慢

涉及到虚拟内存的问题，进程切换涉及虚拟地址空间的切换而线程不会。

因为每个进程都有自己的虚拟地址空间，而线程是共享所在进程的虚拟地址空间的，所以同一个进程中的线程进行线程切换时不涉及虚拟地址空间的转换。

把虚拟地址转换为物理地址需要查找页表，页表查找是一个很慢的过程（至少访问2次内存），因此通常使用Cache来缓存常用的地址映射，这样可以加速页表查找，这个cache就是TLB（快表）。

由于每个进程都有自己的虚拟地址空间，那么显然每个进程都有自己的页表，那么当进程切换后页表也要进行切换，页表切换后TLB就失效了，cache失效导致命中率降低，那么虚拟地址转换为物理地址就会变慢，表现出来的就是程序运行会变慢，而线程切换则不会导致TLB失效，因为线程线程无需切换地址空间，这也就是进程切换要比同进程下线程切换慢的原因。

10、守护进程

守护进程是指在后台运行的，没有控制终端与它相连的进程。它独立于控制终端，周期性地执行某种任务。

Linux的大多数服务器就是用守护进程的方式实现的，如web服务器进程http等。

创建守护进程要点：

（1）让程序在后台执行。

方法是调用fork()产生一个子进程，然后使父进程退出。

（2）调用setsid()创建一个新对话期。

守护进程需要摆脱父进程的影响，方法是调用setsid()使进程成为一个会话组长。setsid()调用成功后，进程成为新的会话组长和进程组长，并与原来的登录会话、进程组和控制终端脱离。

（3）禁止进程重新打开控制终端。

经过1和2，进程已经成为一个无终端的会话组长，但是它可以重新申请打开一个终端。为了避免这种情况发生，可以通过使进程不再是会话组长来实现。再一次通过fork（）创建新的子进程，使调用fork的进程退出。

（4）关闭不再需要的文件描述符。

子进程从父进程继承打开的文件描述符。如不关闭，将会浪费系统资源，造成进程所在的文件系统无法卸下以及引起无法预料的错误。首先获得最高文件描述符值，然后用一个循环程序，关闭0到最高文件描述符值的所有文件描述符。

（5）将当前目录更改为根目录。

（6）子进程从父进程继承的文件创建屏蔽字可能会拒绝某些许可权。

为防止这一点，使用unmask(0)将屏蔽字清零。

（7）处理SIGCHLD信号。

对于服务器进程，在请求到来时往往生成子进程处理请求。如果子进程等待父进程捕获状态，则子进程将成为僵尸进程（zombie），从而占用系统资源。如果父进程等待子进程结束，将增加父进程的负担，影响服务器进程的并发性能。在Linux下可以简单地将SIGCHLD信号的操作设为SIG_IGN。这样，子进程结束时不会产生僵尸进程。

11、僵尸进程

多进程程序，父进程一般需要跟踪子进程的退出状态，当子进程退出，父进程在运行，子进程必须等到父进程捕获到了子进程的退出状态才真正结束。在子进程结束后，父进程读取状态前，此时子进程为僵尸进程。

设置僵尸进程的目的是维护子进程的信息，以便父进程在以后某个时候获取。这些信息至少包括进程ID，进程的终止状态，以及该进程使用的CPU时间。所以当终止子进程的父进程调用wait或waitpid时就可以得到这些信息。

但是子进程停止在僵尸态会占据内核资源，所以需要避免僵尸进程的产生或立即结束子进程的僵尸态。

1. 父进程调用wait/waitpid等函数等待子进程结束，如果尚无子进程退出wait会导致父进程阻塞。waitpid只会等待由pid参数指定的子进程，同时也是非阻塞，目标进程正常退出返回子进程PID，还没结束返回0。
2. 在事件已经发生情况下执行非阻塞调用可以提高程序效率。对waitpid，最好在子进程退出后调用。使用SIGCHLD信号通知父进程，子进程结束。

> 父进程中捕获信号，然后在信号处理函数中调用waitpid以彻底结束子进程

```
static void handle_child(int sig)
{
    pid_t pid;
    int stat;
    while((pid = waitpid(-1, &stat, WNOHANG)) > 0)
    {
        //结束子进程的处理;
    }
}
```

1. 通过signal(SIGCHLD, SIG_IGN)通知内核对子进程的结束不关心，由内核回收。如果不想让父进程挂起，可以在父进程中加入一条语句：signal(SIGCHLD,SIG_IGN);表示父进程忽略SIGCHLD信号，该信号是子进程退出的时候向父进程发送的。
2. 忽略SIGCHLD信号，这常用于并发服务器的性能的一个技巧因为并发服务器常常fork很多子进程，子进程终结之后需要服务器进程去wait清理资源。如果将此信号的处理方式设为忽略，可让内核把僵尸子进程转交给init进程去处理，省去了大量僵尸进程占用系统资源。

12、多进程

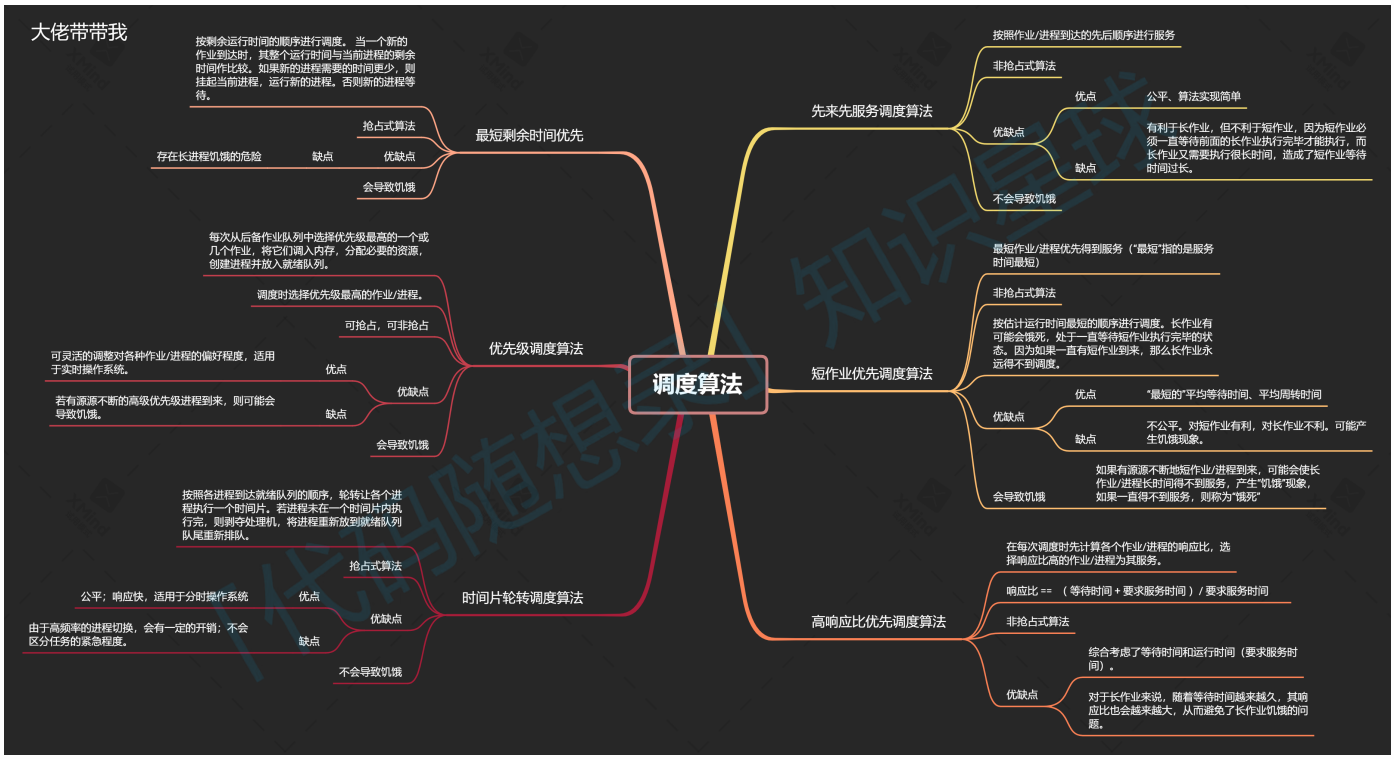
进程结构由以下几个部分组成：代码段、堆栈段、数据段。代码段是静态的二进制代码，多个程序可以共享。

父进程创建子进程之后，父、子进程除了pid外，几乎所有的部分几乎一样。

父、子进程共享全部数据，子进程在写数据时会使用写时复制技术将公共的数据重新拷贝一份，之后在拷贝出的数据上进行操作；不是对同一块数据进行操作；

如果子进程想要运行自己的代码段，还可以通过调用execv()函数重新加载新的代码段，之后就和父进程独立开了。

进程调度算法



1、批处理系统中的调度

（1）先来先服务：

非抢占式的调度算法，按照请求的顺序进行调度。

有利于长作业，但不利于短作业，因为短作业必须一直等待前面的长作业执行完毕才能执行，而长作业又需要执行很长时间，造成了短作业等待时间过长。

（2）最短作业优先：

非抢占式的调度算法，按估计运行时间最短的顺序进行调度。

长作业有可能会饿死，处于一直等待短作业执行完毕的状态。因为如果一直有短作业到来，那么长作业永远得不到调度。

（3）最短剩余时间优先：

最短作业优先的抢占式版本，按剩余运行时间的顺序进行调度。

当一个新的作业到达时，其整个运行时间与当前进程的剩余时间作比较。如果新的进程需要的时间更少，则挂起当前进程，运行新的进程。否则新的进程等待。

2、交互式系统中的调度

（1）时间片轮转调度

将所有就绪进程按 FCFS 的原则排成一个队列，每次调度时，把 CPU 时间分配给队首进程，该进程可以执行一个时间片。当时间片用完时，由计时器发出时钟中断，调度程序便停止该进程的执行，并将它送往就绪队列的末尾，同时继续把 CPU 时间分配给队首的进程。

（2）优先级调度

为每个进程分配一个优先级，按优先级进行调度。为了防止低优先级的进程永远等不到调度，可以随着时间的推移增加等待进程的优先级。

（3）多级队列

一个进程需要执行 100 个时间片，如果采用时间片轮转调度算法，那么需要交换 100 次。

多级队列是为这种需要连续执行多个时间片的进程考虑，它设置了多个队列，每个队列时间片大小都不同，例如 1,2,4,8,..。进程在第一个队列没执行完，就会被移到下一个队列。这种方式下，之前的进程只需要交换 7 次。

每个队列优先权也不同，最上面的优先权最高。因此只有上一个队列没有进程在排队，才能调度当前队列上的进程。

可以将这种调度算法看成是时间片轮转调度算法和优先级调度算法的结合。

（4）最短进程优先

如果我们将每一条命令的执行看作是一个独立的“作业”，则我们可以通过首先运行最短的作业来使响应事件最短

进程通信

1. 通信的类型

1、同一主机：

1. 无名管道
2. 有名管道
3. 信号
4. 消息队列
5. 共享内存
6. 信号量
7. 存储映射

2、不同主机：

8. 不同主机的通信有 socket

2. 无名管道

1、特点

1. 半双工，数据在同一时刻只能在一个方向上流动
2. 数据只能从管道的一端写入，从另一端读出
3. 写入管道中的数据遵循先入先出的规则
4. 管道所传送的数据是无格式的，这要求管道的读出方与写入方必须事先约定好数据的格式，如多少字节算一个消息等
5. 管道不是普通的文件，不属于某个文件系统，其只存在于内存中
6. 管道在内存中对应一个缓冲区。不同的系统其大小不一定相同
7. 从管道读数据是一次性操作，数据一旦被读走，它就从管道中被抛弃，释放空间以便写更多的数据
8. 管道没有名字，只能在具有公共祖先的进程（父进程与子进程，或者两个兄弟进程，具有亲缘关系）之间使用

9. 存在阻塞方式

2、pipe函数

```
#include <unistd.h>
/**
 * 创建无名管道.
 * @param pipefd 为int型数组的首地址，其存放了管道的文件描述符
 * pipefd[0]、pipefd[1].
 * @return 创建成功返回0，创建失败返回-1.
 */
int pipe(int pipefd[2]);
/**
 * 当一个管道建立时，它会创建两个文件描述符 fd[0] 和 fd[1]。其中
 * fd[0] 固定用于读管道，而 fd[1] 固定用于写管道。
 * 一般文件 I/O的函数都可以用来操作管道(lseek() 除外。)
 */
```

3、读写管道四种进程情况

（1）第一种:

1. 如果写端没有关闭，管道中没有数据，这个时候读管道进程去读管道会阻塞;
2. 如果写段没有关闭，管道中有数据，这个时候读管道进程就会将数据读出来，下一次读没有数据汇总阻塞;

（2）第二种:

管道所有的写端关闭，都进程去读管道的内容，读取全部内容，最后返回0;

（3）第三种:

所有读端没有关闭，如果管道被写满了，写管道进程写管道会被阻塞;

（4）第四种:

所有的读端被关闭，写管道进程写管道会收到一个信号，然后退出。

4、读管道

（1）管道中有数据:

read返回实际读到的字节数

（2）管道中无数据:

1. 管道写端被全部关闭，read返回0 (相当于读到文件结尾)
2. 写端没有全部被关闭，read阻塞等待(不久的将来可能有数据递达，此时会让出cpu)

5、写管道

（1）管道读端全部被关闭:

进程异常终止(也可使用捕捉SIGPIPE信号，使进程终止);

（2）管道读端没有全部关闭:

1. 管道已满，write阻塞
2. 管道未满，write将数据写入，并返回实际写入的字节数

6、查看管道缓冲区函数

```
#include <unistd.h>
/**
 * 该函数可以通过name参数查看不同的属性值.
 * @param fd 文件描述符.
 * @param name _PC_PIPE_BUF，查看管道缓冲区大小;
 *  _PC_NAME_MAX，文件名字字节数的上限.
 * @return 成功: name返回的值的决定name的意义; 失败: 返回-1.
 */
long fpathconf(int fd, int name);
```

7、设置为非阻塞的方法

```
// 获取原来的flags
int flags = fcntl(fd[0], F_GETFL);
// 设置新的flags
flag |= O_NONBLOCK;
// flags = flags | O_NONBLOCK;
fcntl(fd[0], F_SETFL, flags);
```

3. 有名管道

命名管道（FIFO）不同于无名管道之处在于它提供了一个路径名与之关联，以 FIFO 的文件形式存在于文件系统中，这样，即使与 FIFO 的创建进程不存在亲缘关系的进程，只要可以访问该路径，就能够彼此通过 FIFO 相互通信，因此，通过 FIFO 不相关的进程也能交换数据.

1、与无名管道(pipe)不同

1. FIFO 在文件系统中作为一个特殊的文件而存在，但 FIFO 中的内容却存放在内存中;
2. 当使用 FIFO 的进程退出后，FIFO 文件将继续保存在文件系统中以便以后使用;
3. FIFO 有名字，不相关的进程可以通过打开命名管道进行通信.

2、通过命令创建有名管道

1. mkfifo fifo;
2. eg创建fifo 文件;
3. fifo形式文件，存在于内容种，查看文件大小位0.

3、通过函数创建有名管道

```c
#include <sys/types.h>
#include <sys/stat.h>
/**
 * 命名管道的创建.
 * @param pathname 普通的路径名，也就是创建后 FIFO 的名字.
 * @param mode 文件的权限,
 * 与打开普通文件的 open() 函数中的 mode 参数相同。(0666).
 * @return 成功: 0 状态码;
 *     失败: 如果文件已经存在，则会出错且返回 -1.
 */
int mkfifo(const char *pathname, mode_t mode);
```

4、注意事项

一个为只读而打开一个管道的进程会阻塞直到另外一个进程为只写打开该管道;

一个为只写而打开一个管道的进程会阻塞直到另外一个进程为只读打开该管道.

（1）读管道:

1. 管道中有数据，read返回实际读到的字节数
2. 管道中无数据:
    1. 管道写端被全部关闭，read返回0 (相当于读到文件结尾);
    2. 写端没有全部被关闭，read阻塞等待.

（2）写管道:

1. 管道读端全部被关闭， 进程异常终止(也可使用捕捉SIGPIPE信号，使进程终止);
2. 管道读端没有全部关闭:
    1. 管道已满，write阻塞;
    2. 管道未满，write将数据写入，并返回实际写入的字节数。

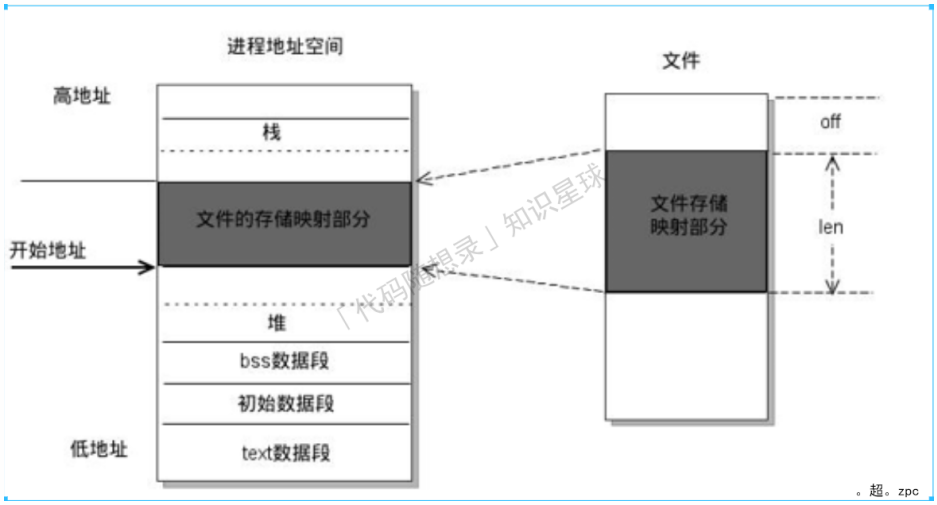
4. 共享存储映射

存储映射I/O (Memory-mapped I/O) 使一个磁盘文件与存储空间中的一个缓冲区相映射.

于是当从缓冲区中取数据，就相当于读文件中的相应字节。于此类似，将数据存入缓冲区，则相应的字节就自动写入文件。

这样，就可在不适用read和write函数的情况下，使用地址（指针）完成I/O操作，进程就可以直接通过读写内存来操作文件.

共享内存可以说是最有用的进程间通信方式，也是最快的IPC形式, 因为进程可以直接读写内存，而不需要任何数据的拷贝.



1、存储映射函数：

（1）mmap函数

```
#include <sys/mman.h>
/**
 * 一个文件或者其它对象映射进内存.
 * @param addr 指定映射的起始地址, 通常设为NULL, 由系统指定.
 * @param length 映射到内存的文件长度.
 * @param prot 映射区的保护方式, 最常用的:
 *              (1) 读: PROT_READ;
 *              (2) 写: PROT_WRITE;
 *              (3) 读写: PROT_READ | PROT_WRITE.
 * @param flags 映射区的特性, 可以是:
 *              (1) MAP_SHARED: 写入映射区的数据会复制回文件,
 *                  且允许其他映射该文件的进程共享。
 *              (2) MAP_PRIVATE: 对映射区的写入操作
 *                   会产生一个映射区的复制(copy - on - write),
 *                   对此区域所做的修改不会写回原文件。
 * @param fd 由open返回的文件描述符, 代表要映射的文件.
 * @param offset 以文件开始处的偏移量,
 *          必须是4k的整数倍, 通常为0, 表示从文件头开始映射.
 * @return 成功: 返回创建的映射区首地址; 失败: MAP_FAILED宏.
 */
void *mmap(void *addr, size_t length,
    int prot, int flags, int fd, off_t offset);
```

> 内存是按照页来区别的，通常一页就是4K

总结：

1. 第一个参数写成NULL
2. 第二个参数要映射的文件大小 > 0
3. 第三个参数：PROT_READ 、PROT_WRITE
4. 第四个参数：MAP_SHARED 或者 MAP_PRIVATE
5. 第五个参数：打开的文件对应的文件描述符
6. 第六个参数：4k的整数倍，通常为0

（2）munmap函数

```
#include <sys/mman.h>
/**
 * 释放内存映射区.
 * @param addr 使用mmap函数创建的映射区的首地址.
 * @param length 映射区的大小.
 * @return 成功返回0; 失败返回-1.
 */
int munmap(void *addr, size_t length);
```

2、注意事项:

1. 创建映射区的过程中，隐含着一次对映射文件的读操作
2. 当MAP_SHARED时，要求：映射区的权限应 <=文件打开的权限(出于对映射区的保护)。而MAP_PRIVATE则无所谓，因为mmap中的权限是对内存的限制
3. 映射区的释放与文件关闭无关。只要映射建立成功，文件可以立即关闭
4. 特别注意，当映射文件大小为0时，不能创建映射区。所以，用于映射的文件必须要有实际大小。mmap使用时常常会出现总线错误，通常是由于共享文件存储空间大小引起的
5. munmap传入的地址一定是mmap的返回地址。坚决杜绝指针++操作
6. 如果文件偏移量必须为4K的整数倍
7. mmap创建映射区出错概率非常高，一定要检查返回值，确保映射区建立成功再进行后续操作

3、匿名映射实现父子进行通信

（1）为什么使用匿名的方式实现通信?

内存映射的需要依赖文件。而建立文件建立好了只会还要unlink close掉，比较麻烦;

（2）有什么好的不能办法进行解决?

直接使用匿名映射来代替;

（3）Linux系统给我们提供了创建匿名映射区的方法，无需依赖一个文件即可创建映射区。同样需要借助标志位参数flags来指定;

（4）使用MAP_ANONYMOUS (或MAP_ANON):

```
int *p = mmap(NULL, 4, PROT_READ|PROT_WRITE, MAP_SHARED|MAP_ANONYMOUS, -1, 0);
```

5. 消息队列

基本原理：A 进程要给 B 进程发送消息，A 进程把数据放在对应的消息队列后就可以正常返回了，B 进程需要的时候再去读取数据就可以了。

特点:

1. 消息队列是保存在内核中的消息链表，每个消息体都是固定大小的存储块。如果进程从消息队列中读取了消息体，内核就会把这个消息体删除。
2. 如果没有释放消息队列或者没有关闭操作系统，消息队列会一直存在。

缺点:

1. 通信不及时，附件也有大小限制。
2. 消息队列不适合比较大数据的传输，每个消息体都有一个最大长度的限制，同时所有队列所包含的全部消息体的总长度也是有上限
3. 消息队列通信过程中，存在用户态与内核态之间的数据拷贝开销

6. 信号

信号是linux进程通信的最古老的方式;

信号是软件中断，它是在软件层次上对中断机制的一种模拟，是一种异步通信的方式 。信号可以导致一个正在运行的进程被另一个正在运行的异步进程中断，转而处理某一个突发事件;

信号可以直接进行用户空间进程和内核空间进程的交互，内核进程可以利用它来通知用户空间进程发生了哪些系统事件.

1、信号的特点

1. 简单
2. 不能携带大量信息
3. 满足某个特定条件才发送

2、一个完整的信号周期

1. 信号的产生
2. 信号在进程种的注册，信号在进程种的注销
3. 执行信号处理函数

3、信号编号

（1）不存在编号为0的信号

1. 其中1-31号信号称之为常规信号（也叫普通信号或标准信号）
2. 34-64称之为实时信号，驱动编程与硬件相关。名字上区别不大。而前32个名字各不相同

（2）不存在编号为0的号，也没有32-33号

（3）比较重要的一些，需要记住的几个信号

1. SIGINT 当用户按下了<Ctrl+C>组合键时，用户终端向正在运行中的由该终端启动的程序发出此信号，终止进程
2. SIGQUIT 用户按下<ctrl+>组合键时产生该信号，用户终端向正在运行中的由该终端启动的程序发出些信号,终止进程
3. SIGSEGV 指示进程进行了无效内存访问(段错误), 终止进程并产生core文件
4. SIGPIPE Broken pipe向一个没有读端的管道写数据,终止进程
5. SIGCHLD 子进程结束时，父进程会收到这个信号,忽略这个信号

4、信号四要素

（1）编号: man 7 signal 查看文档帮助

（2）名称

（3）事件

（4）默认处理动作:

1. Term：终止进程
2. Ign： 忽略信号 (默认即时对该种信号忽略操作)
3. Core：终止进程，生成Core文件。(查验死亡原因，用于gdb调试)
4. Stop：停止（暂停）进程
5. Cont：继续运行进程

特别强调: 9) SIGKILL 和19) SIGSTOP信号，不允许忽略和捕捉，只能执行默认动作。甚至不能将其设置为阻塞.

5、信号的状态

（1）产生

1. 当用户按某些终端键时，将产生信号
2. 硬件异常将产生信号
3. 软件异常将产生信号
4. 调用系统函数(如：kill、raise、abort)将发送信号
5. 运行 kill /killall命令将发送信号

（2）未决状态：没有被处理

（3）递达状态：信号被处理了

6、阻塞信号集和未决信号集

（1）阻塞信号集

将某些信号加入集合，对他们设置屏蔽，当屏蔽x信号后，再收到该信号，该信号的处理将推后(处理发生在解除屏蔽后)

（2）未决信号集合

信号产生，未决信号集中描述该信号的位立刻翻转为1，表示信号处于未决状态。当信号被处理对应位翻转回为0。这一时刻往往非常短暂.

7、信号产生函数

（1）kill函数：

```
#include <sys/types.h>
#include <signal.h>
/**
 * 给指定进程发送指定信号(不一定杀死).
 * @param pid 取值有四种情况:
 *            pid > 0:  将信号传送给进程 ID 为pid的进程.
 *            pid = 0 :  将信号传送给当前进程所在进程组中的所有进程.
 *            pid = -1 : 将信号传送给系统内所有的进程.
 *            pid < -1 : 将信号传给指定进程组的所有进程，这个进程组号等于 pid 的绝对值.
 * @param sig 信号的编号，这里可以填数字编号，也可以填信号的宏定义.
 *            可以通过命令 kill - l("l" 为字母)进行相应查看.
 *            不推荐直接使用数字，应使用宏名，因为不同操作系统信号编号可能不同，但名称一致.
 * @return 成功: 0; 失败: -1.
 * 普通用户基本规则是：发送者实际或有效用户ID == 接收者实际或有效用户ID.
 */
int kill(pid_t pid, int sig);
```

(2) raise函数

```
#include <signal.h>
/**
 * 给当前进程发送指定信号(自己给自己发)，等价于 kill(getpid(), sig).
 * @param sig 信号编号.
 * @return  成功: 0; 失败: 非0值.
 */
int raise(int sig);
```

(3) abort函数

```
#include <stdlib.h>
/**
 * 给自己发送异常终止信号 6) SIGABRT，并产生core文件，等价于kill(getpid(), SIGABRT).
 */
void abort(void);
```

(4) alarm函数(闹钟)

```
#include <unistd.h>
/**
 * 设置定时器(闹钟)。在指定seconds后，内核会给当前进程发送14) SIGALRM信号。进程收到该信号，默认动作终止。每个进程都有且只有唯一的一个定时器;
 * 取消定时器alarm(0)，返回旧闹钟余下秒数.
 * @param seconds 指定的时间，以秒为单位.
 * @return 返回0或剩余的秒数.
 * /
unsigned int alarm(unsigned int seconds);
```

(5) setitimer函数（定时器）

```
#include <sys/time.h>
struct itimerval {
    struct timerval it_interval; // 闹钟触发周期
    struct timerval it_value;    // 闹钟触发时间
};
struct timeval {
    long tv_sec;            // 秒
    long tv_usec;           // 微秒
}
/**
 * 设置定时器(闹钟)。 可代替alarm函数。精度微秒us，可以实现周期定时.
 * @param which 指定定时方式:
 *              (1) 自然定时: ITIMER_REAL → 14) SIGALRM计算自然时间;
 *              (2) 虚拟空间计时(用户空间): ITIMER_VIRTUAL → 26) SIGVTALRM  只计算进程占用cpu的时间;
 *              (3) 虚拟空间计时(用户空间): ITIMER_VIRTUAL → 26) SIGVTALRM  只计算进程占用cpu的时间.
 * @param new_value 负责设定timeout时间.
```

```
 * @param old_value 存放旧的timeout值，一般指定为NULL.
 * @return 成功: 0; 失败: -1.
 */
int setitimer(int which,  const struct itimerval *new_value, struct itimerval
*old_value);
// itimerval.it_value:  设定第一次执行function所延迟的秒数
// itimerval.it_interval:  设定以后每几秒执行function
```

8、自定义信号集函数

```
#include <signal.h>
int sigemptyset(sigset_t *set);    // 将set集合置空
int sigfillset(sigset_t *set);    // 将所有信号加入set集合
int sigaddset(sigset_t *set, int signo);    // 将signo信号加入到set集合
int sigdelset(sigset_t *set, int signo);    // 从set集合中移除signo信号
int sigismember(const sigset_t *set, int signo);    // 判断信号是否存在
```

除sigismember外，其余操作函数中的set均为传出参数。sigset_t类型的本质是位图;

9、阻塞信号集

（1）信号阻塞集也称信号屏蔽集、信号掩码;

（2）信号阻塞集用来描述哪些信号递送到该进程的时候被阻塞;

（3）sigprocmask函数

```
#include <signal.h>
/**
 * 检查或修改信号阻塞集，根据 how 指定的方法对进程的阻塞集合进行修改，
 * 新的信号阻塞集由 set 指定，而原先的信号阻塞集合由 oldset 保存.
 *
 * @param how 信号阻塞集合的修改方法，有 3 种情况:
 *            (1) SIG_BLOCK: 向信号阻塞集合中添加 set 信号集，
 *                新的信号掩码是set和旧信号掩码的并集。相当于 mask = mask|set;
 *            (2) SIG_UNBLOCK: 从信号阻塞集合中删除 set 信号集，
 *                从当前信号掩码中去除 set 中的信号。相当于 mask = mask & ~ set;
 *            (3) SIG_SETMASK: 将信号阻塞集合设为 set 信号集，
 *                相当于原来信号阻塞集的内容清空，然后按照 set 中的信号重新设置信号阻塞集。相当于
mask = set.
 *
 * @set 要操作的信号集地址,若 set 为 NULL，则不改变信号阻塞集合，函数只把当前信号阻塞集合保存到
oldset 中.
 * @oldset 保存原先信号阻塞集地址.
 * @return 成功: 0; 失败: -1，失败时错误代码只可能是 EINVAL，表示参数 how 不合法.
 */
int sigprocmask(int how, const sigset_t *set, sigset_t *oldset);
```

（4）sigpending函数

```
#include <signal.h>
/**
 * 读取当前进程的未决信号集.
 * @param set 未决信号集.
 * @return 成功: 0; 失败: -1.
 */
int sigpending(sigset_t *set);
```

10、信号捕捉

> 注意: SIGKILL 和 SIGSTOP 不能更改信号的处理方式，因为它们向用户提供了一种使进程终止的可靠方法;

（1）sigaction函数

```
#include <signal.h>
/**
 * 检查或修改指定信号的设置（或同时执行这两种操作）.
 * @param signum 要操作的信号.
 * @param act 要设置的对信号的新处理方式（传入参数）.
 * @param oldact：原来对信号的处理方式（传出参数）.
 *
 * 如果 act 指针非空，则要改变指定信号的处理方式（设置），
 * 如果 oldact 指针非空，则系统将此前指定信号的处理方式存入 oldact.
 *
 * @return 成功: 0; 失败: -1.
 */
int sigaction(int signum, const struct sigaction *act, struct sigaction *oldact);
```

（2）sa_handler、sa_sigaction：信号处理函数指针

和 signal() 里的函数指针用法一样，应根据情况给sa_sigaction、sa_handler 两者之一赋值

其取值如下：

1. SIG_IGN：忽略该信号
2. SIG_DFL：执行系统默认动作
3. 处理函数名：自定义信号处理函数

（3）sa_mask：

信号阻塞集，在信号处理函数执行过程中，临时屏蔽指定的信号;

（4）sa_flags：

用于指定信号处理的行为，通常设置为0，表使用默认属性。它可以是一下值的“按位或”组合：

1. A_RESTART：使被信号打断的系统调用自动重新发起（已经废弃）
2. SA_NOCLDSTOP：使父进程在它的子进程暂停或继续运行时不会收到 SIGCHLD 信号
3. SA_NOCLDWAIT：使父进程在它的子进程退出时不会收到 SIGCHLD 信号，这时子进程如果退出也不会成为僵尸进程
4. SA_NODEFER：使对信号的屏蔽无效，即在信号处理函数执行期间仍能发出这个信号
5. SA_RESETHAND：信号处理之后重新设置为默认的处理方式

6. SA_SIGINFO：使用 sa_sigaction 成员而不是 sa_handler 作为信号处理函数

11、struct sigaction结构体

```
struct sigaction {
    void(*sa_handler)(int); //旧的信号处理函数指针
    void(*sa_sigaction)(int, siginfo_t *, void *); //新的信号处理函数指针
    sigset_t   sa_mask;       //信号阻塞集
    int        sa_flags;      //信号处理的方式
    void(*sa_restorer)(void); //已弃用
};
```

12、信号处理函数

```
/**
 * @param signum 信号的编号.
 * @param info 记录信号发送进程信息的结构体.
 * @param context 可以赋给指向 ucontext_t 类型的一个对象的指针
 * 以引用在传递信号时被中断的接收进程或线程的上下文.
 */
void(*sa_sigaction)(int signum, siginfo_t *info, void *context);
```

13、不可重入、可重入函数

如果有一个函数不幸被设计成为这样：那么不同任务调用这个函数时可能修改其他任务调用这个函数的数据，从而导致不可预料的后果。

这样的函数是不安全的函数，也叫不可重入函数;

（1）不可重入函数:

1. 函数体内使用了静态的数据结构;
2. 函数体内调用了malloc() 或者 free() 函数(谨慎使用堆);
3. 函数体内调用了标准 I/O 函数;

（2）可重入函数:

1. 所谓可重入是指一个可以被多个任务调用的过程，任务在调用时不必担心数据是否会出错;
2. 在写函数时候尽量使用局部变量（例如寄存器、栈中的变量）;
3. 对于要使用的全局变量要加以保护（如采取关中断、信号量等互斥方法），这样构成的函数就一定是一个可重入的函数.

14、SIGCHLD信号

1. 子进程终止时;
2. 子进程接收到SIGSTOP信号停止时;
3. 子进程处在停止态，接受到SIGCONT后唤醒时。

15、如何避免僵尸进程

（1）最简单的方法:

父进程通过 wait() 和 waitpid() 等函数等待子进程结束，但是，这会导致父进程挂起;

（2）如果父进程要处理的事情很多，不能够挂起，通过 signal() 函数人为处理信号 SIGCHLD:

只要有子进程退出自动调用指定好的回调函数，因为子进程结束后， 父进程会收到该信号 SIGCHLD ，可以在其回调函数里调用 wait() 或 waitpid() 回收;

（3）如果父进程不关心子进程什么时候结束，那么可以用signal（SIGCHLD, SIG_IGN）通知内核:

自己对子进程的结束不感兴趣，父进程忽略此信号，那么子进程结束后，内核会回收，并不再给父进程发送信号;

7. 守护进程

1、进程组概述

1. 代表一个或多个进程的集合;
2. 每个进程都属于一个进程组;
3. 是为了简化对多个进程的管理。

2、会话

1. 一个会话可以有一个控制终端。这通常是终端设备或伪终端设备;
2. 建立与控制终端连接的会话首进程被称为控制进程;
3. 一个会话中的几个进程组可被分为一个前台进程组以及一个或多个后台进程组;
4. 如果一个会话有一个控制终端，则它有一个前台进程组，其它进程组为后台进程组;
5. 如果终端接口检测到断开连接，则将挂断信号发送至控制进程（会话首进程。

3、创建会话注意事项

1. 调用进程不能是进程组组长，该进程变成新会话首进程(session header);
2. 该调用进程是组长进程，则出错返回;
3. 该进程成为一个新进程组的组长进程;
4. 需有root权限(ubuntu不需要);
5. 新会话丢弃原有的控制终端，该会话没有控制终端;
6. 建立新会话时，先调用fork, 父进程终止，子进程调用setsid。

4、API函数

（1）getsid 函数

```
#include <unistd.h>
/**
 * 获取进程所属的会话ID.
 * @param pid 进程号，pid为0表示查看当前进程session ID.
 * @return 成功: 返回调用进程的会话ID; 失败: -1.
 */
pid_t getsid(pid_t pid);
```

（2）setsid函数

```
#include <unistd.h>
/**
 * 创建一个会话，并以自己的ID设置进程组ID，同时也是新会话的ID。
 * 调用了setsid函数的进程，既是新的会长，也是新的组长.
 *
 * @return 成功：  返回调用进程的会话ID; 失败: -1.
 */
pid_t setsid(void);
```

5、守护进程

守护进程（Daemon Process），也就是通常说的 Daemon 进程（精灵进程），是 Linux 中的后台服务进程

1. 它是一个生存期较长的进程，通常独立于控制终端并且周期性地执行某种任务或等待处理某些发生的事件
2. 一般采用以d结尾的名字
3. 所有的服务存在于 etc/init.d
4. 守护进程是个特殊的孤儿进程
5. 之所以脱离于终端是为了避免进程被任何终端所产生的信息所打断，其在执行过程中的信息也不在任何终端上显示
6. Linux 的大多数服务器就是用守护进程实现的

6、守护进程模型

（1）创建子进程，父进程退出(必须)

所有工作在子进程中进行形式上脱离了控制终端

（2）在子进程中创建新会话(必须)

1. setsid()函数
2. 使子进程完全独立出来，脱离控制

（3）改变当前目录为根目录(不是必须)

1. chdir()函数
2. 防止占用可卸载的文件系统
3. 也可以换成其它路径

（4）重设文件权限掩码(不是必须)

1. umask()函数
2. 防止继承的文件创建屏蔽字拒绝某些权限
3. 增加守护进程灵活性

（5）关闭文件描述符(不是必须)

继承的打开文件不会用到，浪费系统资源，无法卸载

（6）开始执行守护进程核心工作(必须)

守护进程退出处理程序模型

线程

1. 线程的特点

1. 线程是轻量级进程(light-weight process)，也有PCB，创建线程使用的底层函数和进程一样，都是clone;
2. 从内核里看进程和线程是一样的，都有各自不同的PCB;
3. 进程可以蜕变成线程;
4. 在linux下，线程最是小的执行单位；进程是最小的分配资源单位。

实际上，无论是创建进程的fork，还是创建线程的pthread_create，底层实现都是调用同一个内核函数 clone 。

1. 如果复制对方的地址空间，那么就产出一个“进程”;
2. 如果共享对方的地址空间，就产生一个“线程”。

Linux内核是不区分进程和线程的, 只在用户层面上进行区分。所以，线程所有操作函数 pthread_* 是库函数，而非系统调用。

三种线程的实现

（1）用户线程

用户空间实现的线程，由基于用户态的线程库管理

优点：

1. 管理不需要内核直接参与，因此可用于不支持线程技术的操作系统;
2. 用户线程切换由线程库调度，不需要用户态与内核态之间转换。

缺点：

1. 由于不由操作系统调度，一旦用户线程发起系统调用而阻塞，那么此进程下用户线程都无法运行;
2. 一旦某个用户线程正在运行，只有当其交出CPU执行权，其他用户线程才可以运行，无法被打断，因为只有操作系统才有权限打断运行，但是操作系统不直接参与调度;
3. 用户线程操作系统无法感知，CPU资源是直接分配给线程所在的进程，因此分到具体用户线程的时间片就较短了。，其TCB线程控制块无法被操作系统所感知，因此用户线程的创建、终止、同步、调度等都不是由操作系统直接参与。

（2）内核线程：

由操作系统管理、调度，其TCB是存放在内核中。一般由操作系统事先创建内核线程集(类似于线程池)，数量有限。

优点：

1. 当一个内核线程发起系统调用阻塞时不会影响其它内核线程的执行;
2. 操作系统将CPU资源直接分配给内核线程，更多CPU时间。

缺点：

1. 需要由内核来维护内核线程的上下文信息及运行状态等，占用内核资源;
2. 内核线程创建、终止、切换都是在内核中进行，开销比较大。

（3）轻量级线程 LWP：

内核支持的用户线程，像普通进程一样被调度，其类似于进程中的执行线程。

每个LWP都需要一个内核线程的支持(CPU资源分配、调度)，实际上用户线程是运行在LWP上的。

2. 线程共享资源

1. 文件描述符表
2. 每种信号的处理方式
3. 当前工作目录
4. 用户ID和组ID

3. 线程非共享资源

1. 线程id
2. 处理器现场和栈指针(内核栈)
3. 独立的栈空间(用户空间栈)
4. errno变量
5. 信号屏蔽字
6. 调度优先级

4. 线程的优缺点

优点：

1. 提高程序并发性
2. 开销小
3. 数据通信、共享数据方便

缺点：

1. 库函数，不稳定
2. 调试、编写困难、gdb不支持
3. 对信号支持不好

线程如何减少开销

1. 线程创建快，进程创建需要资源管理信息，比如内存管理信息和文件管理信息，而线程创建后是共享其所属进程的资源管理信息;
2. 线程终止时间快，需要回收的仅有少量寄存器和私有的栈区;
3. 线程切换快，因为线程切换仅涉及到少量寄存器和栈区，而进程上下文切换有CPU寄存器和程序计数器(CPU上下文)、虚拟内存空间、页表切换、TLB失效等;
4. 线程因为创建时共享了其所述进程绝大多数资源，因此天生具有很好的线程间通信交互效率。

5. 线程常用操作（线程号）

进程号用 pid_t 数据类型表示，是一个非负整数。线程号则用 pthread_t 数据类型来表示，Linux 使用无符号长整数表示。

1、pthread_self函数

```
#include <pthread.h>
/**
 * 获取线程号.
 * @return 返回调用线程的线程.
 */
pthread_t pthread_self(void);
```

2、pthread_equal函数

```
/**
 * 判断线程号 t1 和 t2 是否相等。为了方便移植，尽量使用函数来比较线程 ID.
 * @param t1 待判断的线程号.
 * @param t2 待判断的线程号.
 * @return 相等: 非0; 不相等: 0.
 */
int pthread_equal(pthread_t t1, pthread_t t2);
```

3、pthread_create函数

```
#include <pthread.h>
/**
 * 创建一个线程.
 * @param thread 线程标识符地址.
 * @param attr 线程属性结构体地址，通常设置为 NULL.
 * @param start_routine 线程函数的入口地址.
 * @param arg 传给线程函数的参数.
 * @return 成功: 0; 失败: 非0.
 */
int pthread_create(pthread_t *thread,
            const pthread_attr_t *attr,
            void *(*start_routine)(void *),
            void *arg );
// 由于pthread_create的错误码不保存在errno中，
// 因此不能直接用perror()打印错误信息，
// 可以先用strerror()把错误码转换成错误信息再打印。
```

4、线程资源回收

pthread_join函数，会阻塞。

```
#include <pthread.h>
/**
 * 等待线程结束（此函数会阻塞），并回收线程资源，类似进程的 wait() 函数。如果线程已经结束，那么该函数会立即返回.
 * @param thread 被等待的线程号.
 * @param retval 用来存储线程退出状态的指针的地址.
 * @return 成功: 0; 失败: 非0.
 */
int pthread_join(pthread_t thread, void **retval);
```

调用该函数的线程将挂起等待，直到id为thread的线程终止。
thread线程以不同的方法终止，通过pthread_join得到的终止状态是不同的。

总结:

1. 如果thread线程通过return返回，retval所指向的单元里存放的是thread线程函数的返回值。
2. 如果thread线程被别的线程调用pthread_cancel异常终止掉，retval所指向的单元里存放的是常数PTHREAD_CANCELED。
3. 如果thread线程是自己调用pthread_exit终止的，retval所指向的单元存放的是传给pthread_exit的参数。

5、线程分离

pthread_detach函数，不会阻塞。

```
#include <pthread.h>
/**
 * 使调用线程与当前进程分离，分离后不代表此线程不依赖与当前进程，
 * 线程分离的目的是将线程资源的回收工作交由系统自动来完成，
 * 也就是说当被分离的线程结束之后，系统会自动回收它的资源。所以，此函数不会阻塞.
 *
 * @param thread 线程号.
 * @return 成功: 0; 失败: 非0.
 */
int pthread_detach(pthread_t thread);
```

总结:

1. 线程终止后，其终止状态一直保留到其它线程调用pthread_join获取它的状态为止。但是线程也可以被置为detach状态，这样的线程一旦终止就立刻回收它占用的所有资源，而不保留终止状态。
2. 不能对一个已经处于detach状态的线程调用pthread_join，这样的调用将返回EINVAL错误。也就是说，如果已经对一个线程调用了pthread_detach就不能再调用pthread_join了

6、线程退出

pthread_exit函数，资源不会被释放。

```
#include <pthread.h>
/**
 * 退出调用线程。一个进程中的多个线程是共享该进程的数据段，因此，通常线程退出后所占用的资源并不会释放.
 * @param retval 存储线程退出状态的指针.
 */
void pthread_exit(void *retval);
```

总结:

1. 线程从执行函数中返回。
2. 线程调用pthread_exit退出线程。
3. 线程可以被同一进程中的其它线程取消。

7、线程取消

pthread_cancel函数

```
#include <pthread.h>
/**
 * 杀死(取消)线程.
 * @param thread 目标线程ID.
 * @return 成功: 0; 失败: 出错编号.
 * 注意: 线程的取消并不是实时的，而又一定的延时。需要等待线程到达某个取消点(检查点)
 */
int pthread_cancel(pthread_t thread);
```

pthread_testcancel(); / / 设置取消点.

6. 线程通信

线程间的通信目的主要是用于线程同步。所以线程没有像进程通信中的用于数据交换的通信机制。

同一进程的不同线程共享同一份内存区域，所以线程之间可以方便、快速地共享信息。只需要将数据复制到共享（全局或堆）变量中即可。但是需要避免出现多个线程试图同时修改同一份信息。

7. 线程属性

1、线程属性初始化和销毁

```
#include <pthread.h>
/**
 * 初始化线程属性函数，注意：应先初始化线程属性，再pthread_create创建线程.
 * @param attr 线程属性结构体.
 * @return 成功: 0; 失败: 错误号.
 */
```

```
int pthread_attr_init(pthread_attr_t *attr);

/**
 * 销毁线程属性所占用的资源函数.
 * @param attr 线程属性结构体.
 * @return 成功: 0; 失败: 错误号.
 */
int pthread_attr_destroy(pthread_attr_t *attr);
```

2、线程分离状态

（1）非分离状态：

线程的默认属性是非分离状态，这种情况下，原有的线程等待创建的线程结束。只有当pthread_join()函数返回时，创建的线程才算终止，才能释放自己占用的系统资源。

（2）分离状态：

分离线程没有被其他的线程所等待，自己运行结束了，线程也就终止了，马上释放系统资源。应该根据自己的需要，选择适当的分离状态。

注意：

1. 如果设置一个线程为分离线程，而这个线程运行又非常快，它很可能在pthread_create函数返回之前就终止了，它终止以后就可能将线程号和系统资源移交给其他的线程使用，这样调用pthread_create的线程就得到了错误的线程号。
2. 要避免这种情况可以采取一定的同步措施，最简单的方法之一是可以在被创建的线程里调用pthread_cond_timedwait函数，让这个线程等待一会儿，留出足够的时间让函数pthread_create返回。

```
#include <pthread.h>
/**
 * 设置线程分离状态.
 * @param attr 已初始化的线程属性.
 * @detachstate(分离状态)
 *      1. PTHREAD_CREATE_DETACHED（分离线程）;
 *      2. PTHREAD_CREATE_JOINABLE（非分离线程）.
 * @return 成功: 0; 失败: 非0.
 */

/**
 * 获取线程分离状态.
 * @param attr 已初始化的线程属性.
 * @detachstate(分离状态)
 *      1. PTHREAD_CREATE_DETACHED（分离线程）;
 *      2. PTHREAD_CREATE_JOINABLE（非分离线程）.
 * @return 成功: 0; 失败: 非0.
 */
int pthread_attr_getdetachstate(const pthread_attr_t *attr, int *detachstate);
```

3、线程栈地址

当进程栈地址空间不够用时，指定新建线程使用由malloc分配的空间作为自己的栈空间。

通过pthread_attr_setstack和pthread_attr_getstack两个函数分别设置和获取线程的栈地址。

```
#include <pthread.h>
/**
 * 设置线程的栈地址.
 * @param attr 指向一个线程属性的指针.
 * @param stackaddr 内存首地址.
 * @param stacksize 返回线程的堆栈大小.
 * @return 成功: 0; 失败: 错误号.
 */
int pthread_attr_setstack(pthread_attr_t *attr, void *stackaddr,  size_t stacksize);

/**
 * 获取线程的栈地址.
 * @param attr 指向一个线程属性的指针.
 * @param stackaddr 返回获取的栈地址.
 * @param stacksize 返回获取的栈大小.
 * @return 成功: 0; 失败: 错误号.
 */
int pthread_attr_getstack(const pthread_attr_t *attr, void **stackaddr,  size_t
*stacksize);-
```

4、线程栈大小

```
#include <pthread.h>
/**
 * 设置线程的栈大小.
 * @param attr 指向一个线程属性的指针.
 * @param stacksize 线程的堆栈大小.
 * @return 成功: 0; 失败: 错误号.
 */
int pthread_attr_setstacksize(pthread_attr_t *attr, size_t stacksize);

/**
 * 获取线程的栈大小.
 * @param attr 指向一个线程属性的指针.
 * @param stacksize 返回线程的堆栈大小.
 * @return 成功: 0; 失败: 错误号.
 */
int pthread_attr_getstacksize(const pthread_attr_t *attr, size_t *stacksize);
```

5、线程使用注意事项

（1）主线程退出其他线程不退出，主线程应调用pthread_exit

（2）避免僵尸线程

1. pthread_join
2. pthread_detach
3. pthread_create指定分离属性

被join线程可能在join函数返回前就释放完自己的所有内存资源，所以不应当返回被回收线程栈中的值

（3）malloc和mmap申请的内存可以被其他线程释放

（4）应避免在多线程模型中调用fork，除非马上exec，子进程中只有调用fork的线程存在，其他线程t在子进程中均pthread_exit

（5）信号的复杂语义很难和多线程共存，应避免在多线程引入信号机制

（6）Cache伪共享：

这种因为多个线程同时读写同一个 Cache Line 的不同变量时，而导致 CPU Cache 失效的现象。避免的方式一般有 Cache Line 大小字节对齐，以及字节填充等方法。在 Linux 内核中存在cachelinealignedin_smp 宏定义，是用于解决伪共享的问题。

8. 多线程

用户态的多线程模型，同一个进程内部有多个线程，所有的线程共享同一个进程的内存空间，进程中定义的全局变量会被所有的线程共享；

i++在计算机中并不是原子操作，涉及内存取数，计算和写入内存几个环节，而线程的切换有可能发生在上述任何一个环节中间，所以不同的操作顺序很有可能带来意想不到的结果。

多线程的好处：

主要原因是许多应用中同时发生多个活动，某些活动随着时间推移而阻塞，将这些应用程序分解成并发运行的多个线程，简化设计模型。同时多线程有共享同一地址空间和可用数据的能力，这是多进程没有的。

线程比进程开销小，更容易创建和释放。

多个线程是IO密集型时，多线程可以使这些活动彼此重叠运行，可以加快程序执行的速度。

对于线程需要考虑：

线程之间有无先后访问顺序（线程依赖关系）

多个线程共享访问同一变量（同步互斥问题）

同一进程的多个线程共享进程的资源，除了标识线程的tid，每个线程还有自己独立的栈空间，线程彼此之间是无法访问其他线程栈上内容的。

互斥与同步

互斥锁Mutex

互斥锁（mutex）：

也叫互斥量，互斥锁是一种简单的加锁的方法来控制对共享资源的访问，互斥锁只有两种状态,即加锁(lock)和解锁(unlock)

1. 在访问共享资源后临界区域前，对互斥锁进行加锁。

2. 在访问完成后释放互斥锁导上的锁。
3. 对互斥锁进行加锁后，任何其他试图再次对互斥锁加锁的线程将会被阻塞，直到锁被释放。

互斥锁的数据类型是： pthread_mutex_t

1. 创建互斥锁

pthread_mutex_init()

```
#include <pthread.h>
/**
 * 初始化一个互斥锁.
 * @param mutex 互斥锁地址。类型是 pthread_mutex_t.
 * @param attr 设置互斥量的属性，通常可采用默认属性，即可将 attr 设为 NULL.
 * @return 成功: 0 成功申请的锁默认是打开的; 失败: 非0(错误码).
 */
int pthread_mutex_init(pthread_mutex_t *restrict mutex,
const pthread_mutexattr_t *restrict attr);
// 这种方法等价于使用 NULL 指定的 attr 参数调用 pthread_mutex_init() 来完成动态初始化，
// 不同之处在于 PTHREAD_MUTEX_INITIALIZER 宏不进行错误检查。
```

2. 销毁互斥锁

pthread_mutex_destroy()

```
#include <pthread.h>
/**
 * 销毁指定的一个互斥锁。互斥锁在使用完毕后，必须要对互斥锁进行销毁，以释放资源.
 * @param mutex 互斥锁地址。类型是 pthread_mutex_t.
 * @return 成功: 0; 失败: 非0(错误码).
 */
int pthread_mutex_destroy(pthread_mutex_t *mutex);
```

3. 互斥锁上锁

pthread_mutex_lock()

```
#include <pthread.h>
/**
 * 对互斥锁上锁，若互斥锁已经上锁，则调用者阻塞，直到互斥锁解锁后再上锁.
 * @param mutex 互斥锁地址.
 * @return 成功: 0; 失败: 非0(错误码).
 */
int pthread_mutex_lock(pthread_mutex_t *mutex);

/**
 * 调用该函数时，若互斥锁未加锁，则上锁，返回 0;
 * 若互斥锁已加锁，则函数直接返回失败，即 EBUSY.
 */
int pthread_mutex_trylock(pthread_mutex_t *mutex);
```

4. 互斥锁解锁

pthread_mutex_unlock()

```
#include <pthread.h>
/**
 * 对指定的互斥锁解锁.
 * @param mutex 互斥锁地址.
 * @return 成功: 0; 失败: 非0(错误码)
 */
int pthread_mutex_unlock(pthread_mutex_t *mutex);
```

死锁（DeadLock）

如果一个进程集合中的每一个进程都在等待只能由该进程集合中的其他进程才能引发的事件，那么，该进程集合就是死锁

1. 资源

1、可抢占资源

可以从拥有它的进程中抢占而不会产生任何副作用，存储器就是一类可抢占资源

2、不可抢占资源

是指在不引起相关计算失败的情况下，无法把它从占有它的进程处抢占过来

2. 必要条件

1、互斥

每个资源要么已经分配给一个进程，要么就是可用的

2、占有和等待

已经得到了某个资源的进程可以再请求新的资源

3、不可抢占

已经分配给一个进程的资源不能强制性地被抢占，它只能被占有它的进程显式地释放

4、环路等待

死锁发生时，系统中一定有由两个或两个以上的进程组成的一条环路，该环路中的每个进程都在等待着下一个进程所占有的资源。

3. 处理方法

1、鸵鸟算法

把头埋在沙子里，假装根本没发生问题。

因为解决死锁问题的代价很高，因此鸵鸟算法这种不采取任务措施的方案会获得更高的性能。

当发生死锁时**不会对用户造成多大影响**，或发生死锁的概率很低，可以采用鸵鸟算法。

2、死锁检测与死锁恢复

（1）每种类型一个资源的死锁检测

检测算法：

通过检测有向图中是否存在环来实现，从一个节点出发进行深度优先搜索，对访问过的节点进行标记，如果访问了已经标记的节点，就表示有向图存在环，也就是检测到死锁发生

（2）每种类型多个资源的死锁检测

锁检测算法如下：

每个进程最开始时都不被标记，执行过程有可能被标记。当算法结束时，任何没有被标记的进程都是死锁进程。

1. 寻找一个没有标记的进程Pi，它所请求的资源小于或等于A
2. 如果真找到这样一个进程，那么将C矩阵的第i行向量加到A中，标记该进程，并转回第1步
3. 如果没有这样的进程，那么算法终止

3、从死锁中恢复

（1）利用抢占恢复

将进程挂起，强行取走资源给另一个进程使用，用完再放回

（2）利用回滚恢复

复位到更早的状态，那时它还没有取得所需的资源

（3）通过杀死进程恢复

杀掉环中的一个进程或多个，牺牲掉一个环外进程

4、死锁预防

（1）破坏互斥条件

例如假脱机打印机技术允许若干个进程同时输出，唯一真正请求物理打印机的进程是打印机守护进程。

（2）破坏占有个等待条件

1. 规定所有进程在开始执行前请求所需要的全部资源。
2. 要求当一个进程请求资源时，先暂时释放其当前占用的所有资源，然后在尝试一次获得所需的全部资源。

（3）破坏不可抢占条件

1. 保证每一个进程在任何时刻只能占用一个资源，如果请求另一个资源必须先释放第一个资源

2. 将所有的资源统一编号，进程可以在任何时刻提出资源请求，但是所有请求必须按照资源编号的顺序(升序)提出

（4）破坏环路等待

5、死锁避免

（1）安全状态

如果没有死锁发生，并且即使所有进程突然请求对资源的最大需求，也仍然存在某种调度次序**能够使得每一个进程运行完毕**，则称该状态是安全的。

（2）单个资源的银行家算法

一个小城镇的银行家，他向一群客户分别承诺了一定的贷款额度，算法要做的是判断对**请求的满足是否会进入不安全状态**，如果是，就拒绝请求；否则予以分配。

（3）多个资源的银行家算法

检查一个**状态是否安全**的算法

1. 查找右边的矩阵是否存在一行小于等于向量 A。如果不存在这样的行，那么系统将会发生死锁，状态是不安全的。
2. 假若找到这样一行，将该进程标记为终止，并将其已分配资源加到 A 中。
3. 重复以上两步，直到所有进程都标记为终止，则状态时安全的。

如果一个状态不是安全的，需要拒绝进入这个状态。

读写锁

在对数据的读写操作中，更多的是读操作，写操作较少，例如对数据库数据的读写应用。

为了满足当前能够允许多个读出，但只允许一个写入的需求，线程提供了读写锁来实现。

读写锁的特点

1、如果有其它线程读数据，则允许其它线程执行读操作，但不允许写操作

2、如果有其它线程写数据，则其它线程都不允许读、写操作

读写锁分为读锁和写锁，规则如下

1. 如果某线程申请了读锁，其它线程可以再申请读锁，但不能申请写锁。
2. 如果某线程申请了写锁，其它线程不能申请读锁，也不能申请写锁。

POSIX 定义的读写锁的数据类型是： pthread_rwlock_t。

1. 初始化读写锁

pthread_rwlock_init()

```
/**
 * 用来初始化 rwlock 所指向的读写锁.
 * @param rwlock 指向要初始化的读写锁指针.
 * @param attr：读写锁的属性指针。如果 attr 为 NULL 则会使用默认的属性初始化读写锁，否则使用指定的
attr 初始化读写锁.
 * 可以使用宏 PTHREAD_RWLOCK_INITIALIZER 静态初始化读写锁，比如:
 * pthread_rwlock_t my_rwlock = PTHREAD_RWLOCK_INITIALIZER;
 * 这种方法等价于使用 NULL 指定的 attr 参数调用 pthread_rwlock_init() 来完成动态初始化，
 * 不同之处在于PTHREAD_RWLOCK_INITIALIZER 宏不进行错误检查.
 * @return 成功: 0，读写锁的状态将成为已初始化和已解锁; 失败: 非 0 错误码.
 */
#include <pthread.h>
int pthread_rwlock_init(pthread_rwlock_t *restrict rwlock,
const pthread_rwlockattr_t *restrict attr);
```

2. 销毁读写锁

pthread_rwlock_destroy()

```
/**
 * 用于销毁一个读写锁，并释放所有相关联的资源（所谓的所有指的是由 pthread_rwlock_init() 自动申请的
资源.
 * @param rwlock 读写锁指针.
 * @return 成功: 0; 失败: 非0错误码.
 */
#include <pthread.h>
int pthread_rwlock_destroy(pthread_rwlock_t *rwlock);
```

3. 读锁定

pthread_rwlock_rdlock()

```
#include <pthread.h>
/**
 * 以阻塞方式在读写锁上获取读锁（读锁定）.
 * 如果没有写者持有该锁，并且没有写者阻塞在该锁上，则调用线程会获取读锁.
 * 如果调用线程未获取读锁，则它将阻塞直到它获取了该锁。一个线程可以在一个读写锁上多次执行读锁定.
 * 线程可以成功调用 pthread_rwlock_rdlock() 函数 n 次，但是之后该线程必须调用
pthread_rwlock_unlock() 函数 n 次才能解除锁定.
 * @param rwlock 读写锁指针.
 * @return 成功: 0; 失败: 非0错误码.
 */
int pthread_rwlock_rdlock(pthread_rwlock_t *rwlock);

/**
 * 用于尝试以非阻塞的方式来在读写锁上获取读锁.
```

```
 * 如果有任何的写者持有该锁或有写者阻塞在该读写锁上，则立即失败返回.
 */
int pthread_rwlock_tryrdlock(pthread_rwlock_t *rwlock);
```

4. 写锁定

pthread_rwlock_wrlock()

```
#include <pthread.h>
/**
 * 在读写锁上获取写锁（写锁定）.
 * 如果没有写者持有该锁，并且没有写者读者持有该锁，则调用线程会获取写锁.
 * 如果调用线程未获取写锁，则它将阻塞直到它获取了该锁.
 * @param rwlock 读写锁指针.
 * @return 成功: 0; 失败: 非0错误码.
 */
int pthread_rwlock_wrlock(pthread_rwlock_t *rwlock);

/**
 * 用于尝试以非阻塞的方式来在读写锁上获取写锁.
 * 如果有任何的读者或写者持有该锁，则立即失败返回.
 */
int pthread_rwlock_trywrlock(pthread_rwlock_t *rwlock);
```

5. 解锁

pthread_rwlock_unlock()

```
#include <pthread.h>
/**
 * 无论是读锁或写锁，都可以通过此函数解锁.
 * @param rwlock 读写锁指针.
 * @return 成功: 0; 失败: 非0错误码.
 */
int pthread_rwlock_unlock(pthread_rwlock_t *rwlock);
```

条件变量

与互斥锁不同，条件变量是用来等待而不是用来上锁的，条件变量本身不是锁！

条件变量用来自动阻塞一个线程，直到某特殊情况发生为止。通常条件变量和互斥锁同时使用。

条件变量的两个动作：

1. 条件不满, 阻塞线程
2. 当条件满足, 通知阻塞的线程开始工作

条件变量的类型: **pthread_cond_t**

1. 创建条件变量

pthread_cond_init()

```
#include <pthread.h>
/**
 * 初始化一个条件变量.
 * @param cond 指向要初始化的条件变量指针.
 * @param attr 条件变量属性，通常为默认值，传NULL即可.
 *              也可以使用静态初始化的方法，初始化条件变量;
 *              pthread_cond_t cond = PTHREAD_COND_INITIALIZER.
 * return 成功: 0; 失败: 非0错误号.
 */
int pthread_cond_init(pthread_cond_t *restrict cond,
const pthread_condattr_t *restrict attr);
```

2. 删除条件变量

pthread_cond_destroy()

```
#include <pthread.h>
/**
 * 销毁一个条件变量.
 * @param cond 指向要初始化的条件变量指针.
 * @return  成功: 0; 失败: 非0错误号.
 */
int pthread_cond_destroy(pthread_cond_t *cond);
```

3. 唤醒线程

pthread_cond_signal()

```
#include <pthread.h>
/**
 * 唤醒至少一个阻塞在条件变量上的线程.
 * @param cond 指向要初始化的条件变量指针.
 * @return  成功: 0; 失败: 非0错误号.
 */
int pthread_cond_signal(pthread_cond_t *cond);

/**
 * 唤醒全部阻塞在条件变量上的线程.
 * @param cond 指向要初始化的条件变量指针.
 * @return  成功: 0; 失败: 非0错误号.
 */
int pthread_cond_broadcast(pthread_cond_t *cond);
```

4. 阻塞线程

pthread_cond_wait()

```
#include <pthread.h>
struct timespec {
    time_t tv_sec;      /* seconds */ // 秒
    long   tv_nsec; /* nanosecondes*/ // 纳秒
}
// time_t cur = time(NULL);         //获取当前时间。
// struct timespec t;               //定义timespec 结构体变量t
// t.tv_sec = cur + 1;              // 定时1秒
// pthread_cond_timedwait(&cond, &t);
/**
 * 阻塞等待一个条件变量
 *      1. 阻塞等待条件变量cond（参1）满足
 *      2. 释放已掌握的互斥锁（解锁互斥量）相当于pthread_mutex_unlock(&mutex);
 *      ((1), (2)两步为一个原子操作)
 *      3. 当被唤醒，pthread_cond_wait函数返回时，解除阻塞并重新申请获取互斥锁
pthread_mutex_lock(&mutex);
 * @param cond 指向要初始化的条件变量指针.
 * @param mutex 互斥锁.
 * @return  成功: 0; 失败: 非0错误号.
 */
int pthread_cond_wait(pthread_cond_t *restrict cond,
    pthread_mutex_t *restrict mutex);

/**
 * 限时等待一个条件变量.
 * @param cond 指向要初始化的条件变量指针.
 * @param mutex 互斥锁.
 * @param abstime 绝对时间.
 * @return  成功: 0; 失败: 非0错误号.
 */
int pthread_cond_timedwait(pthread_cond_t *restrict cond,
    pthread_mutex_t *restrict mutex,
    const struct *restrict abstime);
```

5. 条件变量流程分析

1、条件变量的优缺点

相较于mutex而言，条件变量可以减少竞争。

如直接使用mutex，除了生产者、消费者之间要竞争互斥量以外，消费者之间也需要竞争互斥量;

但如果汇聚（链表）中没有数据，消费者之间竞争互斥锁是无意义的。
有了条件变量机制以后，只有生产者完成生产，才会引起消费者之间的竞争。提高了程序效率。

2、条件变量流程分析

场景: 你是个老板，招聘了三个员工，但是你不是有了活才去招聘员工，而是先把员工招来，没有活的时候员工需要在那里等着，一旦有了活，你要去通知他们，他们要去抢活干，干完了再等待，你再有活，再通知他们

代码如下：

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#define NUM_OF_TASKS 3
#define MAX_TASK_QUEUE 11
char tasklist[MAX_TASK_QUEUE]="ABCDEFGHIJ";
int head = 0;
int tail = 0;
int quit = 0;
pthread_mutex_t g_task_lock;
pthread_cond_t g_task_cv;

void *coder(void *notused)
{
    pthread_t tid = pthread_self();
    while(!quit){
        pthread_mutex_lock(&g_task_lock);
        while(tail == head){
            if(quit){
                pthread_mutex_unlock(&g_task_lock);
                pthread_exit((void *)0);
            }
            printf("No task now! Thread %u is waiting!\n", (unsigned int)tid);
            pthread_cond_wait(&g_task_cv, &g_task_lock);
            printf("Have task now! Thread %u is grabing the task !\n", (unsigned
int)tid);
        }
        char task = tasklist[head++];
        pthread_mutex_unlock(&g_task_lock);
        printf("Thread %u has a task %c now!\n", (unsigned int)tid, task);
        sleep(5);
        printf("Thread %u finish the task %c!\n", (unsigned int)tid, task);
    }
    pthread_exit((void *)0);
}

int main(int argc, char *argv[])
{
    pthread_t threads[NUM_OF_TASKS];
    int rc;
    int t;
    pthread_mutex_init(&g_task_lock, NULL);
    pthread_cond_init(&g_task_cv, NULL);
    for(t=0;t<NUM_OF_TASKS;t++){
        rc = pthread_create(&threads[t], NULL, coder, NULL);
        if (rc){
            printf("ERROR; return code from pthread_create() is %d\n", rc);
```

```
            exit(-1);
        }
    }
    sleep(5);
    for (t=1;t<=4;t++) {
        pthread_mutex_lock(&g_task_lock);
        tail+=t;
        printf("I am Boss, I assigned %d tasks, I notify all coders!\n", t);
        pthread_cond_broadcast(&g_task_cv);
        pthread_mutex_unlock(&g_task_lock);
        sleep(20);
    }
    pthread_mutex_lock(&g_task_lock);
    quit = 1;
    pthread_cond_broadcast(&g_task_cv);
    pthread_mutex_unlock(&g_task_lock);
    pthread_mutex_destroy(&g_task_lock);
    pthread_cond_destroy(&g_task_cv);
    pthread_exit(NULL);
}
```

下面是对于每个输出的分析：

```
No task now! Thread 1461831424 is waiting!
No task now! Thread 1453438720 is waiting!
No task now! Thread 1445046016 is waiting!
```

这是主线程新建的三个工作线程，这时候工作线程都是先获得锁，判断tail==head，阻塞，cond_wait释放了锁，三个工作线程都等待信号来唤醒

```
I am Boss, I assigned 1 tasks, I notify all coders!
```

主线程增加了一个任务，引起了head!=tail，这时候broadcast唤醒了所有工作线程，之后主线程释放锁，开始睡眠20s

```
Have task now! Thread 1453438720 is grabing the task !

```

1工作线程在竞争中抢到了锁，退出了while(tail==head)循环，并开始完成A任务，这时候把头加1，释放锁

```
Have task now! Thread 1445046016 is grabing the task !
No task now! Thread 1445046016 is waiting!
```

1工作线程时间片到了，2，3工作线程对锁进行竞争，2得到了锁，但是这时候已经没有工作了。while(tail==head)循环没有退出，因此释放锁，等待唤醒

```
Thread 1453438720 has a task A now!
```

调度程序调度到1线程执行，打印了这句话，开始睡眠5s

```
Have task now! Thread 1461831424 is grabing the task !
No task now! Thread 1461831424 is waiting!
```

3工作线程抢到了锁（虽然这时候只有它在抢锁），3得到了锁，但是这时候已经没有工作了。while(tail==head)循环没有退出，因此释放锁，等待唤醒

```
Thread 1453438720 finish the task A!
No task now! Thread 1453438720 is waiting!
```

20工作线程执行完任务，这时候还是没有工作。先上锁，后进入while(tail==head)循环没有退出，因此释放锁，等待唤醒

```
I am Boss, I assigned 2 tasks, I notify all coders!
```

主线程醒了，又找了两个工作B,C，唤醒了所有工作线程去做

```
Have task now! Thread 1461831424 is grabing the task !
Thread 1461831424 has a task B now!
```

24号线程抢到了锁，执行头加1，释放锁，然后睡眠了

```
Have task now! Thread 1445046016 is grabing the task !
Thread 1445046016 has a task C now!
```

16号线程抢到了锁，执行头加1，释放锁，然后睡眠了

```
Have task now! Thread 1453438720 is grabing the task !
No task now! Thread 1453438720 is waiting!
```

20号线程抢到了锁，没任务了，就释放锁，等待唤醒

```
Thread 1445046016 finish the task C!
No task now! Thread 1445046016 is waiting!
```

调度16号线程执行，16号线程完成任务，得到锁，发现没任务，释放锁，等待唤醒

```
Thread 1461831424 finish the task B!
No task now! Thread 1461831424 is waiting!
```

调度24号线程执行，24号线程完成任务，得到锁，发现没任务，释放锁，等待唤醒

```
I am Boss, I assigned 3 tasks, I notify all coders!
```

主线程醒了，又找了三个个工作D,E,F，唤醒了所有工作线程去做

```
Have task now! Thread 1461831424 is grabing the task !
Thread 1461831424 has a task D now!
```

24号线程抢到了锁，执行头加1，释放锁，然后睡眠了

```
Have task now! Thread 1445046016 is grabing the task !
Thread 1445046016 has a task E now!
```

16号线程抢到了锁，执行头加1，释放锁，然后睡眠了

```
Have task now! Thread 1453438720 is grabing the task !
Thread 1453438720 has a task F now!
```

20号线程抢到了锁，执行头加1，释放锁，然后睡眠了

```
Thread 1461831424 finish the task D!
No task now! Thread 1461831424 is waiting!
```

调度24号线程执行，24号线程完成任务，得到锁，发现没任务，释放锁，等待唤醒

```
Thread 1445046016 finish the task E!
No task now! Thread 1445046016 is waiting!
```

调度16号线程执行，16号线程完成任务，得到锁，发现没任务，释放锁，等待唤醒

```
Thread 1453438720 finish the task F!
No task now! Thread 1453438720 is waiting!
```

调度20号线程执行，20号线程完成任务，得到锁，发现没任务，释放锁，等待唤醒

```
I am Boss, I assigned 4 tasks, I notify all coders!
```

主线程醒了，又找了三个个工作G,H,I,J，唤醒了所有工作线程去做

```
Have task now! Thread 1453438720 is grabing the task !
Thread 1453438720 has a task G now!
```

20号线程抢到了锁，执行头加1，释放锁，然后睡眠了

```
Have task now! Thread 1461831424 is grabing the task !
Thread 1461831424 has a task H now!
```

24号线程抢到了锁，执行头加1，释放锁，然后睡眠了

```
Have task now! Thread 1445046016 is grabing the task !
Thread 1445046016 has a task I now!
```

16号线程抢到了锁，执行头加1，释放锁，然后睡眠了

```
Thread 1453438720 finish the task G!
Thread 1453438720 has a task J now!
```

20号线程执行完任务，下一轮while中，持有了锁，发现还有任务，就继续做J，执行头加1，释放了锁，开始睡眠5s

```
Thread 1461831424 finish the task H!
No task now! Thread 1461831424 is waiting!
```

调度24号线程执行，24号线程完成任务，得到锁，发现没任务，释放锁，等待唤醒

```
Thread 1445046016 finish the task I!
No task now! Thread 1445046016 is waiting!
```

调度16号线程执行，16号线程完成任务，得到锁，发现没任务，释放锁，等待唤醒

```
Thread 1453438720 finish the task J!
No task now! Thread 1453438720 is waiting!
```

20线程完成任务，下一轮while中持有锁，发现没任务了，释放锁，等待唤醒
主线程抢锁，设置退出标志quit为1，并通知三个工作线程

```
Have task now! Thread 1461831424 is grabing the task !
Have task now! Thread 1445046016 is grabing the task !
Have task now! Thread 1453438720 is grabing the task !
```

三个工作线程分别抢到了锁，发现quit为1，就释放锁，然后终止各自的线程
主线程销毁了锁和条件变量
终止了main线程

信号量

信号量广泛**用于进程或线程间的同步和互斥**，信号量本质上是一个非负的整数计数器，它被用来控制对公共资源的访问。

编程时可根据操作信号量值的结果判断是否对公共资源具有访问的权限，当信号量值大于 0 时，则可以访问，否则将阻塞。

PV 原语是对信号量的操作，一次 P 操作使信号量减 1 ，一次 V 操作使信号量加 1 。

信号量数据类型为：sem_t

1. 创建信号量

sem_init()

```
#include <semaphore.h>
/**
 * 创建一个信号量并初始化它的值。一个无名信号量在被使用前必须先初始化.
 * @param sem 信号量的地址.
 * @param pshared 等于 0，信号量在线程间共享（常用）；不等于0，信号量在进程间共享.
 * @param value 信号量的初始值.
 * @return 成功: 0; 失败: -1.
 */
int sem_init(sem_t *sem, int pshared, unsigned int value);
```

2. 删除信号量

sem_destroy()

```
#include <semaphore.h>
/**
 * 删除 sem 标识的信号量.
 * @return 成功: 0; 失败: -1.
 */
int sem_destroy(sem_t *sem);
```

3. P操作

sem_wait()(P操作)

```
#include <semaphore.h>
/**
 * 将信号量的值减 1。操作前，先检查信号量（sem）的值是否为 0;
 * 若信号量为 0，此函数会阻塞，直到信号量大于 0 时才进行减 1 操作。
 * @param sem 信号量的地址.
 * @return 成功: 0; 失败: -1.
 */
int sem_wait(sem_t *sem);

/**
 * 以非阻塞的方式来对信号量进行减 1 操作。
 * 若操作前，信号量的值等于 0，则对信号量的操作失败，函数立即返回。
 */
int sem_trywait(sem_t *sem);

/**
 * 限时尝试将信号量的值减 1
 * abs_timeout：绝对时间
 */
int sem_timedwait(sem_t *sem, const struct timespec *abs_timeout);
```

4. V操作

sem_post()(V操作)

```
#include <semaphore.h>
/**
 * 将信号量的值加 1 并发出信号唤醒等待线程(sem_wait()).
 * @param sem 信号量的地址.
 * @return 成功: 0; 失败: -1.
 */
int sem_post(sem_t *sem);
```

5. 获取信号量的值

sem_getvalue()

```
#include <semaphore.h>
/**
 * 获取 sem 标识的信号量的值, 保存在 sval 中.
 * @param sem 信号量的地址.
 * @param sval 保存信号量值的地址.
 * @return 成功: 0; 失败: -1.
 */
int sem_getvalue(sem_t *sem, int *sval);\
```

管程

使用信号量机制实现的生产者消费者问题需要客户端代码做很多控制，
而管程把控制的代码独立出来，不仅不容易出错，也使得客户端代码调用更容易。

下面是经典的IPC问题

1. 哲学家进餐问题

下面是一种错误的解法，如果所有哲学家同时拿起左手边的筷子，那么所有哲学家都在等待其它哲学家吃完并释放自己手中的筷子，导致死锁。

```
#define N 5

void philosopher(int i) {
    while(TRUE) {
        think();
        take(i);       // 拿起左边的筷子
        take((i+1)%N); // 拿起右边的筷子
        eat();
        put(i);
        put((i+1)%N);
    }
}
```

为了防止死锁的发生，可以设置两个条件：

1. 必须同时拿起左右两根筷子
2. 只有在两个邻居都没有进餐的情况下才允许进餐

```
#define N 5
#define LEFT (i + N - 1) % N // 左邻居
#define RIGHT (i + 1) % N    // 右邻居
#define THINKING 0
#define HUNGRY   1
#define EATING   2
typedef int semaphore;
int state[N];                // 跟踪每个哲学家的状态
semaphore mutex = 1;         // 临界区的互斥，临界区是 state 数组，对其修改需要互斥
semaphore s[N];              // 每个哲学家一个信号量

void philosopher(int i) {
    while(TRUE) {
        think(i);
        take_two(i);
        eat(i);
        put_two(i);
    }
}

void take_two(int i) {
    down(&mutex);
    state[i] = HUNGRY;
    check(i);
    up(&mutex);
    down(&s[i]); // 只有收到通知之后才可以开始吃，否则会一直等下去
}

void put_two(i) {
    down(&mutex);
    state[i] = THINKING;
    check(LEFT); // 尝试通知左右邻居，自己吃完了，你们可以开始吃了
    check(RIGHT);
```

```
    up(&mutex);
}

void eat(int i) {
    down(&mutex);
    state[i] = EATING;
    up(&mutex);
}

// 检查两个邻居是否都没有用餐，如果是的话，就 up(&s[i])，使得 down(&s[i]) 能够得到通知并继续执行
void check(i) {
    if(state[i] == HUNGRY && state[LEFT] != EATING && state[RIGHT] !=EATING) {
        state[i] = EATING;
        up(&s[i]);
    }
}
```

2. 读者-写者问题

允许多个进程同时对数据进行读操作，但是不允许读和写以及写和写操作同时发生。

一个整型变量 count 记录在对数据进行读操作的进程数量，一个互斥量 count_mutex 用于对 count 加锁，一个互斥量 data_mutex 用于对读写的数据加锁。

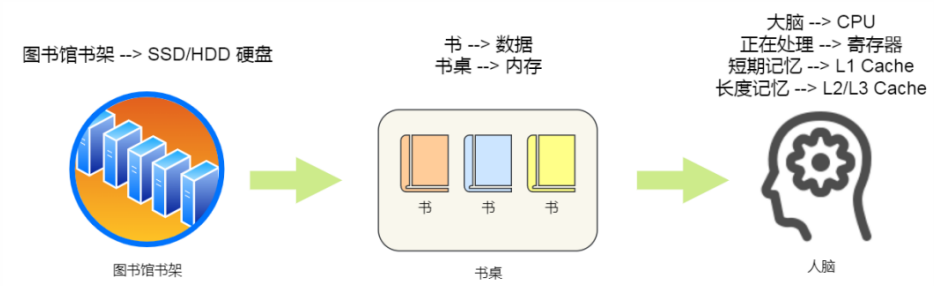
```
typedef int semaphore;
semaphore count_mutex = 1;
semaphore data_mutex = 1;
int count = 0;

void reader() {
    while(TRUE) {
        down(&count_mutex);
        count++;
        if(count == 1) down(&data_mutex); // 第一个读者需要对数据进行加锁，防止写进程访问
        up(&count_mutex);
        read();
        down(&count_mutex);
        count--;
        if(count == 0) up(&data_mutex);
        up(&count_mutex);
    }
}

void writer() {
    while(TRUE) {
        down(&data_mutex);
        write();
        up(&data_mutex);
    }
}
```

存储系统

存储系统层级

思维导图：

1、存储器的层次结构



我们从图书馆书架取书，把书放到桌子上，再阅读书，我们大脑就会记忆知识点，然后再经过大脑思考，这一系列过程相当于，数据从硬盘加载到内存，再从内存加载到 CPU 的寄存器和 Cache 中，然后再通过 CPU 进行处理和计算。

对于存储器，它的速度越快、能耗会越高、而且材料的成本也是越贵的，以至于速度快的存储器的容量都比较小。

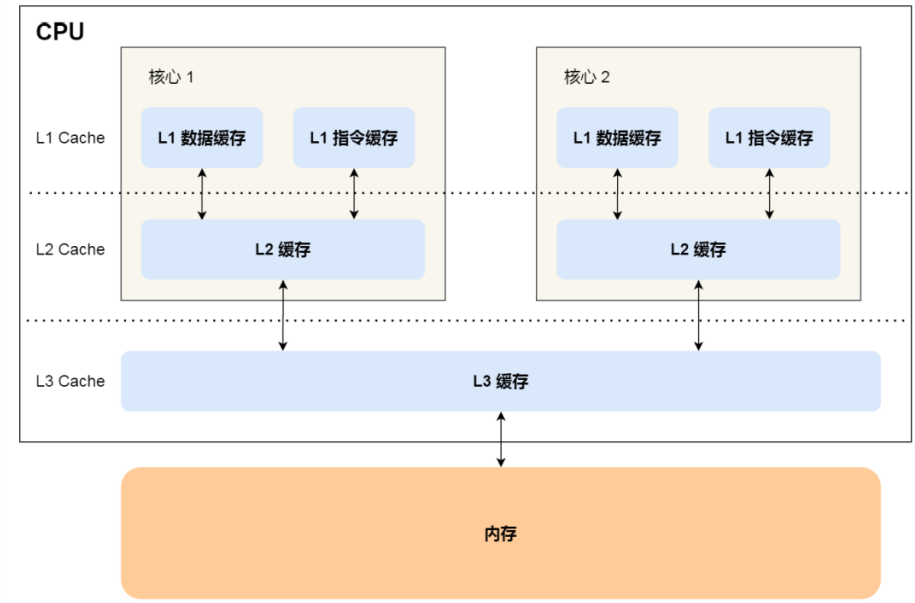
（1）寄存器

寄存器的访问速度非常快，一般要求在半个 CPU 时钟周期内完成读写，CPU 时钟周期跟 CPU 主频息息相关，比如 2 GHz 主频的 CPU，那么它的时钟周期就是 1/2G，也就是 0.5ns（纳秒）。

（2）CPU Cache（用的是一种叫 SRAM（Static Random-Access Memory，静态随机存储器）的芯片）

SRAM 之所以叫「静态」存储器，是因为只要有电，数据就可以保持存在，而一旦断电，数据就会丢失了。

CPU Cache 通常分为三层，如下图所示：



查看各级Cache的大小:

```
(base)  ~$ cat /sys/devices/system/cpu/cpu0/cache/index0/size
32K
(base) jiaoqq@ubuntu-GPU:~$ cat /sys/devices/system/cpu/cpu0/cache/index1/size
32K
(base) jiaoqq@ubuntu-GPU:~$ cat /sys/devices/system/cpu/cpu0/cache/index2/size
1024K
(base) jiaoqq@ubuntu-GPU:~$ cat /sys/devices/system/cpu/cpu0/cache/index3/size
14080K
```

（3）内存

内存用的芯片和 CPU Cache 有所不同，它使用的是一种叫作 DRAM （Dynamic Random Access Memory，动态随机存取存储器） 的芯片。相比 SRAM，DRAM 的密度更高，功耗更低，有更大的容量，而且造价比 SRAM 芯片便宜很多。

DRAM 存储一个 bit 数据，只需要一个晶体管和一个电容就能存储，但是因为数据会被存储在电容里，电容会不断漏电，所以需要「定时刷新」电容，才能保证数据不会被丢失，这就是 DRAM 之所以被称为「动态」存储器的原因，只有不断刷新，数据才能被存储起来。

DRAM 的数据访问电路和刷新电路都比 SRAM 更复杂，所以访问的速度会更慢，内存速度大概在 200~300 个 时钟周期之间。

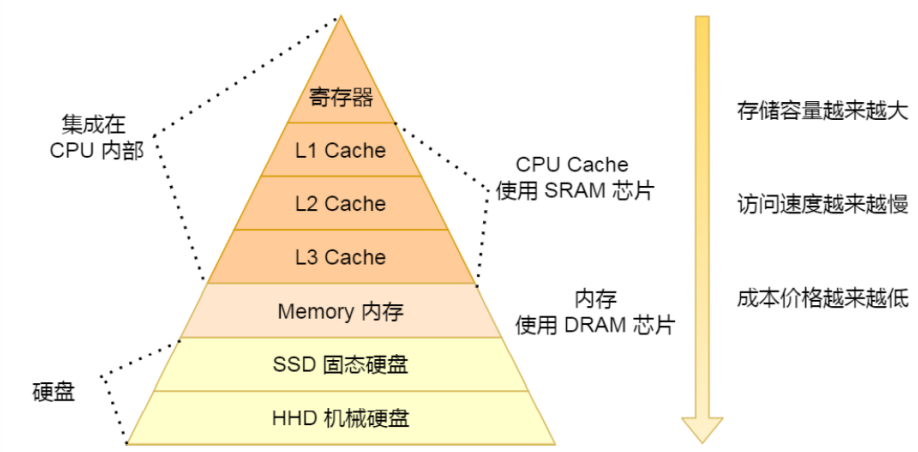
（4）SSD/HDD硬盘:

SSD（Solid-state disk） 就是我们常说的固体硬盘，结构和内存类似，但是它相比内存的优点是断电后数据还是存在的，而内存、寄存器、高速缓存断电后数据都会丢失。内存的读写速度比 SSD 大概快 10~1000 倍。

机械硬盘（Hard Disk Drive, HDD），它是通过物理读写的方式来访问 数据的，因此它访问速度是非常慢的，它的速度比内存慢 10W 倍左右。

2、存储器的层次关系

CPU 并不会直接和每一种存储器设备直接打交道，而是每一种存储器设备只和它相邻的存储器设备打交道。



所以，每个存储器只和相邻的一层存储器设备打交道，并且存储设备为了追求更快的速度，所需的材料成本必然也是更高，也正因为成本太高，所以 CPU 内部的寄存器、L1\L2\L3 Cache 只好用较小的容量，相 反内存、硬盘则可用更大的容量，这就我们今天所说的存储器层次结构。

3、存储器之间的实际价格和性能差距

不同层级的存储器之间的成本对比图：

存储器	硬件介质	单位成本（美元/MB）	随机访问延时
L1 Cache	SRAM	7	1ns
L2 Cache	SRAM	7	4ns
Memory	DRAM	0.015	100ns
Disk	SSD（NAND）	0.0004	150μs
Disk	HDD	0.00004	10ms

机械硬盘（HDD)到底有多慢：

- **SSD 比机械硬盘快 70 倍左右；**
- **内存比机械硬盘快 100000 倍左右；**
- **CPU L1 Cache 比机械硬盘快 10000000 倍左右；**

页面置换算法

1、最佳页面置换算法(OPT)

置换在「未来」最长时间不访问的页面,但是实际系统中无法实现，因为程序访问页面时是动态的

我们是无法预知每个页面在「下一次」访问前的等待时间，因此作为实际算法效率衡量标准。

2、先进先出置换算法(FIFO)

顾名思义，将页面以队列形式保存，先进入队列的页面先被置换进入磁盘。

3、最近最久未使用的置换算法（LRU）

根据页面未被访问时长用升序列表将页面排列，每次将最久未被使用页面置换出去。

4、时钟页面置换算法

把所有的页面都保存在一个类似钟面的「环形链表」中，页面包含一个访问位。

当发生缺页中断时，顺时针遍历页面，如果访问位为1，将其改为0，继续遍历，直到访问到访问位为0页面，进行置换。

5、最不常用算法

记录每个页面访问次数，当发生缺页中断时候，将访问次数最少的页面置换出去，此方法需要对每个页面访问次数统计，额外开销。

分段

虚拟内存采用的是分页技术，也就是将地址空间划分成固定大小的页，每一页再与内存进行映射。

如果使用分页系统的一维地址空间，动态增长的特点会导致覆盖问题的出现。

分段的做法是把每个表分成段，一个段构成一个独立的地址空间。

每个段的长度可以不同，并且可以动态增长。

1、纯分段

分段和分页本质上是不同的，页面是定长的而段不是。

优点：

(1) 共享：有助于几个进程之间共享过程和数据。 比如共享库
(2) 保护：每个段都可以独立地增大或减小而不会影响其他的段

2、分段和分页结合

程序的地址空间划分成多个拥有独立地址空间的段，每个段上的地址空间划分成大小相同的页。

这样既拥有分段系统的共享和保护，又拥有分页系统的虚拟内存功能.

3、分段与分页的比较

对程序员的透明性：

分页透明，但是分段需要程序员显式划分每个段。

地址空间的维度：

分页是一维地址空间，分段是二维的。

大小是否可以改变：

页的大小不可变，段的大小可以动态改变。

出现的原因：

分页主要用于实现虚拟内存，从而获得更大的地址空间；分段主要是为了使程序和数据可以被划分为逻辑上独立的地址空间并且有助于共享和保护。

虚拟内存

简述

属于计算机系统内存管理的一种技术，虚拟地址空间构成虚拟内存，它使得应用程序认为自己拥有连续的可用内存空间，但实际上是被分隔的多个物理内存页、以及部分暂时存储在磁盘上的交换分区所构成的。虚拟内存的实现通过硬件异常、硬件地址翻译、主存、磁盘以及内核软件共同完成

1、地址空间：

地址空间是物理内存的抽象，是一个进程可用于寻址内存的一套地址集合。

2、分页：

地址空间被分割成多个块，每一块称作一页或页面(Page)。每一页有连续的地址范围，这些页被映射到连续的物理内存(页框)。

3、页表：

页表的目的是把虚拟页面(虚拟地址)映射为页框(物理地址)。页表给出了虚拟地址与物理地址的映射关系。从数学的角度说页表是一个函数，他的参数是虚拟页号，结果是物理页页框号

4、加速分页：

（1）TLB加速分页

概念：将虚拟地址直接映射到物理地址，而不必再访问页表，这种设备被称为转换检测缓冲区（TLB）、相联存储器或快表

工作过程：将一个虚拟地址放入MMU中进行转换时，硬件首先通过将该虚拟页号与TLB中所有表项同时进行匹配，判断虚拟页面是否在其中：

1. 虚拟页号在TLB中。如果MMU检测一个有效的匹配并且访问操作并不违反保护位，则将页框号直接从TLB中取出而不必访问页表。
2. 虚拟页号不在TLB中。如果MMU检测到没有有效的匹配项就会进行正常的页表查询。接着从TLB中淘汰一个表项，然后用新的页表项替换它。

（2）软件TLB管理

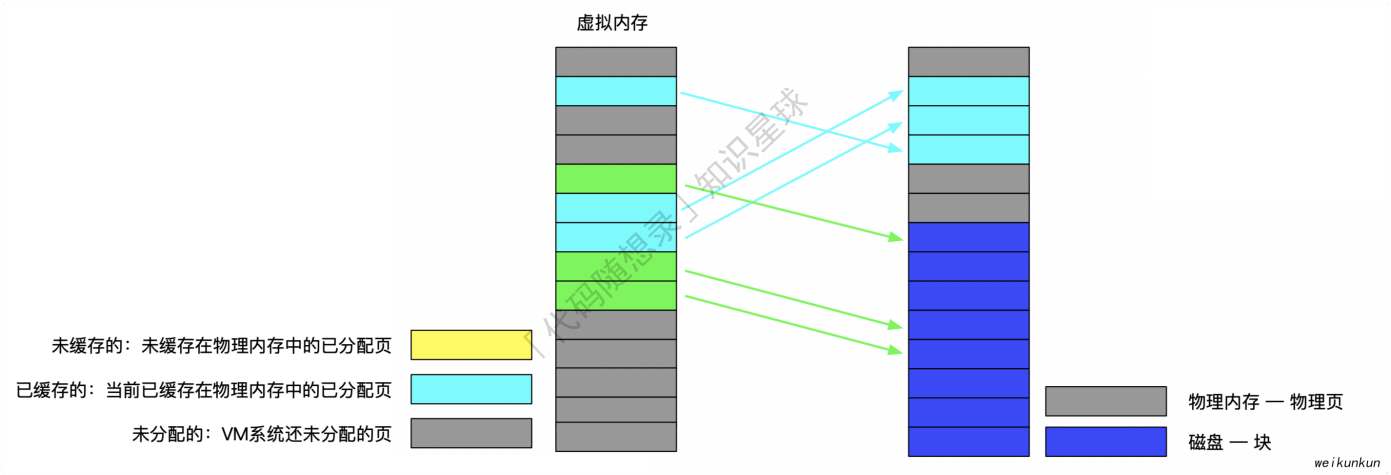
5、针对大内存的页表

（1）多级页表
（2）倒排页表

三个重要的能力：

1. 高速缓存

按照存储器层次结构中的缓存划分来看，将位于k层的更小更快的存储设备作为存储在k+1层的更大更慢的存储设备的缓存。具体如下：



在虚拟内存中的虚拟页共分为三种类型：

1. 未分配（没有被进程申请使用的，也就是空闲的虚拟内存， 不占用虚拟内存磁盘的任何空间）
2. 未缓存（仅仅加载到磁盘中的页）
3. 已缓存（已经加载到内存中的内存页（页框））

2. 内存管理

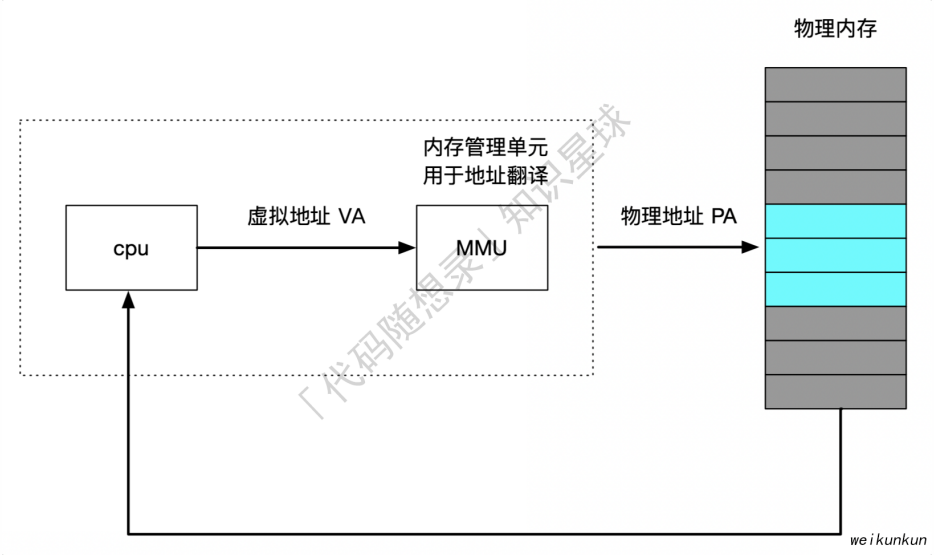
3. 内存保护

现代操作系统中，用户进程不应该被允许修改它的只读代码段;

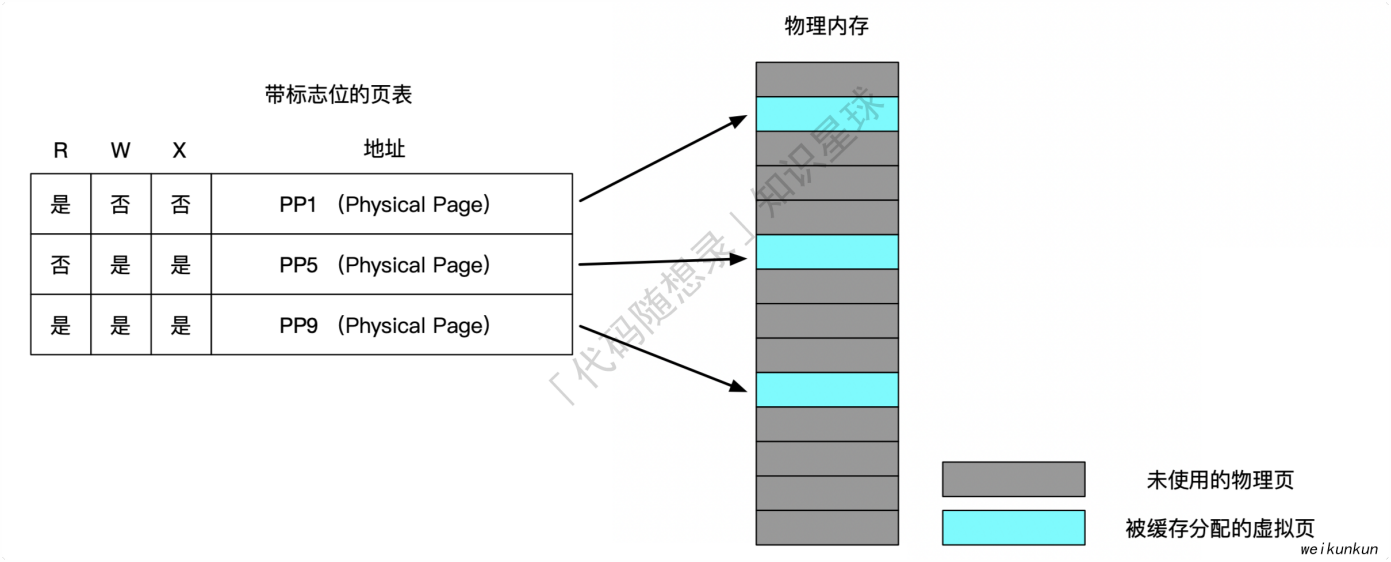
而且也不应该允许它读取或修改任何内核中的代码和数据结构;

并且也不允许其读取或者修改其他进程的私有内存，以及修改和其他进程共享的虚拟页面。

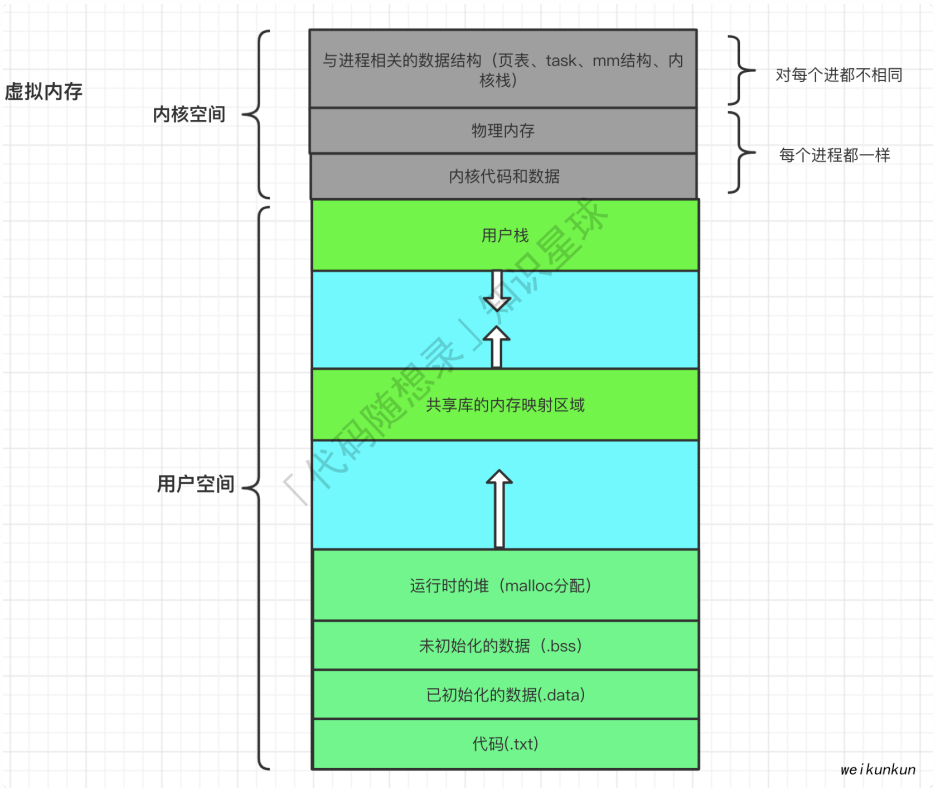
如果不对进程的内存访问进行限制，攻击者就能够访问和修改其他进程的私有内存，进而导致整个系统崩溃。大体的虚拟寻址如下：



通过MMU，每次都会读取页表中的一个页表条目(PTE)，通过在这些页表条目(PTE)中添加一些标志位，就能够实现对一个虚拟页的访问控制权限。譬如：



相关的其他概念



1. 用户态和内核态

为了使操作系统内核提供一个无懈可击的进程抽象，处理器必须提供一种机制，限制一个应用可以执行的指令以及他可以用来访问的地址空间范围。
处理器通常是用某个控制寄存器中的一个模式位来提供这种观功能的，该寄存器描述了进程当前享有的特权。

当设置了模式位时：
进程就运行在内核态中。
运行在内核态中的进程可以执行指令集中的任何指令，并且可以访问系统中的任何内存位置。

没有设置模式位时：
进程就运行在用户态。用户模式中的进程不允许执行特权指令。

比如：停止处理器、改变模式位、或者发起一个I/O操作。同时也不允许用户态下的进程直接引用地址空间中内核空间的代码和数据。
任何这样的尝试都会导致致命的保护故障。用户态下的程序必须通过系统调用接口间接的访问内核代码和数据。

2. 上下文切换 '

上下文切换属于一种较高层形式的异常控制流，进行上下切换涉及到的内容如下：

（1）各种寄存器，TLB 也是其中之一。

（2）程序计数器

（3）用户栈

（4）内核栈

（5）内核中的各种数据结构

1. 页表
2. task_struct
3. mm_struct

小总结

虚拟内存的思想，整体来看就是：**通过结合磁盘和内存各自的优势，利用中间层对资源进行更合理地调度，充分提高资源的利用率。并提供和谐以及统一的抽象。**

为什么需要虚拟内存：（摘自 为什么Linux需要虚拟内存一文）

1. 虚拟内存可以结合磁盘和物理内存的优势为进程提供看起来速度足够快并且容量足够大的存储
2. 虚拟内存可以为进程提供独立的内存空间并引入多层的页表结构将虚拟内存翻译成物理内存，进程之间可以共享物理内存减少开销，也能简化程序的链接、装载以及内存分配过程;
3. 虚拟内存可以控制进程对物理内存的访问，隔离不同进程的访问权限，提高系统的安全性。

文件系统

文件

1. 文件

文件是一种抽象机制，它提供了一种在磁上保存信息而且方便以后读取的方法。这种方法可以使用户不必了解存储信息的方法、位置和实际磁盘工作方式等有关细节

2. 文件命名

win95、win98用的都是MS-DOS的文件系统，即FAT-16， win98扩展了FAT-16成为FAT-32。

较新版的操作系统NTFS,win8配备ReFS。微软优化FAT,叫作exFAT。prog.c，圆点后面的部分称为文件扩展名。

3. 文件结构

1、字节结构

把文件看成字节序列为操作系统提供了最大的灵活度

2、记录序列

文件结构上的第一步改进，这种模型中，文件是具有固定长度记录的序列

3、树

文件在这种结构中由一棵记录树构成，每个记录不必具有相同的长度，记录的固定位置上有一个键字段。这棵树按“键”字段进行排序，从而可以对特定“键”进行快速查找。

4. 文件类型

1. 普通文件
2. 目录
3. 字符特殊文件（UNIX）
4. 块特殊文件（UNIX）

5. 文件访问

1、顺序访问

按顺序读取文件的全部字节，早期操作系统只有这种访问方式

2、随机访问文件

当用磁盘存储文件时，可以以任何次序读取其中字节或记录的文件。许多应用程序需要这种类型文件

6. 文件属性

除了文件名和数据外，所有操作系统还会保存其他的文件相关信息，如创建日期、时间和大小等，这些附加的信息称为文件属性

举例：

创建者：创建文件者ID
所有者：当前所有者
当前大小：文件字节数
…：…

7. 文件操作

使用文件的目的是**存储信息并方便以后检索**。对于存储和检索，不同系统提供了不同的操作。

常见的文件操作（系统调用）：

1、create

创建不包含任何数据的文件

2、delete

当不再需要某个文件时，必须删除该文件以释放磁盘空间

3、open

在使用文件之前，必须先打开文件

4、close

访问结束后，不再需要文件属性和磁盘地址，这时应该关闭文件以释放内部表空间

5、read

在文件中读取数据

6、write

向文件写数据，写操作一般也是从文件当前位置开始

7、append

此调用是write的限制形式，他只能在文件末尾添加数据

8、seek

对于随机访问文件，要指定从何处开始获取数据，通常的方法是用seek系统调用把当前位置指针指向文件中特定的位置。

9、get attributes

进程运行常需要读取文件属性

10、set attributes

某些属性是可由用户设置的，甚至在文件创建之后

11、rename

用户尝尝要改变已有的名字，rename系统调用用于这一目的

目录

文件系统通常提供目录或文件夹用于记录文件的位置，在很多操作系统中目录本身也是文件

1. 一级目录系统

在一个目录中包含所有文件，这有时称为根目录

2. 层级目录系统

当用户有着数以千计的文件，为了寻找方便。需要层次结构（即一个目录树）

3. 路径名

1、绝对路径名

它由从根目录到文件的路径组成。

Windows： `\usr\ast\maibox`
UNIX： `/usr/ast/mailbox`

路径名的第一个字符是分隔符，则这个路径是绝对路径

2、相对路径名

它常和工作目录(working directory)(也和当前目录(acurrent directory))一起使用

例：UNIX

```
cp /usr/ast/mailbox /usr/ast/mailbox/mailbox.bak
```

和命令

```
cp mailbox mailbox.bak # 具有相同的含义
```

3、特殊目录项

`.` dot：指当前目录
`..` dotdot 指其父目录

eg:

```
cp ../lib/dictionay .
```

将usr/lib/下的dictionay复制到当前目录下

4. 目录操作

管理目录的系统调用

举例(Unix):

```
create      # 创建目录
delete      # 删除目录
opendir     # 目录内容可被读取
closedir    # 读目录结束
readdir     # 返回打开目录下一个目录项
rename      # 改变目录名
link        # 链接技术允许在多个目录中出现同一个文件
unlink      # 删除目录项
```

文件系统的实现

文件存储实现的关键问题是记录各个文件分别用到哪些磁盘块。不同的操作系统用到不同的方法

1. 文件系统布局

文件系统存放在磁盘上。多数磁盘划分为一个或多个分区，每个分区中国有一个独立的文件系统。

磁盘0号扇区称为主引导记录(MBR)，用来引导计算机。在MBR结尾是分区表。该表给出每个分区的起始结束地址。

表中的一个分区被标记为活动区。在计算机被引导时，BIOS读入并执行MBR。MBR做的第一件事是确定活动分区，读入它的第一个块，称为引导块，并执行之。

引导块中的程序将装载该分区中的操作系统。

2. 文件的实现

1、连续分配

最简单的分配是把每个文件作为一连串连续数据块存储在磁盘

优势:

（1）实现简单

记录每个文件用到磁盘块简化为只需记住两个数字即可：第一块的磁盘地址和文件的块数。给定了第一块编号，一个简单的加法就可以找到任何其他块的编号

（2）操作性能较好

因为单个操作中就能从磁盘上读出整个文件。只需一次寻找。

2、链表分配

为每个文件构造磁盘块链表。每一块的第一个字作为指向下一块的指针，块的其他部分存放数据。

优势：

（1）可以充分利用磁盘块，不会因为磁盘碎片浪费存储内存
（2）随机访问快。

缺点：

指针占用一些字节，每个磁盘块存储数据的字节数不再是2的整数次幂。怪异的大小降低了系统的运行效率，每个块前几个字节被指向下一个块的指针所占据，需要从两个磁盘中获取拼接信息，这就因复制而引发额外的开销。

3、采用内存中的表进行链表分配

取出每个磁盘块的指针字，把他放在内存的一个表中，解决链表分配的不足。内存中这样的表格称为文件分配表(File Allocation Table，FAT)

4、i节点

最后一个记录各文件分别包含哪些磁盘块的方法是给每个文件赋予一个称为i节点的数据结构，其中列出了文件属性和文件的磁盘地址。

3. 目录的实现

在读文件之前，必须先打开文件。打开文件时，操作系统利用用户给出的路径名找到相应的目录项。

简单目录：包含固定大小的目录，在目录项中有磁盘地址和属性

采用i节点的系统：把文件属性存放在i节点中而不是目录项中。这种情形下，目录项会更短。

4. 共享文件

当几个用户同在一个项目里工作时，他们常常需要共享文件。

共享文件与目录的联系称为一个链接（link）。这样文件系统本身就是一个有向无环图（DAG），而不是一棵树。

5. 日志结构文件系统

CPU运行速度越来越快，磁盘容量越来越大，价格越来越便宜（但磁盘速度并没有增快多少），同时内存容量也以指数形式增长。

而没有得到快速发展的参数是磁道的寻道时间。这些成为了文件系统性能的瓶颈，为了解决这一问题设计了全新的文件系统即日志结构文件系统（LFS）。

虽然是一个很吸引人的想法，由于它们和文件系统不匹配，该文件系统并没有被广泛应用。

6. 日志文件系统

基本的想法是保存一个用于记录系统下一步将要做什么的日志。

这样当系统在完成他们即将完成的任务前崩溃时，重新启动后，可以通过查看日志，获取崩溃前计划完成的任务，并完成它们。

这样的文件日志系统，并已经被实际应用。微软的NTFS、Linux的 ext3和ReiserFS文件系统都使用日志。

7. 虚拟文件系统

将多个文件系统整合到一个统一的结构中。

一个Linux系统可以用ext2作为根文件系统，ext3分区装载在/usr下，另一块采用ReiserFS文件系统的硬盘装载在/home下，以及一个ISO 9660的CD-ROM临时装载在/mnt下。从用户的观点来看，只有一个文件系统层级。

它们事实上是多种文件系统，对于用户和进程是不可见的。绝大多数Unix操作系统都在使用虚拟文件系统（Virtual File System, VFS）

文件系统的管理和优化

1. 磁盘空间管理

几乎所有文件系统都把文件分割成固定大小的块来存储，各块之间不一定相邻

1、块大小

从历史的观点上来说，文件系统将大小设在1~4KB之间，但现在随着磁盘超过了1TB，还是将块的大小提升到64KB并且接受浪费的磁盘孔空间，这样也许更好。磁盘空间几乎不再会短缺。

2、记录空闲块

（1）磁盘块链表

链表的每个块中包含尽可能多的空闲磁盘块号。
通常情况下，采用空闲块存放空闲表，这样不会影响存储器

（2）位图

在位图中，空闲块用1表示，已分配块用0表示。

（3）磁盘配额

为防止人们贪心而占有太多的磁盘空间，用户操作系统常常提供一种强制性磁盘配额机制。
其思想是系统管理员分给每个用户拥有文件和块的最大数量，操作系统确保每个用户不超过分给他们的配额。（配额表、打开文件表）

2. 文件系统备份

做磁盘备份主要是处理好两个潜在问题中的一个

1. 从意外的灾难中恢复
2. 从错误的操作中恢复

3. 文件系统的一致性

很多文件系统读取磁盘块，进行修改后，再写回磁盘。

如果在修改过的磁盘块全部写回之前系统崩溃，则文件系统有可能处于不一致状态。

如果一些未被写回的块是i节点块、目录块或者是包含有空闲表的块时，这个问题尤为严重

4. 文件系统性能

1、高速缓存

减少磁盘访问次数技术是块高速缓存（block cache）或者缓冲区高速缓存（buffer cache）。

本书中，高速缓存指的是一系列的块，它们在逻辑上属于磁盘，但实际上基于性能的考虑被保存在内存中。

2、块提取读

在需要用到块之前，试图提前将其写入高速缓存，从而提高命中率。

块提前读策略只适用于实际顺序读取的文件。对随机访问文件，提前读丝毫不起作用。

3、减少磁盘臂运动

把可能顺序访问的块放一起，当然最好是同一柱面上，从而减少磁盘臂的移动次数。

5. 磁盘碎片整理

移动文件使它们相邻，并把所有的空闲空间放在一个或多个大的连续区域内。

磁盘

生磁盘的使用

生磁盘：根据**盘块号**来使用磁盘；
熟磁盘：根据**文件**来使用磁盘。

磁盘如何使用？（仍然是 3 件事）

1. 发送 out 指令；
2. 文件视图；
3. 发送 中断

磁盘的结构

盘面：类似于多个堆叠在一起的 DVD 光盘
磁道：每个 DVD 光盘上的同心圆
扇区：每个同心圆上的 一段儿 小区域

磁盘 IO 过程

寻道：将磁头移动到 指定 磁道
旋转：将磁头旋转到磁道中的指定 扇区
传输：对指定扇区 进行 磁盘数据 IO 读写

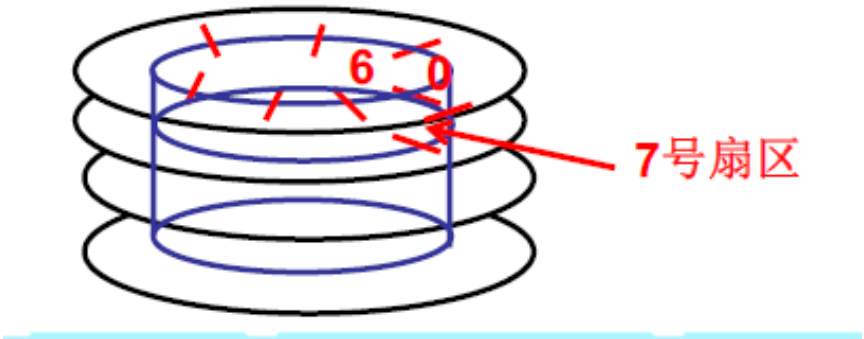
一层抽象

通过盘块号读写磁盘

对于用户来说，如果直接利用 盘/柱面 C、磁头 H、扇区 S 来进行 磁盘 IO 不太友好，所以通过一层抽象，即 用户只用利用 盘块号 block 来进行 磁盘IO，然后由系统根据 block 计算出 C、H、S 即可

磁盘编址

由于寻道时间 通常时间更长一些，所以，为了让 磁盘 IO 时间尽可能短一些，则需要使 寻道时间尽量段。即，把当一个 磁道编址用完后，向下一个 柱面 继续编号，而不是向 下一个 磁道继续编号。



计算公式：盘块号 block = C *(Heads* Sectors) + H * Sectors + S

二层抽象

多个进程通过队列使用磁盘

虽然一层抽象（利用 盘块号）可以达到使用磁盘的目的，但是如果有多个磁盘访问请求出现在请求队列怎么办？此时，操作系统会根据 磁盘调度算法 来挑选一个 磁盘请求服务

常见磁盘调度算法

1、FCFS：找最先来的请求

优点：公平
缺点：并不提供 最快的服务

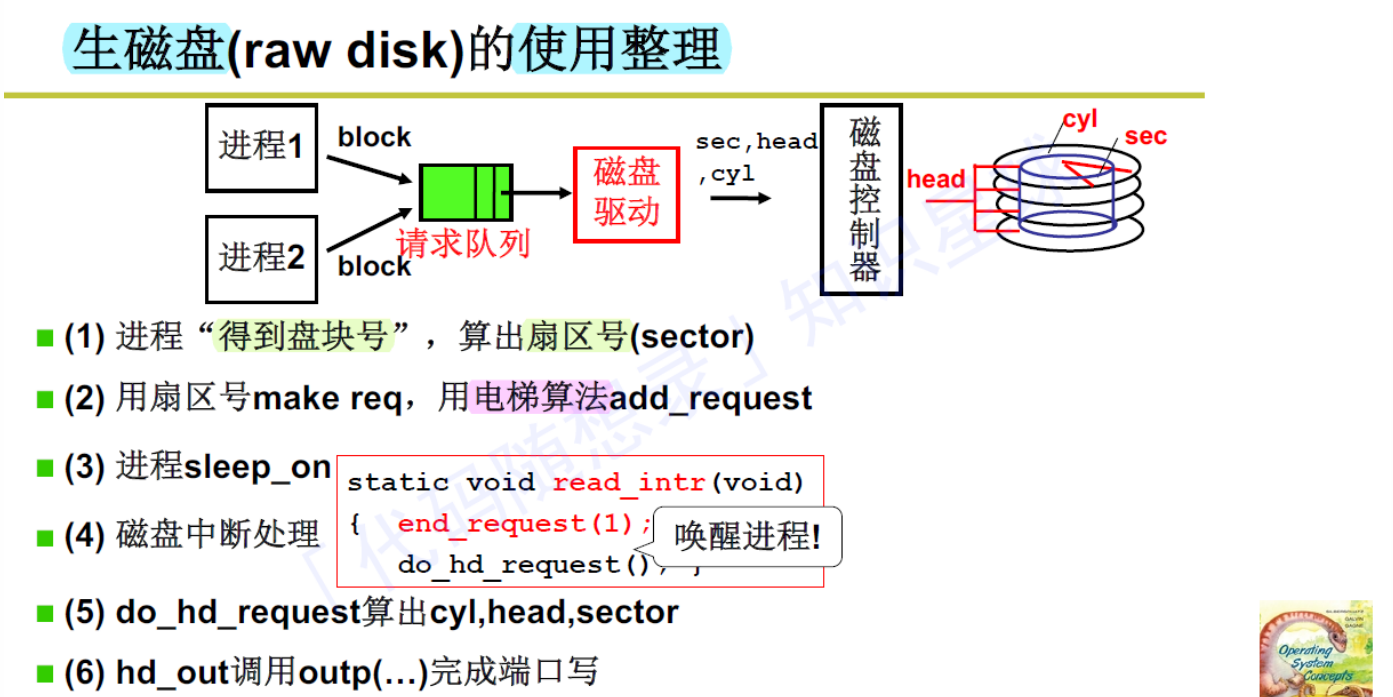
FCFS 在寻道过程中，可能已经遇到一些以后可能需要访问的 磁道，但是会跳过，而造成 访问磁道 耗费时间较多。

2、SSTF：类似于 SJF，每次选择距离当前磁头最近的待处理请求

优点：寻道时间最短
缺点：可能造成部分请求“饥饿”（当某个请求的磁盘距离磁头较远，而一直有比其更近的请求时，这个请求一直无法执行）

3、SCAN 算法：类似电梯，先处理向上的请求，然后再处理反方向的请求

优点：不会造成部分请求的“饥饿”



posper

三层抽象

利用**文件**使用磁盘，就成了熟磁盘

让普通用户使用 raw disk： 许多人连扇区都不知道是什么? 要求他们根据盘块号来访问磁盘不太便利
需要在盘块上引入更高一层次的抽象概念! 文件

用户眼里的“文件”：一堆字符序列（字符流）
磁盘上的“文件”：多个盘块构成的集合
所以，如果要**利用文件来进行磁盘IO**的话，就需要建立字符流到盘块集合的映射关系

常见的文件映射关系（文件实现方式）有以下 3 种：

1. 顺序结构
2. 链式结构
3. 索引

顺序结构-实现文件

FCB：记录文件在磁盘中的起始块

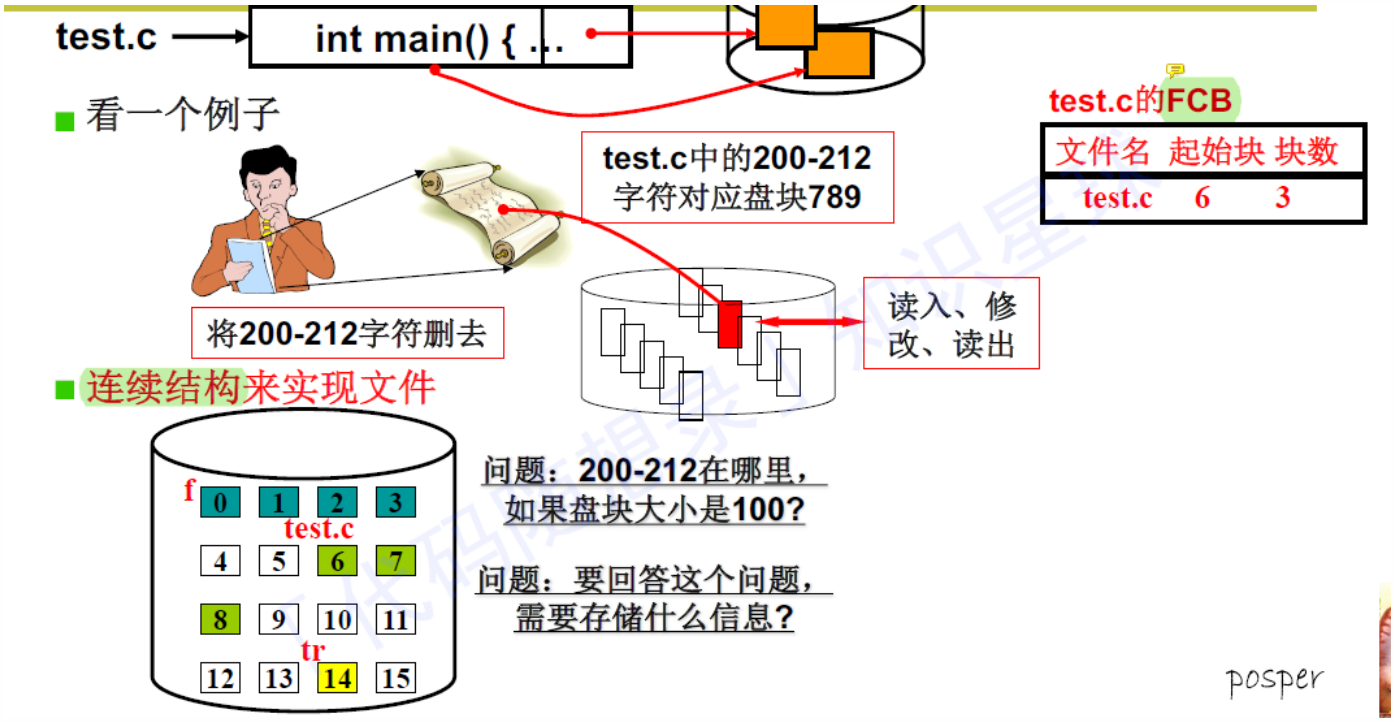
文件->盘块号

如何根据 文件 找到 盘块号?

将文件按照 连续结构存放；（下 左图）；
对每个文件建立一个 FCB，记录该文件在磁盘中的 起始块号；
根据文件（字符流）中的第 n 个待查字符 + FCB 计算 （n / 每个盘块的大小 + FCB中的 起始块号】出其在磁盘上对应的 盘块号 【得到盘块号，然后采用 SCAN 算法]

这种方法的缺点：

1. 利用顺序结构存储文件，不利用文件的修改（因为 顺序存储的 增/删 代价非常大）
2. 所以，当需要 动态修改 文件时，最好使用 链式存储文件



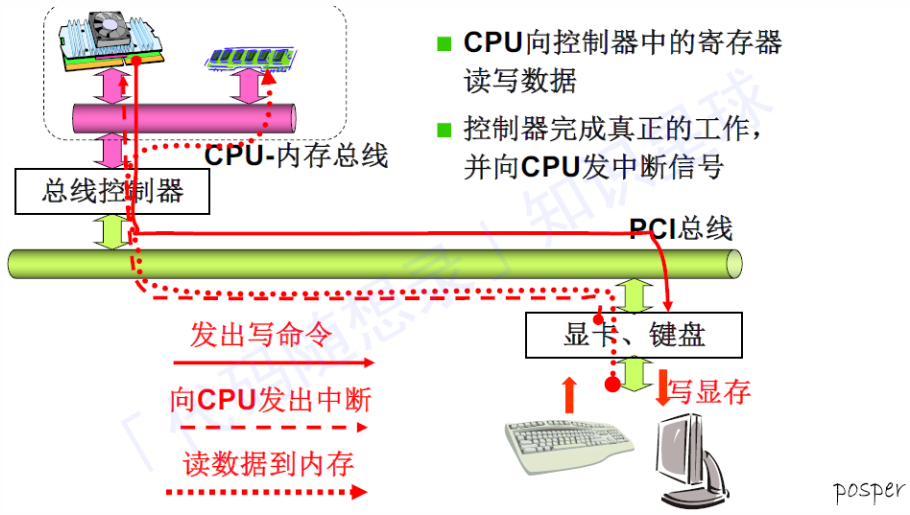
链式结构-实现文件

优点：利于 增/删
缺点：不利于查询

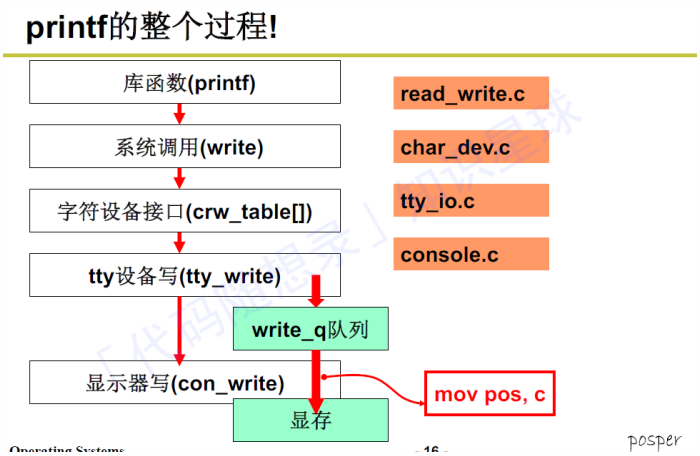
外设管理

如何让外设动起来

1. CPU 向 外设的控制器发送指令，即 out 指令
2. 形成 文件视图（为了统一 out 指令的形式）
3. 中断（外设处理完事后，需要通知 cpu 继续接手 下一步处理）

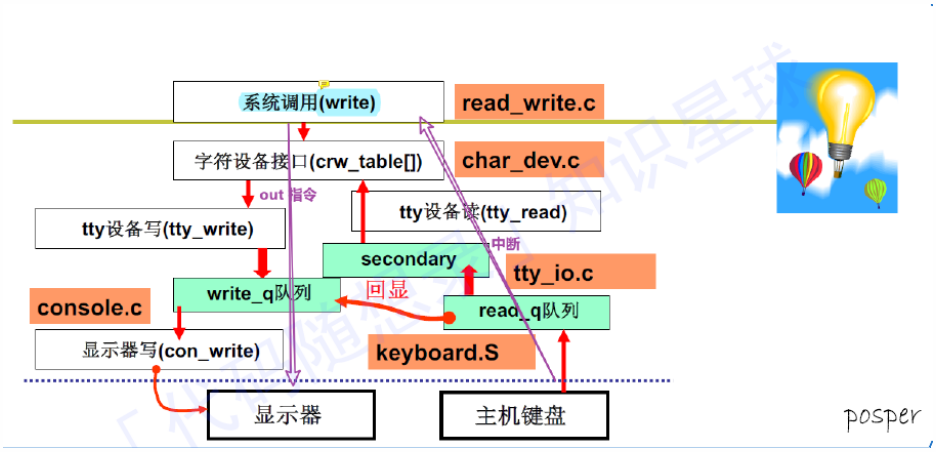


显示器如何工作？即，printf 函数的工作流程



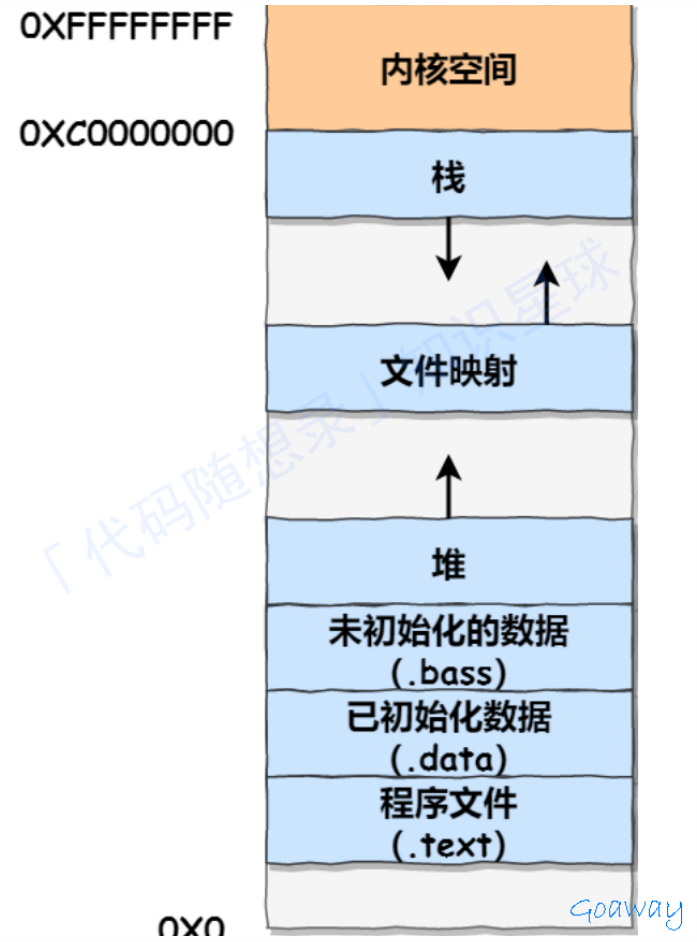
键盘如何工作?

中断处理（根据扫描码 获取 对应的 ascii 码）；
将对应的 ascii 码 加入 缓冲队列 read_que 中，等待上层程序调用



Linux

Linux 内存管理



1、程序文件段

包括程序的二进制可执行代码，只读；TEXT

2、已初始化数据段

包括已初始化的全局变量和静态常量；DATA

3、未初始化数据段

包括未初始化的静态变量和全局变量；BSS

4、堆段

包括动态分配的内存，从低地址开始向上增长；

5、文件映射段

包括动态库、共享内存等，从低地址开始向上增长（跟硬件和内核版本有关）；

6、栈段

包括函数的参数值和局部变量、函数调用的上下文等。栈的大小是固定的，一般是 8 MB 。当然系统也提供了参数，以便我们自定义大小；栈区是从高地址位向低地址位增长的

Linux虚拟内存

对32位处理器，虚拟内存空间为4G，每个进程都认为自己拥有4G的空间，实际上，在虚拟内存对应的物理内存上，可能只对应的一点点的物理内存。

进程得到的这4G虚拟内存是一个连续的地址空间（这也只是进程认为），而实际上，它通常是被分隔成多个物理内存碎片，还有一部分存储在外部磁盘存储器上，在需要时进行数据交换。

由于存在两个内存地址，因此一个应用程序从编写到被执行，需要**进行两次映射**。第一次是**映射到虚拟内存空间**，第二次时**映射到物理内存空间**。在计算机系统中，第两次映射的工作是由硬件和软件共同来完成的。承担这个任务的**硬件部分叫做存储管理单元 MMU**，**软件部分**就是操作系统的**内存管理模块**了。

1、如何处理虚拟地址和物理地址的关系

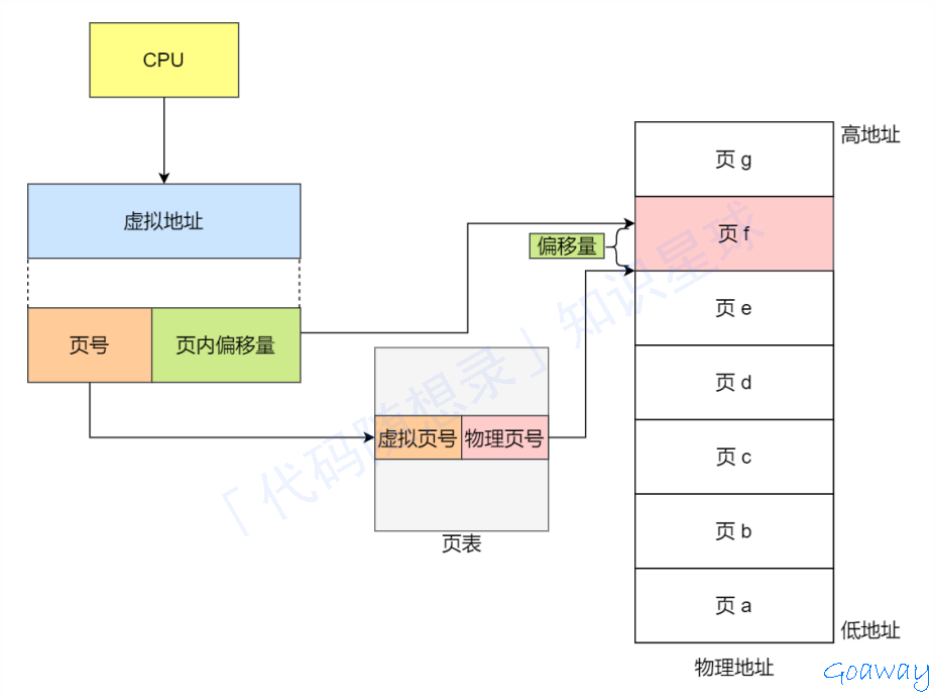
（1）内存分页

分页就是把整个虚拟和物理内存切成一段段固定大小的空间，连续且尺寸固定的内存空间叫页，Linux下每一页大小4KB。虚拟内存和物理内存之间通过页表来映射。

当进程访问的虚拟地址在页表中查不到时，系统会产生一个缺页异常，进入系统内核空间分配物理内存、更新进程页表，最后再返回用户空间，恢复进程的运行。

1. 采用了分页，那么释放的内存都是以页为单位释放的，也就不会产生无法给进程使用的小内存，解决了内存碎片的问题
2. 内存空间不足时，操作系统将正在运行的进程中，最近没使用的内存页面释放（暂时写入硬盘）需要的时候再加载进来，一次性只有少数的页，解决了交换效率低的问题
3. 分页使我们在加载程序时，不用一次性加载到物理内存，可以只有在程序运行中，需要用到对应虚拟内存页里面的指令和数据时，再加载到物理内存里面去

2、分页虚拟地址和物理地址是如何映射的



虚拟地址分为：页号和页内偏移。

页号作为页表的索引，页表包含物理页每页所在物理内存的基地址，这个基地址与页内偏移的组合就形成了物理内存地址。

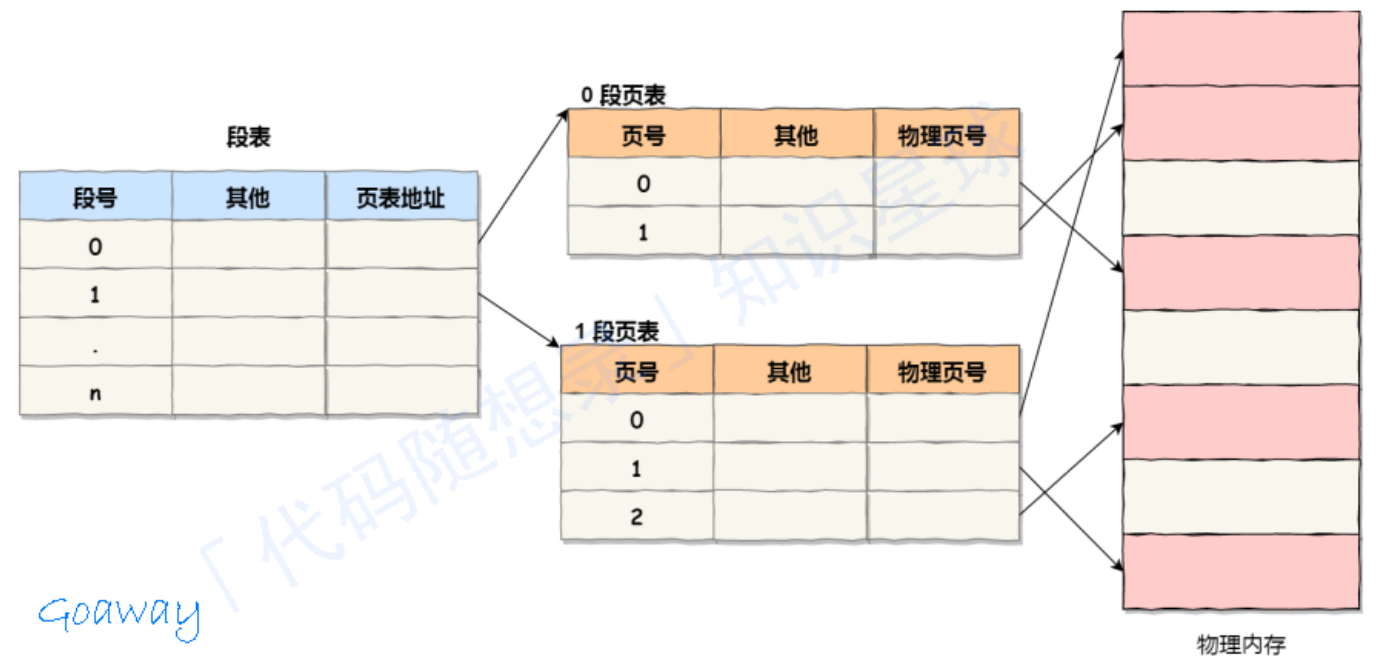
3、简单的分页缺陷

空间上的缺陷：

32位，单进程一个页4KB，虚拟内存4GB，就有2^20个页，一个页表项4字节，结果是4GB空间映射需要4*2^20 = 4MB存储页表。多进程100，需要400MB。

以页表一定要覆盖全部虚拟地址空间，不分级的页表就需要有 100 多万个页表项来映射，二级分页则只需要 1024 个页表项（此时一级页表覆盖到了全部虚拟地址空间，二级页表在需要时创建）。

4、段页式内存管理



地址结构就由段号、段内页号和页内位移三部分组成。

1. 第一次访问段表，得到页表起始地址
2. 第二次访问页表，得到物理页号
3. 第三次将物理页号与页内位移组合，得到物理地址

可用软、硬件相结合的方法实现段页式地址变换，这样虽然增加了硬件成本和系统开销，但提高了内存的利用率。

Linux 信号

信号是用户、系统、进程发送给目标进程的信息，通知某个状态的改变或系统异常。

linux常用信号：

1. SIGHUP控制终端挂起
2. SIGPIPE向读端关闭的通道或socket连接中写数据
3. SIGURG：socket连接上收到紧急数据

Linux设计

1、MutiTask 多任务

多个任务同时执行，同时可以是并发或并行。

2、SMP 对称多处理

每个CPU地位相等，使用权限相同，多个CPU共享一个内存，每个 CPU 都可以访问完整的内存和硬件资源。

3、ELF 可执行文件链接格式

Linux中可执行文件的存储格式。

4、Monolithic Kerne宏内核的特征是系统内核的所有模块

比如进程调度、内存管理、文件系统、设备驱动等，都运行在内核态。Linux 的内核是一个完整的可执行程序，且拥有最高的权限。

IO复用

1. I/O(数据交换过程)

1、背景

I/O即Input/Output（输入和输出），由于程序和运行时数据是在内存中驻留，由CPU这个超快的计算核心来执行，涉及到数据交换的地方，通常是磁盘、网络等，就需要IO接口。

IO编程中涉及到流，其是相对于内存而言的，所以，Input Stream就是数据从外面（磁盘、网络）流进内存，Output Stream就是数据从内存流到外面去。

2、概念

I/O操作就是在运行代码的过程中，可能需要对文件的读写，即将文件输入（Input）到内存和将代码执行结果产生的文件输出（Output）到外设（网络、磁盘）的过程。

3、分类

1. 网络I/O：通过网络进行数据的拉取和输出;
2. 磁盘I/O：主要是对磁盘进行读写工作。

2. 五种IO模型

1、输入阶段一般操作

1. 等待数据准备好
2. 从内核向进程复制数据

2、例如：

1. 等待数据从网络中到达：所等分组到达时，被复制到内核中的某个缓冲区
2. 把数据从内核缓冲区复制到应用进程缓冲区

3、分类

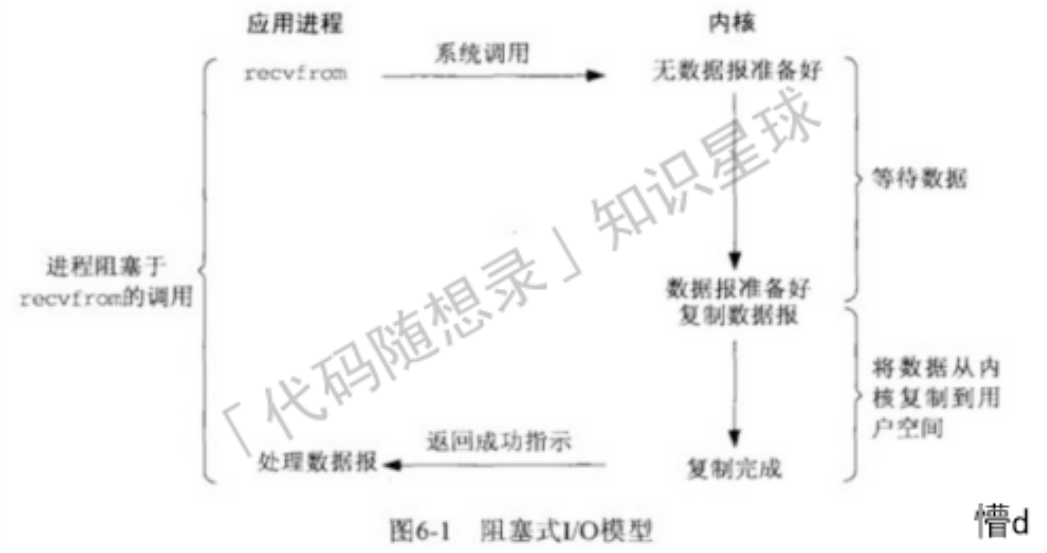
1. 阻塞式I/O
2. 非阻塞式I/O
3. I/O复用（select和poll）
4. 信号驱动式I/O（SIGIO）
5. 异步I/O（POSIX的aio_系列函数）

4、同步I/O

（1）阻塞式I/O

进程或线程等待某个条件，如果条件不满足，则一直等下去。条件满足，则进行下一步操作。（默认情况下）

应用进程通过系统调用 recvfrom 接收数据，但由于内核还未准备好数据报，应用进程就会阻塞住，直到内核准备好数据报，recvfrom 完成数据报复制工作，应用进程才能结束阻塞状态。



「代码随想录」知识星球

图6-1　阻塞式I/O模型

槽d

优点：

设备文件不可操作时，可进入休眠状态，将CPU资源让出；当设备文件可以操作的时候就必须唤醒进程，一般在中断函数中完成唤醒工作;

缺点：

耗费时间，适合并发低，时效性要求低的情况。

（2）非阻塞式I/O

应用进程与内核交互，目的未达到之前，不再一味的等着，而是直接返回;

通过轮询的方式，不停的去问内核数据有没有准备好。如果某一次轮询发现数据已经准备好了，那就把数据拷贝到用户空间中。

前三次调用 recvfrom 时没有数据可返回，内核立即返回 EWOULDBLOCK 错误

第四次调用 recvfrom 时，已有一个数据报准备好，将其从内核复制到应用进程缓冲区， recvfrom 成功返回;

轮询（polling）：

应用进程持续轮询内核，以查看某个操作是否就绪;

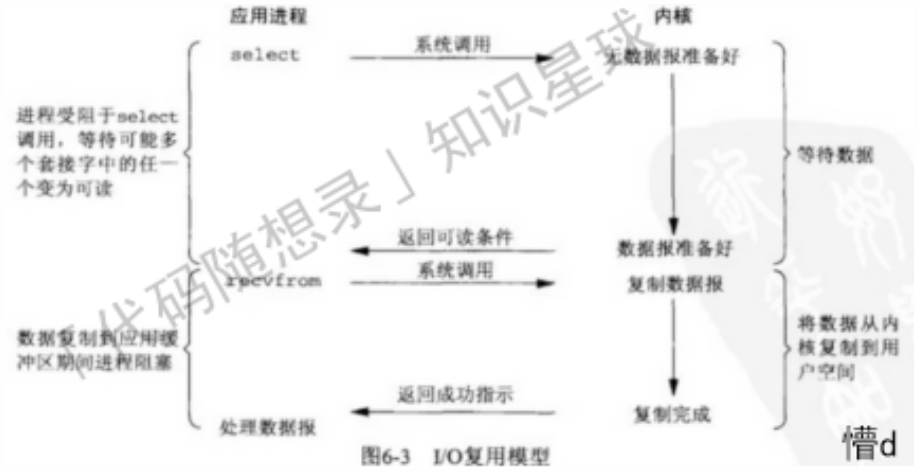
缺点：往往耗费大量CPU时间。

（3）IO复用

通过调用 select 或 poll，阻塞在这两个系统调用中的某一个之上，而不是阻塞在真正的I/O系统调用上。

阻塞于 select 调用，等待数据报套接字变为可读，当 select 返回套接字可读这一条件时，调用 recvfrom 把所读数据报复制到应用进程缓存区。

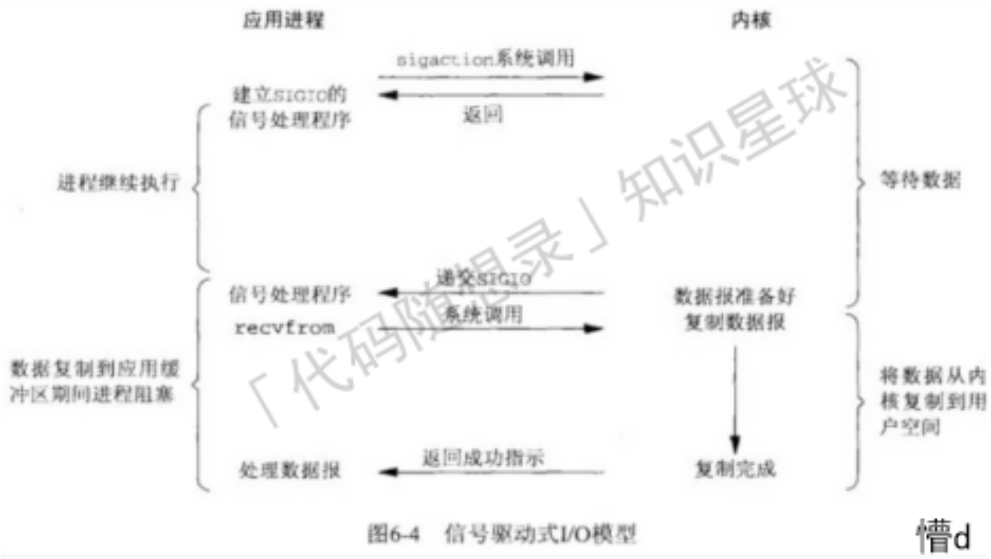
多个进程I/O注册到同一个 select，当用户进程调用 select，select 监听所有注册好的IO：



1. 若所有被监听的I/O需要的数据未准备好，则阻塞
2. 任意一个所需数据准备好后，select 调用返回
3. 用户进程通过 recvfrom 进行数据拷贝

优点：可以等待多个描述符就绪。

（4）信号驱动I/O



开启套接字信号驱动I/O功能，通过 sigaction 系统调用安装一个信号处理函数，该系统函数立即返回，不阻塞;

数据报准备好后，内核为该进程产生一个 SIGIO 信号递交给进程;

可以在信号处理函数中调用 recvfrom 读取数据报，通知主循环数据已准备好待处理;

可以立即通知主循环，读取数据报。

优点:

等待数据报到达期间进程不被阻塞。主循环可以继续执行，等待来自信号处理函数的通知：既可以是数据已经准备好被处理，也可以是数据报已经准备好被读取。

（5）异步IO

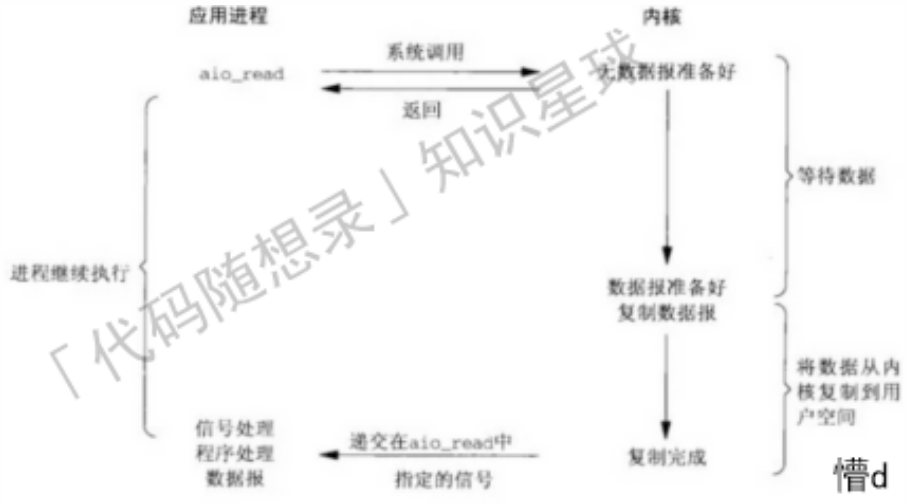
用户进程告知内核启动某个操作，并由内核在整个操作中完成后通知用户进程。

与信号驱动I/O的区别:

1. 信号驱动I/O是由内核通知我们何时可以启动一个I/O操作
2. 异步I/O是由内核通知我们I/O操作何时完成

步骤:

1. 用户进程调用aio_read函数，给内核传递描述符、缓冲区指针、缓冲区大小和文件偏移，告诉内核整个操作完成时如何通知我们，然后就立刻去做其他事情;
2. 当内核收到aio_read后，会立刻返回，然后内核开始等待数据准备，数据准备好以后，直接把数据拷贝到用户空间，然后再通知进程本次IO已经完成。



3. 各种IO模型的比较

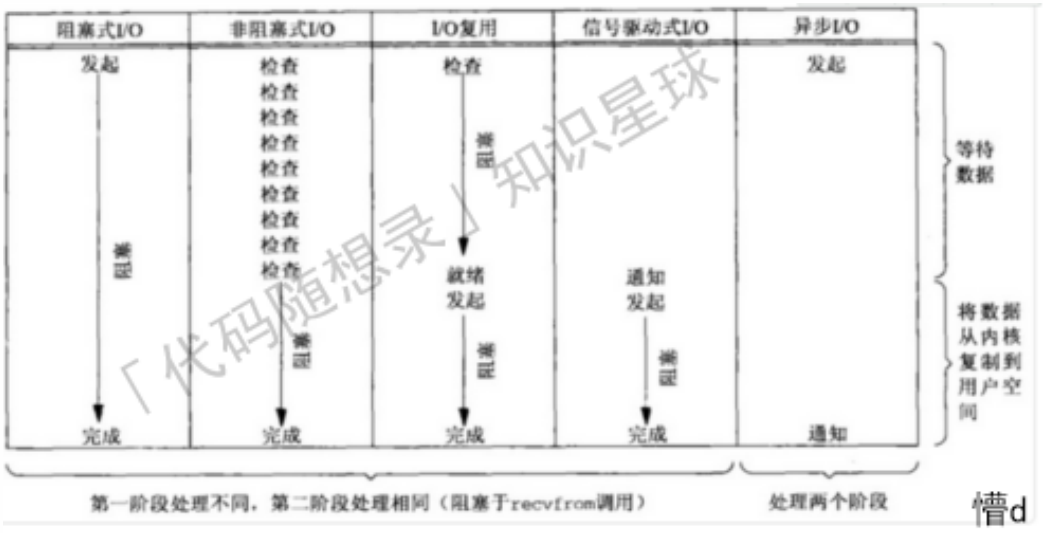
1、同步I/O

导致请求进程阻塞，直到I/O操作完成;

四种同步模型的区别在于第一阶段等待数据的处理方式不同，第二阶段均为将数据从内核空间复制到用户空间缓冲区期间，进程阻塞于recvfrom调用。

2、异步I/O

不导致请求进程阻塞。



3、I/O复用函数

多路复用接口 select/poll/epoll ，内核提供给用户态的多路复用系统调用，进程可以通过一个系统调用函数从内核中获取多个事件:

（1）**select-->两次遍历+两次拷贝**

把已连接的socket放在一个文件描述符集合，调用 select 函数将文件描述符集合拷贝到内核里，让内核来检查是否有网络事件产生;

通过遍历，有事件产生就把此socket标记为可读/可写，然后再整个拷贝回用户态;

用户态还需要遍历找到刚刚标记的socket。

（2）**poll**

动态数组，以链表形式来组织，相比于select，没有文件描述符个数限制，当然还会受到系统文件描述符限制。

（3）**epoll(event poll)-->红黑树**

在内核里使用红黑树来跟踪进程所有待检测的文件描述字。

调用epoll_ctl() 函数，把需要监控的 socket 加入内核中的红黑树里：（红黑树的增删查时间复杂度是O(logn)，不需要每次操作都传入整个集合，只需要传入一个待检测的socket，减少了内核和用户空间的大量数据拷贝和内存分配）

epoll 使用事件驱动的机制，内核里维护了一个链表来记录就绪事件（当某个 socket 有事件发生时，通过回调函数，内核会将其加入到这个就绪事件列表中）

当用户调用 epoll_wait() 函数时，只会返回有事件发生的文件描述符的个数，不需要像 select/poll 那样轮询扫描整个 socket 集合，大大提高了检测的效率。

（4）**epoll触发模式**

epoll支持的事件触发模式：

(1)边缘触发ET
(2)水平触发LT

边缘触发模式ET：

当被监控的 Socket 描述符上有可读事件发生时，服务器只会从 epoll_wait中苏醒一次，即使进程没有调用 read 函数从内核读取数据，也依然只苏醒一次，因此我们程序要保证一次性将内核缓冲区的数据读取完，只有第一次满足条件的时候才触发，之后就不会再传递同样的事件了。

水平触发模式LT：

当被监控的 Socket 上有可读事件发生时，服务器不断地从 epoll_wait中苏醒，直到内核缓冲区数据被 read 函数读完才结束，目的是告诉我们有数据，只要满足事件的条件，比如内核中有数据需要读，就一直不断地把这个事件传递给用户。

（5）**select和epoll的区别**

1. select 和 poll 采用轮询的方式检查就绪事件，每次都要扫描整个文件描述符，复杂度O(N);
2. epoll 采用回调方式检查就绪事件，只会返回有事件发生的文件描述符的个数，复杂度O(1);
3. select 只工作在低效的LT模式，epoll 可以在 ET 高效模式工作;
4. epoll 是 Linux 所特有，而 select 则应该是 POSIX 所规定，一般操作系统均有实现;
5. select 单个进程可监视的fd数量有限，即能监听端口的大小有限，64位是2048；epoll 没有最大并发连接的限制，能打开的 fd 的上限远大于2048（1G的内存上能监听约10万个端口）；
6. select 内核需要将消息传递到用户空间，都需要内核拷贝动作；epoll通过内核和用户空间共享一块内存来实现的。

网络

> 如何在基本网络通信模型基础上提升服务器自身服务响应?

1、多进程服务器

> 每次的请求父进程fork出子进程进行业务处理，父进程用于处理请求连接

（1）父子进程的生命周期绝大多数都不相同，如何、何时进行销毁，才能确保不会大量出僵尸进程

（2）子父进程之间如何进行通讯?

进程的创建、切换都需要都会带来大量的开销，同时每个进程都具有自己独立的虚拟内存空间，所以需要使用到IPC机制，完成进程间的数据交换

2、I/O复用服务器

1. 在不额外创建进程的基础上同时为多个请求端提供请求
2. select、poll、epoll实现优缺点

3、多线程服务器

> 同一进程下的多个线程共享进程的系统资源，所以相比于多进程，同进程下的多线程通信更加方便，并且切换的开销小很多。

1. 如何创建、销毁线程
2. 如何保证线程的安全
3. 线程间的通信

数据库

MySQL基础

1、什么是主键

拿来做唯一标识的字段

2、主键、外键、索引的区别

定义:

主键：唯一标识一条记录，不允许重复，不允许为空
外键：外键表示另一张表的主键，允许重复，可以是空值
索引：没有重复值，但可以有一个空值

作用:

主键：用来保存数据的完整性
外键：主要用于和其它表建立联系
索引：为了提高查询排序的速度

6、表的优化

在建表之初就需要考虑拆分逻辑

字段优化：

1. 尽量使用TINYINT，SMALLINT，MEDIUM_INT替代INT类型，如果是非负则加上UNSIGNED
2. VARCHAR的长度只分配真正需要的空间
3. 尽量使用整数或者枚举替代字符串类型
4. 时间类型尽量使用TIMESTAMP而非DATETIME
5. 单表不要放太多字段
6. 尽量少使用NULL，很难查询优化而且占用额外索引空间

7、慢查询

基础：

（1）设置指定超时时间，超过这个时间的sql语句被称为慢查询
（2）long_query_time：默认10秒
优化
（1）运行语句，找到慢查询的sql
（2）查询区分度最高的字段
（3）explain：显示mysql如何使用索引来处理select语句以及连接表，可以帮助选择更好的索引、写出更优化的查询语句
（4）order by limit形式的sql语句，让排序的表优先查
（5）考虑建立索引原则
（6）了解业务使用场景

1. MySQL与MongoDB的区别，海量数据的存储

数据库模型
MySQL：关系型
MongoDB：非关系型

存储方式
MySQL：不同引擎有不同的存储方式
MongoDB：虚拟内存+持久化

查询语句
MySQL：sql语句
MongoDB：使用MongoDB自己的查询方式

常见架构特点
MySQL：单点、Master-Slave，MHA，Cluster
MongoDB：通过副本集与分片来实现高可用

数据处理方式
MySQL：不同引擎有不同处理特点
MongoDB：基于内存，将热数据存储在物理内存中，从而达到高速读写的目的

海量数据的存储
MySQL效率相对较低，但是MongoDB不支持事务

MySQL表连接

前导知识

连接：

将各个表中的记录都取出来进行依次匹配，将匹配后的结果发给客户端

笛卡尔积：

连接查询的结果中包含一个表的每一条记录与另一个表中每一条记录的组合，这样的查询结果就是笛卡尔积

比如表a有5条记录；表b有6条记录；a和b的笛卡尔积就是30

连接过程

1、确定第一个需要查询的表，此表为驱动表

2、从驱动表中取每一条符合搜索条件的记录，到接下来的表中查找匹配的记录；驱动表之后的那个表就叫被驱动表

只需要访问驱动表一次，可能会多次访问被驱动表

每获得一条满足条件的驱动表记录，马上到被驱动表中寻找匹配的记录

内/外连接

内连接：

驱动表中的记录在被驱动表中找不到匹配的记录，那么驱动表的这条记录不会加入到最后的结果中

```
select * from 驱动表, 被驱动表;
select * from 驱动表 join 被驱动表;
select * from 驱动表 inner join 被驱动表;
select * from 驱动表 cross join 被驱动表;
```

外连接：

驱动表中的记录在被驱动表中找不到匹配的记录，也仍需要加入到最后结果中

左外连接：语句左侧的表为驱动表

```
select * from 驱动表 left join 被驱动表 on 连接条件;
```

右外连接：语句右侧的表为驱动表

```
select * from 被驱动表 right join 驱动表 on 连接条件;
```

对于内连接，驱动表和被驱动表的顺序可以更换；对于外连接，这个顺序不能随意更换

过滤条件：

where：

不论内外连接，只要是不符合where子句的记录都不会加入到最后的结果中

on：

在内连接中与where等价；

在外连接中，如果驱动表中的记录在被驱动表中没有记录可以匹配，该驱动表记录仍会加入到结果中，对应的被驱动表字段以null填充

嵌套循环连接

如果有3个表进行连接，那么表1和表2完成连接后的结果作为驱动表，将表3作为被驱动表进行连接查询

使用索引

在根据驱动表的一条记录去被驱动表中查询时，相当于确定搜索条件的单表查询，可以使用索引优化单表查询

在被驱动表上使用二级索引进行查询时，可能连接查询的变量和过滤条件都是二级索引的部分列，可以不用回表直接覆盖索引，所以最好不要用 * 作为查询列表，而是使用真正需要查询的列

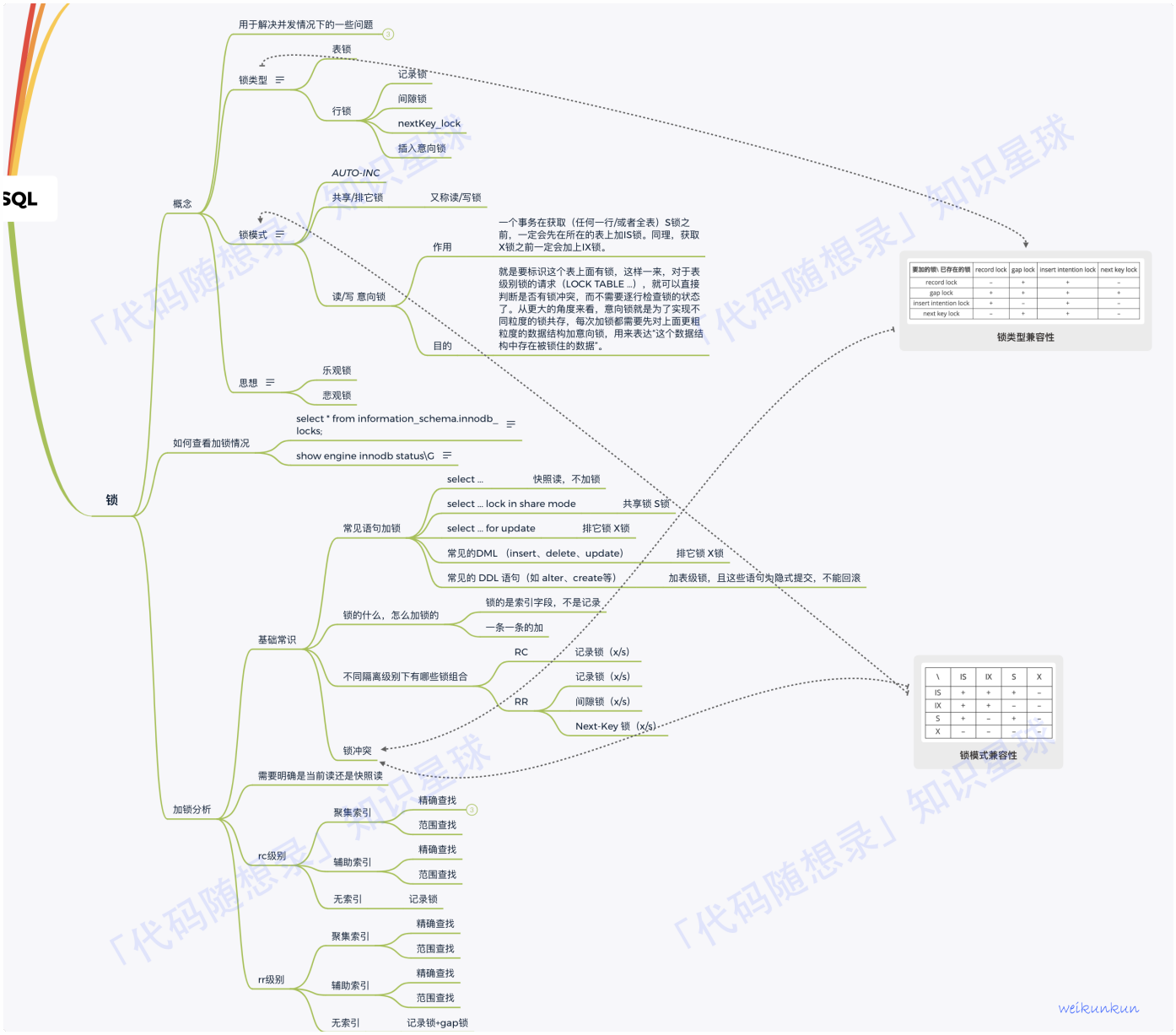
嵌套循环连接

减少被驱动表的访问次数

Join Buffer连接缓冲区：在执行连接查询前申请一块固定大小的内存，先把若干条驱动表结果集中的记录放在buffer中；在扫描被驱动表时，每一条被驱动表的记录与buffer中的多条驱动表记录进行匹配

MySQL锁

多个事物在并发情况下会出现一些经典的问题，如脏读、不可重复读、幻读、丢失更新。为了不同程度的解决这些问题，出现了不同的隔离级别，锁就是实现各种隔离级别的一种方式。



Innodb使用表锁还是行锁?
对于innodb，绝大部分情况应该使用行锁

使用表锁的情况
（1）表比较大，事务需要更新全部或者大部分数据
（2）事务涉及到多个表，比较复杂，可能引起死锁，造成大量的事务回滚

锁的种类

1、全局锁

顾名思义，全局锁就是对整个数据库实例加锁。典型使用场景：全库逻辑备份，即把整个库的表都select出来存成文本

MySQL 提供了一个加全局读锁的方法，命令是 Flush tables with read lock (FTWRL)。

当你需要让整个库处于只读状态的时候，可以使用这个命令，之后其他线程的以下语句会被阻塞：

1. 数据更新语句（数据的增删改）
2. 数据定义语句（包括建表、修改表结构等）
3. 更新类事务的提交语句

2、表级锁

MySQL 里面表级别的锁有两种：

（1）表锁

特点：

1. 每次操作锁住整张表
2. 开销小，加锁快
3. 并发度最低

表锁的语法是 lock tables … read/write。

与 FTWRL 类似，可以用 unlock tables 主动释放锁，也可以在客户端断开的时候自动释放。

需要注意： lock tables 语法除了会限制别的线程的读写外，也限定了本线程接下来的操作对象。

（2）元数据锁（meta data lock，MDL）

MDL 不需要显式使用，在访问一个表的时候会被自动加上。

MDL 的作用：保证读写的正确性。

你可以想象一下，如果一个查询正在遍历一个表中的数据，而执行期间另一个线程对这个表结构做变更，删了一列，那么查询线程拿到的结果跟表结构对不上，肯定是不行的。

在 MySQL 5.5 版本中引入了 MDL

当对一个表做增删改查操作的时候，加 MDL 读锁；
当要对表做结构变更操作的时候，加 MDL 写锁。

读锁之间不互斥，因此你可以有多个线程同时对一张表增删改查。读写锁之间、写锁之间是互斥的，用来**保证变更表结构操作的安全性**。因此，如果有两个线程要同时给一个表加字段，其中一个要等另一个执行完才能开始执行。

事务中的 MDL 锁，在语句执行开始时申请，但是语句结束后并不会马上释放，而会等到整个事务提交后再释放。（这可能会产生死锁的问题）

（3）行锁

顾名思义，行锁就是针对数据表中行记录的锁（也有人称为记录锁）。

这很好理解，比如事务 A 更新了一行，而这时候事务 B 也要更新同一行，则必须等事务 A 的操作完成后才能进行更新。

特点：

1. 每次操作锁住一行数据
2. 开销大，加锁慢

3. 发生锁冲突的概率是最低的，并发度是最高的

在 InnoDB 事务中，行锁是在需要的时候才加上的，但并不是不需要了就立刻释放，而是要等到事务结束时才释放。这个就是**两阶段锁协议**。

知道了这个设定，对我们使用事务有什么帮助呢？那就是，如果你的事务中需要锁多个行，要把最可能造成锁冲突、最可能影响并发度的锁尽量往后放。

3、加锁规则

加锁规则里面，包含了两个“原则”、两个“优化”和一个“bug”

两个原则：

1. 加锁的基本单位是 next-key lock。希望你还记得，next-key lock 是前开后闭区间。
2. 查找过程中访问到的对象才会加锁。

两个优化：

1. 索引上的等值查询，给唯一索引加锁的时候，next-key lock 退化为行锁。
2. 索引上的等值查询，向右遍历时且最后一个值不满足等值条件的时候，next-key lock 退化为间隙锁。

一个"bug"：

1. 唯一索引上的范围查询会访问到不满足条件的第一个值为止。（MySQL8.0之后的版本已经修复）

以上规则是在可重复读隔离级别 (repeatable-read) 下验证的。同时，可重复读隔离级别遵守两阶段锁协议，所有加锁的资源，都是在事务提交或者回滚的时候才释放的。如果切换到读提交隔离级别 (read-committed) 的话，就好理解了，过程中去掉间隙锁的部分，也就是只剩下行锁的部分。

锁的划分

1. 数据库角度

1、共享锁（共享锁也叫读锁或 S 锁 ）

共享锁锁定的资源可以被其他用户读取，但不能修改。

在进行SELECT的时候，会将对象进行共享锁锁定，当数据读取完毕之后，就会释放共享锁，这样就可以保证数据在读取时不被修改。

如果我们想要给某一行加上共享锁呢，比如想对 user_id=10的数据行加上共享锁，可以像下面这样：

```
SELECT user_id FROM product_comment WHERE user_id = 10 LOCK IN SHARE MODE;
```

2、排他锁（排它锁也叫独占锁、写锁或 X 锁）

排它锁锁定的数据只允许进行锁定操作的事务使用，其他事务无法对已锁定的数据进行查询或修改。

如果我们想要在某个数据行上添加排它锁，比如针对 user_id=10的数据行，则写成如下这样：

```
SELECT user_id FROM product_comment WHERE user_id = 10 FOR UPDATE;
```

另外当我们对数据进行更新的时候，也就是INSERT、DELETE或者UPDATE的时候，数据库也会自动使用排它锁，防止其他事务对该数据行进行操作。

3、共享锁与排他锁

共享锁和排他锁不仅可以锁住一行，也可以锁住一张表，如下所示：

比如我们想给 product_comment 在表上加共享锁，可以使用下面这行命令

```
LOCK TABLE product_comment READ;
```

如果我们想要对表上的共享锁进行解锁，可以使用下面这行命令：

```
UNLOCK TABLE;
```

如果我们想给 product_comment 数据表添加排它锁，可以使用下面这行命令

```
LOCK TABLE product_comment WRITE;
```

这时只有获得排它锁的事务可以对 product_comment 进行查询或修改，其他事务如果想要在 product_comment 表上查询数据，则需要等待。

你可以自己开两个 MySQL 客户端来模拟下。这时我们释放掉排它锁，使用这行命令即可

```
UNLOCK TABLE;
```

4、意向锁（Intent Lock）

简单来说就是给更大一级别的空间示意里面是否已经上过锁。

举例：

你可以给整个房子设置一个标识，告诉它里面有人，即使你只是获取了房子中某一个房间的锁。这样其他人如果想要获取整个房子的控制权，只需要看这个房子的标识即可，不需要再对房子中的每个房间进行查找。这样是不是很方便?

返回数据表的场景，如果我们给某一行数据加上了排它锁，数据库会自动给更大一级的空间，比如数据页或数据表加上意向锁，告诉其他人这个数据页或数据表已经有人上过排它锁了，这样当其他人想要获取数据表排它锁的时候，只需要了解是否有人已经获取了这个数据表的意向排他锁即可。

如果事务想要获得数据表中某些记录的共享锁，就需要在数据表上添加意向共享锁。同理，事务想要获得数据表中某些记录的排他锁，就需要在数据表上添加意向排他锁。这时，意向锁会告诉其他事务已经有人锁定了表中的某些记录，不能对整个表进行全表扫描。

2. 程序员角度

1、乐观锁（Optimistic Locking）

认为对同一数据的并发操作不会总发生，属于小概率事件，**不用每次都对数据上锁**，也就是不采用数据库自身的锁机制，而是通过程序来实现。在程序上，我们可以采用版本号机制或者时间戳机制实现。

2、悲观锁（Pessimistic Locking）

也是一种思想，对数据被其他事务的修改持**保守态度**，会通过数据库自身的锁机制来实现，从而保证数据操作的排它性。

3、乐观锁和悲观锁的适用场景

（1）乐观锁

适合读操作多的场景，相对来说写的操作比较少。它的优点在于程序实现，不存在死锁问题，不过适用场景也会相对乐观，因为它阻止不了除了程序以外的数据库操作。

（2）悲观锁

适合写操作多的场景，因为写的操作具有排它性。采用悲观锁的方式，可以在数据库层面阻止其他事务对该数据的操作权限，防止读 - 写和写 - 写的冲突。但是加锁的时间会比较长，可能会长时间限制其他用户的访问，也就是说他的并发访问性不好。

MySQL事务

1. 基本概念

1. 一个最小的不可再分的工作单元
2. 通常情况下，一个事务对应一个完整的业务

2. MySQL事务特性

ACID

1、A 原子性（Atomicity）

一个事务的所有操作，要么全部完成，要么都没完成，不能结束在中间环节。如果事务在执行过程中发生错误，会被回滚到事务开始之前的状态

2、C 一致性（Consistency）

在事务开始之前以及事务结束之后，数据库的完整性不能被破坏

3、I 隔离性（Isolation）

允许多个并发事务同时对数据进行修改和读写的能力，它可以防止由于多个事务并发执行时由于交叉执行而导致的数据不一致

4、D 持久性（Durability）

事务处理结束了以后，对数据的修改是永久的，即使是发生了系统故障，数据也不会丢失

3. MySQL四大隔离级别

术语说明：

脏读： 一个事务读取另一个事务还没有提交的数据
不可重复读： 同一个事务内，两个相同的查询返回了不同的结果

1、RU 读不提交（Read Uncommited）

在该隔离级别下，事务之间完全不隔离，会产生脏读，一般情况不会使用

2、RC 读提交（Read Commited）

本事务读取到的是其它事务提交的最新数据，但有一个问题在同一个事务中，前后两相同的select可能会读到不同的结果

3、RR 可重复读（Repeatable Read）

在同一个事务中，select的结果是事务开始时时间点的状态，因此，同一个事务同样的select操作可以读到一致的结果

4、serializable串行化

读操作会隐式获取共享锁，保证不同事务之间的互斥

分布式

1、幂等操作

用户对同一操作发起一次请求或者多次请求的结果是一致的，不会因为多次点击而产生副作用

2、lvs负载均衡

Linux virtual server：一个实现负载均衡的开源软件项目

目标： 通过LVS的负载均衡技术与Linux操作系统实现一个高性能，高可用的Linux服务器集群

逻辑架构

（1）调度层
（2）server集群层
（3）共享存储

基本工作原理

关键词

IPVS：使用用户定义的策略生效
ipvsadn：用于用户定义和集群服务管理的工具

原理

IVS的IP负载均衡技术主要通过IPVS实现，IPVS虚拟一个IP地址，用户必须通过这个虚拟的IP地址访问服务器。该虚拟IP被称为LVS的VIP，访问的请求首先经过LVS的VIP，访问的请求首先经过LVS的VIP，到达负载调度器，由负载调度器从real server列表中选取一个服务节点响应用户的请求

3、lvs和Nginx的区别

负载均衡

LVS：4层负载均衡
Nginx：7层负载均衡

技术原理

LVS：IP+TCP端口
Nginx：URL应用层（内容交换）

优点：

LVS： 配置简单，效率高
Nginx： 智能化，可以对客户端请求服务端响应进行自定义修改。极大的提升了应用系统在网络层的灵活性

缺点

LVS： 不理解http，ftp等应用协议，满足不了特定需求，比如动静分离等
Nginx： 对负载均衡的设备要求很高，配置复杂

4、consul、etcd区别

分布式微服务架构：一个应用可能由一组职责单一的服务组成

consul与etcd都提供了相关的服务管理功能

区别

服务健康检查

consul：服务状态、内存、硬盘等
etcd：连接心跳

多数据中心

consul：支持
etcd：不支持

使用接口

consul：http/dns
etcd：http/grpc

5、分布式事务

保证在分布式环境中，不同的数据的数据一致性

6、分布式锁

将多线程的锁机制应用到了分布式的部署环境中

特点

（1）互斥性
（2）可重入：同一个节点上的同一个线程，获取到锁之后还可以两次获取这个锁
（3）锁超时：支持锁超时，防止死锁
（4）高效，高可用
（5）支持阻塞和非阻塞
（6）支持公平锁和非公平锁

公平锁：按照请求加锁的顺序获取锁
非公平锁：无序获取锁

常见分布式锁

（1）MySQL
（2）Redis
（3）ZK（Zookeeper）

7、淘汰算法

LRU（Least recently used）

概念：最近最少使用，如果数据最近被访问过，那么将来被访问的几率也高
实现：
（1）设计一个链表来缓存数据
（2）新数据插入到链表头部
（3）每当缓存命中（也就是缓存数据被访问），将数据移动到链表头部
当链表满的时候，将链表尾部的数据丢弃

LFU（Latest frequently used）

原理：最不经常使用，如果一个数据在最近一段时间内使用次数很少，那么它在将来的一段时间内被使用的可能性也很小
实现
（1）LFU每个数据块设计一个引用计数
（2）所有数据块按照引用计数排序，具有相同引用计数的数据块按照时间排序
（3）新加入的数据插入到队列尾部
（4）在队列中某一个数据被访问，该数据引用计数加1，队列重新排序
需要删除数据时，将已排序的列表最后的数据库删除

FIFO（First in first out）

原理：先进先出，如果一个数据最先进入缓存中，就最早淘汰掉
实现
直接用队列实现

8、一致性hash算法

作用：该算法通常用于负载均衡中要求资源被均匀的分布到所有节点上，并且对资源的请求能够快速路由到对应的节点上面
特点
（1）平衡性：哈希结果要均匀的分配在整个输出空间中
（2）单调性：当数据发生变动的时候，对相同的数据始终映射到相同的节点，或者新增加的缓冲节点中，避免无法找到原来的数据
（3）稳定性：当出现节点坏掉或者需要动态扩容时，尽量减少数据的移动
原理

关键词
（1）哈希环：将整个哈希的输出空间设置为一个环形区域
（2）设置哈希环
（3）将服务器进行哈希，可以考虑使用服务器的编号或者ip等作为输入，得到一个输出值，将该输出值映射到输出空间的环形区域上
（4）对用户数据进行同样的hash操作，映射在环形区域上。然后让数据按照顺时针方向移动，遇到的第一个服务器就是它分配的服务器
（5）通过将整个哈希输出空间设置为一个环形区别，可以有效的减小输出空间的变化对于哈希结果的影响

9、CAP原理

基本概念：

（1）C（一致性）：对于分布在不同节点上的数据，如果一个节点更新数据之后，其他节点都能读到这个最新的数据
（2）A（可用性）：非故障节点在合理的时间内返回合理的响应
（3）P（分区容错性）：出现网络错误，系统还能继续工作

CAP不能同时共存：

CA：分布式中一般不会选择
CP：CP的代表是Zookeeper，放弃可用性，追求一致性和容错性
AP：追求分区容错性与可用性

10、BASE理论

Basically Available（基本可用）：分布式系统在出现故障时，允许损失部分可用功能
Soft State（软状态）：允许系统中存在中间状态，这个状态不影响系统可用性
Eventually consistent（最终一致性）：在经过一段时间之后，所有节点的数据都会达到一致

MySQL索引

索引：索引的出现其实就是为了提高数据查询的效率，就像书的目录一样

MySQL 在查询方面主要就是**两种方式：**

1. 全表扫描（一个一个挨个找）
2. 根据索引检索

1、创建索引

```
create index 索引名 on 表名(列名);
# 也可以有多个列名构成索引（即，组合索引）
```

2、删除索引

```
drop index 索引名 on 表名;
```

在mysql当中，怎么查看一个SQL语句是否使用了索引进行检索?

1. 在 SQL 语句前，添加 explian 关键字
2. 当 type=ALL 时，表示使用 全表查询（未使用索引）
3. 当 type=RES时，表示使用索引

索引失效的情况：

1. 模糊匹配当中以 “%” 开头时，索引失效
2. OR 有一边的条件字段没有索引时，索引失效
3. 使用复合索引的时候，没有使用左侧的列查找，索引失效
4. 在 where 当中索引列参加了运算，索引失效
5. 在where当中索引列使用了函数，索引失效

Note:

1. 在任何数据库当中主键上都会自动添加索引对象
2. 在mysql当中，一个字段上如果有unique约束的话，也会自动创建索引对象

3、建立索引注意事项

1. 索引不是越多越好，虽然索引会提高 select 效率，但是也降低了insert以及update的效率
2. 数据量小的表不需要建立索引，会增加额外的索引开销
3. 不经常使用的列不要建立索引
4. 频繁更新的列不要建立索引，会影响更新的效率

4、MySQL的索引有几种

1. 普通索引：最基本的索引，没有任何限制
2. 唯一索引：与普通索引类似，但索引列的值必须是唯一的，允许空值
3. 主键索引：一种特殊的唯一索引，一个表只能有一个主键，不允许有空值
4. 组合索引：在多个字段上创建的索引，只有在查询条件中使用了创建索引的第一个字段，索引才会被使用
5. 全文索引：主要用来查找文本中的关键字，类似于搜索引擎

5、索引优化

1. 尽量避免在where字句中对字段进行空值判断，这会导致引擎放弃使用索引，进行全表扫描
2. 字段值分布很稀少的字段，不适合建立索引
3. 不要用字符字段做主键
4. 字符字段只建立前缀索引
5. 不要用外键和UNIQUE
6. 使用多列索引时，注意顺序和查询条件保持一致，同时删除不必要的单列索引

索引常见类型

哈希表

使用 key-value 对，存储数据

但是，可能存在 hash 冲突（多个 value 对应同一个 key）
拉链法 解决 hash 冲突 的方法之一

适用情景：只有等值查询，如Mencached、NoSQL

hash 表：

优点：

key 无序，插入数据时无需维护顺序（直接在最后一个元素后追加，即可），效率较高；

缺点：

因为不是有序的，所以哈希索引做区间查询的速度是很慢

Note：

哈希表适用于只有等值查询的场景，而不适用频繁 区间查找

有序数组

优点：

等值查询 和 区间查询 性能都挺6 （有序数组 适合 查询）

缺点：

有序数组不适合 频繁 增/删 记录的场景 （但是，有序数组不合适 增/删）

Note：

有序数组只适用于静态存储引擎，在 等值查询 和 范围（区间）查询 场景中的性能非常优秀

等值查询：

二分（前提是，有序 + 顺序存储），查询时间复杂度 O(log N)

区间查询：

1. 先用 二分查找 “左边界”（不存在的时，则找第一个比 它大的记录）
2. 然后，再向右扫描，直至 大于右边界

二叉搜索树

BST 是为了 保留了 “有序数组” 查询（二分，O(logN)）性能好的优点，同时解决“有序数组” 不适合 增/删 的缺点

BST 查询的时间复杂度：O(log N)
但是，为了维持 O(log(N)) 的查询复杂度，就需要保持这棵树是平衡二叉树。（维护 BST 是 AVL 的时间复杂度也是 O(log(N))）

二叉树是搜索效率最高的，但是实际上大多数的数据库存储却并不使用二叉树。其原因是，索引不止存在内存中，还要写到磁盘上

N 叉树

case:

> 你可以想象一下一棵 100 万节点的平衡二叉树，树高 20。一次查询可能需要访问 20 个数据块。在机械硬盘时代，从磁盘随机读一个数据块需要 10 ms 左右的寻址时间。也就是说，对于一个 100 万行的表，如果使用二叉树来存储，单独访问一个行可能需要 20 个 10 ms 的时间，这个查询可真够慢的。

> 为了让一个查询尽量少地读磁盘，就必须让查询过程访问尽量少的数据块。那么，我们就不应该使用二叉树，而是要使用“N 叉”树。这里，“N 叉”树中的“N”取决于数据块的大小。
>
> 以 InnoDB 的一个整数字段索引为例，这个 N 差不多是 1200。这棵树高是 4 的时候，就可以存 1200 的 3 次方个值，这已经 17 亿了。考虑到树根的数据块总是在内存中的，一个 10 亿行的表上一个整数字段的索引，查找一个值最多只需要访问 3 次磁盘。其实，树的第二层也有很大概率在内存中，那么访问磁盘的平均次数就更少了。 posper

为了让一个查询尽量少地读磁盘，就必须让查询过程访问尽量少的数据块

那么，我们就不应该使用二叉树，而是要使用“N 叉”树（N 取决于数据块的大小）

即，通过使用 N 叉树 来降低 树的高度，即 减少读取磁盘的次数（IO是很慢的），提高查询效率 ⭐

N 叉树特点：

折中 考虑了“查询性能”和“读取磁盘的次数”，广泛应用于 数据库引擎中

MySQL 中，索引是在存储引擎层实现的，而且不同存储引擎的索引的工作方式并不一样。而即使多个存储引擎支持同一种类型的索引，其底层的实现也可能不同。

下面以InnoDB 为例，和你分析一下其中的索引模型

InnoDB 的索引模型:

InnoDB 使用了 B+ 树索引模型，每一个索引在 InnoDB 里面对应一棵 B+ 树

假设建表语句如下，则对应的 2 棵 索引树（主键 自动创建一颗；字段 k 创建 一颗索引树），如下所示

```
 create table T
(
    id int primary key, # 主键 自动创建索引
    k int not null,
    name varchar(16),
    index (k)
) # 给字段 k 条件 索引

engine=InnoDB;
```

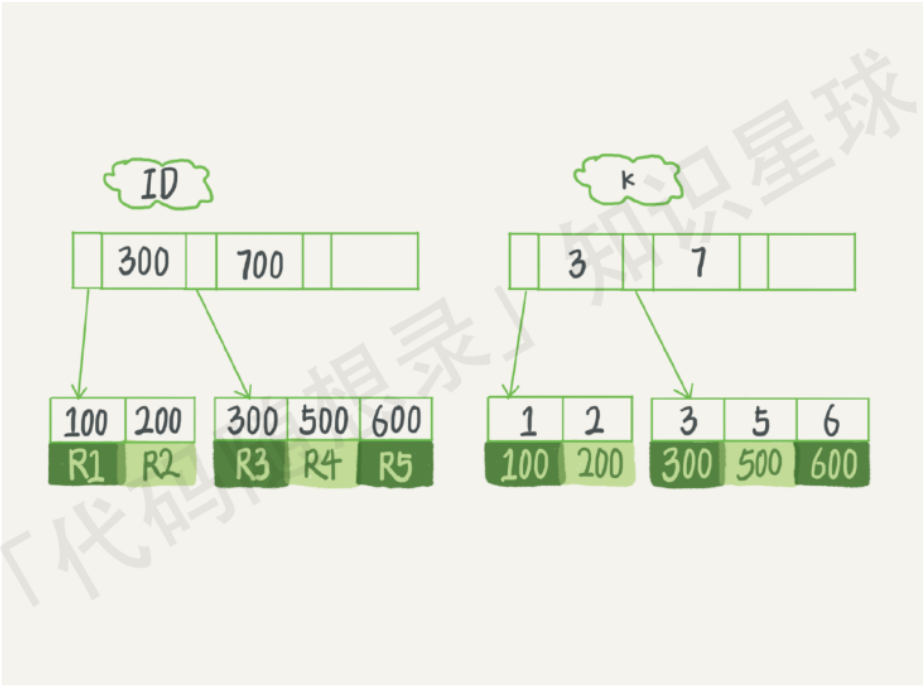


图 4 InnoDB 的索引组织结构

posper

根据叶子节点的内容，索引类型分为主键索引和非主键索引：

主键索引的叶子节点存的是整行数据
在 InnoDB 里，主键索引也被称为聚簇索引（clustered index）

非主键索引的叶子节点存的是主键的值
在 InnoDB 里，非主键索引也被称为二级索引（secondary index）

主键查询 vs. 普通索引查询

主键查询：

直接在 主键索引 所在的 B+ 树中查询，然后直接返回查询到的叶子节点（此时，叶子节点里面就是整行记录）

普通索引查询：

首先，在普通索引所在的 B+ 树中，查询到待查询记录的 主键；
然后，再根据这些查到的 主键，执行“主键查询”（即，回表）

基于非主键索引的查询需要多扫描一棵索引树。因此，我们在应用中应该尽量使用主键查询。

索引维护：

B+ 树为了维护索引有序性，如果待插入记录所在的数据页已经满了，则可能造成“页分裂”（申请一个新的数据页，然后将部分数据 挪过去）

页分裂的缺点：

1. 影响系统性能
2. 影响数据页的利用率
3. 主键长度越小，普通索引的叶子节点就越小，普通索引占用的空间也就越小
4. 所以，应该尽量使用 自增主键，减少空间存储消耗。（但是，事无绝对，只是大多情况使用 自增主键，下面也有 特例）

适合用 业务字段 直接做主键，而不是使用 自增主键的场景：（特例）

1. 只有一个索引
2. 该索引必须是唯一索引

覆盖索引：

当执行如下语句时，这时只需要查 ID 的值，而 ID 的值已经在 k 索引树上了，因此可以直接提供查询结果，不需要回表

```
select ID from T where k between 3 and 5
```

覆盖索引：

在 普通索引树中，就可以直接查到待查结果，而不需要 回表

由于覆盖索引可以减少树的搜索次数，显著提升查询性能，所以使用覆盖索引是一个常用的性能优化手段。

最左前缀原则：

场景： 当单独为一个不频繁的请求创建索引时，会造成空间浪费；但是，如果直接让它走 全表查询(一个一个找)，那效率也太低了吧？此时，应该怎么做？

答：**B+** 树这种索引结构，可以利用索引的“最左前缀”，来定位记录。

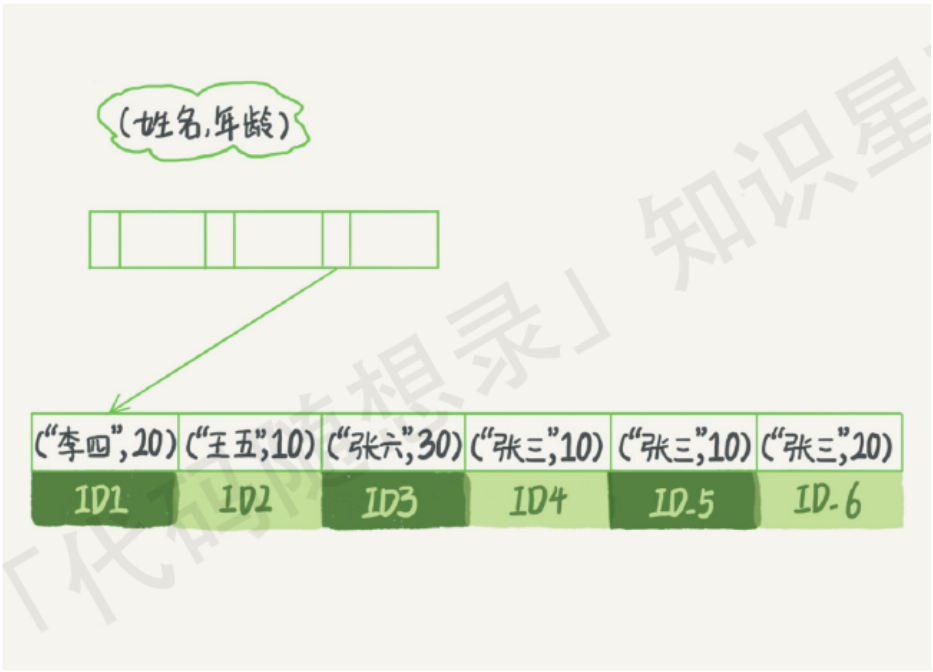


图 2 （ name，age ）索引示意图 posper

使用 "最左前缀" 查询：

当查询语句是 where name like ‘张 %’时，也能够用上这个索引

1. 首先，查找到第一个符合条件的记录是 ID3；
2. 然后，向后遍历，直到不满足条件为止

但是，使用 'like %张" 时，索引失效（即，必须使用 最左前缀）

最左前缀可以是联合索引的最左 N 个字段，也可以是字符串索引的最左 M 个字符。

Note：考虑到“最左前缀原则”，可以通过调整组合索引中的字段顺序，可以少维护一个索引

索引下推:

那些不符合最左前缀的部分，会怎么样呢?

比如，执行如下语句时候，则存在不符合最左前缀的部分：

```
select * from tuser
where name like '张 %'
and age=10
and ismale=1;
```

根据前缀索引规则，这个语句在搜索索引树的时候，只能用 “张”，找到第一个满足条件的记录 ID3

然后在 MySQL 5.6 之前，只能从 ID3 开始一个个回表，到主键索引上找出数据行，再对比字段值。

而 MySQL 5.6 引入的索引下推优化（index condition pushdown）

索引下推：

可以在索引遍历过程中，对索引中包含的字段先做判断，直接**过滤掉不满足条件的记录**，减少回表次数

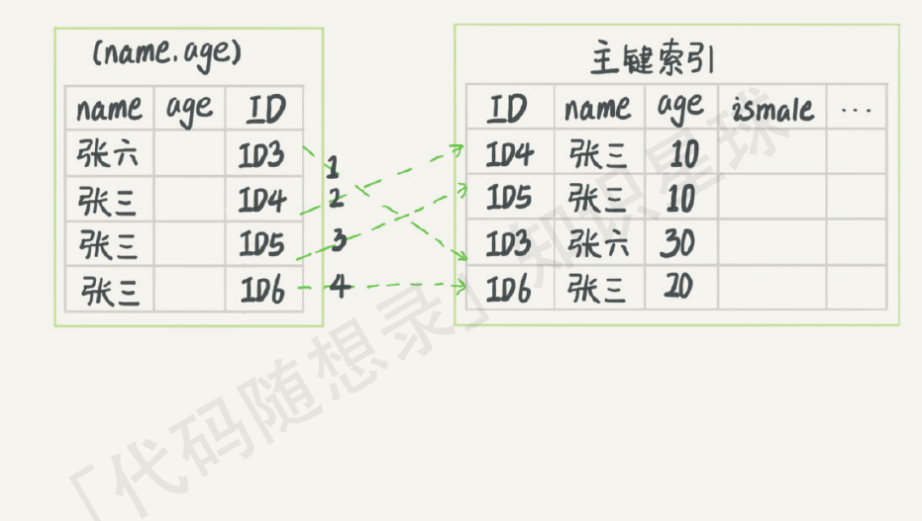


图 3 无索引下推执行流程 posper

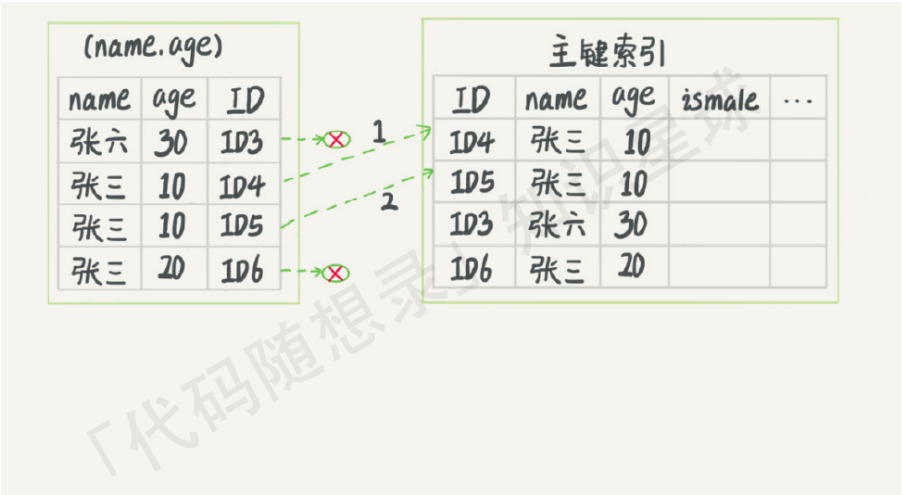


图 4 索引下推执行流程 posper

由图3可知，本例中 无索引优化时，需要 回表 4 次；

图4中，采用 索引下推 优化，直接在索引遍历的过程中，过滤了不符合条件的记录（ID3、ID6），只用回表 2 次

索引的优缺点

索引的优缺点

优点

使用索引可以大大加快数据的检索速度（大大减少检索的数据量），这也是创建索引的最主要的原因。

但是注意使用索引不一定能够提高查询性能，因为如果数据库的数据量不大，那么使用索引也不一定能够带来很大提升。其余大多数情况下，索引查询比全表扫描要快。

通过创建唯一性索引，可以保证数据库表中每一行数据的唯一性。

缺点

空间消耗，一个索引对应的就是一棵 b+树，每一个节点都是一个 16KB 大小的页。**占用的空间较大。**

创建索引和维护索引需要耗费许多时间，当对表中的数据进行增删改的时候，如果数据有索引，那么索引也需要动态的修改，会降低 SQL 执行效率。

举例子： 向数据库中增加一条记录，如果记录应该存储的页恰好满了，此时就要进行对应的页面分裂操作。

InnoDB收集统计数据

表的统计数据：show table status;

索引的统计数据：show index;

统计数据的存储：

1、永久性存储

存储于磁盘上，服务器重启后数据仍然存在

实际存储在磁盘内的两个表中

1. innodb_index_stats
2. innodb_table_stats

非永久性存储

存储于内存中，服务器重启时，重新收集统计数据

MySQL用户管理

目的：为了保证MySql服务器的安全，每个MySql的用户应该对他们需要的数据具有适当的访问权;

例如:

1. 多数用户只能够用到读写权限，只有少数用户能够创建表格和删除表格
2. 一些用户只能访问某个数据库而不能访问其他的数据库
3. 一些用户可以管理其他用户的账号，但大多数人不需要管理别人的账号

每个用户需要用到什么权限，就只给他们什么权限

如果每个用户的权限都很高，那数据库会变得十分危险，随时都需要面临很多误操作，所以要进行访问权限控制

1. 用户账号存储

在 MySql 下，所有的用户的账号，密码，对数据库的操作权限，都存储在 mysql 下（数据库的名字就叫 mysql）的 user 表格中。
user 表格有一个 user 属性，user 属性就是当前所有的用户名，通过以下操作可以查看当前 mysql 的用户一共有多少。

```
use mysql
select user from user
```

```
mysql> select user from user
    -> ;
+---------------+
| user          |
+---------------+
| root          |
| ysj           |
| mysql.session |
| mysql.sys     |
| root          |
+---------------+
5 rows in set (0.00 sec)
```

charon 知识随想录 | 知识星球

2. 用户账号管理

既然 mysql 中的 user 表格中存放着当前 mysql 的用户，那么新增用户只需要在 user 表中使用 insert 语句新增一条记录即可。

不过为了安全起见不建议这样做，mysql 数据库存放着很重要的数据，这样直接对 mysql 数据库直接操作可能会伤害到 mysql 服务器。

2.1 新增用户

新增一个用户的语法：

```
CREATE USER [user_name] IDENTIFIED BY [user_pwd]
```

例如：新增用户 zhangsan，pwd=123456

```
CREATE USER zhangsan IDENTIFIED BY '123456'
```

2.2 修改用户名

修改用户名的语法：

```
RENAME USER [user_name] TO [new_user_name]
```

例如：例如将账号名从 zhangsan 改为 lisi

```
RENAME USER zhangsan TO lisi
```

2.3 修改用户密码

修改用户密码语法：

```
SET PASSWORD FOR [user_name] = Password('[new_pwd]')
```

这里使用 password 函数对新密码进行加密。

如果不指定用户名：

```
SET PASSWORD = Password('[new_pwd]')
```

默认为修改当前登陆的用户的密码。

例如：修改 lisi 账号的密码：

```
SET PASSWORD FOR lisi = Password('135790')
```

3. 用户账号删除

删除用户账号语法：

```
DROP USER [user_name]
```

例如：删除 lisi 的账号

```
DROP USER lisi
```

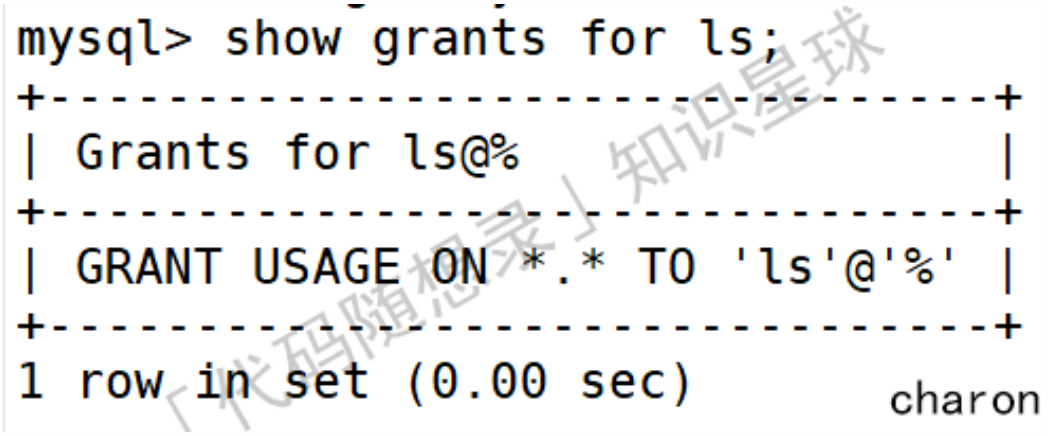
如果mysql版本低于mysql5，需要先revoke用户所有的权限，才能删除账号。更高版本则可以一次性删除权限和账号。

4. 用户权限控制

4.1 查看用户权限

刚创建的的用户是没有任何权限的，使用以下语法查看某用户所拥有的权限：

```
SHOW GRANTS FOR [user_name]
```



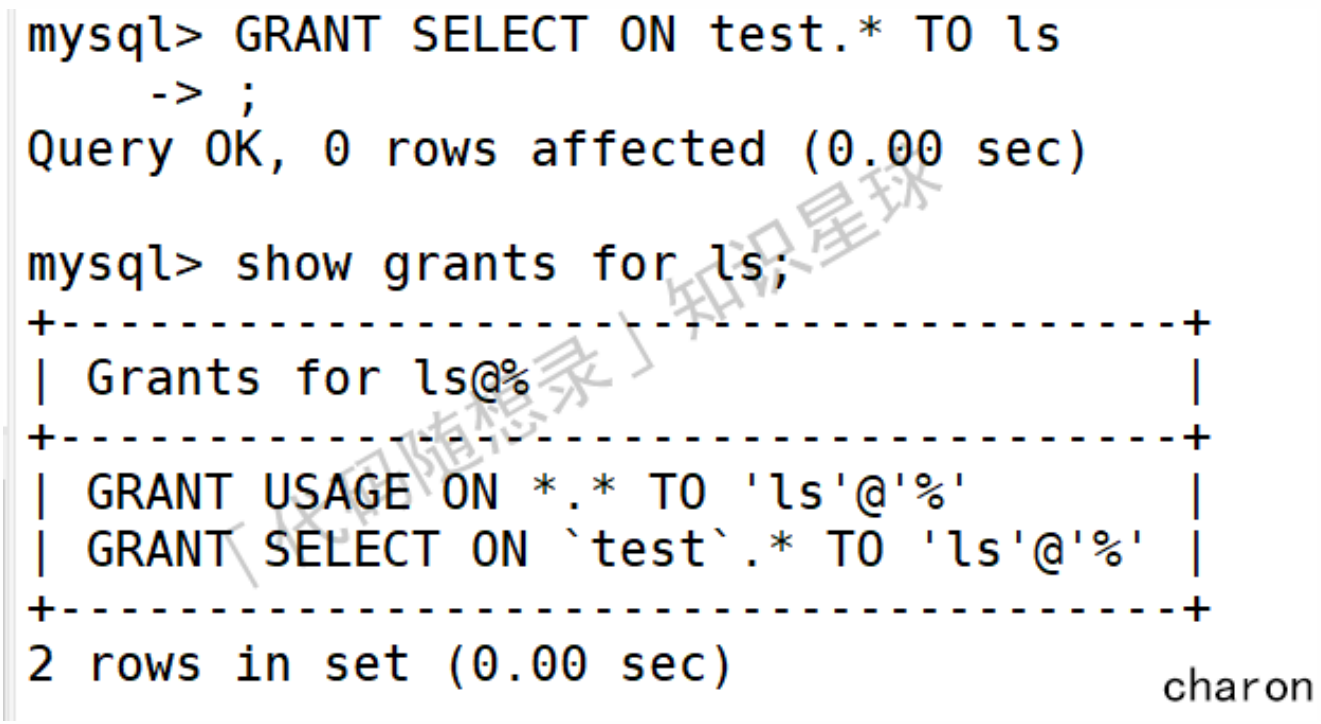
而GRANT USAGE ON *.* TO 'ls'@'%'则代表了用户ls没有任何权限。

4.2 用户授权

给用户授权的语法：

```
GRANT [权限名] ON [数据库名].[表名] TO [用户名]
```

例如：用户 'ls' 查找权限 select



从表格中可以看到 'ls' 新增的权限。

4.3 撤销授权

撤销用户权限的语法：

```
REVOKE [权限名] ON [数据库名].[表名] FROM [用户名]
```

收回用户 ls 查找权限 select

```
mysql> revoke select on test.* from ls;
Query OK, 0 rows affected (0.00 sec)

mysql> show grants for ls;
+-----------------------------------+
| Grants for ls@%                   |
+-----------------------------------+
| GRANT USAGE ON *.* TO 'ls'@'%' |
+-----------------------------------+
1 row in set (0.00 sec)
```

「代码随想录」知识星球

charon

从表格中可以看初 ls 新增的权限消失了

MySQL语句执行

查询语句运行

1、连接器

负责跟客户端建立连接、获取权限、维持和管理连接

Mysql 会定期自动清理"空闲"连接，由参数 wait_timeout 控制的，默认值是 8 小时。
由于建立连接比较复杂，所以尽量使用长连接，而不是 短连接（少量查询后，就断开连接）
但是，当 长连接 过多时，可能导致内存占用太大，被系统强行杀掉（OOM），即会导致 MySQL 异常重启

解决方案为:

1. 定期断开长连接
2. MySQL 5.7以后版本，可以通过执行 mysql_reset_connection 来重新初始化连接资源

2、查询缓存

执行查询语句前，先看下查询缓存中是否有结果

- 如果有，则不必执行查询语句，直接取出缓存结果
- 如果没命中缓存，则执行查询语句，并将执行查询语句后的结果，放入查询缓存中

Note: 不建议使用查询缓存（当 数据表频繁更新时，最新查询结果可能和查询缓存中存放的结果不一致）

MySQL 8.0 版本直接将查询缓存的整块功能删掉

3、分析器

1. 词法分析
2. 语法分析

4、优化器

优化器是在表里面有多个索引的时候，决定使用哪个索引；或者在一个语句有多表关联（join）的时候，决定各个表的连接顺序

5、执行器

执行语句时，首先判断当前用户是否有执行权限

- 如果没有执行权限，则终止
  * 如果有执行权限，则执行器就会根据表的引擎定义，去使用这个引擎提供的接口。

更新语句执行

1、redo log VS. binlob （3点不同)

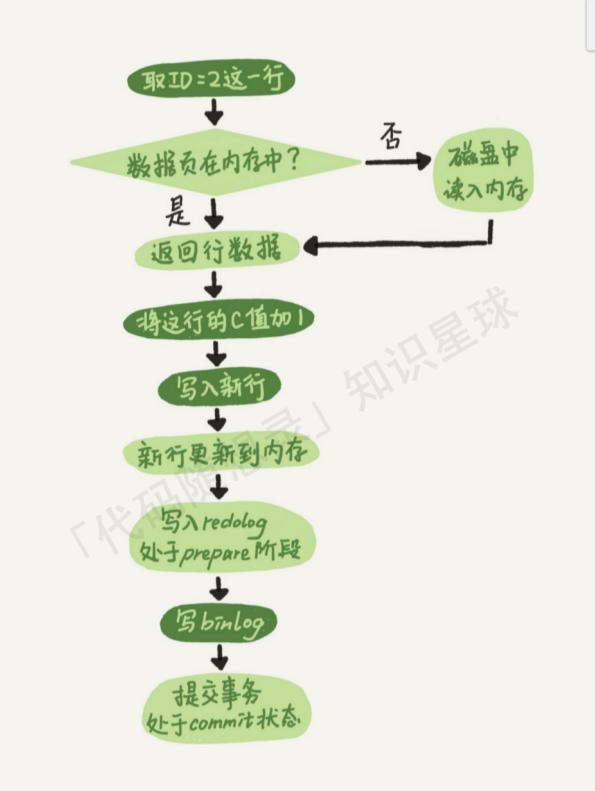
1. redo log 是 InnoDB 引擎特有的；binlog 是 MySQL 的 Server 层实现的，所有引擎都可以使用。
2. redo log 是物理日志，记录的是“在某个数据页上做了什么修改”；binlog 是逻辑日志，记录的是这个语句的原始逻辑，比如“给 ID=2 这一行的 c 字段加 1
3. redo log 是循环写的，空间固定会用完；binlog 是可以追加写入的。“追加写”是指 binlog 文件写到一定大小后会切换到下一个，并不会覆盖以前的日志。

2、数据恢复

先找到最新的 全量备份，恢复之；
然后，依次执行 该全量备份以后得所有 binlog

3、为什么必须有“两阶段提交”呢?

这是为了让两份日志之间的逻辑一致性



update 语句执行流程 posper

MySQL查询优化

查询优化就是MySQl会将程序员编写的一些比较耗费性能的语句进行查询重写

1、条件化简

2、移除不必要的括号

3、常量传递：

（1）某个表达式是某个列和某个常量的等值匹配，会直接用常量替换列名

4、移除没用的条件：

（1）比如永远为true或false：

5、表达式计算：

（1）表达式只包含常量的话，值会被计算出来
（2）如果某个列在函数中或者以运算形式出现，优化器不会进行化简

6、having和where子句的合并：

（1）查询语句中没有sum、max这样的聚集函数以及group子句的话，优化器会将having和where子句合并
（2）having子句用于分组后过滤，where用于分组前合并

7、常量表检测：

（1）使用主键等值匹配、使用唯一二级索引列等值匹配进行查询的表称为常量表
（2）优化器会优先执行常量表查询，因为速度非常快

8、外连接消除

9、优化器会将右连接转换为左连接查询

10、空值拒绝

（1）在外连接查询中，指定的where子句中包含被驱动表的列不为null值的条件（就是不允许查出来的记录中含有null值）
（2）被驱动表的where子句符合空值拒绝的条件后，外连接和内连接可以相互转换

11、子查询优化

12、子查询简介

在一个查询语句中的某个位置可以出现另一个查询语句，这另一个查询就叫子查询

（1）按出现位置分类

- 在select子句中

```
select (select m1 from t1 limit 1);
```

- 在from子句中

```
select m, n from (select ...);
```

将这种子查询的结果当做一个表，在from子句中的子查询称为派生表

- 在where或on子句中

```
select * from t1 where m1 in (select ...);
```

（2）按返回的结果集分类

- 标量子查询

只返回一个单一值

```
select (select m1 from t1 limit 1);
```

- 行子查询

返回一条记录，需要包含多个列

```
select * from t1 where (m1, n1) = (select m2, n2 from t2 limit 1);
```

使用limit 1保证子查询的结果只有一条记录

- 列子查询

返回一个列的数据，可能包含多条纪录

```
select * from t1 where m1 in (select m2 from t2);
```

- 表子查询

子查询的结果既有多条纪录，又有多个列

```
select * from t1 where (m1, n1) = (select m2, n2 from t2);
```

- 按外层查询关系分类
- 不相关子查询

子查询可以单独运行出结果，不依赖于外层查询的值

上面都是例子

- 相关子查询

子查询的执行需要依赖于外层查询的值

```
select * from t1 where m1 in (select m2 from t2 where n1 = n2);
```

子查询的注意事项：

1. 必须用小括号括起来
2. select子句中的子查询必须是标量子查询
3. 要想得到标量子查询/行子查询，应该使用limit 1
4. 对于[not] in/any/som/all子查询来说，子查询不允许出现limit
5. 子查询中不必使用order by：子查询相当于一个集合，集合没必要排序
6. 子查询中不必使用distinct，因为集合也不需要去重
7. 子查询中没有聚集函数以及having子句时，不必使用group by子句

13、子查询的执行方式

（1）不优化

- 对于不相关子查询，先执行子查询，再将子查询的结果作为外层查询的参数
- 对于相关子查询，先从外层查询中取一条记录，取相关列进行子查询，如此循环

（2）in子查询的优化

（3）物化

将子查询的结果写入临时表中，该临时表就是物化表

基于内存的物化表建立哈希索引，基于磁盘的物化表建立B+树索引

（4）半连接

s1表和s2表半连接的意思是：对于s1表的某条肌理，只关心在s2表中是否存在与之匹配的记录，而不关心与多少条记录与之匹配

这只是MySQL内部执行子查询的一种方式，不面向用户

（5）实现方式

（6）table pullout（表上拉）

查询列表只有主键或唯一索引列是，将子查询中的表上拉到外层查询的from子句中

（7）重复值消除

1. 用临时表消除半连接结果集的重复值
2. 松散扫描
3. 半连接物化

首次匹配

MySQL高性能

1. MySQL主从复制

1.1 概念

数据可以从一个MySQL数据库服务器主节点复制到一个或者多个从节点。

MySQL默认采用异步复制方式，这样从节点就不用一直访问主服务器来更新最新数据。

从节点可以复制主节点数据库中的所有数据库、特定的数据库或者特定的表。

1.2 用途

1、数据实时备份

当系统中某个节点发生故障时，可以方便故障切换

2、读写分离

在开发过程中，如果遇到某个sql语句需要锁表，导致暂时不能使用读的服务

使用主从复制，让主数据库负责写，从数据库负责读，即使主库出现锁表的情景，也可以通过从库正常读数据

3、架构扩展

随着系统中业务访问量的增加，如果是单机部署数据，会导致I/O访问频率过高

通过主从复制，增加多个数据存储结点，将负载分布在多个从节点上，降低单机的I/O访问频率，提高单机的I/O性能

1.3 原理

MySQL主从复制涉及到三个线程

一个运行在主节点： binary log dump thread
两个运行在从节点： I/O thread、SQL thread

1、binary log dump

当从节点连接主节点的时候，主节点创建该线程，用于发送bin-log内容

2、I/O thread

当从节点执行“start slave”命令之后，从节点会创建一个I/O线程用来连接主节点，请求其中的数据。I/O线程接收到主节点binlog dump的更新数据之后，保存在本地的relay log中

3、SQL线程

该线程负责读取relay log中的内容，解析或具体的操作并执行，最终保证主从数据的一致性

1.4 基本过程

1、从节点I/O进程连接主节点

请求指定日志文件的指定位置后面的内容

2、主节点接收到请求之后

通过负责复制的I/O进程根据请求的信息读取指定的日志位置之后的日志信息，返回给从节点。返回信息中除了日志所包含的指定日志信息还包含了本次返回信息的 bin-log file 以及 bin-log position

3、从节点的I/O线程接收到内容之后

将接收到的日志内容更新到本机的 relay log 中，并且把读取到的 binary log 文件名和位置保存到 master-info 文件中，方便下一次告知 master 从节点需要更新的位置

4、Slave 的 SQL 线程检测到 relay-log 中新增了内容

将 relay-log 的内容解析成在主节点上实际执行的操作，并在数据库中执行

主从复制模式：

1. 异步模式
2. 半同步模式
3. 全同步模式
4. GTID复制模式

2. 分库分表

2.1 原因

1、单库太大

单个数据库处理能力有限，所在的服务器上的磁盘空间也有限，单库存在I/O操作瓶颈。

主要方案：切分成更多更小的库

2、单表太大

CRUD都成问题，索引膨胀，查询超时。

主要方案：切分成多个数据集更小的表

2.2 拆分方案

1、垂直拆分

（1）垂直分表

1. “大表拆小表“，基于列的字段进行
2. 一般表中字段较多，将不常用的，数据较大的，长度较长的，拆分到“扩展表”

（2）垂直分库

1. 一般情况下，针对一个系统中不同业务进行拆分
2. 一般我们拆分之后，放到多个服务器上

2、水平拆分

（1）水平分表

针对数据量巨大的单张表（比如订单），按照某种规则（RANGE，HASH取模），切分到多张表中，这些表还在一个数据库中

（2）水平分库

将单张表的数据切分到多个服务器上，每个服务器都有相应的库和表，只是表中的数据集合不同

水平分库能够有效的缓解单机和单库的性能瓶颈，I/O，连接数和硬件资源等瓶颈

水平分库分表切分规则：

range： 根据范围，比如0-1000一个表，1001到2000一个表

hash取模： 比如取ID，进行hash取模，根据模数分配到不同的数据库中

地理区域： 按照地理范围进行划分

时间范围： 按照时间进行切分

MySQL高可用方案类型：

（1）基于主从复制

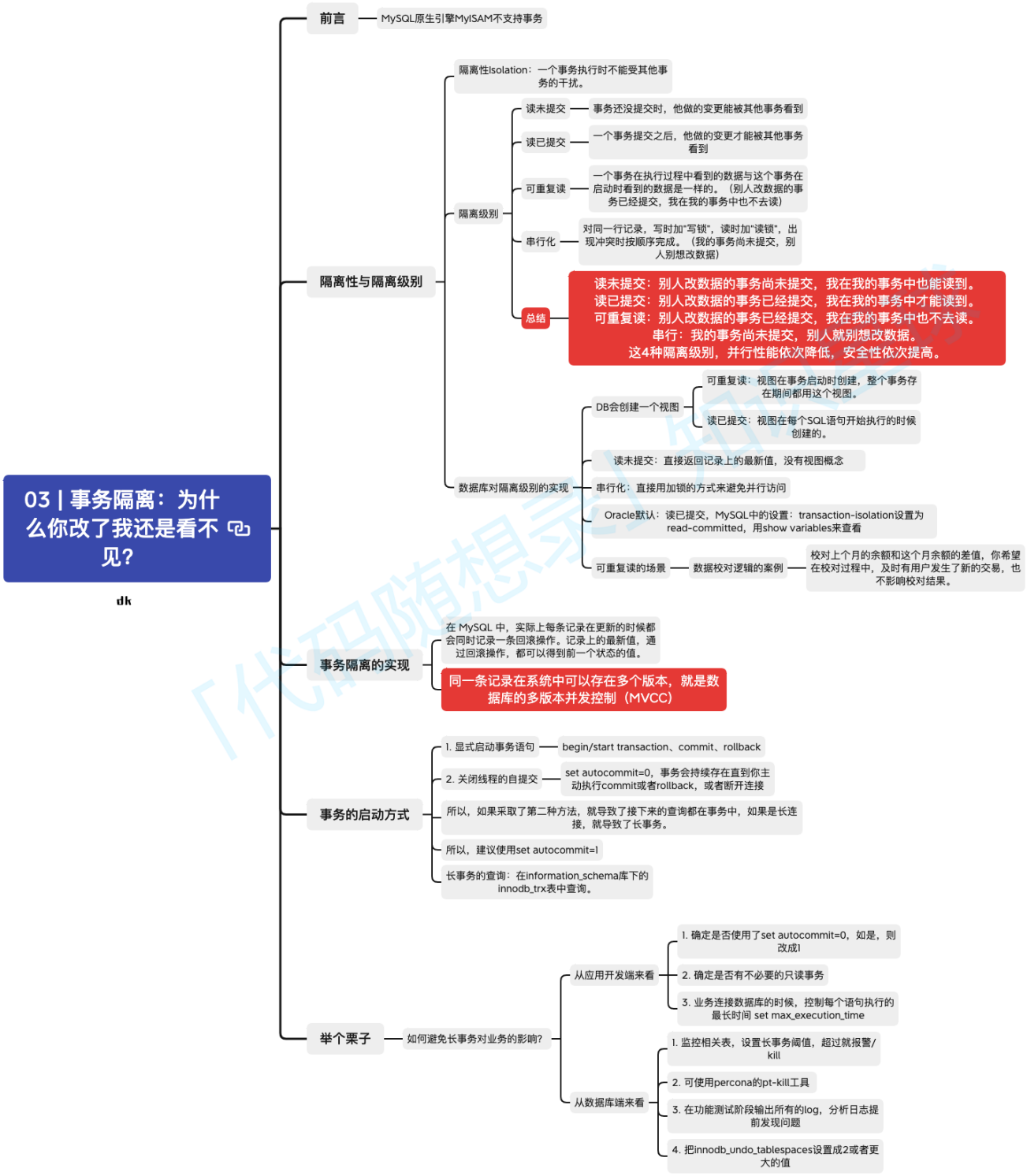
1. 一般情况下，采用双节点主从+keepalived/heartbeat方案
2. 在master节点发生故障以后，复用keepalived/heartbeat的高可用机制实现快速切换到slave节点

（2）基于Galera协议
（3）基于NDB协议
（4）基于中间件/proxy
（5）基于共享存储
（6）基于主机高可用

MySQL事务隔离

思维导图：

原图链接：「代码随想录」知识星球



MySQL原生引擎MyISAM不支持事务，所以被InnoDB取代。

事务， ACID（Atomicity、Consistency、Isolation、Durability，即原子性、一致性、隔离性、持久性）

redis 的隔离性

redis 的事务中的隔离性并没有保证原子性，已经做过的操作是不会rollback的。

它的隔离性是指其他事务不会干扰到它自己的事务。

隔离性与隔离级别

隔离性Isolation：

一个事务执行时不能受其他事务的干扰。

隔离级别

1、读未提交 read uncommitted

事务还没提交时，他做的变更能被其他事务看到

2、读已提交 read committed

一个事务提交之后，他做的变更才能被其他事务看到

3、可重复读 repeatable read

一个事务在执行过程中看到的数据与这个事务在启动时看到的数据是一样的。（别人改数据的事务已经提交，我在我的事务中也不去读）

4、串行化 serializable

对同一行记录，写时加“写锁”，读时加“读锁”，出现冲突时按顺序完成。（我的事务尚未提交，别人别想改数据）

总结：

读未提交：别人改数据的事务尚未提交，我在我的事务中也能读到。

读已提交：别人改数据的事务已经提交，我在我的事务中才能读到。

可重复读：别人改数据的事务已经提交，我在我的事务中也不去读。

串行：我的事务尚未提交，别人就别想改数据。

这4种隔离级别，并行性能依次降低，安全性依次提高。

数据库对隔离级别的实现

DB会创建一个视图

可重复读：

视图在事务启动时创建，整个事务存在期间都用这个视图。

读已提交：

视图在每个SQL语句开始执行的时候创建的。

读未提交：

直接返回记录上的最新值，没有视图概念

串行化：

直接用加锁的方式来避免并行访问

Oracle默认：

读已提交，MySQL中的设置：transaction-isolation设置为read-committed，用show variables来查看
可重复读的场景

数据校对逻辑的案例：

校对上个月的余额和这个月余额的差值，你希望在校对过程中，及时有用户发生了新的交易，也不影响校对结果。

事务隔离的实现

在 MySQL 中，实际上每条记录在更新的时候都会同时记录一条回滚操作。记录上的最新值，通过回滚操作，都可以得到前一个状态的值。

同一条记录在系统中可以存在多个版本，就是数据库的多版本并发控制（MVCC）

事务的启动方式

显式启动事务语句

```
begin/start transaction、commit、rollback
```

关闭线程的自提交

```
set autocommit=0;
```

事务会持续存在直到你主动执行commit或者rollback，或者断开连接，所以，如果采取了第二种方法，就导致了接下来的查询都在事务中，如果是长连接，就导致了长事务。

所以，建议使用 `set autocommit=1`

长事务的查询：

在informationschema库下的innodbtrx表中查询。

举个栗子

如何避免长事务对业务的影响?

从应用开发端来看:

1. 确定是否使用了set autocommit=0，如是，则改成1
2. 确定是否有不必要的只读事务
3. 业务连接数据库的时候，控制每个语句执行的最长时间 set maxexecutiontime

从数据库端来看:

1. 监控相关表，设置长事务阈值，超过就报警/kill
2. 可使用percona的pt-kill工具
3. 在功能测试阶段输出所有的log，分析日志提前发现问题
4. 把innodbundotablespaces设置成2或者更大的值

MySQL常见的分库分表

垂直分表

需求:

商品表字段太多，每个字段访问频次不一样，浪费了IO资源，需要进行优化

垂直分表介绍:

也就是“大表拆小表”，基于列字段进行的，拆分原则一般是表中的字段较多，将不常用的或者数据较大，长度较长的拆分到“扩展表 如text类型字段，访问频次低、字段大的商品描述信息单独存放在一张表中，访问频次较高的商品基本信息单独放在一张表中。

垂直拆分原则:

1. 把不常用的字段单独放在一张表;
2. 把text，blob等大字段拆分出来放在附表中;
3. 业务经常组合查询的列放在一张表中

垂直分库

需求:

C端项目里面，单个数据库的CPU、内存长期处于90%+的利用率，数据库连接经常不够，需要进行优化

垂直分库讲解:

垂直分库针对的是一个系统中的不同业务进行拆分， 数据库的连接资源比较宝贵且单机处理能力也有限，没拆分之前全部都是落到单一的库上的，单库处理能力成为瓶颈，还有磁盘空间，内存，tps等限制

拆分之后，避免不同库竞争同一个物理机的CPU、内存、网络IO、磁盘，所以在高并发场景下，垂直分库一定程度上能够突破IO、连接数及单机硬件资源的瓶颈

垂直分库可以更好解决业务层面的耦合，业务清晰，且方便管理和维护

一般从单体项目升级改造为微服务项目，就是垂直分库

仍然存在的问题：

垂直分库分表可以提高并发，但是依然没有解决单表数据量过大的问题

水平分表

需求：

当一张表的数据达到几千万时，查询一次所花的时间长，需要进行优化，缩短查询时间

都是大表拆小表

垂直分表：表结构拆分
水平分表：数据拆分

水平分表：

把一个表的数据分到一个数据库的多张表中，每个表只有这个表的部分数据，核心是把一个大表，分割N个小表，每个表的结构是一样的，数据不一样，全部表的数据合起来就是全部数据

针对数据量巨大的单张表（比如订单表），按照某种规则（RANGE,HASH取模等），切分到多张表里面去，但是这些表还是在同一个库中，所以单数据库操作还是有IO瓶颈，主要是解决单表数据量过大的问题

减少锁表时间，没分表前，如果是DDL(create/alter/add等)语句，当需要添加一列的时候mysql会锁表，期间所有的读写操作只能等待

水平分库

需求：

高并发的项目中，水平分表后依旧在单个库上面，1个数据库资源瓶颈 CPU/内存/带宽等限制导致响应慢，需要进行优化

水平分库：

把同个表的数据按照一定规则分到不同的数据库中，数据库在不同的服务器上。

水平分库是把不同表拆到不同数据库中，它是对数据行的拆分，不影响表结构，每个库的结构都一样,但每个库的数据都不一样，没有交集，所有库的并集就是全量数据

水平分库的粒度，比水平分表更大

分库分表总结

1、技术负责人前瞻性思维

1. 需要提前考虑系统一年到两年左右的业务增长情况
2. 对数据库服务器的QPS、连接数、容量等做合理评估和规划
3. 很多人觉得“分库分表”是宜早不宜迟-----其实不然

常规开发里面单表建议1千万内，推荐是百万级别单表存储，常规sql和索引优化先行，然后结合缓存+异步+nosql+mq

2、垂直角度（表结构不一样）

垂直分表：

将一个表字段拆分多个表，每个表存储部分字段

好处：

避免IO时锁表的次数，分离热点字段和非热点字段，避免大字段IO导致性能下降

原则：

业务经常组合查询的字段一个表；不常用字段一个表；text、blob类型字段作为附属表

垂直分库：

根据业务将表分类，放到不同的数据库服务器上

好处：

避免表之间竞争同个物理机的资源，比如CPU/内存/硬盘/网络IO

原则：

根据业务相关性进行划分，领域模型，微服务划分一般就是垂直分库

3、水平角度（表结构一样）

水平分库：

把同个表的数据按照一定规则分到不同的数据库中，数据库在不同的服务器上

好处：

多个数据库，降低了系统的IO和CPU压力

原则：

- 选择合适的分片键和分片策略，和业务场景配合
- 避免数据热点和访问不均衡、避免二次扩容难度大

水平分表：

- 同个数据库内，把一个表的数据按照一定规则拆分到多个表中，对数据进行拆分，不影响表结构
- 单个表的数据量少了，业务SQL执行效率高，降低了系统的IO和CPU压力

原则：

- 选择合适的分片键和分片策略，和业务场景配合
- 避免数据热点和访问不均衡、避免二次扩容难度大

4、互联网公司实际使用和跳槽面试

公司业务稳定发展，多数情况是为了解决【单库单表】数据量过多问题

重点是水平角度的【分库分表】

分库分表的问题

分库分表解决了问题，同时也带来了问题。

如跨库查询、分布式事务、分库之后的排序（翻页，函数的计算问题）、全局主键问题、容量规划（二次扩容问题）、技术选型问题

以下是一些常见的问题

1、跨节点数据库Join关联查询

数据库切分前：

多表关联查询，可以通过sql join进行实现

分库分表后：

数据可能分布在不同的节点上，sql join带来的问题就比较麻烦

2、分库操作带来的分布式事务问题

操作内容同时分布在不同库中，不可避免会带来跨库事务问题，即分布式事务

3、执行的SQL排序、翻页、函数计算问题

分库后：

数据分布再不同的节点上， 跨节点多库进行查询时，会出现limit分页、order by排序等问题

而且当排序字段非分片字段时，更加复杂了，要在不同的分片节点中将数据进行排序并返回，然后将不同分片返回的结果集进行汇总和再次排序（也会带来更多的CPU/IO资源损耗）

4、数据库全局主键重复问题

常规表的id是使用自增id进行实现，分库分表后，由于表中数据同时存在不同数据库中，如果用自增id，则会出现冲突问题

5、容量规划,分库分表后二次扩容问题

业务发展快，初次分库分表后，满足不了数据存储，导致需要多次扩容

6、分库分表技术选型问题

市场分库分表中间件相对较多，框架各有各的优势与短板，应该如何选择

数据库排序思路

1. 思路

整体的思路和海量数据排序类似，不过考虑更加好一些

正常思路：

在分割段的阶段，使用内部排序，生成n个大小等于可用内存的顺串，最后再进行归并，使得数据整体有序。

但是，为了**避免I/O操作带来的影响**，所以**使用替换-选择排序的方式**，可以使得在分割段阶段生成的顺串大小大于可用内存大小。

同时为了能够再次减小I/O开销，合并阶段，可适量的增加归并的路数

不过增大路数也就意味着内部每个缓冲区数量增加（一个缓冲区放一个顺串），每次要在更多的缓冲区中选取最小值，所以引入败者树（和堆排很像），减少增加归并路数，而影响内部归并。

2. 练习题与小总结

1. 练习内部排序的各种算法
   LC_排序数组
2. 练习二路归并
   LC_归并排序

总结：

为了提升效率，避免速度不匹配的问题，真的是精益求情，OS里面有各种高速缓存、缓冲区等，数据库会利用磁盘的预读，顺序性读，以及设计特殊的数据结构。

MySQL常见问题

1-1：什么是幻读?

幻读是指在同一个事务中，存在前后两次查询同一个范围的数据，但是第二次查询却看到了第一次查询没看到的行。

注意，幻读出现的场景：

1. 事务的隔离级别为可重复读，且是当前读
2. 幻读仅专指新插入的行

1-2：幻读带来的问题?

1. 对行锁语义的破坏
2. 破坏了数据一致性

1-3：怎么避免幻读?

存储引擎采用加间隙锁的方式来避免出现幻读

1-4：为啥会出现幻读?

行锁只能锁定存在的行，针对新插入的操作没有限定

1-5：间隙锁是啥？它怎么避免出现幻读的？它引入了什么新的问题?

间隙锁：

是专门用于解决幻读这种问题的锁，它锁的了行与行之间的间隙，**能够阻塞新插入的操作**

间隙锁的引入也带来了一些新的问题，比如：降低并发度，可能导致死锁。

注意，读读不互斥，读写/写读/写写是互斥的，但是间隙锁之间是不冲突的，间隙锁会阻塞插入操作

另外，间隙锁在可重复读级别下才是有效的

间隙锁和 next-key lock：

行锁和间隙锁合称 next-key lock，这个锁是左开右闭的区间。

MySQL 为了解决幻读问题，在线程更新数据并 next-key lock 的过程中，首先必须在可重复读的隔离级别下，执行以下的原则和优化：

原则：

原则 1：加锁的基本单位是 next-key lock，next-key lock 是前开后闭区间。
原则 2：查找过程中访问到的对象才会加锁。

优化：

优化 1：索引上的等值查询，给唯一索引加锁的时候，next-key lock 退化为行锁，如果不存在这个索引，退化为间隙锁。

优化 2：索引上的等值查询，向右遍历时且最后一个值不满足等值条件的时候，next-key lock 退化为间隙锁。

注意：

1. 非唯一索引的范围查询：范围查询都会访问到不满足条件的第一个值，并且不会执行上述的两个优化。唯一索引的范围查询仍旧会执行上述的优化。
2. delete 语句的和查询的加锁方式相同
3. limit 语句，遍历到满足条件的 n 条数据后，之后不再加 next-key lock

死锁：不同线程可以给同一个间隙加锁。

意思时只有我这个线程可以操控这个间隙，其他线程不能使用，加锁时不会检测是否冲突的，但是如果两个都给同一个间隙上锁，之后两个线程都没办法在这个间隙上更新数据了，都会陷入等待另一个线程的间隙锁释放，也就是死锁。

如果使用读提交隔离级别，那么只加行锁，不加间隙锁，语句执行过程中加上的行锁，在语句执行完成后，就要把“不满足条件的行”上的行锁直接释放了，不需要等到事务提交才释放。

2-1：for update的使用场景

一般这些操作都是很长一串并且是开启事务的。

如果库存刚开始读的时候是1，而立马另一个进程进行了update将库存更新为0了，而事务还没有结束，会将错的数据一直执行下去，就会有问题。

需要 for upate 进行数据加锁防止高并发时候数据出错，即 **使事务保持 当前读** 状态。

```
SELECT * FROM table LIMIT [offset,] rows | rows OFFSET offset;
# 如果只给定一个参数，表示记录数。
```

```
mysql> SELECT * FROM orange LIMIT 5;     # 检索前5条记录(1-5)
```

相当于

```
mysql> SELECT * from orange LIMIT 0,5;
```

两个参数，第一个参数表示offset, 第二个参数为记录数。

```
mysql> SELECT * FROM orange LIMIT 10,15; // 检索记录11-25
```

如果你想要清除一些MySQL使用内部缓存，你应该使用FLUSH命令

2-2：事务隔离

innodb 支持 RC 和 RR 隔离级别实现是用的一致性视图(consistent read view)

事务在启动时会拍一个快照,这个快照是基于整个库的。

基于整个库的意思就是说一个事务内,整个库的修改对于该事务都是不可见的(对于快照读的情况)

如果在事务内 select t 表,另外的事务执行了DDL t表,根据发生时间,只有两种情况

1. 报错
2. 锁住

事务是如何实现mvcc的?

（1）每个事务都有一个事务ID,叫做transaction id(严格递增)

（2）事务在启动时,找到已提交的最大事务ID记为up_limit_id。

（3）事务在更新一条语句时

比如 id=1 改为了 id=2.会把 id=1 和该行之前的 row trx_id 写到 undo log 里, 并且在数据页上把 id 的值改为 2,并且把修改这条语句的transaction id 记在该行行头

（4）再定一个规矩

一个事务要查看一条数据时,必须先用该事务的 up_limit_id 与该行的transaction id 做比对

- 如果 up_limit_id >= transaction id,那么可以看
- 如果 up_limit_id < transaction id,则只能去 undo log 里去取。

去 undo log 查找数据的时候,也需要做比对,必须 up_limit_id > transaction id，才返回数据

2-3：什么是当前读

由于当前读都是先读后写,只能读当前的值,所以为当前读会更新事务内的 up_limit_id 为该事务的 transaction

你提到了隐藏列有一个DB_ROW_ID，是干嘛的？那假设有10个update，到第九个回滚了，DB_ROLL_PTR如何做的，那提交了是否更新DB_ROLL_PTR?

1、为什么 rr 能实现可重复读而 rc 不能,分两种情况

（1）快照读的情况下：

rr不能更新事务内的 up_limit_id ,而 rc 每次会把 up_limit_id 更新为快照读之前最新已提交事务的 transaction id,则 rc 不能可重复读

（2）当前读的情况下：

rr 是利用 record lock+gap lock来实现的,而 rc 没有 gap,所以 rc 不能可重复读

2-3：选择普通索引还是唯一索引?

1、对于查询过程来说：

（1）普通索引

查到满足条件的第一个记录后，继续查找下一个记录，知道第一个不满足条件的记录

（2）唯一索引

由于索引唯一性，查到第一个满足条件的记录后，停止检索

但是，两者的性能差距微乎其微。因为InnoDB根据数据页来读写的。

2、对于更新过程来说：

概念：change buffer

当需要更新一个数据页：

- 如果数据页在内存中就直接更新
- 如果不在内存中，在不影响数据一致性的前提下，InnoDB 会将这些更新操作缓存在change buffer 中。

下次查询需要访问这个数据页的时候，将数据页读入内存，然后执行 change buffer 中的与这个页有关的操作。

change buffer 是**可以持久化**的数据。在内存中有拷贝，也会被写入到磁盘上

purge：将 change buffer 中的操作应用到原数据页上，得到最新结果的过程，成为 purge

访问这个数据页会触发 purge，系统有后台线程定期 purge，在数据库正常关闭的过程中，也会执行 purge

唯一索引的更新不能使用 change buffer

change buffer 用的是 buffer pool 里的内存，change buffer 的大小，可以通过参数 innodb_change_buffer_max_size 来动态设置。这个参数设置为50的时候，表示 change buffer 的大小最多只能占用 buffer pool 的50%。

将数据从磁盘读入内存涉及随机IO的访问，是数据库里面成本最高的操作之一。

change buffer 因为减少了随机磁盘访问，所以对更新性能的提升很明显。

change buffer使用场景：

在一个数据页做purge之前，change buffer记录的变更越多，收益就越大。

对于写多读少的业务来说，页面在写完以后马上被访问到的概率比较小，此时change buffer的使用效果最好。这种业务模型常见的就是账单类、日志类的系统。

反过来，假设一个业务的更新模式是写入之后马上会做查询，那么即使满足了条件，将更新先记录在change buffer，但之后由于马上要访问这个数据页，会立即触发purge过程。

这样随机访问IO的次数不会减少，反而增加了change buffer的维护代价。所以，对于这种业务模式来说，change buffer反而起到了副作用。

索引的选择和实践：

尽可能使用普通索引。

redo log主要节省的是随机写磁盘的IO消耗(转成顺序写)，而change buffer主要节省的则是随机读磁盘的IO消耗。

2-4：char 和 varchar 的区别？

char 是固定长度类型，比如：定义 char(10)，当你输入"abc"三个字符的时候，它们占的空间还是 10 个字节，其他 7 个是空字节。varchar 长度可变

存储情况不同

以 compact 行格式为例：

一条完整的记录由两部分组成

1. 变长字段的真正数据内容
2. 该变长字段所占用的字节数

该变长字段所占用的字节数被放在变长字段列表中，并且按照列的顺序逆序存放。

而 char 类型的数据则只需要存储其真正的数据内容。

对于 类型为 char(M) 的列

如果采用的是变长编码的字符集：

那么该列的值占用的字节数也会被存储到变长字段列表中，采用 变长编码字符集的 CHAR(M) 类型的值要求至少占用 M 个字节，但是 varchar(M) 没有这个要求。

2-5：MySQL抖一下是什么意思？

因为运行的不正常了，或者不稳定了，要花费更多的资需源处理别的事情，会使SQL语句的执行效率明显变慢。

针对innoDB导致MySQL抖的原因，主要是InnoDB 会在后台刷脏页，而刷脏页的过程是要将内存页写入磁盘。所以，无论是你的查询语句在需要内存的时候可能要求淘汰一个脏页，还是由于刷脏页的逻辑会占用 IO 资源并可能影响到了你的更新语句，都可能是造成你从业务端感知MySQL"抖"了一下的原因。

2-6：MySQL抖一下有啥问题？

很明显系统不稳定，性能突然下降对业务端是很不友好的

2-7：怎么让MySQL不抖？

设置合理参数配配置，尤其是设置 好innodb_io_capacity 的值，并且平时要多关注脏页比例，不要让它经常接近 75%

3-1：啥是脏页?

当内存数据页跟磁盘数据页内容不一致的时候，我们称这个内存页为“脏页”。

3-2：啥是干净页?

内存数据写入到磁盘后，内存和磁盘上的数据页的内容就一致了，称为“干净页”。

3-3：脏页是咋产生的?

因为使用了WAL技术，这个技术会把数据库的随机写转化为顺序写，但副作用就是会产生脏页。

3-4：啥是随机写？为啥那么耗性能?

随机写我的理解是，这次写磁盘的那个扇区和上一次没啥关系，需要重新定位位置，机械运动是很慢的即使不是机械运动重新定位写磁盘的位置也是很耗时的。

3-6：啥是顺序写?

顺序写我的理解是，这次写磁盘那个扇区就在上一次的下一个位置，不需要重新定位写磁盘的位置速度当然会快一些。

3-7：WAL怎么把随机写转化为顺序写的?

写redolog是顺序写的，先写redolog等合适的时候再写磁盘，间接的将随机写变成了顺序写，性能确实会提高不少。

3-8：为啥删除了表的一半数8据，表文文件大小没耍化？ 、

因为delete 命令其实只是把记录的位置，或者数据页标记为了“可复用”，但磁盘文件的大小是不会变的。也可以认为是一种逻辑删除，所以物理空间没有实际释放，只是标记为可复用，表文件的大小当然是不变的啦！

4-1：表的数据信息存在哪里?

表数据信息可能较小也可能巨大无比，可以存储在共享表空间里，也可以单独存储在一个以.ibd为后缀的文件里，由参数 innodb_file_per_table 来控制，建议总是作为一个单独的文件来存储，这样非常容易管理，并且在不需要的时候，使用 drop table 命令也能直接把对应的文件删除，如果存储在共享空间之中即使表删除了空间也不会释放。

4-2：表的结构信息存在哪里?

首先，表结构定义占有的存储空间比较小

MySQL8.0 之前：表结构的定义信息存在以.frm为后缀的文件里
MySQL8.0 之后：则允许把表结构的定义信息存在系统数据表之中

系统数据表，主要用于存储MySQL的系统数据，比如：数据字典、undo log(默认)等文件

4-3：如何才能删除表数据后，表文件大小就耍小?

重建表，消除表因为进行大量的增删改操作而产生的空洞，使用如下命令：

```
alter table t engine=InnoDB
optimize table t( 等于 recreate+analyze)。
truntace table t (等于drop+create)
```

5-1：空洞是啥？咋产生的？

空洞就是那些被标记可复用但是还没被使用的存储空间。

使用delete命令删除数据会产生空洞，标记为可复用

插入新的数据可能引起页分裂，也可能产生空洞

修改操作，有时是一种先删后插的动作也可能产生空洞

按照效率排序的话：

count(字段) < count(主键 id) < count(1) ≈ count(*)

5-2：count(*)这么慢，我该怎么办？

要么忍，要么自己动手记录。。。

5-3：count()的语义是啥？

首先，不同的存储引擎实现方式不同

MyISAM 引擎把一个表的总行数存在了磁盘上，因此执行 count(*) 的时候会直接返回这个数，效率很高；

而 InnoDB 引擎就麻烦了，它执行 count(*) 的时候，需要把数据一行一行地从引擎里面读出来，然后累积计数。

以下针对innodb来说

count() 是一个聚合函数，对于返回的结果集，一行行地判断，如果 count 函数的参数不是 NULL，累计值就加 1，否则不加，最后返回累计值。

5-4：count(字段)怎么计数？

1、如果这个“字段”是定义为 not null 的话

一行行地从记录里面读出这个字段，判断不能为 null，按行累加；

2、如果这个“字段”定义允许为 null

执行的时候，判断到有可能是 null，还要把值取出来再判断一下，不是 null 才累加。

从引擎返回的字段会涉及到解析数据行，以及拷贝字段值的操作。

5-5：count(主键 id)怎么计数？

对于 count(主键 id) 来说，InnoDB 引擎会遍历整张表，把每一行的 id 值都取出来，返回给 server 层。

server 层拿到 id 后，判断是不可能为空的，就按行累加。从引擎返回的 主键id 会涉及到解析数据行，以及拷贝字段值的操作。

5-5：count(1)怎么计数？

对于 count(1) 来说，InnoDB 引擎遍历整张表，但不取值。server 层对于返回的每一行，放一个数字“1”进去，判断是不可能为空的，按行累加。

5-6：count(*)怎么计数?

对于count(*)来说，并不会把全部字段取出来，而是专门做了优化，不取值。

count(*) 肯定不是 null，按行累加。

这些count()的原则如下：

server 层要什么就给什么；

InnoDB 只给必要的值；

现在的优化器只优化了 count(*) 的语义为“取行数”，其他“显而易见”的优化并没有做。

6-1：order by 是怎样工作的?

example：

```
select city,name,age from t where city='杭州' order by name limit 1000;
```

涉及到用户语句的排序，mysql 会给每个线程分配一块内存用于排序，也就是 sort_buffer。

这条语句的执行逻辑是：

1、先初始化 sort_bufer

2、然后放入 city,name,age 字段，不断地由主键id索引到整行再到三个字段的值，匹配查找的值存入 sort_buff

3、然后按 name 排序，返回前 1000 个值

但是如果 sort_buffer_size 设置的太小，无法存放所有匹配的字段，排序就无法在内存中完成

需要借鉴磁盘临时文件辅助排序，可以通过 number_of_tmp_files 这个标识来判断是否使用，其实这个原理和对超大数据的排序相同。

如果要记录的字段太长，这样内存里能够同时放下的行数很少，要分成很多个临时文件，排序的性能会很差。

这时会换一个算法，叫做rowid排序，顾名思义，就是对主键 id 以及排序字段进行存放，这样就节省了空间，但是最后需要通过主键 id 去找到之前未取出的字段。对比全字段排序，rowid 排序多访问了一次表 t 的主键索引。

结论：

如果 MySQL 实在是担心排序内存太小，会影响排序效率，才会采用 rowid 排序算法，这样排序过程中一次可以排序更多行，但是需要再回到原表去取数据。

如果 MySQL 认为内存足够大，会优先选择全字段排序，把需要的字段都放到 sort_buffer 中，这样排序后就会直接从内存里面返回查询结果了，不用再回到原表去取数据。

如果想要避免排序，可以建立对应字段的索引。

如果想要进一步避免主键 id 的回表查询，可以使用覆盖索引，这种情况的索引建立成本会比较大，需要你去自己权衡是否使用

6-2：如何正确地显示随机消息?

example：

英语学习 App 首页有一个随机显示单词的功能，也就是根据每个用户的级别有一个单词表，然后这个用户每次访问首页的时候，都会随机滚动显示三个单词。他们发现随着单词表变大，选单词这个逻辑变得越来越慢，甚至影响到了首页的打开速度

1、order by rand() 实现

创建临时表→按主键顺序取出所有的单词，并给他们一个随机小数→初始化 sort_buffer，从内存临时表取出数据放入 sort_buffer，按随机数排序，取出前三个，总扫描行数是2×n+3

2、磁盘临时表

如果 sort_buffer_size 设置的小，就会使用到磁盘临时表用于辅助排序，当然 mysql 高版本使用了优先队列排序的方法，就是只取三个值，构成一个堆，然后不断把剩下的值输入，用于更新队列。

这个过程不需要临时文件，因此对应的 number_of_tmp_files 是 0。当然，如果堆的大小设置的很大，sort_buffer 放不下，就会舍弃优先队列，重新使用临时表+归并排序来实现，number_of_tmp_files 就不再为 0。

3、随机id方法

我们实际上并不需要真正的为每行赋予一个随机值，然后进行排序，这样的成本就太高了。

可以借鉴主键id的唯一性，生成一个介于主键id最小值和最大值之间的随机数，然后取不小于这个随机数的主键id的那一行即可。总扫描行数是3，的那hi是如果主键id有空洞，就会导致选择不同行的概率不同，不是真正的随机

4、严格随机id方法

取出表的总行数，生成一个总行数对应的随机值，取不小于这个随机数的主键id的那一行，总扫描行数是C+Y+1

6-3：为什么SQL语句逻辑相同，性能却差异巨大?

案例一：条件字段函数操作

```
select count(*) from tradelog where month(t_modified)=7;
```

并且在 t_modified 上建立了索引

你以为对字段生成了索引，应该会返回的很快，但是mysql有一个规则：就是如果对字段做了函数计算，就用不上索引了。

实际上，mysql使用的B+ 树提供的对于索引的快速定位能力，来源于同一层兄弟节点的有序性。对索引字段做函数操作，可能会破坏索引值的有序性，因此优化器就决定放弃走树搜索功能，转而进行全表扫描，所以运行就变慢了

案例二：隐式类型转换：

如果在限制语句中错误的将字符串与数字进行比较，mysql 会将可以转化成数字的字符串转化成数字，没法转换的话，就会变成数字 0，因此如果出现这种输入的失误，相当于调用了字符串转数字的函数，还是会使得优化器会放弃走树搜索功能，进行全表扫描

案例三：隐式字符编码转换

如果执行两个表的联合查询，两个表通过外键进行联结，如果两个表使用的字符集不同，会对低水平的字符集执行升级转换函数，（相当于将int赋给double的这个思路）。

如果限制语句是 where d.tradeid=l.tradeid，如果d的字符集低水平，就会执行函数操作，使得优化器会放弃走树搜索功能，进行全表扫描

总结：对索引字段做函数操作，可能会破坏索引值的有序性，因此优化器就决定放弃走树搜索功能

6-4：为什么我只查一行的语句，也执行这么慢?

原因从大到小可分为三种情况

第一 MySQL 数据库本身被堵住了，比如：系统或网络资源不够

第二 SQL 语句被堵住了，比如：表锁，行锁等，导致存储引擎不执行对应的 SQL 语句

第三确实是索引使用不当，没有走索引，类比上一节

第四是表中数据的特点导致的，走了索引，但由于一致性读，需要回滚多次才能读取出当前事务对应的值。

6-5：为什么临时表可以重名?

在实际应用中，临时表一般用于处理比较复杂的计算逻辑。

由于临时表是每个线程自己可见的，所以不需要考虑多个线程执行同一个处理逻辑时，临时表的重名问题。在线程退出的时候，临时表也能自动删除，省去了收尾和异常处理的工作。

在 binlog_format='row'的时候，临时表的操作不记录到 binlog 中，也省去了不少麻烦，防止出现主备不一致的情况

这里说到的临时表是用户临时表，而不是内存临时表

6-5：group-by使用原则

如果对 group by 语句的结果没有排序要求，要在语句后面加 order by null；

尽量让 group by 过程用上表的索引，确认方法是 explain 结果里没有 Using temporary 和 Using filesort；

如果 group by 需要统计的数据量不大，尽量只使用内存临时表；也可以通过适当调大 tmp_table_size 参数，来避免用到磁盘临时表；

如果数据量实在太大，使用 SQL_BIG_RESULT 这个提示，来告诉优化器直接使用排序算法得到 group by 的结果，不要使用内存临时表。

6-6：一张表可以存储多少条信息

假设采用的是InnoDB引擎

那么数据结构的角度来看，本着**索引即数据**的思路，就是求一个 B+树 能够存储多少数据，根据 B+树 定义也就转变成了 **2 个子问题**：

1. 每个叶子结点存储能存储多少条记录
2. 一共有多少个叶子结点

1、每个叶子结点能存储多少条记录

假设一条记录的大小为1kb，InnoDB中默认每页为16kb，则一个叶子节点可存储16条数据

2、一共有多少个叶子结点

根据B+树的形式，确定了该B+树为几阶的B+树，以及树的高度，就能确定叶子节点有最多有多少个。

非叶子节点存储的数据为关键子+指针。关键字也就是我们常说的主键，如果为int类型则4字节，指针大小也假设为4字节。

以关键字+指针为基本单位，那么一个非叶子节点最多能够存储 16384 / 8 = 2048 个，2048个指针能够引出2048个子节点，如果该B+树的高度为2，则存储的记录数量为2048 *16 = 32768条记录，如果高度为3则有2048* 2048 * 16 = 67108864条记录。（也能反向得出，一般索引高度都在3左右吧）

6-5：有一个联合索引(A,B,C),select * from test where a,c会不会使用索引?

不会，根据索引的创建过程，这棵 b+ 树的叶子节点的记录以及 二级索引记录 是按照 abc 的规则来排序的，也就是说，先按照 a的大小来排序，a 相等的情况下，再按照 b 的大小来排序，b 相等的话，最后再是 c 。这样的话，符合 where 条件的 记录不一定相邻，

也就是说，并不能在某一个区间中找，还是得一条条找，查询效率并没有变高。

MySQL架构升级之路

数据库的演变升级

1、单机

（1）请求量大查询慢
（2）单机故障导致业务不可用

2、主从

（1）数据库主从同步，从库可以水平扩展，满足更大读需求
（2）但单服务器TPS，内存，IO都是有限的
（3）可以考虑用主节点负责业务，从节点负责统计

3、双主

（1）用户量级上来后，写请求越来越多
（2）一个Master是不能解决问题的，添加多了个主节点进行写入
（3）多个主节点数据要保存一致性，写操作需要2个master之间同步更加复杂

4、分库和分表

自研工具类、tddl、shardingsphere、mycat等

【面试题】业务增长-数据库性能优化：

这边有个数据库-单表1千万数据，未来1年还会增长多500万，性能比较慢，说下你的优化思路

思路：

千万不要一上来就说分库分表，这个是最忌讳的事项，一定要根据实际情况分析，两个角度思考

不分库分表：

（1）软优化：

1. 数据库参数调优
2. 分析慢查询SQL语句，分析执行计划，进行sql改写和程序改写
3. 优化数据库索引结构
4. 优化数据表结构优化
5. 引入NOSQL和程序架构调整

（2）硬优化

提升系统硬件（更快的IO、更多的内存）：带宽、CPU、硬盘

（3）分库分表

根据业务情况而定，选择合适的分库分表策略（没有通用的策略），外卖、物流、电商领域。

先看只分表是否满足业务的需求和未来增长，数据库分表能够解决单表数据量很大的时,数据查询的效率问题。

无法给数据库的并发操作带来效率上的提高，分表的实质还是在一个数据库上进行的操作，受数据库IO性能的限制
如果单分表满足不了需求，再分库分表一起。

结论：在数据量及访问压力不是特别大的情况，首先考虑缓存、读写分离、索引技术等方案，如果数据量极大，且业务持续增长快，再考虑分库分表方案。

5、MySQL数据库分库分表后带来的优点

分库分表解决的现状问题：

可以解决数据库本身的瓶颈以及系统本身的瓶颈

解决数据库本身瓶颈：

连接数：

连接数过多时，就会出现‘too many connections’的错误，访问量太大或者数据库设置的最大连接数太小的原因
Mysql默认的最大连接数为100可以修改，而mysql服务允许的最大连接数为16384。

数据库分表可以解决单表海量数据的查询性能问题

数据库分库可以解决单台数据库的并发访问压力问题

解决系统本身IO、CPU瓶颈

磁盘读写IO瓶颈，热点数据太多，尽管使用了数据库本身缓存，但是依旧有大量IO,导致sql执行速度慢

网络IO瓶颈，请求的数据太多，数据传输大，网络带宽不够，链路响应时间变长
CPU瓶颈，尤其在基础数据量大单机复杂SQL计算，SQL语句执行占用CPU使用率高，也有扫描行数大、锁冲突、锁等待等原因

可以通过 show processlist; 、show full processlist，发现 CPU 使用率比较高的SQL

常见的对于查询时间长，State 列值是 Sending data，Copying to tmp table，Copying to tmp table on disk，Sorting result，Using filesort 等都是可能有性能问题SQL，清楚相关影响问题的情况可以kill掉

也存在执行时间短，但是CPU占用率高的SQL，通过上面命令查询不到，这个时候最好通过执行计划分析explain进行分析

Redis基础

Redis基本结构

String：

可以是字符串、整数或者浮点数

Redis 的字符串有两种存储方式，在长度特别短时，使用 embstr 形式存储(embeded)，而当长度超过 44 字节时，使用 raw 形式存储。

命令：

get、set、del

list：

一个链表，链表上的每个节点都包含了一个字符串

命令：

rpush、lrange、index、lpop

set：

包含字符串的无序收集器，并且被包含的每个字符串都是独一无二的

命令：

sadd、smember、sismember、srem

hash：

包含键值对的无序散列表

命令：

hset、hget、hgetall、hdel

zset：

字符串的成员与浮点数分值之间有序映射元素的顺序由分值大小决定

命令：

zadd、zrange、zrangebyscore、zrem

跳表（skiplist）

Redis zset 是一个复合结构，一方面它需要一个 hash 结构来存储 value和 score 的对应关系，另一方面需要提供按照 score 排序的功能，还需要能够指定 score 的范围来获取 value 列表的功能。所以需要跳表来帮忙实现

Redis 的跳跃列表共有64 层，可容纳 2 ^ 64 个元素。

kv 之间使用指针串起来形成了双向链表结构，它们是有序排列的，从小到大。不同的 kv 层高可能不 样，层数越高的 kv 越少。同一层的 kv 会使用指针串起来。每一个层元素的遍历都是从 kv header 出发。

Hash

1、介绍

字典，又称为符号表(symbol table)、关联数组(associative array)或映射(map), 是一种用于保存键值对(key-value pair)的抽象数据结构，Redis的字典是使用哈希作为底层实现的。

哈希对象的编码可以是ziplist或者hashtable。当哈希对象可以同时满足以下两个条件时，哈希对象使用ziplist编码:

- 哈希对象保存的所有键值对的键和值的字符串长度都小于64字节;
- 哈希对象保存的键值对数量小于512个;

不能满足这两个条件的哈希对象需要使用hashtable编码。

通过hash-max-ziplist-value和hash-max-ziplist-entries选项可以更改两个条件的上限值，这个和list使用ziplist或者quicklist编码条件类似。

2、Zplist介绍

介绍

3、hash与ziplist

当我们为某个key第一次执行 hset key field value 命令的时候，Redis会创建一个hash结构，这个新创建的hash底层就是一个ziplist。每执行一次hset命令，插入的field和value分别作为一个新的数据项插入到ziplist中（即每次hset产生两个数据项）。

zaplist 使用上面的配置进行限制原因:

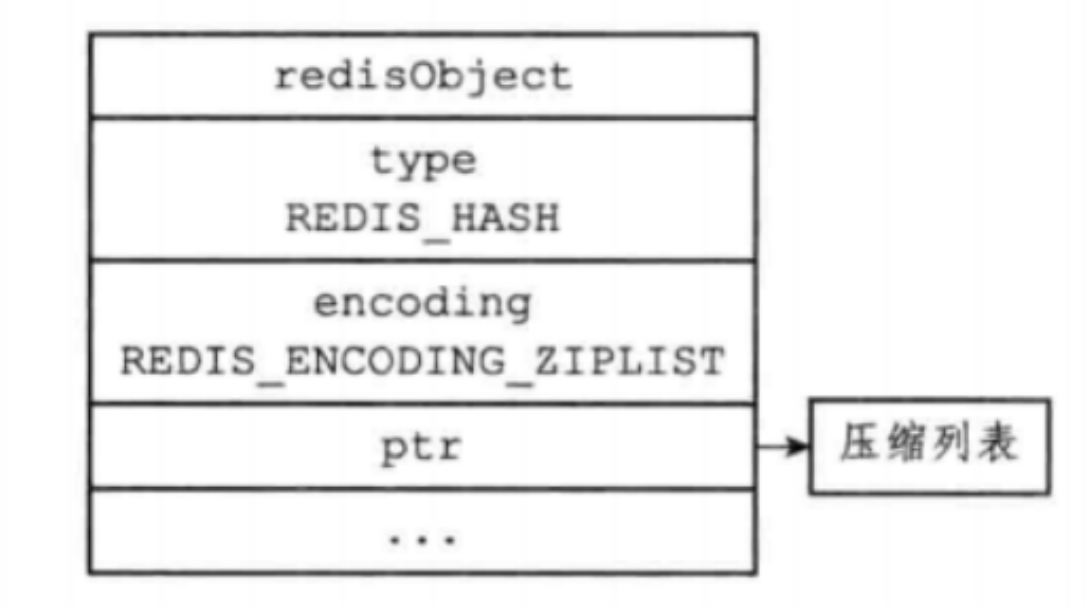
每次插入或修改引发的realloc操作会有更大的概率造成内存拷贝，从而降低性能。

一旦发生内存拷贝，内存拷贝的成本也相应增加，因为要拷贝更大的一块数据。

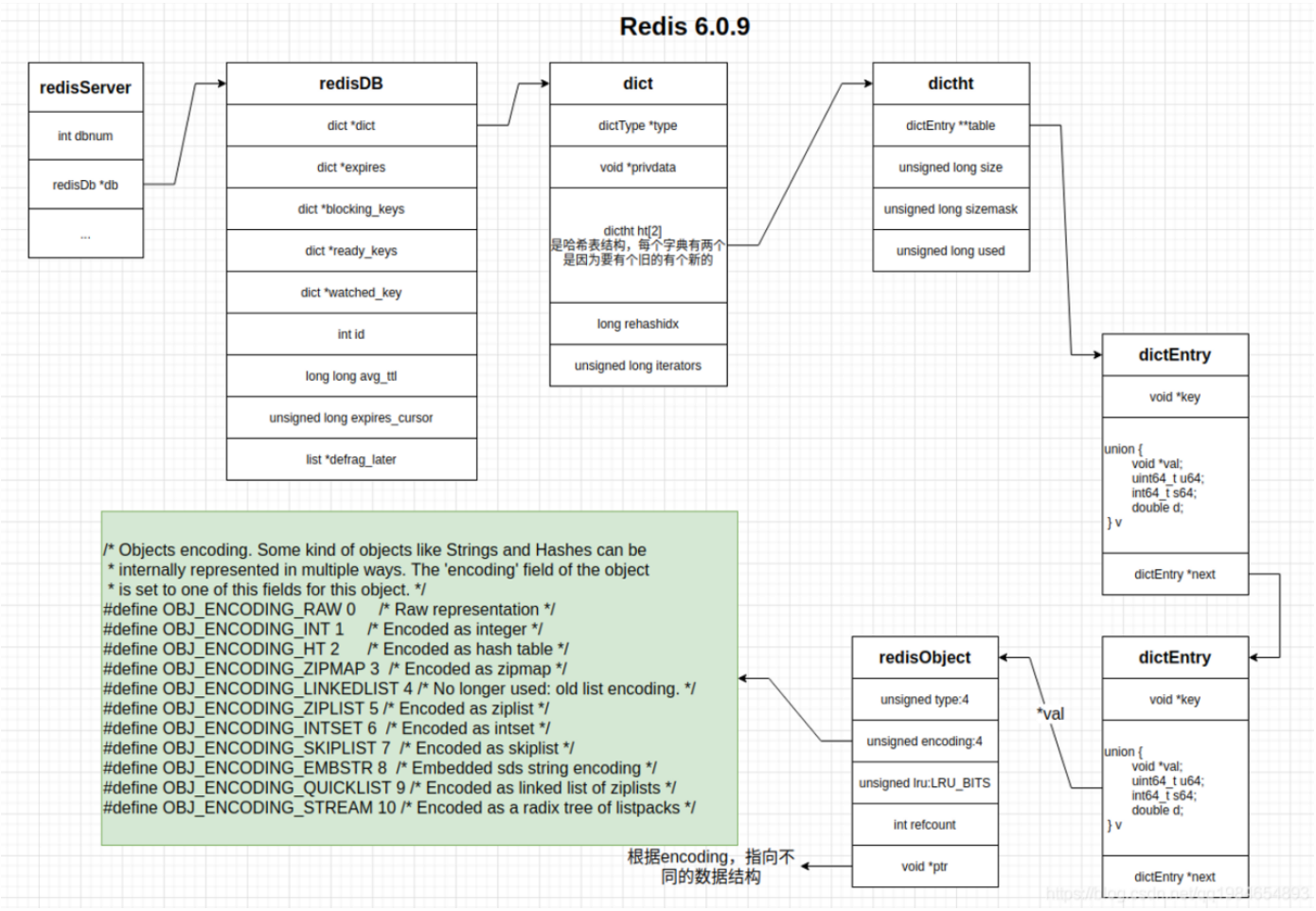
当ziplist数据项过多的时候，在它上面查找指定的数据项就会性能变得很低，因为ziplist上的查找需要进行遍历。

如果 ziplist 占据内存太大，重新分配内存和拷贝内存就会有很大的消耗，所以zip list 不适合存储大型字符串，存储的元素也不宜过多。

ziplist编码的hash对象:



配合Redis整体图食用更佳：



4、dict相关结构体

dict结构：

```
typedef struct dict {
    dictType *type;  // 类型特定函数
    void *privdata;  // 私有数据
    dictht ht[2];    // hash表
    long rehashidx; /* rehash索引，不在rehash时值为-1 */
    unsigned long iterators; /*当前正在进行遍历的iterator的个数*/
} dict;
```

解释：

type属性和privdata属性是针对不同类型的键值对，为创建多态字典而设置的。type属性是一个指向dictType结构的指针，每个dictType结构保存了一簇用于操作特定类型键值对的函数，Redis 会为用途不同的字典设置不同的类型特定函数。privdata属性保存需要传给那些类型特定函数的可选参数

dictType结构：

```
typedef struct dictType {
    uint64_t (*hashFunction)(const void *key); //计算hash值的函数
    void *(*keyDup)(void *privdata, const void *key);//复制键的函数
    void *(*valDup)(void *privdata, const void *obj);//复制值的函数
    int (*keyCompare)(void *privdata, const void *key1, const void *key2);//对比键的函数
    void (*keyDestructor)(void *privdata, void *key);//销毁键的函数
    void (*valDestructor)(void *privdata, void *obj);//销毁值的函数
} dictType;
```

hash结构：

```
typedef struct dictht {
dictEntry **table;   // hash表
unsigned long size;  // hash表大小
    unsigned long sizemask;//
    unsigned long used;
} dictht;
```

hash节点：

```
typedef struct dictEntry {
    void *key;   // key
    union {
        void *val;  -->指向redisObject类型 然后根据需要选择对应的type和encoding
        uint64_t u64;
        int64_t s64;
        double d;
    } v;
    struct dictEntry *next; //hash冲突时指向另一个 dictEntry 结构的指针
} dictEntry;
```

三者关系图：

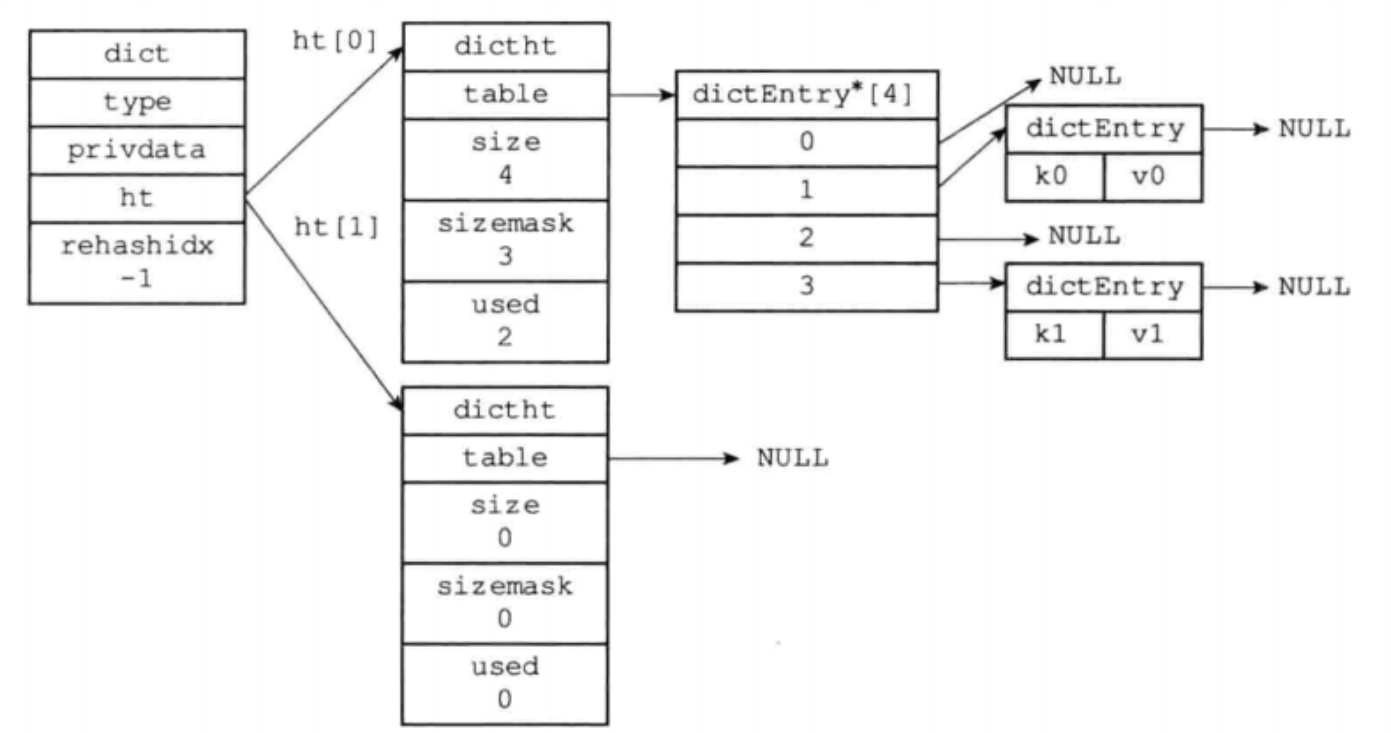
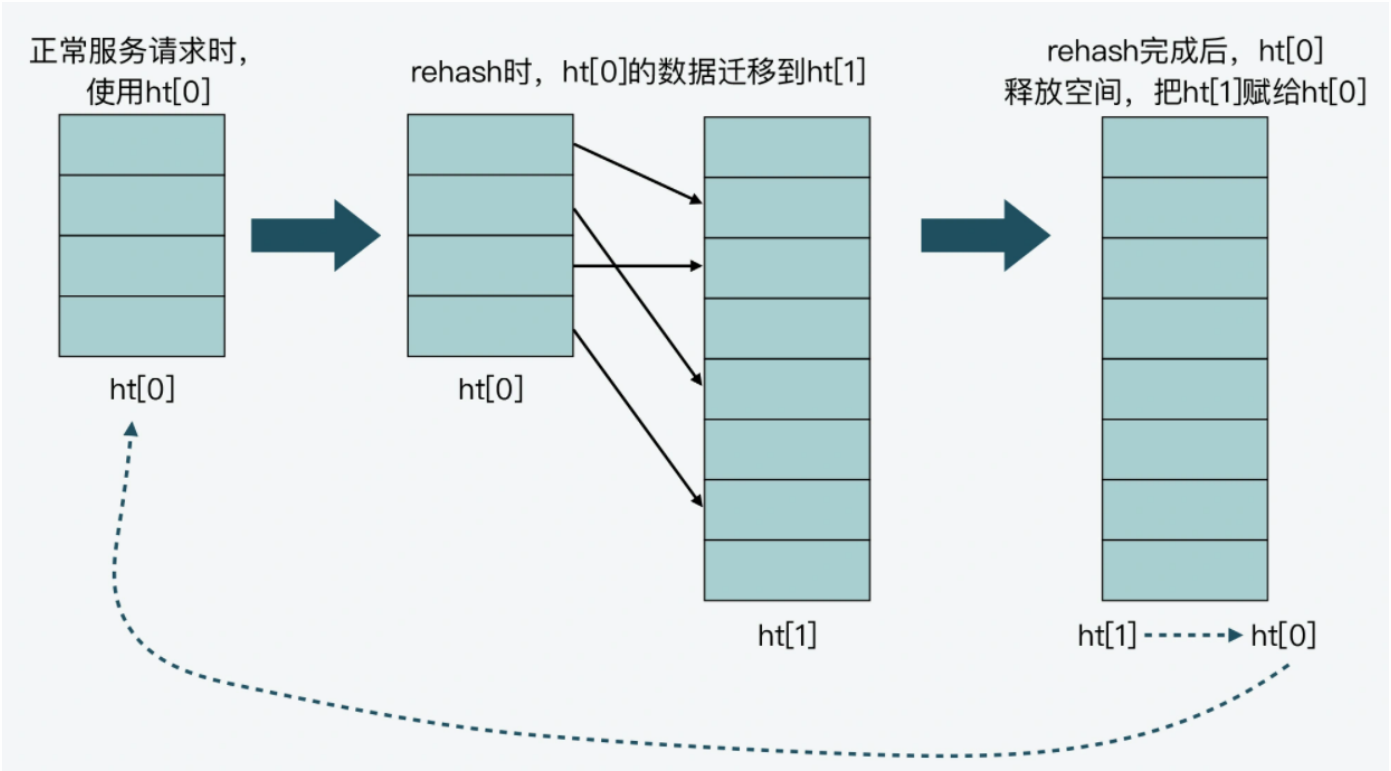


图 4-3　普通状态下的字典

5、Rehash

当Hash 表的桶过少而key值较多时，就会导致不同的key通过hash函数映射到了相同的的桶，这些key通过*Next形成链表，若hash冲突太严重时，当查找时某个key可能会导致时间复杂度从O(1)到O(N),为了解决这种情况，当hash中节点个数和桶个数触发负载因子时候，就需要发生ReHash。

Redis 准备了两个哈希表，用于 rehash 时交替保存数据即上图的`ht[0]`和`ht[1]`



6、什么时候触发rehash

Redis 用来判断是否触发 rehash 的函数是 *dictExpandIfNeeded*。dictExpandIfNeeded 函数中定义了三个扩容条件。

条件一：`ht[0]`的大小为 0。Redis初始化后就不满足了
条件二：`ht[0]`承载的元素个数已经超过了`ht[0]`的大小，同时 Hash 表可以进行扩容
条件三：`ht[0]`承载的元素个数，是`ht[0]`的大小的 dict_force_resize_ratio 倍，其中，dict_force_resize_ratio 的默认值是 5。

实际上二和三和负载因子有关。而负载因子 = 哈希表已保存节点数量/哈希表大小。

- 若负载因子>5 Hash 表已经过载比较严重了，需要立刻进行库扩容
- 而当负载因子大于等于 1 时，Redis 还会再判断 dict_can_resize 这个变量值，查看当前是否可以进行扩容

7、dict_can_resize内容

dict_can_resize 这个变量值是在 dictEnableResize 和 dictDisableResize 两个函数中设置的，它们的作用分别是启用和禁止哈希表执行 rehash 功能，这两个函数又被封装在了 updateDictResizePolicy 函数中。

updateDictResizePolicy 函数是用来启用或禁用 rehash 扩容功能的，这个函数调用 dictEnableResize 函数启用扩容功能的条件是：当前没有 RDB 子进程，并且也没有 AOF 子进程。

_dictExpandIfNeeded 函数会根据 Hash 表的负载因子以及能否进行 rehash 的标识，判断是否进行rehash，而 updateDictResizePolicy 函数会根据 RDB 和 AOF 的执行情况，启用或禁用rehash。

所以说rehash情况：

当负载因子>5：需要马上进行扩容；
当负载因子>=1：若此时Redis正在发生持久化操作，就禁止hash扩容，否则可以扩容；
当负载因子<0.1：则需要进行缩容，减少桶节点。

rehash 扩容扩多大?

如果当前表的已用空间大小为 size，那么就将表扩容到 size*2 的大小。

8、rehash 如何执行

Hash 表在执行 rehash 时，由于 Hash 表空间扩大，原本映射到某一位置的键可能会被映射到一个新的位置上，因此，很多键就需要从原来的位置拷贝到新的位置。而在键拷贝时，由于 Redis 主线程无法执行其他请求，所以键拷贝会阻塞主线程，这样就会产生 rehash 开销。

所以采用渐进式rehash。渐进式rehash采用的是一种分而治之的方式，将rehash的操作分摊在每一个的访问中，避免集中式rehash而带来的庞大计算量。redis中的hash则是执行的单步rehash的过程。

渐进式 rehash有两个关键函数：

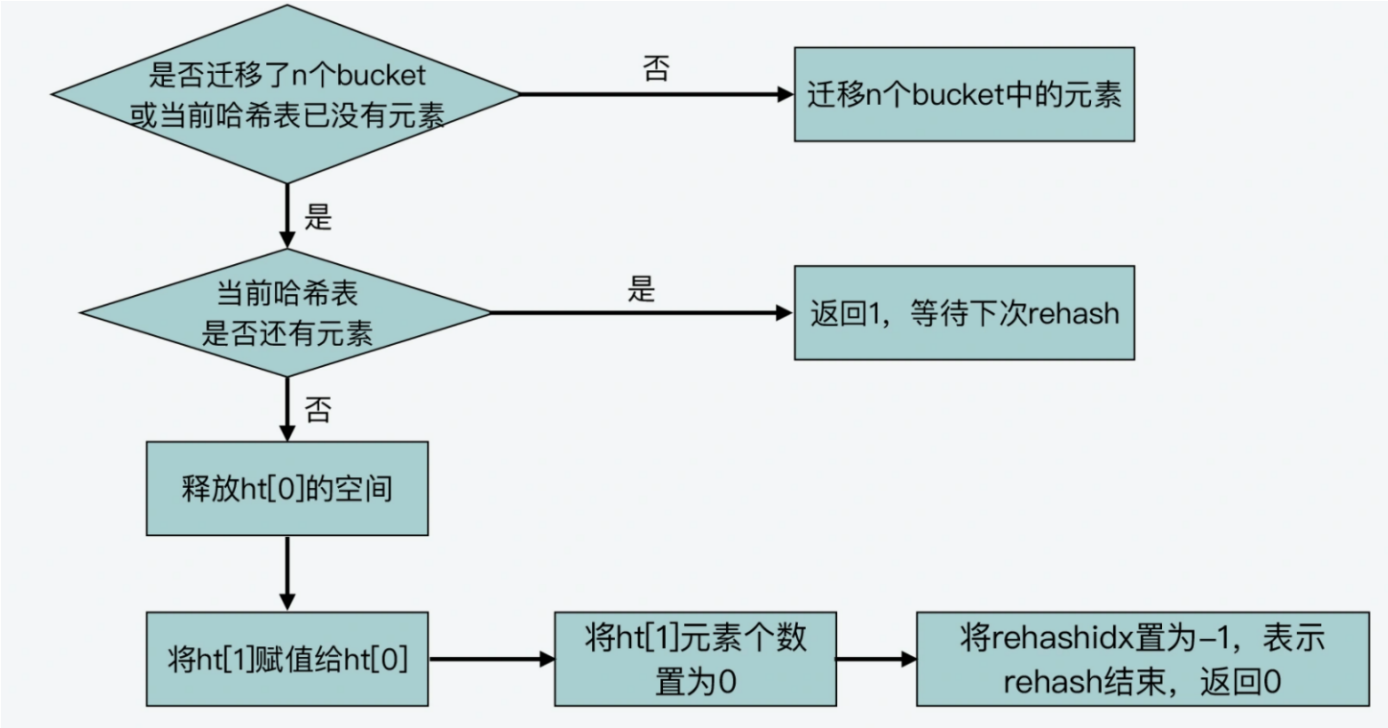
dictRehash 和 _dictRehashStep。

dictRehash主要执行流程：

主要代码

```
int dictRehash(dict *d, int n) {
    int empty_visits = n*10;
    // ...
    //主循环，根据要拷贝的bucket数量n，循环n次后停止或ht[0]中的数据迁移完停止
    while(n-- && d->ht[0].used != 0) {
```

```
        ...
    }
    //判断ht[0]的数据是否迁移完成
    if (d->ht[0].used == 0) {
        //ht[0]迁移完后，释放ht[0]内存空间
        zfree(d->ht[0].table);
        //让ht[0]指向ht[1]，以便接受正常的请求
        d->ht[0] = d->ht[1];
        //重置ht[1]的大小为0
        _dictReset(&d->ht[1]);
        //设置全局哈希表的rehashidx标识为-1，表示rehash结束
        d->rehashidx = -1;
        //返回0，表示ht[0]中所有元素都迁移完
        return 0;
    }
    //返回1，表示ht[0]中仍然有元素没有迁移完
    return 1;
}
```



渐进式 rehash 是如何按照 bucket 粒度拷贝数据的，这其实就和全局哈希表 dict 结构中的 rehashidx 变量相关。rehashidx 变量表示的是当前 rehash 在对哪个 bucket 做数据迁移而 dictRehash 函数的主循环，首先会判断 rehashidx 指向的 bucket 是否为空，如果为空（可能本来就没存数据），那就将 rehashidx 的值加 1，检查下一个 bucket。

因为可能连续几个 bucket 都为空，所以渐进式 rehash 在执行时设置了一个变量 empty_visits，用来表示已经检查过的空 bucket，当检查了一定数量的空 bucket 后，这一轮的 rehash 就停止执行，转而继续处理外来请求，避免了对 Redis 性能的影响。

而如果 rehashidx 指向的 bucket 有数据可以迁移，那么 Redis 就会把这个 bucket 中的哈希项依次取出来，并根据 ht[1]的表空间大小，重新计算哈希项在 ht[1]中的 bucket 位置，然后把这个哈希项赋值到 ht[1]对应 bucket 中。

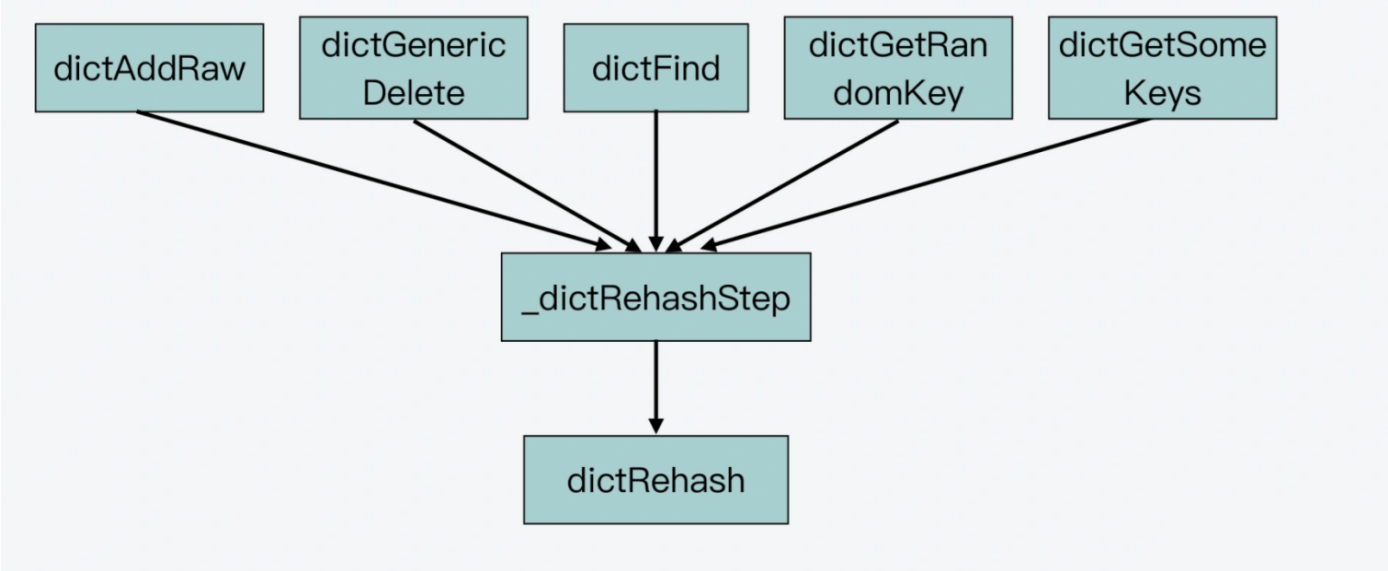
_dictRehashStep

_dictRehashStep 这个函数实现了每次只对一个 bucket 执行 rehash。

dictAddRaw，dictGenericDelete，dictFind，dictGetRandomKey，dictGetSomeKeys通过调用_dictRehashStep进而调用dictRehash。其中，dictAddRaw 和 dictGenericDelete 函数，分别对应了往 Redis 中增加和删除键值对，而后三个函数则对应了在 Redis 中进行查询操作。

这 5 个函数调用的 _dictRehashStep 函数，给 dictRehash 传入的循环次数变量 n 的值都为 1，每次迁移完一个 bucket，Hash 表就会执行正常的增删查请求操作，这就是在代码层面实现渐进式 rehash 的方法。

几者关系图：



Kaito大佬的部分评论总结：

全局哈希表 在触发渐进式 rehash 的情况有 2 个：

增删改查哈希表时：

每次迁移 1 个哈希桶(_dictRehashStep函数)

定时 rehash：

如果 dict 一直没有操作，无法渐进式迁移数据，那主线程会默认每间隔 100ms 执行一次迁移操作。

dict 在负载因子超过 1 时（used: bucket size >= 1），会触发 rehash。但如果 Redis 正在 RDB 或 AOF rewrite，为避免父进程大量写时复制，会暂时关闭触发 rehash。但这里有个例外，如果负载因子超过了 5（哈希冲突已非常严重），依旧会强制做 rehash

dict 在 rehash 期间，查询旧哈希表找不到结果，还需要在新哈希表查询一次。

redis分片

概念：将数据拆分到多个redis实例的过程

功能：

1. 允许使用很多电脑的内存总和来支持更大的数据库
2. 允许伸缩计算能力到多核或者多服务器，伸缩网络带宽到多服务器或者多网络适配器

redis分片方式：哈希分片

1. 使用哈希函数对键名转换为一个数字
2. 对这个数字进行取模，取模的基数等于redis实例的对象

旁路缓存

什么情况下考虑使用缓存:

1. 不需要实时更新又极其消耗数据库的数据
2. 需要实时更新，但更新的频率不高

简单来说，就是需要在应用程序中新增缓存逻辑处理的代码。Redis就是旁路缓存，因为需要应用程序调用它。电脑内存里的磁盘文件就不是旁路缓存，因为是自动调用的，对应用程序透明。

Redis 做缓存的两种模式:

1、只读缓存

加强读请求性能。查询数据时，缓存缺失需要从DB加载。更新数据时到DB更新，Redis上的老数据直接删除。

2、读写缓存：

读操作和只读缓存一样。写操作分为同步直写和异步写回两种模式，根据实际的业务场景需求来进行选择：

（1）同步直写模式

Redis和DB同时删改写回。侧重于保证数据可靠性。

（2）异步写回模式

只写Redis，数据要被淘汰时再写回DB。侧重于提供低延迟访问。

替换策略

(缓存满了怎么办)

Redis总共有**8种淘汰策略**：

1、不进行数据淘汰的策略，

1种： no-eviction

no-enviction：禁止驱逐数据

2、会进行数据淘汰的策略：

（1）在设置了过期时间的数据中进行淘汰

4种：volatile-ttl、volatile-random、volatile-lru、volatile-lfu。

1. volatile-lru：从已设置过期时间的数据集中，挑选最近最少使用的数据淘汰
2. volatile-ttl：从已设置过期时间的数据集中，选择将要过期的数据进行淘汰
3. volatile-random：从已设置过期时间的数据集中，随机选择数据进行淘汰

（2）所有数据范围内进行淘汰

3种：allkeys-random、allkeys-lru、allkeys-lfu。

1. allkeys-lru：从数据集中，挑选最近最少使用的数据淘汰
2. allkeys-random：从数据集中，随机选择数据进行淘汰

传统的LRU算法，需要维护一个大链表，随着数据访问，更新数据在链表中的位置。

Redis 中对 LRU 算法进行了简化，简单来说就是**增加了一个随机选取候选集合的操作**：

1、记录每个数据的访问时间

Redis 默认会记录每个数据的最近一次访问的时间戳（由键值对数据结构 RedisObject 中的 lru 字段记录）。

2、随机选取一个集合

Redis 在决定淘汰的数据时，第一次会随机选出 N 个数据，把它们作为一个候选集合。

3、淘汰集合中lru字段最小的数据

接下来，Redis 会比较这 N 个数据的 lru 字段，把 lru 字段值最小的数据从缓存中淘汰出去。

4、需要再次淘汰数据时

选取集合外，lru 字段值小于候选集合中最小的 lru 值的数据，进入候选集合，直到选满集合。重复步骤3。

这样，Redis就不用维护一个大链表了，也不用频繁的进行链表数据交换操作。

Redis缓存大小选取建议：结合实际应用的数据总量、热数据的体量，以及成本预算，设置在总数据量的 15% 到 30% 这个区间。

redis与memcache的区别

1、存储内容

1. redis只能存正常数据
2. memcache还可以存图片，视频等

2、支持类型

1. redis不仅仅支持k/v结构
2. memcache是k/v系统

3、数据安全

1. redis可以持久化
2. memcache不能持久化

4、应用场景

1. redis不仅仅能作为NoSQL数据库使用，还能用途消息队列、数据堆栈、数据缓存等
2. memcache适合缓存sql语句，数据集，用户临时数据等

Redis 备份

持久化机制：

RDB：

在指定的时间间隔内将内存中的数据集快照写入磁盘

AOF

以日志的形式记录服务器所处理的每一个写操作，redis服务器启动之初，会读取该日志来重新构建数据库，以保证启动后的数据库是完整的。

AOF 日志是写后日志，"写后" 的意思是 Redis 是先执行命令，把数据写入内存，然后才记录日志，Redis是内存和日志(写后日志)，mysql是磁盘数据和日志(写前日志)。

Redis 使用写后日志这一方式的好处：

1. 可以避免出现记录错误命令的情况。
2. 它是在命令执行后才记录日志，所以不会阻塞当前的写操作。

AOF三种写回策略：

Always，同步写回；Everysec，每秒写回；No，操作系统控制的写回。

DPNO

配置项	写回时机	优点	缺点
Always	同步写回	可靠性高，数据基本不丢失	每个写命令都要落盘，性能影响较大
Everysec	每秒写回	性能适中	宕机时丢失1秒内的数据 最多是丢失2s
No	操作系统控制的写回	性能好	宕机时丢失数据较多

「代码随想录」知识星球

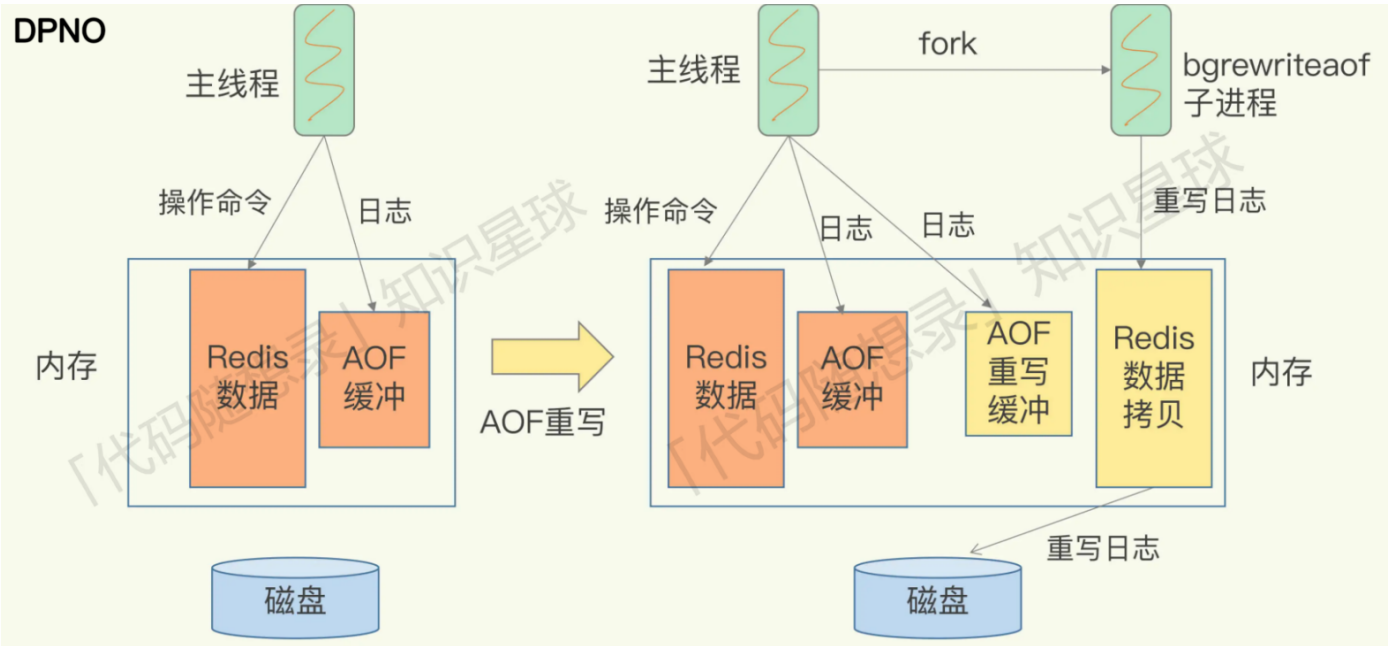
写回策略选择：

想要获得高性能，就选择 No 策略；如果想要得到高可靠性保证，就选择 Always 策略；如果允许数据有一点丢失，又希望性能别受太大影响的话，那么就选择 Everysec 策略。

AOF重写

AOF通过AOF重写的方式进行减小AOF文件大小

AOF重写过程：一处拷贝，两处日志



AOF 重写的四个触发时机：

- 时机一：bgrewriteaof 命令被执行。
- 时机二：主从复制完成 RDB 文件解析和加载（无论是否成功）。
- 时机三：AOF 重写被设置为待调度执行。
- 时机四：AOF 被启用，同时 AOF 文件的大小比例超出阈值，以及 AOF 文件的大小绝对值超出阈值。

另外，这里还需要注意，在这四个时机下，其实都**不能有正在执行的 RDB 子进程和 AOF 重写子进程**，否则的话，AOF 重写就无法执行了。

AOF 重写的基本执行过程。AOF 重写和 RDB 创建的过程类似，它也是创建了一个子进程来完成重写工作。这是因为 AOF 重写操作，实际上需要遍历 Redis server 上的所有数据库，把每个键值对以插入操作的形式写入日志文件，而日志文件又要进行写盘操作。所以，Redis 源码使用子进程来实现 AOF 重写，这就避免了阻塞主线程，也减少了对 Redis 整体性能的影响。

Redis 扩容

集群：

动态增加 Redis 节点

主从同步、读写分离：

对 Redis 的访问分为读和写

hash算法：

分成多个实例进行存储，增加 Redis 服务器数量。在客户端对存储的 key 进行 hash 运算，存入不同的服务器中，读取时，也进行相同的 hash 运算，找到对应的 redis 服务器

Redis 主从复制

Redis 提供了主从库模式，以保证数据副本的一致，主从库之间采用的是读写分离的方式。

读操作：主库、从库都可以接收；

写操作：首先到主库执行，然后，主库将写操作同步给从库。

全量复制

主从库间进行第一次复制操作（全量复制）

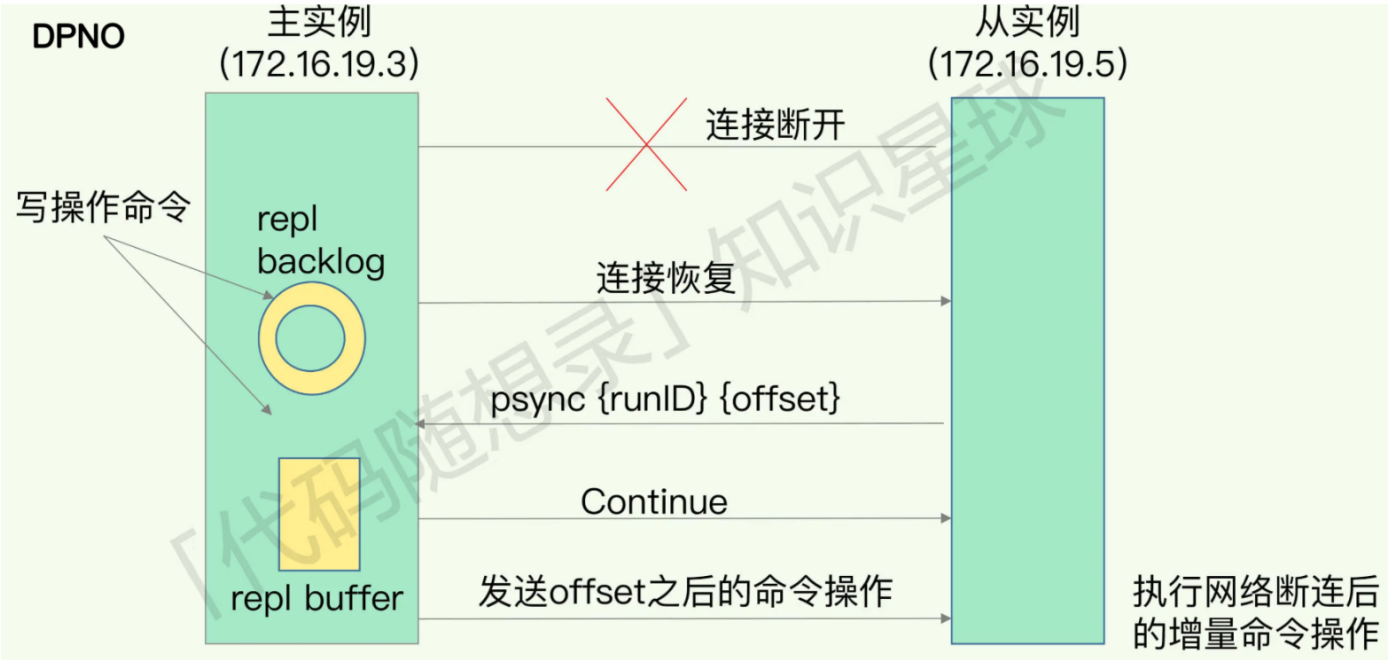
第一阶段是从库和主库建立起连接，并告诉主库即将进行同步，主库确认回复后，主从库间就可以开始同步了。

第二阶段主库收到 psync 命令后，会用 FULLRESYNC 响应命令带上两个参数：主库 runID 和主库目前的复制进度 offset，返回给从库。

第三个阶段，主库会把第二阶段执行过程中新收到的写命令，再发送给从库。

增量复制

从库发生宕机，重新连接后数据的同步操作（增量复制）



若从库发生宕机，主库会把断连期间收到的写操作命令，写到repl_backlog_buffer中，当从库重连后，从库首先会给主库发送 psync 命令，并把自己当前的 slave_repl_offset 发给主库，主库会判断自己的 master_repl_offset 和 slave_repl_offset 之间的差距。若此时从库相差 > **repl_backlog_buffer**说明可能一个从库如果和主库断连时间过长，不能够进行增量复制(因为前面的值被覆盖的了，会导致数据不一致)，所以直接进行全量复制。

而从库相差 < **repl_backlog_buffer**，则将master_repl_offset 和 slave_repl_offset 之间的差距发送给从库进行执行。

Redis 的主从库同步的基本原理，总结来说，有三种模式：全量复制、基于长连接的命令传播，以及增量复制。

- 第一次同步(本来就是全量复制)或者增量复制时master_repl_offset 和 slave_repl_offset差值>repl_backlog_buffer时(repl_backlog_size这个配置参数）就无法增量复制，就被迫导致全量复制。
- 主从库正常运行后的常规同步阶段，在这个阶段中，主从库之间通过命令传播实现同步。
- 从库断开重连后进行增量复制。

主从全量同步使用RDB而不使用AOF的原因：

1、RDB文件内容是经过压缩的二进制数据（不同数据类型数据做了针对性优化），文件很小。而AOF文件记录的是每一次写操作的命令，写操作越多文件会变得很大，其中还包括很多对同一个key的多次冗余操作。

2、打开AOF就要选择文件刷盘的策略，选择不当会严重影响Redis性能。而RDB只有在需要定时备份和主从全量同步数据时才会触发生成一次快照。

redis 容灾

采用master-slave方式：

slave 同时开启快照和 AOF 进行持久化，保证数据的安全性

（1）在 master 挂掉时，修改 slave 为 master
（2）恢复原 master 数据，修改 master 为 slave，启动 slave

如果 master 和 slave 都挂掉，调用命令通过 AOF 和 snapshot 进行恢复

redis 哨兵

监控：

监控主从是否正常

通知：

出现问题的时候，通知相关人员

故障迁移：

自动主从切换

统一的配置管理：获取主从地址

Redis 消息队列

Redis 做消息队列 就是用 list 数据结构来实现，使用 lpush, rpush,操作来实现入队，lpop, rpop 来实现出队。

在客户端，维护一个死循环来从队列中读取消息，并处理，如果队列中有消息，就读取，没有消息，就会陷入死循环，知道下一次有消息进入，这种死循环会造成大量的资源浪费，可以使用 lbpop,也就是阻塞式弹出

延迟消息队列

用 Zset,其中 Zset 的score 就是时间。 将 value 存到 redis 中，然后通过 轮询的方式，去不断的读取消息出来。然后处理。

布隆过滤器

由于 HyperLogLog 没有判断是否包含或者是否存在的方法。

但是有这样的业务需求：刷微博/今日头条，推送的内容有相似的，但是没有相同的，如何做到在推送的时候去重呢？

解决这个问题，就引入了 布隆过滤器

1、介绍

Bloom Filter 专门用来处理去重问题，它相当于一个不那么精确的 set 集合，可以利用 它的 contains 方法去判断某一个对象是否存在，但是判断结果不是很精确，如果 contains 判断某个值不存在，那就一定不存在，如果判断为存在，那么不一定存在。

那么上面的问题，可以这么解决，将用户的浏览记录装进 Bloom Filter ,将要推送的东西 先进入布隆过滤器判断是否已经推送过，如果结果是没送过，那么就是一定没有，可以推送，反之，不推送，当然这条消息可能没有推送过，现在由于错误判断不推送了。这样的概率很低。

2、原理

每一个 布隆过滤器，在 Redis 中都对应了一个大型的位数组以及几个不同的 hash 函数

所谓的 add 操作是这样的： 首先根据几个不同的 hash 函数给元素进行 hash 运算一个整数索引值，拿到这个索引值之后，对位数 组的长度进行取模运算，得到一个位置，每一个 hash 函数都会得到一个位置，将位数组中对应的位置 设置位 1 ，这样就完成了添加操作。

当判读元素是否存在时，同样的。先对元素进行 hash 运算，将运算的结果和位数组取模，然后去对应的位置查看是否有相应的数据，如果有，表示元素可能存在（因为这个有数据的地方也可能是其他元素存 进来的），如果没有表示元素一定不存在。

Bloom Filter 中，误判的概率和位数组的大小有很大关系，位数组越大，误判概率越小，当然占用的存储空间越大；位数组越小，误判概率越大，当然占用的存储空间就小。

3、基本用法

添加命令 和判断是否存在命令

bf.add / bf.madd 添加和批量添加
bf.exists / bf.mexists 判断是否存在和批量判断

默认情况，布隆过滤器的错误率是 0.01 ，默认的元素的大小是 100，

这两个参数可在 bf.reserve方法进行配置

bf.reserve key 0.0001 1000000

第一个参数是key 第二个参数是错误率，错误率越低，占用的空间越大，第三个参数预计存储的数量，当实际存储的数量超过预计值时，错误率会上升。

4、另一个场景

解决缓存穿透，或者缓存击穿问题

假设有 1亿 条用户数据，现在查询用户要去数据库中查，效率低而且数据库压力大，所以我们会把请求首先在 Redis 中处理（活跃用户存在 Redis 中），Redis 中没有的用户，再去数据库中查询。

现在可能会存在一种恶意请求，这个请求携带上了很多不存在的用户，这个时候 Redis 无法拦截下来请求，

所以请求会直接跑到数据库里去。这个时候，这些恶意请求会击穿我们的缓存，甚至数据库，进而引起“雪崩效应”。

为了解决这个问题，我们就可以使用布隆过滤器。将 1亿条用户数据存在 Redis 中不现实，但是可以存 在布隆过滤器中，请求来了，首先去布隆过滤器中判断数据是否存在，如果存在，再去数据库中查询，否则就不去数据库中查询。

Redis String

使用：

设置指定 key 的值

```
SET key value
```

获取指定 key 的值

```
GET key
```

获取所有(一个或多个)给定 key 的值

```
MGET key1 [key2..]
```

将值 value 关联到 key ， 并将 key 的过期时间设为 seconds (以秒为单位)

```
SETEX key seconds value |  SET key value EX seconds
```

将 key 中储存的数字值增或减一

```
INCR/DECR key
```

将 key 所储存的值加上给定的增量值（increment）

```
INCRBY key increment
```

1、SDS结构

```
typedef char *sds;

struct __attribute__ ((__packed__)) sdshdr8 {

    uint8_t len; /* used */

    uint8_t alloc; /* excluding the header and null terminator */

    unsigned char flags; /* 3 lsb of type, 5 unused bits */

    char buf[];

};
```

Sdshdr8和Sdshdr16类型的sds图

「代码随想录」知识星球

DPND

根据string能够存储字节大小不同，将 sdsheader分为5,8,16,32,64类型。sds和传统的C语言字符串保持类型兼容，sds本质还是字符数组，只是在字符数组基础上增加了额外的元数据。在 Redis 中需要用到字符数组时，就直接使用 sds 这个别名。

字段含义：

len 表示字符数组现在的长度，alloc 字符数组分配的空间长度

flags 表示sds的类型，char 表示字符数组

一个sds字符串的完整结构，由在内存地址上前后相邻的两部分组成：

一个header

包含长度(len)、最大容量(alloc)和flags。

一个字符数组

这个字符数组的长度等于最大容量+1。

2、SDS 操作上提升效率

例如 a1追加a2 在C和redis中的函数 即C的strcat和Redis的 sdscatlen

C的strcat：

```
char *strcat(char *dest, const char *src) {

    //将目标字符串复制给tmp变量
    char *tmp = dest;

    //用一个while循环遍历目标字符串，直到遇到“\0”跳出循环，指向目标字符串的末尾
    while(*dest) {
        dest++;
    }
```

```
    //将源字符串中的每个字符逐一赋值到目标字符串中，直到遇到结束字符
    while((*dest++ = *src++) != '\0' )
    return tmp;
}
```

开发人员调用strcat前需要先进行判断目标字符串是否存在可用的空间，保证可用可用空间充足，然后调用strcat将源字符串追加到目标字符串后，不充足则进行申请空间后在调用strcat。

Redis的 sdscatlen：

```
sds sdscatlen(sds s, const void *t, size_t len) {

    //获取目标字符串s的当前长度
    size_t curlen = sdslen(s);

    //根据要追加的长度len和目标字符串s的现有长度，判断是否要增加新的空间
    s = sdsMakeRoomFor(s,len);

    if (s == NULL) return NULL;

    //将源字符串t中len长度的数据拷贝到目标字符串结尾
    memcpy(s+curlen, t, len);

    //设置目标字符串的最新长度：拷贝前长度curlen加上拷贝长度
    sdssetlen(s, curlen+len);

    //拷贝后，在目标字符串结尾加上\0
    s[curlen+len] = '\0';
    return s;
}
```

首先，获取目标字符串的当前长度

其次，在保证了目标字符串的空间足够后，将源字符串中指定长度 len 的数据追加到目标字符串。

最后，设置目标字符串的最新长度。

和 C 语言中的字符串操作相比，SDS 通过记录字符数组的使用长度和分配空间大小，避免了对字符串的遍历操作（时间复杂度由O(N)到O(1)以空间换时间），降低了操作开销，并且因为预先分配了空间，所以若追加的长度不大于此时sds的长度时，不需要在进行分配空间，减少操作, c因为我不熟悉，在go中若提前分配好切片的大小，之后进行使用时若追加长度不大于cap(容量)，也不需要申请空间。所以说，使用元数据记录字符串数组长度和封装操作的这种设计思想，使得redis追加string时效率更高。

3、紧凑的方式分配内存

SDS 一共设计了 5 种类型，分别是 sdshdr5、sdshdr8、sdshdr16、sdshdr32 和 sdshdr64。这 5 种类型的主要区别就在于，它们数据结构中的字符数组现有长度 len 和分配空间长度 alloc，这两个元数据的数据类型不同。

SDS 之所以设计不同的结构头（即不同类型），是为了能灵活保存不同大小的字符串，从而有效节省内存空间。

使用较大的sdshdr 会导致len和alloc占用更大的字节。

4、Redis存储string时编码方式

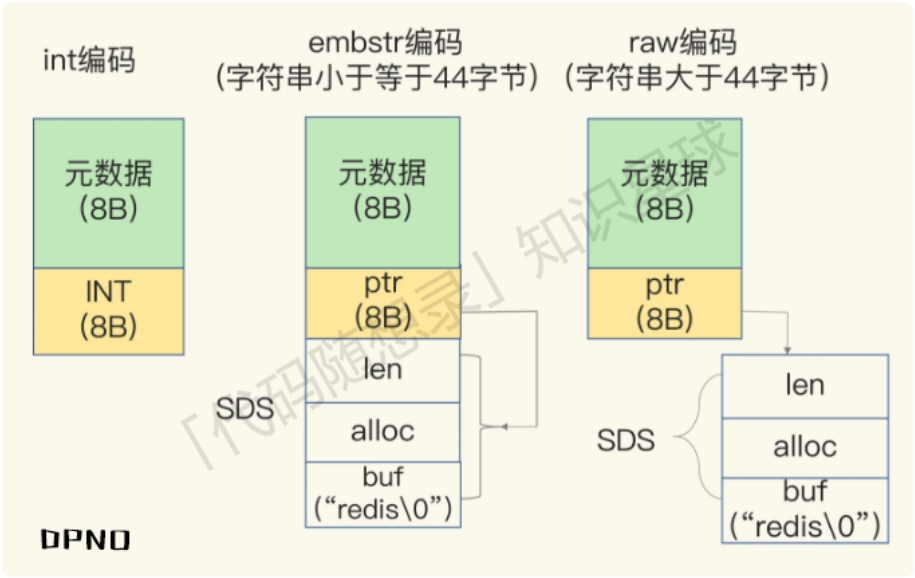
set key val

当 val为int/long 时redisObject enconding为int

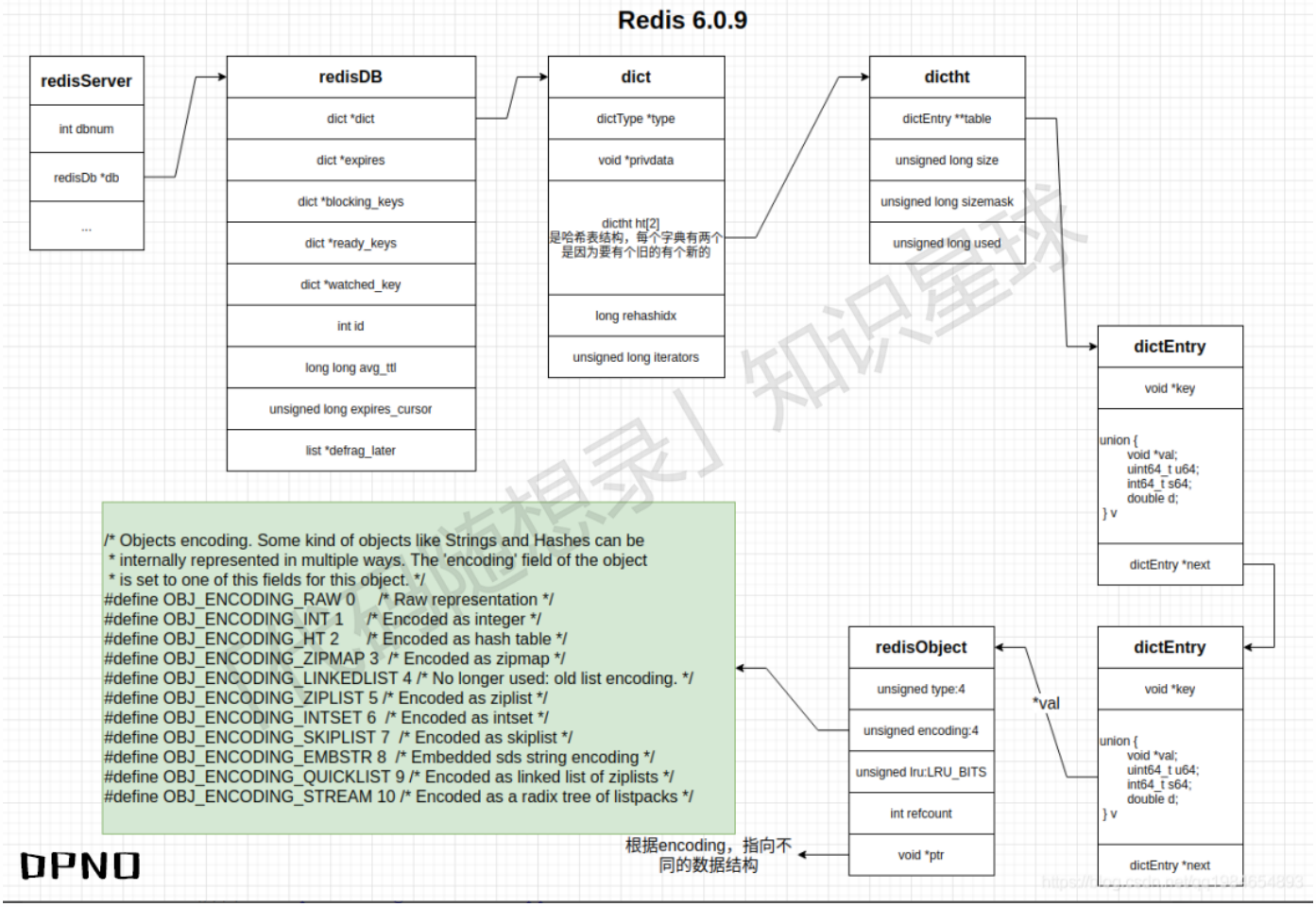
当 len(val)<=44字节时 redisObject enconding为embstr 编码方式

当 len(val)>44字节时　redisObject enconding为raw 编码方式

三种编码的示意图



int类型存储时：



当val为int能够直接使用指针*ptr存此时的值，不需要在创建一个sds通过指针进行指向，因为该指针大小为8个字节所以最大存64位大小的数。

若存储的为string时 则 *ptr 指向sds，然后根据string大小选择embstr和raw编码方式。

embstr和raw编码方式区别是，embstr时RedisObject 中的元数据、指针和 SDS 是一块连续的内存区域，这样就可以避免内存碎片，而当string>44字节时，Redis 就不再把 SDS 和 RedisObject 布局在一起了，而是会给 SDS 分配独立的空间，并用指针指向 SDS 结构。

为什么是44字节为界限呢？(注意这里是以sdshdr8的情况 所以len和alloc占用1个字节)

因为按照需求申请的为sdshdr8这种SDS 即alloc=64字节

并且新版的sds做了优化，len、alloc、flag使用的都是int8类型，只占用1个字节，这样64-16（ReadObject头部）-3（sds头部）-1（buf末尾\0）= 44。

5、Redis在操作SDS时，空间预分配，扩容，缩容

为了避免频繁操作字符串时，每次「申请、释放」内存的开销，还做了这些优化：

内存预分配：

根据需要大小使用sdshdr8,sdshdr16等类型

SDS 扩容：

当free不够时，会多申请一些内存（小于1MB翻倍扩容，大于1MB按 1MB 扩容）

缩容时：

惰性空间释放，不释放多余的内存，下次使用可直接复用这些内存，当 SDS 调用 API 需要缩短 SDS 保存的字符串时，程序并不立即使用内存重分配来回收缩短后多出来的字节，而是使用 free属性将这些字节的数量记录起来，并等待将来使用。

小结：

Redis中string API的使用

Redis使用SDS交于C的*char优点

常数复杂度获取字符串长度

减少修改字符串长度时所需的内存重分配次数

二进制安全

添加val 的编码类型 int embstr raw

内存相关

疑问?

没看到 sdshdr5 使用到的地方,或者说例子。

张铁蕾个人博客中：sdshdr5与其它几个header结构不同，它不包含alloc字段，而长度使用flags的高5位来存储。因此，它不能为字符串分配空余空间。如果字符串需要动态增长，那么它就必然要重新分配内存才行。所以说，这种类型的sds字符串更适合存储静态的短字符串（长度小于32）。

这里说的是惰性空间释放，那什么时候才会真正空间释放呢?

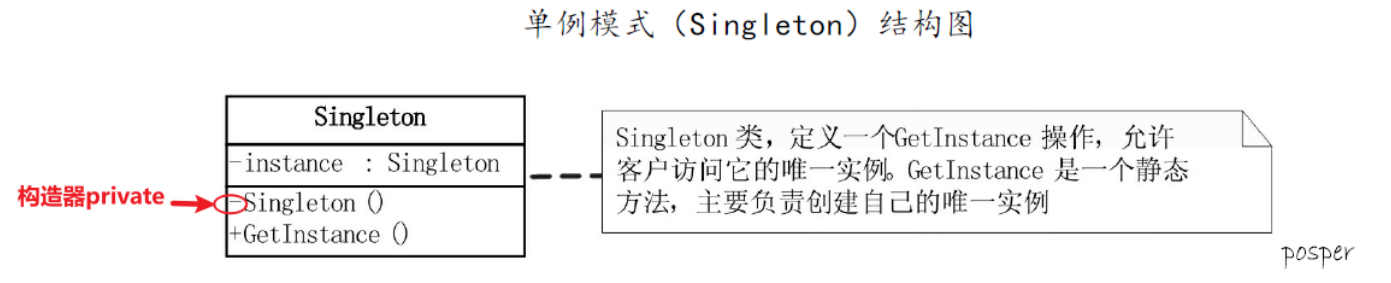
问题：

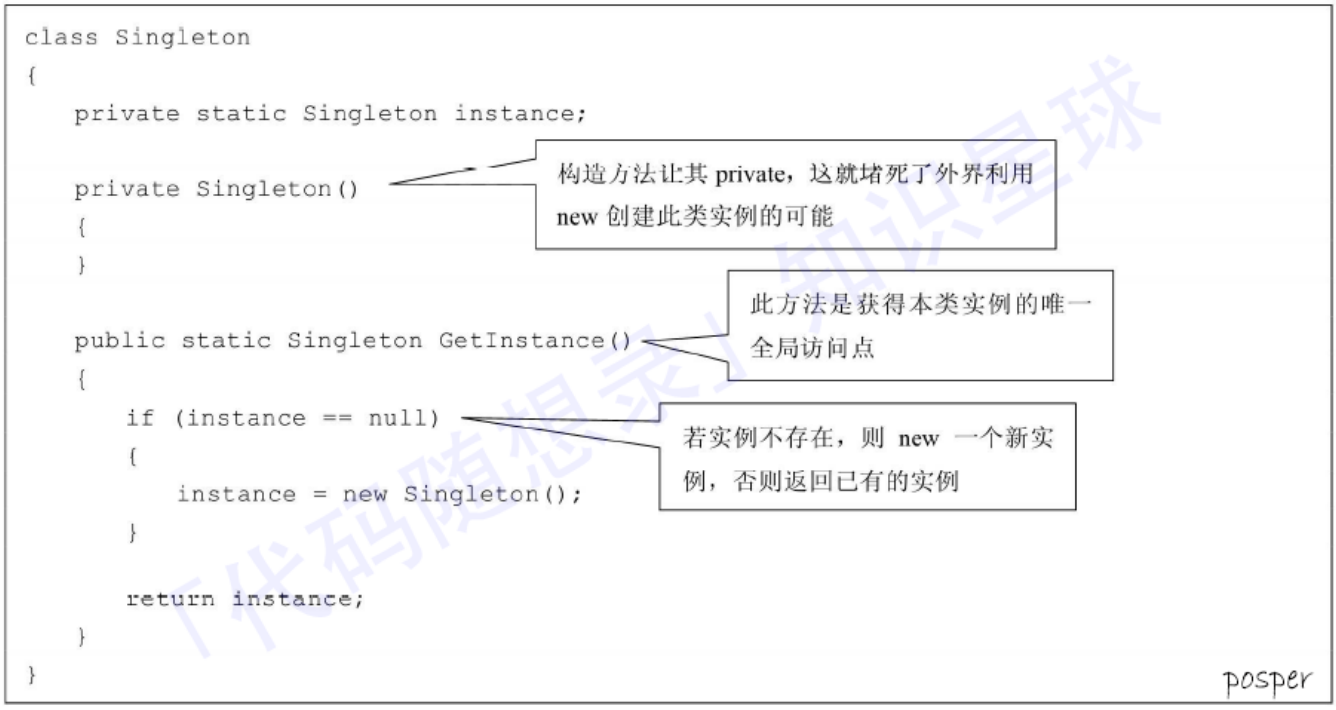
SDS 字符串在 Redis 内部模块实现中也被广泛使用，你能在 Redis server 和客户端的实现中，找到使用 SDS 字符串的地方么？（来自极客Redis 源码剖析与实战）

设计模式

单例模式

保证一个类仅有一个实例，并提供一个访问它的全局访问点





确保一个类只有一个实例，并提供该实例的全局访问点。

优点：

有些实例，全局只需要一个就够了，使用单例模式就可以避免一个全局使用的类，频繁的创建与销毁，耗费系统资源。

设计要素：

一个私有构造函数（确保只能单例类自己创建实例）
一个私有静态变量（确保只有一个实例）
一个公有静态函数（给使用者提供调用方法）

简单来说就是，单例类的构造方不让其他人修改和使用；并且单例类自己只创建一个实例，这个实例，其他人也无法修改和直接使用；然后单例类提供一个调用方法，想用这个实例，只能调用。这样就确保了全局只创建了一个实例。

单例模型6种实现及各实现的优缺点

1、懒汉式（线程不安全）

```
public class Singleton {

    private static Singleton uniqueInstance;
    private Singleton() {}

    public static Singleton getUniqueInstance() {
        if (uniqueInstance == null) {
            uniqueInstance = new Singleton();
        }
        return uniqueInstance;
    }
}
```

说明：

先不创建实例，当第一次被调用时，在创建实例，所以被称为懒汉式。

优点：

延迟了实例化，如果不需要使用该类，就不会被实例化，节约了系统资源。

缺点：

线程不安全，多线程环境下，如果多个线程同时进入了 if (uniqueInstance == null) ，若此时还未实例化，也就是 uniqueInstance == null，那么就会有多个线程执行 uniqueInstance = new Singleton(); ，就会实例化多个实例；

2、饿汉式（线程安全）

实现：

```
public class Singleton{

    private static Singleton uniqueInstance = new Singleton();
    private Singleton(){}

    public static Singleton getUniqueInstance(){
        return uniqueInstance;
    }
}
```

说明：

先不管需不需要使用这个实例，直接先实例化好实例（饿死鬼一样，所以称为饿汉式），然后当需要使用的时候，直接调方法就可以使用了

优点：

提起实例化好了一个实例，避免了线程不安全问题的出现，

缺点：

直接实例化了实例，不再延迟实例化；若系统没有使用这个实例，或者系统运行很久之后才需要使用这个实例，都会使操作系统的资源浪费。

3、懒汉式（线程安全）

实现：

```
public class Singleton {
    private static Singleton uniqueInstance;
    private static singleton() {}
    private static synchronized Sing leton getUinqueInstance() {
        if (uniqueInstance == null) {
            uniqueInstance = new Singleton();
        }
        return uniqueInstance;
    }
}
```

说明：

实现和线程不安全的懒汉式 几乎一样，唯一不同的点是，在get方法上 加了一把锁。如此一来，多个线程访问，每次只有拿到锁的的线程能够进入该方法，避免了多线程不安全问题的出现。

优点：

延迟实例化，节约了资源，并且是线程安全的。

缺点：

虽然解决了线程安全问题，但是性能降低了。因为，即使实例已经实例化了，既后续不会再出现线程安全问题了，但是锁还在，每次还是只能拿到锁的线程进入该方***使线程阻塞，等待时间过长。

4、双重检查锁实现（线程安全）

实现：

```
public class Singleton {
    private volatile static Singleton uniqueInstance;

    private Singleton() {}

    public static Singleton getUniqueInstance() {
        if (uniqueInstance == null) {
            synchronized (Singleton.class) {
                if (uniqueInstance == null) {
                    uniqueInstance = new Singleton();
                }
            }
        }
        return uniqueInstance;
    }
}
```

说明：

双重检查数相当于是改进了线程安全的懒汉式。线程安全的懒汉式的缺点是性能降低了，造成的原因是因为即使实例已经实例化，依然每次都会有锁。

而现在，我们将锁的位置变了，并且多加了一个检查。也就是，先判断实例是否已经存在，若已经存在了，则不会执行判断方法内的有锁方法了。 而如果，还没有实例化的时候，多个线程进去了，也没有事，因为里面的方法有锁，只会让一个线程进入最内层方法并实例化实例。如此一来，最多最多，也就是第一次实例化的时候，会有线程阻塞的情况，后续便不会再有线程阻塞的问题。

为什么使用 volatile 关键字修饰了 uniqueInstance 实例变量?

```
uniqueInstance = new Singleton();
```

这段代码执行时分为三步：

1. 为 uniqueInstance 分配内存空间
2. 初始化 uniqueInstance
3. 将 uniqueInstance 指向分配的内存地址

正常的执行顺序当然是 1>2>3 ，但是由于 JVM 具有指令重排的特性，执行顺序有可能变成 1>3>2。

单线程环境时，指令重排并没有什么问题；多线程环境时，会导致有些线程可能会获取到还没初始化的实例。

例如：线程A 只执行了 1 和 3 ，此时线程B来调用 getUniqueInstance()，发现 uniqueInstance 不为空，便获取 uniqueInstance 实例，但是其实此时的 uniqueInstance 还没有初始化。

解决办法就是加一个 volatile 关键字修饰 uniqueInstance ，volatile 会禁止 JVM 的指令重排，就可以保证多线程环境下的安全运行。

优点：

延迟实例化，节约了资源；线程安全；并且相对于线程安全的懒汉式，性能提高了。

缺点：

volatile 关键字，对性能也有一些影响。

5、静态内部类实现（线程安全）

实现：

```
public class Singleton {
    private Singleton() {}

    private static class SingletonHolder {
        private static final Singleton INSTANCE = new Singleton();
    }

    public static Singleton getUniqueInstance() {
        return SingletonHolder.INSTANCE;
    }
}
```

说明：

首先，当外部类 Singleton 被加载时，静态内部类 SingletonHolder 并没有被加载进内存。当调用 getUniqueInstance() 方法时，会运行 return SingletonHolder.INSTANCE;

触发了 SingletonHolder.INSTANCE ，此时静态内部类 SingletonHolder 才会被加载进内存，并且初始化 INSTANCE 实例，而且 JVM 会确保 INSTANCE 只被实例化一次。

优点：

延迟实例化，节约了资源，且线程安全，性能也提高了。

6、枚举类实现（线程安全）

实现：

```
public enum Singleton {
    INSTANCE;
    //添加自己需要的操作
    public void doSomeThing() {}
}
```

说明：

默认枚举实例的创建就是线程安全的，且在任何情况下都是单例。

优点：

写法简单，线程安全，天然防止反射和反序列化调用。

防止反序列化

序列化：

把java对象转换为字节序列的过程；

反序列化：

通过这些字节序列在内存中新建java对象的过程；

说明：

反序列化将一个单例实例对象写到磁盘再读回来，从而获得了一个新的实例。

我们要防止反序列化，避免得到多个实例，枚举类天然防止反序列化。

其他单例模式 可以通过 重写 readResolve() 方法，从而防止反序列化，使实例唯一重写

```
readResolve();
private Object readResolve();
throws
ObjectStreamException {
    return singleton;
}
```

单例模式的应用场景

应用场景举例：

1. 网站计数器
2. 应用程序的日志应用
3. Web项目中的配置对象的读取
4. 数据库连接池
5. 多线程池

.....

使用场景总结：

（1）频繁实例化然后又销毁的对象，使用单例模式可以提高性能

（2）经常使用的对象，但实例化时耗费时间或者资源多，如数据库连接池，使用单例模式，可以提高性能，降低资源损坏

（3）使用线程池之类的控制资源时，使用单例模式，可以方便资源之间的通信

工厂模式

概述：

工厂模式是一种非常常用的创建型设计模式，其提供了创建对象的最佳方式。在创建对象时，不会对客户端暴露对象的创建逻辑，而是通过使用共同的接口来创建对象。其用来封装和管理类的创建，本质是对获取对象过程的抽象。

一般情况下，工厂模式分为三种更加细分的类型：简单工厂、工厂方法和抽象工厂。

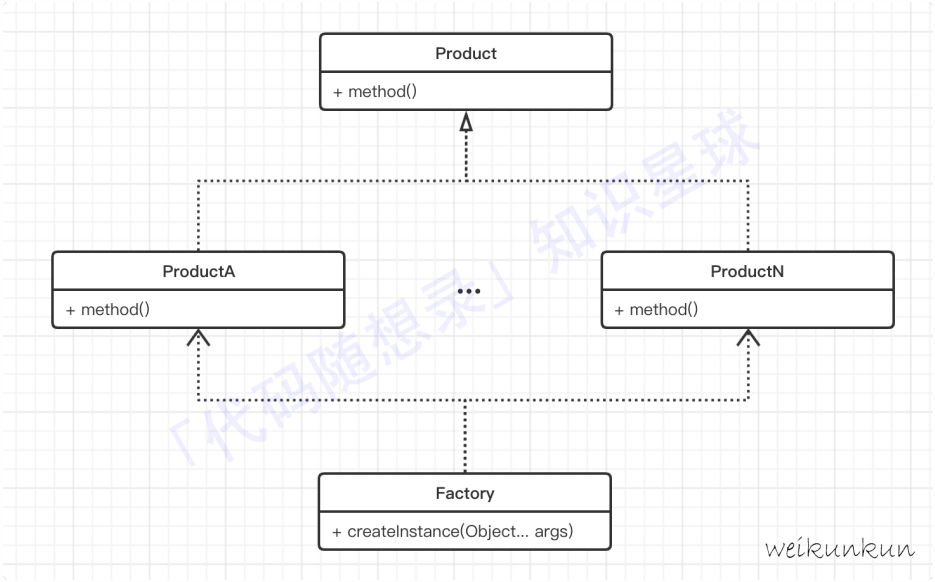
不过，在 GoF 的《设计模式》一书中，它将简单工厂模式看作是工厂方法模式的一种特例，所以工厂模式只被分成了工厂方法和抽象工厂两类

优点：

- 解耦：将对象的创建和使用进行分离
- 可复用：对于创建过程比较复杂且在很多地方都使用到的对象，通过工厂模式可以提高对象创建的代码的复用性。
- 降低成本：由于复杂对象通过工厂进行统一管理，所以只需要修改工厂内部的对象创建过程即可维护对象，从而达到降低成本的目的。

简单工厂模式（静态工厂方法模式）

UML



概念

是创建型的一种。

在简单工厂模式中，可以根据实际的参数不同返回不同的实例。同时在简单工厂模式中会定义一个类负责创建其他类的实例，被创建的实例也通常具有共同的父类。

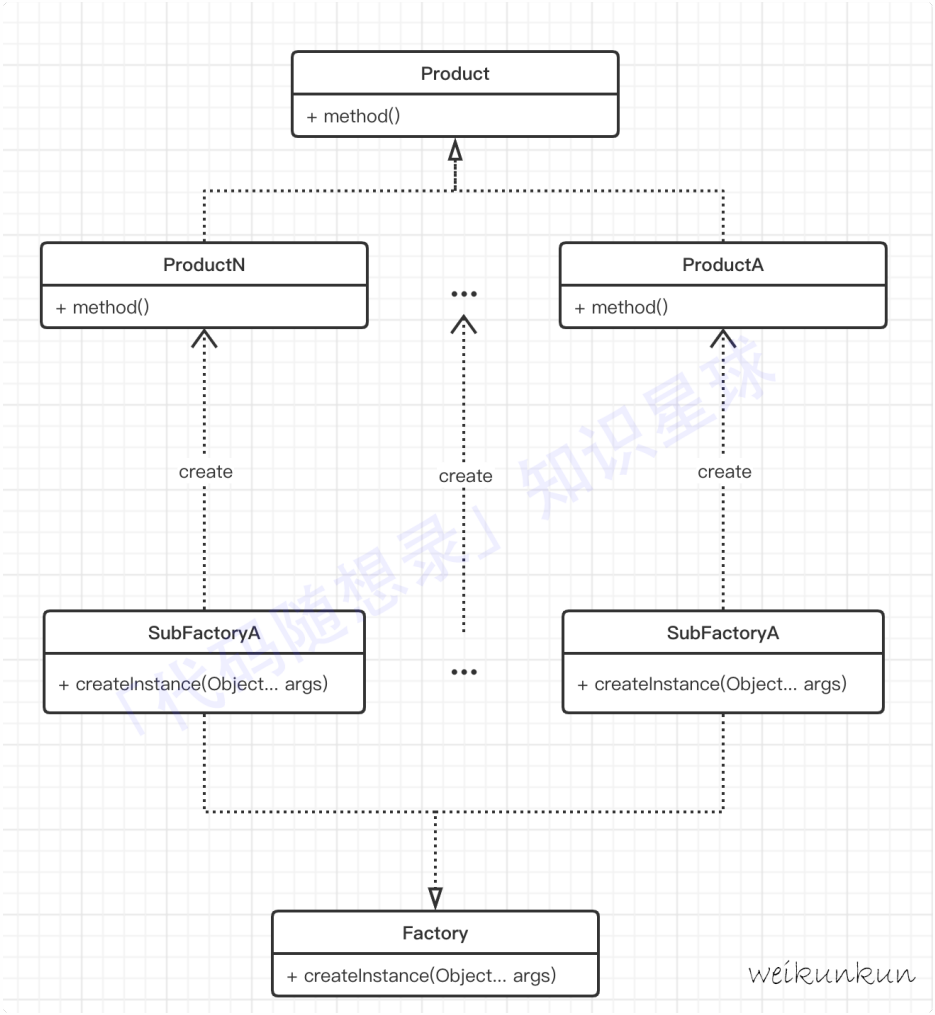
虽然实现了对象的创建和使用的分离，但是不够灵活，工厂类集合了所有产品的创建逻辑，职责过重，同时新增一个产品就需要在原工厂类内部添加一个分支，违反了开闭原则。**并且若是有多个判断条件共同决定创建对象，则后期修改会越来越复杂。**

实际应用

JDK中的DateFormate、Calendar类都有使用，通过不同参数返回我们需要的对象。

工厂方法模式

UML



概念

工厂方法模式中，将简单工厂中的工厂类变为一个抽象接口。负责给出不同工厂应该实现的方法，自身不再负责创建各种产品，而是将具体的创建操作交给实现该接口的子工厂类来做。

通过多态的形式解决了简单工厂模式过多的分支问题。虽然在新增产品时不仅要新增一个产品类还要实现与之对应的子工厂，但是相较于简单工厂模式更符合开闭原则

实际应用

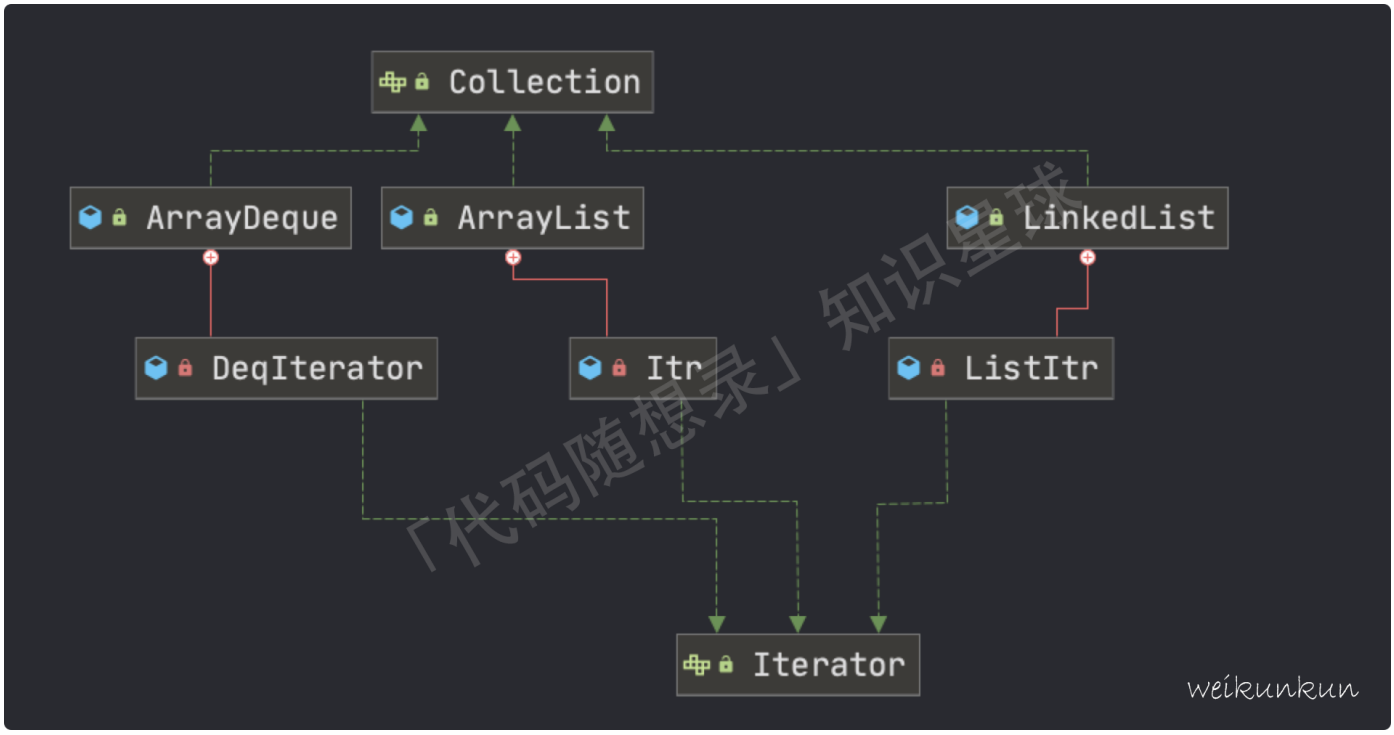
JDK中的Collection接口中Iterator的实现。Collection中不同的实现类生产适合于自己的迭代器对象

1、Factory：Collection

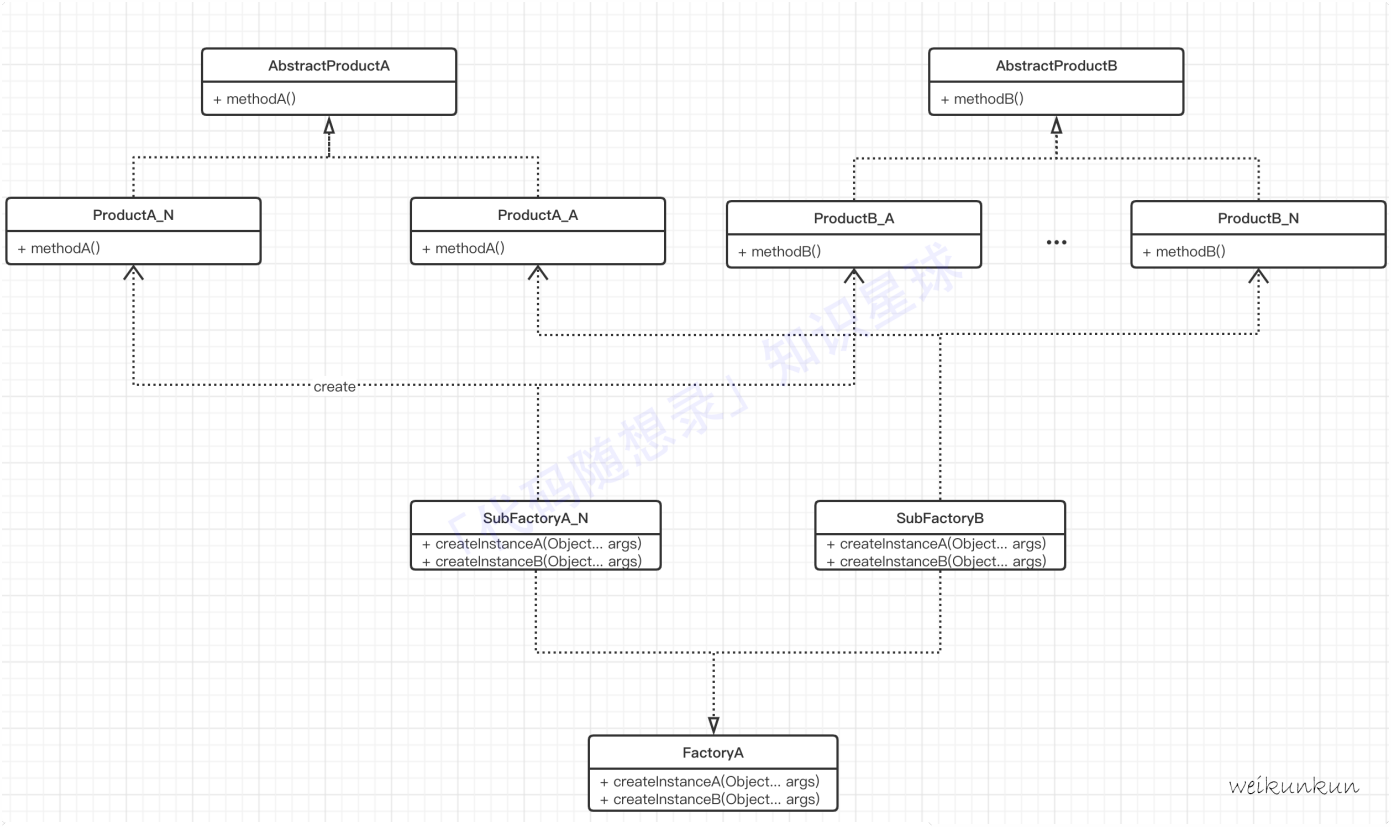
1. SubFactoryA：LinkedList
2. SubFactoryB：ArrayList

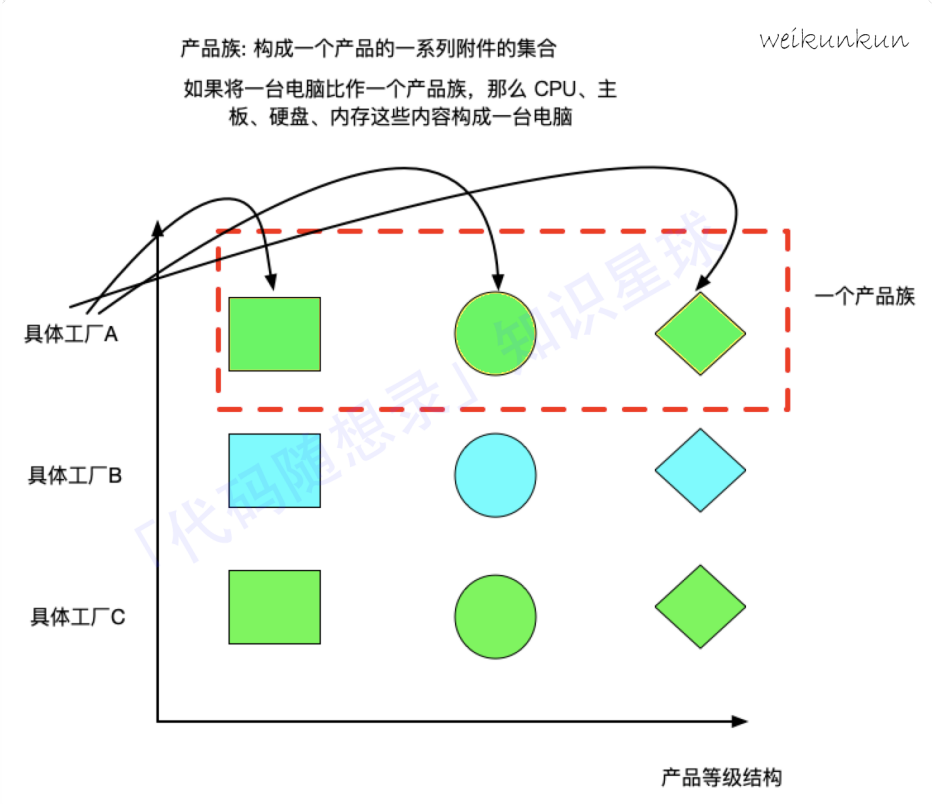
2、Product：Iterator

1. ProductA：ListItr
2. ProductB：Itr


Collection
ArrayDeque
ArrayList
LinkedList
DeqIterator
Itr
ListItr
Iterator
「代码随想录」知识星球
weikunkun

UML





概念

工厂方法模式通过引入工厂等级结构，解决了简单工厂模式中工厂类职责太重的问题，但由于工厂方法模式中的每个工厂只生产一类产品，可能会导致系统中存在大量的工厂类，势必会增加系统的开销。抽象工厂模式为工厂方法模式的进一步延伸，其将一些相关的产品组成一个“产品族”，由同一个工厂来统一生产。虽然对于新增一个产品族很方面，并且也符合开闭原则，但是新增一个产品等级结构，会对整个工厂结构进行大改

应用

Spring中的BeanFactory

策略模式

定义一系列的算法，把它们单独封装起来，并且使它们可以互相替换，使得算法可以独立于使用它的客户端而变化，也是说这些算法所完成的功能类型是一样的，对外接口也是一样的，只是不同的策略为引起环境角色表现出不同的行为。

相比于使用大量的if...else，使用策略模式可以降低复杂度，使得代码更容易维护。

缺点：可能需要定义大量的策略类，并且这些策略类都要提供给客户端。

主要构成

Stragety（策略的抽象，接口）
ConcreteStragetyA、ConcreteStrategyB...（具体的策略实现类）
Context（用来操作策略的上下文环境类）：环境类的构造函数包含了Strategy类，通过多态传进来不同的具体策略（ConcreteStrategyA。ConcreteStrategyB）来调用不同策略的方法

优点：

使用策略模式可以避免使用多重条件转移语句。多重转移语句将算法或行为的逻辑混合在一起，不易维护

缺点：

客户端必须知道所有的策略类，并自行决定使用哪一个策略类，策略模式只适用于客户端知道所有的算法或行为的情况。

观察者模式

属于行为型模式的一种，它定义了一种一对多的依赖关系，让多个观察者对象同时监听某一个主题对象。这个主题对象在状态变化时，会通知所有的观察者对象，使他们能够自动更新自己。

主要构成

Subject（抽象被观察者）：

将所有观察者对象保存到一个集合里(通常是ArrayList)，主题可以有任意数量的观察者，抽象主题提供增加、删除观察者对象的接口（attach、detach）

Observer（抽象观察者）：

它定义了一个更新接口方法（update），使得在得到主题更改通知时更新自己

ConcreteSubject（具体被观察者）：

有关状态存入具体观察者对象，具体被观察者的内部状态发生改变时，给所有加入过的观察者发送通知。

ConcreteObserver（具体观察者）：

实现抽象观察者定义的更新接口，以便在得到主题更改通知时更新自身的状态。

案例：

Carl哥的微信公众号 是被观察者，微信用户 是观察者，有多个微信用户关注了Carl哥的公众号，当Carl哥的公众号更新订阅时就会通知这些订阅了的微信用户。

优点：

解除耦合，让耦合的双方都依赖于抽象，从而使得各自的变换都不会影响另一边的变换

缺点：

调试复杂，而且在Java中消息的通知一般是顺序执行，那么一个观察者卡顿，会影响整体的执行效率，在这种情况下，一般会采用异步实现。

装饰模式

结构图：

第6章 穿什么有这么重要？——装饰模式

6.1 穿什么有这么重要？

6.2 小菜扮靓第一版

6.3 小菜扮靓第二版

6.4 装饰模式

6.5 小菜扮靓第三版

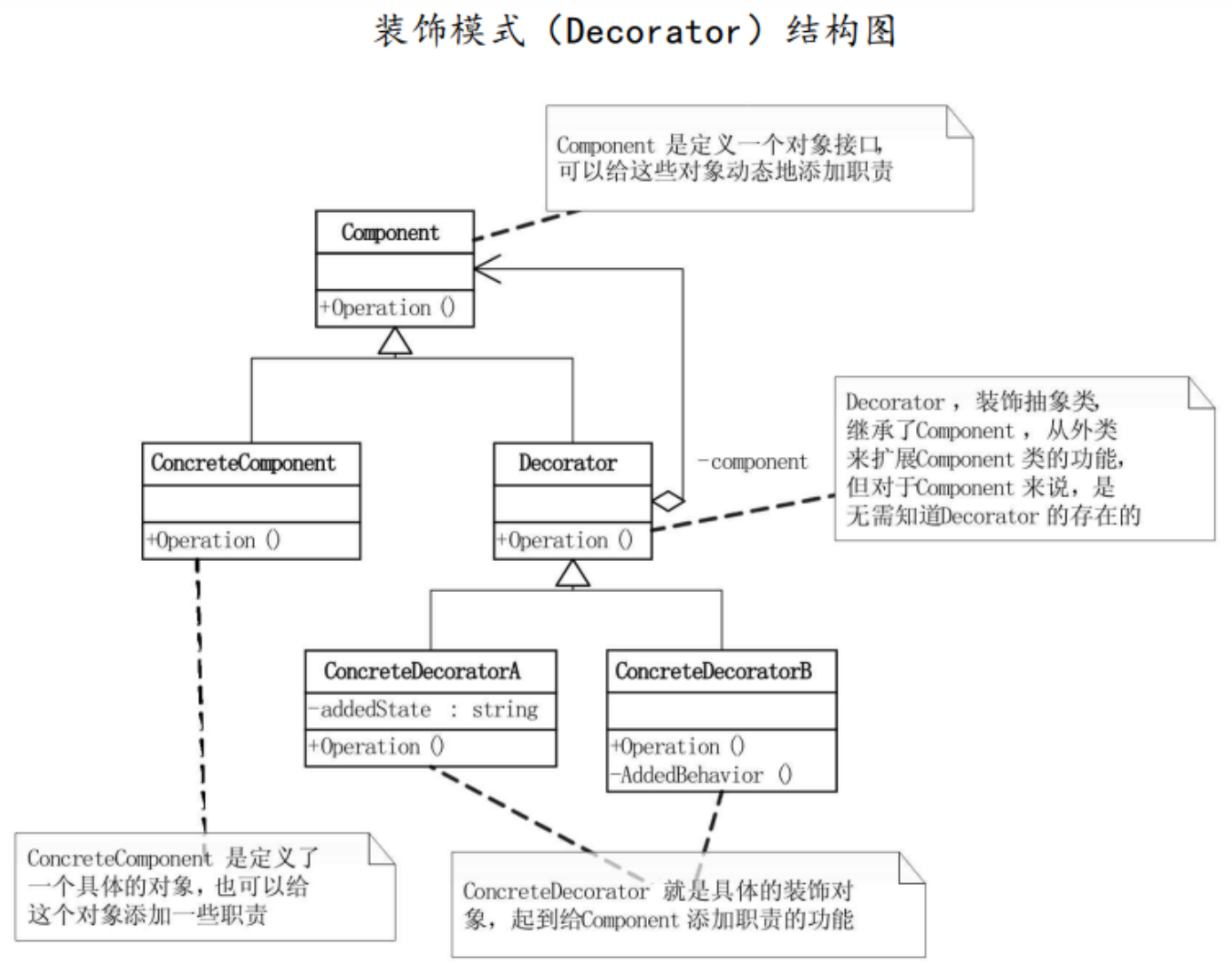
6.6 装饰模式总结

1、装饰模式的定义

装饰模式（Decorator）：

动态地给一个对象添加一些额外的职责，就增加功能来说，装饰模式比生成子类更为灵活。

2、装饰模式结构图



3、基本代码实现

（1）Component类

```
abstract class Component
{
    public abstract void Operation () ;
}
```

（2）ConcreteComponent类

```
class ConcreteComponent : Component
{
    public override void Operation()
    {
        Console.WriteLine("具体对象的操作");
    }
}
```

（3）Decorator类

```
abstract class Decorator : Component
{
    protected Component component;

    public void SetComponent(Component component)
    {
        this.component = component;
    }

    public override void Operation()
    {
        if (component != null)
        {
            component.Operation();
        }
    }
}
```

设置 Component

重写 Operation()，实际执行的是 Component 的 Operation()

（4）ConcreteDecoratorA类 和 ConcreteDecoratorB类

```
class ConcreteDecoratorA : Decorator
{
    private string addedState;

    public override void Operation()
    {
        base.Operation();
        addedState = "New State";
        Console.WriteLine("具体装饰对象 A 的操作");
    }
}

class ConcreteDecoratorB : Decorator
{
    public override void Operation()
    {
        base.Operation();
        AddedBehavior();
        Console.WriteLine("具体装饰对象 B 的操作");
    }

    private void AddedBehavior()
    {

    }
}
```

本类的独有功能，以区别于 ConcreteDecoratorB

首先运行原 Component 的 Operation()，再执行本类的功能，如 addedState，相当于对原 Component 进行了装饰

首先运行原 Component 的 Operation()，再执行本类的功能，如 AddedBehavior()，相当于对原 Component 进行了装饰

本类独有的方法，以区别于 ConcreteDecoratorA

（5）客户端代码

```
static void Main(string[] args)
{
    ConcreteComponent c = new ConcreteComponent();
    ConcreteDecoratorA d1 = new ConcreteDecoratorA();
    ConcreteDecoratorB d2 = new ConcreteDecoratorB();

    d1.SetComponent(c);
    d2.SetComponent(d1);
    d2.Operation();

    Console.Read();
}
```

装饰的方法是：首先用 ConcreteComponent 实例化对象 c，然后用 ConcreteDecoratorA 的实例化对象 d1 来包装 c，再用 ConcreteDecoratorB 的对象 d2 包装 d1，最终执行 d2 的 Operation()

4、装饰模式在什么情况下使用

装饰模式把每个要装饰的功能放在单独的类中，并让这个类包装它所要装饰的对象。因此，当需要执行特殊行为时，客户代码就可以在运行时根据需要有选择地、按顺序地使用装饰功能包装对象了。

代理模式

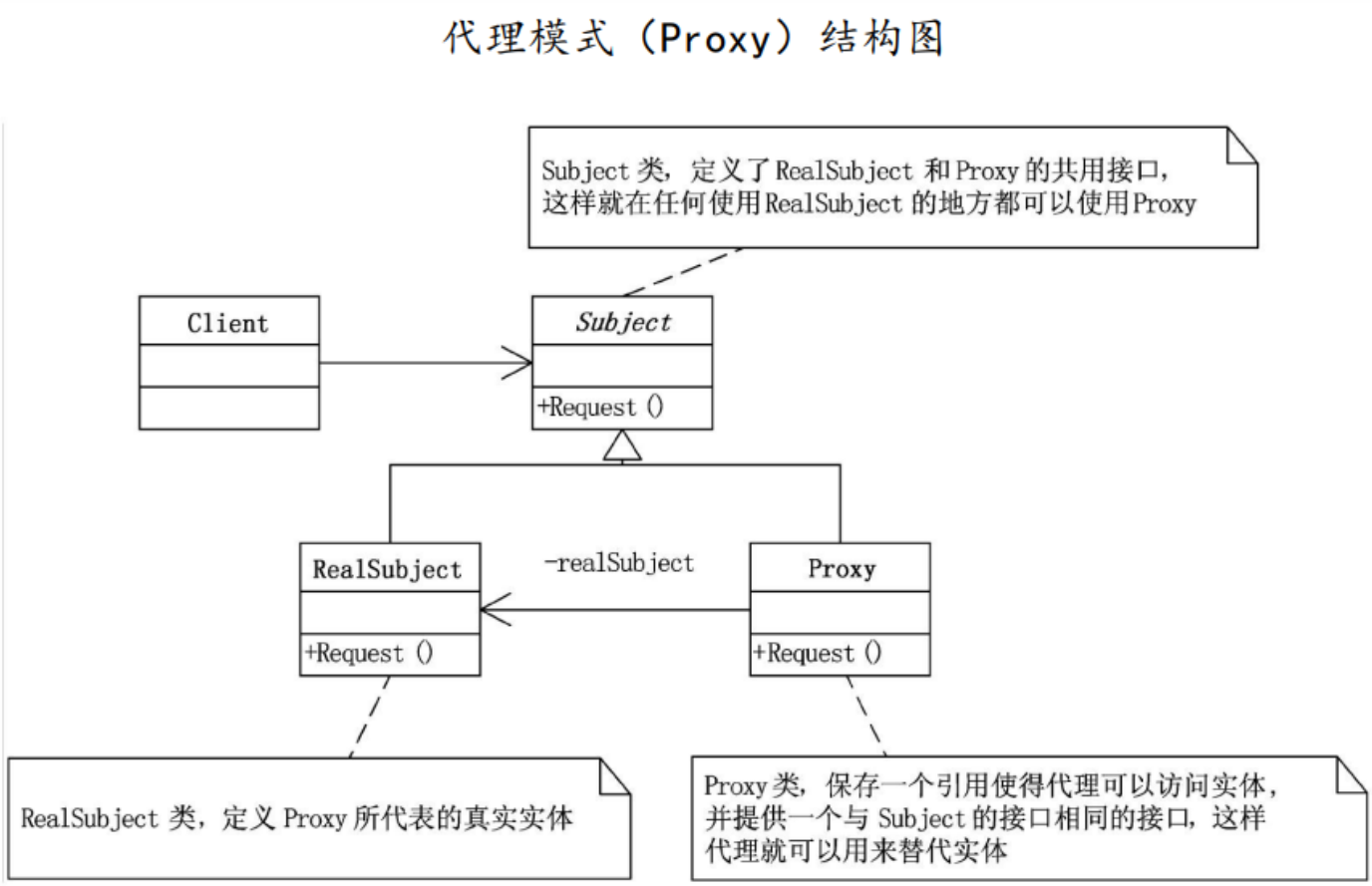
结构图：

7.2 没有代理的代码
7.3 只有代理的代码
7.4 符合实际的代码
7.5 代理模式
7.6 代理模式应用
7.7 秀才让小六代其求婚

1、代理模式定义

代理模式，为其他对象提供一种代理以控制对这个对象的访问。

2、代理模式结构图



（1）Subject类

```
abstract class Subject
{
    public abstract void Request（）；
}
```

（2）RealSubject 类，

定义Proxy 所代表的真实实体

```
static void Main（string[] args）
{
    Proxy proxy = new Proxy（）；
    proxy.Request（）；

    Console.Read（）；
}
```

（3）Proxy 类

保存一个引用使得代理可以访问实体，并提供一个与Subject的接口相同的接口，这样代理就可以用来替代实体。

```
class RealSubject : Subject
{
    public override void Request（）
    {
        Console.WriteLine（"真实的请求"）；
    }
}
```

（4）客户端代码

```
class Proxy : Subject
{
    RealSubject realSubject;
    public override void Request（）
    {
        if（realSubject == null）
        {
            realSubject = new RealSubject（）；
        }
        realSubject.Request（）；
    }
```

3、代理模式应用

（1）远程代理

一个对象在不同的地址空间提供局部代表。这样可以隐藏一个对象存在于不同地址空间的事实。

（2）虚拟代理

根据需要创建开销很大的对象。通过它来存放实例化需要很长时间的真实对象，这样就可以达到性能的最优化。比如说你打开一个很大的HTML网页时，里面可能有很多的文字和图片，但你还是可以很快打开它，此时你所看到的是所有的文字，但图片却是一张一张地下载后才能看到。那些未打开的图片框，就是通过虚拟代理来替代了真实的图片，此时代理存储了真实图片的路径和尺寸。

（3）安全代理

用来控制真实对象访问时的权限。一般用于对象应该有不同的访问权限的时候。

（4）智能指引

是指当调用真实的对象时，代理处理另外一些事。如计算真实对象的引用次数，这样当该对象没有引用时，可以自动释放它；或当第一次引用一个持久对象时，将它装入内存；或在访问一个实际对象前，检查是否已经锁定它，以确保其他对象不能改变它。它们都是通过代理在访问一个对象时附加一些内务处理。

组合复用原则

1、概念：

优先使用组合 contains a（聚合 has a），而不是继承 is a 来达到目的

2、原因：

继承会将实现细节暴露给子类，继承复用破坏了封装性，是白箱复用
使用继承时需要考虑里氏替换原则

3、优点：

新类对象存取成员对象只通过成员对象的接口，是黑箱复用，系统更灵活，降低耦合度，可以在运行时动态进行，新对象可动态引用与成员对象类型相同的对象

4、缺点：

需要管理较多对象

迪米特法则

概念：

一个对象应当对其他对象有尽可能少的了解，即不和陌生人说话

“朋友圈”概念（以下情况是该对象朋友）：

1、this：

1. 该对象方法中的参数
2. 实例变量直接引用的对象

3. 实例变量如果是一个聚集（聚合对象），聚集中的元素
4. 该对象方法中创建的变量

2、要求：

1. 优先考虑将一个类设计成不变类
2. 尽量降低一个类的访问权限
3. 谨慎使用 Serializable（持久化，通过序列化一个对象，将其写入磁4. 盘，以后程序调用时重新恢复该对象）
4. 尽量降低成员的访问权限

3、优点： 降低类之间的耦合

4、缺点： 过多使用迪米特法则，会产生大量中介类，设计变复杂

23种设计模式的概念

创建型模式

1、工厂方法(factory method)模式：

定义一个创建对象的接口，但由子类决定需要实例化哪一个类。工厂方法使得子类实例化的过程推迟

2、抽象工厂(abstract factory)模式：

提供一个接口，可以创建一系列相关或相互依赖的对象，而无需指定他们具体的类

3、原型(prototype)模式：

用原型实例指定创建对象的类型，并且通过拷贝这个原型来创建新的对象

4、单例(singleton)模式：

保证一个类只有一个实例，并提供一个访问它的全局访问点

5、构建器(builder)模式：

将一个复杂类的表示与其构造相分离，使得相同的构建过程能够得出不同的表示

结构型模式

1、适配器(adapter)模式：

将一个类的接口转换成用户希望得到的另一个接口。它使原本不相容的接口得以协同工作——速记关键字:转换接口

2、桥接(bridge)模式：

将类的抽象部分和它的实现部分分离开来，使它们可以独立地变化——速记关键字：继承树拆分

3、组合(composite)模式：

将对象组合成树型结构以表示“整体-部分”的层次结构，使得用户对单个对象和组合对象的使用具有一致性——速记关键字：树形目录结构

4、装饰(decorator)模式：

动态地给一个对象添加一些额外的职责。它提供了用子类扩展功能的一个灵活的替代，比派生一个子类更加灵活——速记关键字：附加职责

5、外观(facade)模式：

定义一个高层接口，为子系统中的一组接口提供一个一致的外观，从而简化了该子系统的使用——速记关键字：对外统一接口

6、享元(flyweight)模式：

提供支持大量细粒度对象共享的有效方法

7、代理(proxy)模式：

为其他对象提供一种代理以控制这个对象的访问

行为型模式

1、职责链(chain of responsibility)模式：

通过给多个对象处理请求的机会，减少请求的发送者与接收者之间的耦合。将接收对象链接起来，在链中传递请求，直到有一个对象处理这个请求——速记关键字：传递职责

2、命令(command)模式：

将一个请求封装为一个对象，从而可用不同的请求对客户进行参数化，将请求排队或记录请求日志，支持可撤销的操作——速记关键字：日志记录，可撤销

3、解释器(interpreter)模式：

给定一种语言，定义它的文法表示，并定义一个解释器，该解释器用来根据文法表示来解释语言中的句子

4、迭代器(iterator)模式：

提供一种方法来顺序访问一个聚合对象中的各个元素而不需要暴露该对象的内部表示

5、中介者(mediator)模式：

用一个中介对象来封装一系列的对象交互。它使各对象不需要显式地相互调用，从而达到低耦合，还可以独立地改变对象间的交互——速记关键字：不直接引用

6、备忘录(memento)模式：

在不破坏封装性的前提下，捕获一个对象的内部状态，并在该对象之外保存这个状态，从而可用在以后将该对象恢复到原先保存的状态

7、观察者(observer)模式：

定义对象间的一种一对多的依赖关系，当一个对象的状态发生改变时，所有依赖于它的对象都得到通知并自动更新

8、状态(state)模式：

允许一个对象在其内部状态改变时改变它的行为——速记关键字：状态变成类

9、策略(strategy)模式：

定义一系列算法，把它们一个个封装起来，并且使它们之间可互相替换，从而让算法可以独立于使用它的用户而变化

10、模板方法(template method)模式：

定义一个操作中的算法骨架，而将一些步骤延迟到子类中，使得子类可以不改变一个算法的结构即可重新定义算法的某些特定步骤

11、访问者(visitor)模式：

表示一个作用于某对象结构中的各元素的操作，使得在不改变各元素的类的前提下定义作用于这些元素的新操作

计算机系统

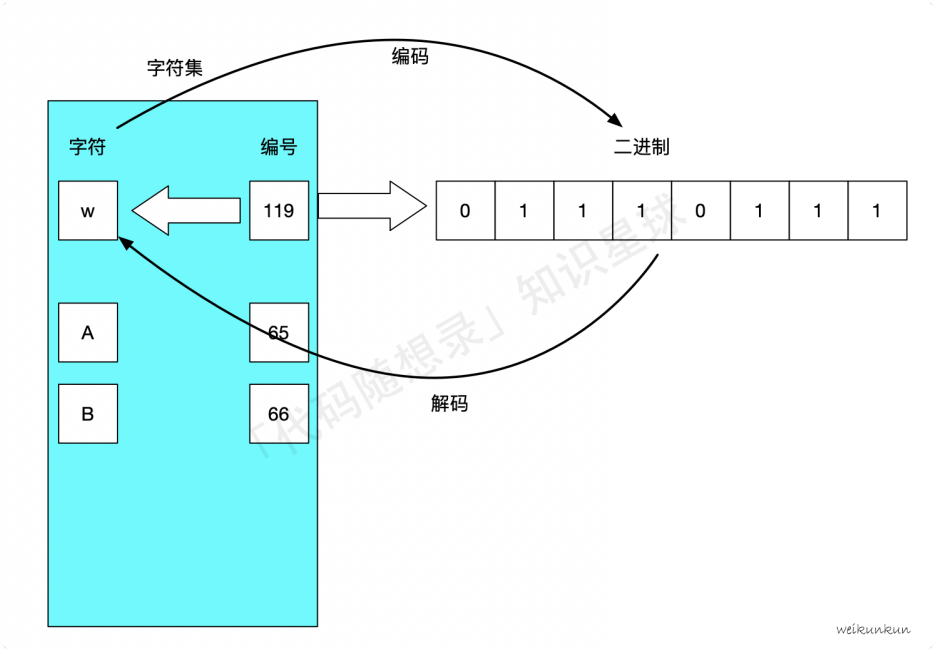
字符编码

我们知道一个字节有8位，8位在不考虑正负的情况下，能够表示256个数字。同理2个字节就能表示出65536个数字。那么对于字符该如何表示？

通过计算机领域的至理名言：**计算机科学领域的任何问题都可以通过增加一个间接的中间层来解决**。

我们通过在字符和二进制编码之间嵌套一个数字编号，通过数字编号，就能实现字符、二进制编码之间的关系绑定。

像这样收录了很多的字符并行一一编号得到的一个对照表，我们称之为字符集。从二进制 -> 字符 称为解码，从字符 -> 二进制 称为编码



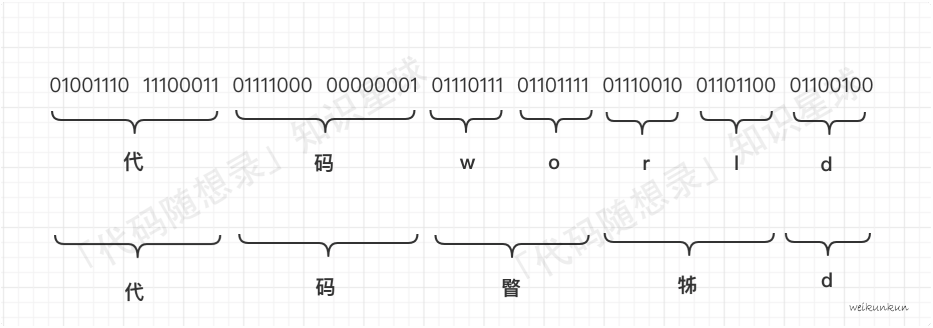
从ASCII字符集表示英文、控制符号这些，再到GB2312能表示出中文，目前存在着很多的字符集，这也就出现了一个问题，同一个二进制，在不同的字符集可能对应的字符不同，这也就导致了同一个文件，通过不同的字符集进行解码，呈现的结果不同（也就是我们所说的乱码）。如果有一种统一的编码形式（很全的字符集），对所有的字符都能提供唯一的二进制编码，那么就能解决上述的问题。Unicode就是这个很全的字符集。

虽然通过Unicode我们实现了对字符的唯一表示，但是随之而来的还有另一个问题，这些二进制编码该如何存储?

字符	编码	二进制
代	20195	01001110 11100011
码	30721	01111000 00000001
w	119	00000000 01110111
o	111	00000000 01101111
r	114	00000000 01110010
l	108	00000000 01101100
d	100	00000000 01100100

weikunkun

以代码world为例，将二进制直接照搬成一行，按照不同的步长进行解析，则会得出不同的结果，如何解决?



固定长度编码

最直接思路，就是固定长度，不管编号多大多小，统一根据最长位数来，位数不够，高位补零。

字符	编码	二进制
代	20195	00000000 00000000 01001110 11100011
码	30721	00000000 00000000 01111000 00000001
w	119	00000000 00000000 00000000 01110111
o	111	00000000 00000000 00000000 01101111
r	114	00000000 00000000 00000000 01110010
l	108	00000000 00000000 00000000 01101100
d	100	00000000 00000000 00000000 01100100

weikunkun

优点：高效、快速进行编码、解码

缺点：固定长度的形式，会导致内存的浪费，并且随着编码越来越大，编码之间的跨度也就越大，这样定长编码造成的浪费就更加显著

可变长度编码

根据固定长度编码的缺点，可以采用变长的形式进行解决，小编号少占用字节，大编号多占字节。不过如何划分边界是一个问题

一种解决方案：

编号	编码模板
[0, 127]	0XXXXXXX
[128, 2047]	110XXXXX 10XXXXXX
[2048, 65535]	1110XXXX 10XXXXXX 10XXXXXXXX

weikunkun

可以采用类似IPV4中的分类编制的形式，在二进制中占用几位作为标识。

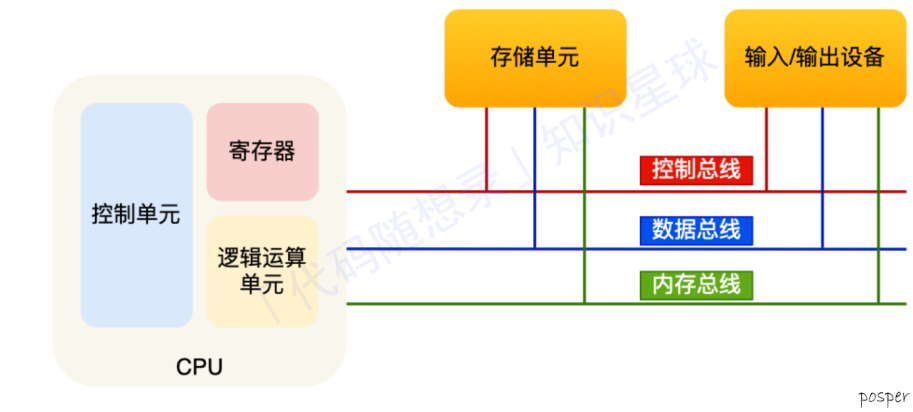
w	119	01110111
代	228	11000011 10100100
码	30721	11100111 10100000 10000001

weikunkun

上述的解决方案，其实就是我们所熟知的UTF-8编码，同时也是Go默认的编码方式

冯诺依曼模型

冯诺依曼模型分为 5 大部件：内存、CPU、总线、输入设备、输出设备



posper

内存

从 0 开始编号，最先存储单位是字节（byte）
程序和数据都是在内存中的

CPU

CPU 位宽： CPU 一次能处理多少字节的数据

32位CPU一次可以计算4个字节;
64位CPU一次可以计算8个字节;

为了能一次计算大数的运算，CPU需要支持多个byte一起计算，所以CPU位宽越大，可以计算的数值就越大。
比如说32位CPU能计算的最大整数是4294967295。

CPU内部还有一些组件，常见的有寄存器、控制单元和逻辑运算单元等

控制单元： 控制CPU工作；
逻辑运算单元： 负责计算；
寄存器： 在 CPU 中，计算速度快

常见寄存器类型：

通用寄存器：放运算数据
程序记数器（PC）：用来存放 CPU 下一条要执行指令的 内存地址；
指令寄存器：用来存放 PC 执行的指令，即 指令本身（取值后放入到 指令寄存器中）

总线

用于 CPU 和内存以及其他设备之间的通信，**总线可分为 3 种:**

地址总线：

用于指定CPU将要操作的内存地址;

数据总线：

用于读写内存的数据;

控制总线：

用于发送和接收信号，比如中断、设备复位等信号，CPU收到信号后自然进行响应，这时也需要控制总线;

当CPU要读写内存数据的时候．一般需要通过两个总线:

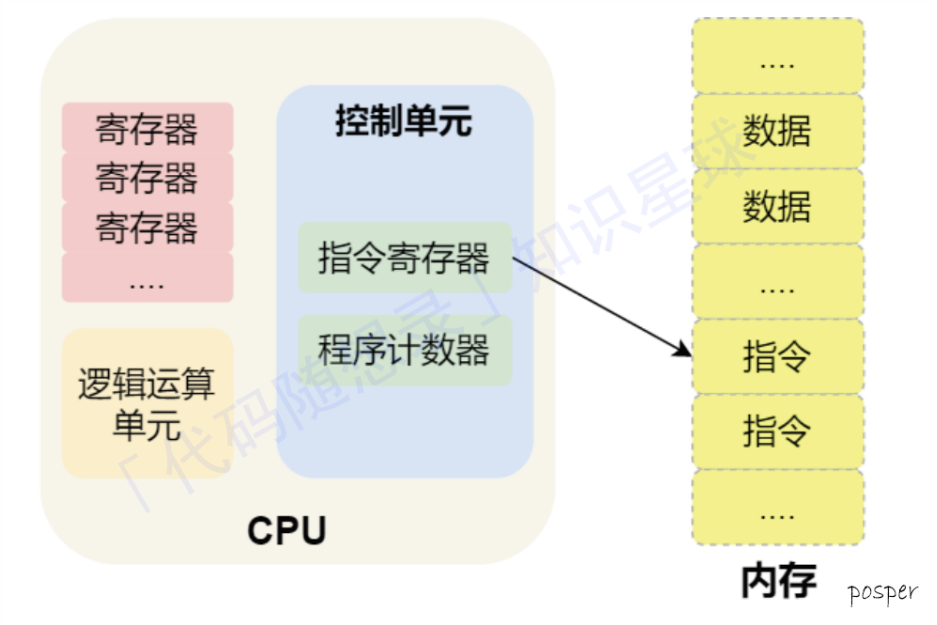
首先要通过「地址总线」来指定内存的地址;
再通过「数据总线」来传输数据;

程序的执行过程

现代大多数 CPU 都使用来流水线的方式来执行指令，
所谓的流水线就是把一个任务拆分成多个小任务，于是一条指令通常分为 4 个阶段，称为 4 级流水线

1. 取指令
2. CPU 的 控制单元向 地址总线 发送 PC 中的地址，从内存中找到 待执行指令；
3. 并将 待执行指令 通过 数据总线 传入到 CPU 中的 指令寄存器 中

4. 自动更新 PC，指向下一条待执行指令的地址
5. 译指（分析指令）
6. CPU分析「指令寄存器」中的指令，确定指令的类型和参数：
    1. 如果是计算类型的指令，就把指令交给「逻辑运算单元」运算;
    2. 如果是存储类型的指令，则交由「控制单元」执行
7. 执行指令
8. 根据 译指的结果，把指令交给 CPU 中响应的控件（逻辑运算单元/控制单元）执行
9. 数据回写
0. CPU 将计算结果存回寄存器或者将寄存器的值存入内存



不同的阶段其实是由计算机中的不同组件完成的：

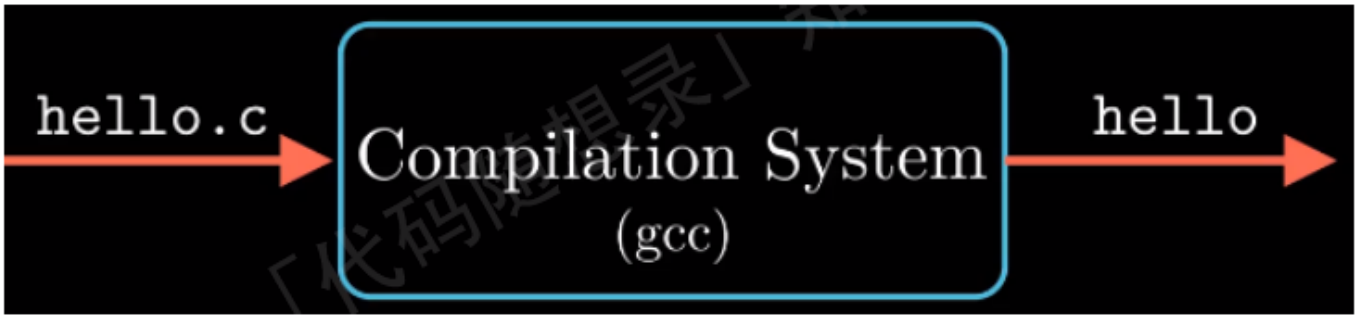
取指令、译指令：由 控制器 进行；

指令执行：

1. 算术操作、逻辑操作、数据传输、条件分支都是由 逻辑运算单元（运算器)完成
2. 跳转语句：由 控制器（控制单元）完成

编译系统

```
linux> gcc -o hello hell.c
```



青

编译系统如何工作

了解编译系统如何工作是大有益处的

1、理解编译系统可以优化程序的性能。

例如：一个switch语句是不是要比一连串的if-else要高效得多？函数调用的开销有多大？while循环比for循环更高效么？

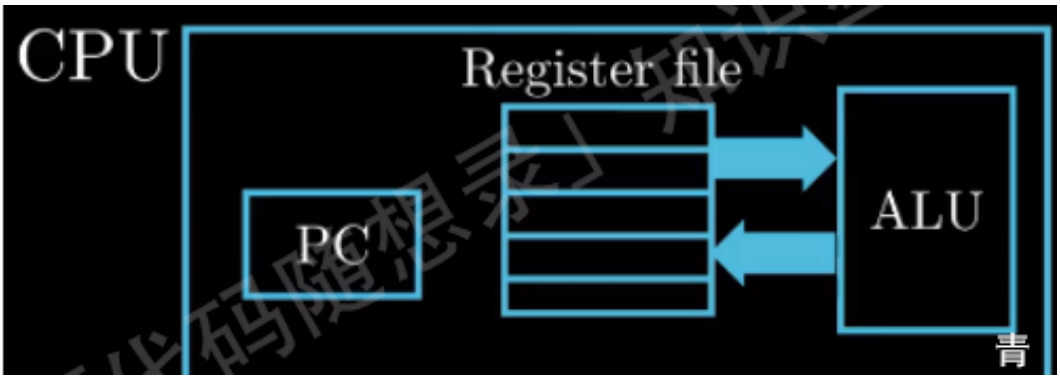
2、理解编译系统可以帮助我们理解链接过程中出现的错误

例如：静态变量和全局变量的区别是什么？静态库和动态库的区别是什么？

3、避免安全漏洞

例如：缓冲区溢出（buffer overflow）是导致互联网安全漏洞的主要原因，如何避免写出的代码存在安全漏洞，第一步就是要理解数据和控制信息在程序栈上是如何存储的，了解不严谨不规范的书写方式会引起什么样的后果

系统的硬件组成



PC 实质上是一个大小为一个字的存储区域。对于32位的机器，一个字是4个字节；对于64位的机器，一I/O设备个字就是8个字节。说白了，PC就是一个4字节或是8字节的存储空间，里面存放的是某一条指令的地址。

从系统上电的那一瞬间，直到系统断电，处理器就不断地在执行 PC 指向的指令，然后更新 PC，使其指向下一条要执行的指令（注意：这个下一条指令与刚刚执行的指令不一定是相邻的）

通俗的讲，寄存器可以理解为一个临时存放数据的空间。

例如我们计算两个变量 a+b 的和，处理器从内存中读取 a 的值暂存在寄存器 X 中，读取 B 的值暂存在寄存器 Y 中，这个操作会覆盖寄存器中原来的数值，处理器完成加载的操作后，ALU（Arithmatic/logic Unit）会从复制寄存器 X 和 Y 中保存的数值，然后进行算术运算，得到的结果会保存到寄存器 X 或者寄存器 Y 中，此时寄存器中原来的数值会被新的数值覆盖

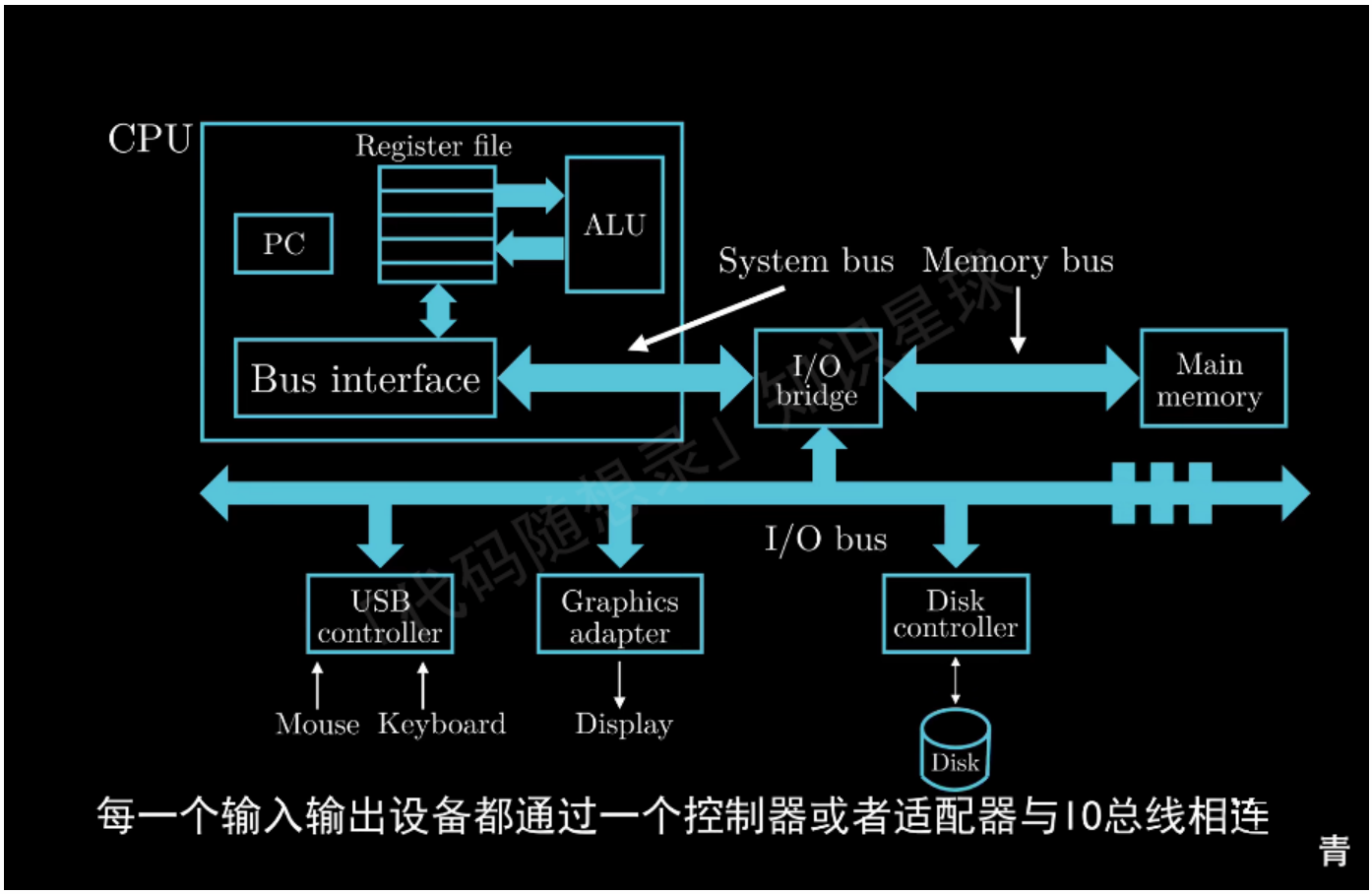
1、内存

处理器在执行程序时，内存主要存放程序指令以及数据

内存和处理器之间通过总线来进行数据传递。实际上，总线贯穿了整个计算机系统，它负责将信息从一个部件传递到另外一个部件。

2、I/O设备

除了处理器，内存以及总线，计算机系统还包含各种输入输出设备，如键盘、鼠标、显示器以及磁盘等等。



控制器与适配器主要区别是在于它们的封装方式，无论是控制器还是适配器，它们的功能都是在 IO 设备与 IO 总线之间传递数据

处理器解释指令

处理器读并解释储存在内存中的指令

运行hello程序

hello.c 经过编译系统得到可执行目标文件 hello，此时可执行目标文件 hello 已经存放在系统的磁盘上，那么，如何运行这个可执行文件呢?

在 linux 系统上运行可执行程序：打开一个 shell 程序，然后在 shell 中输入相应可执行程序的文件名。

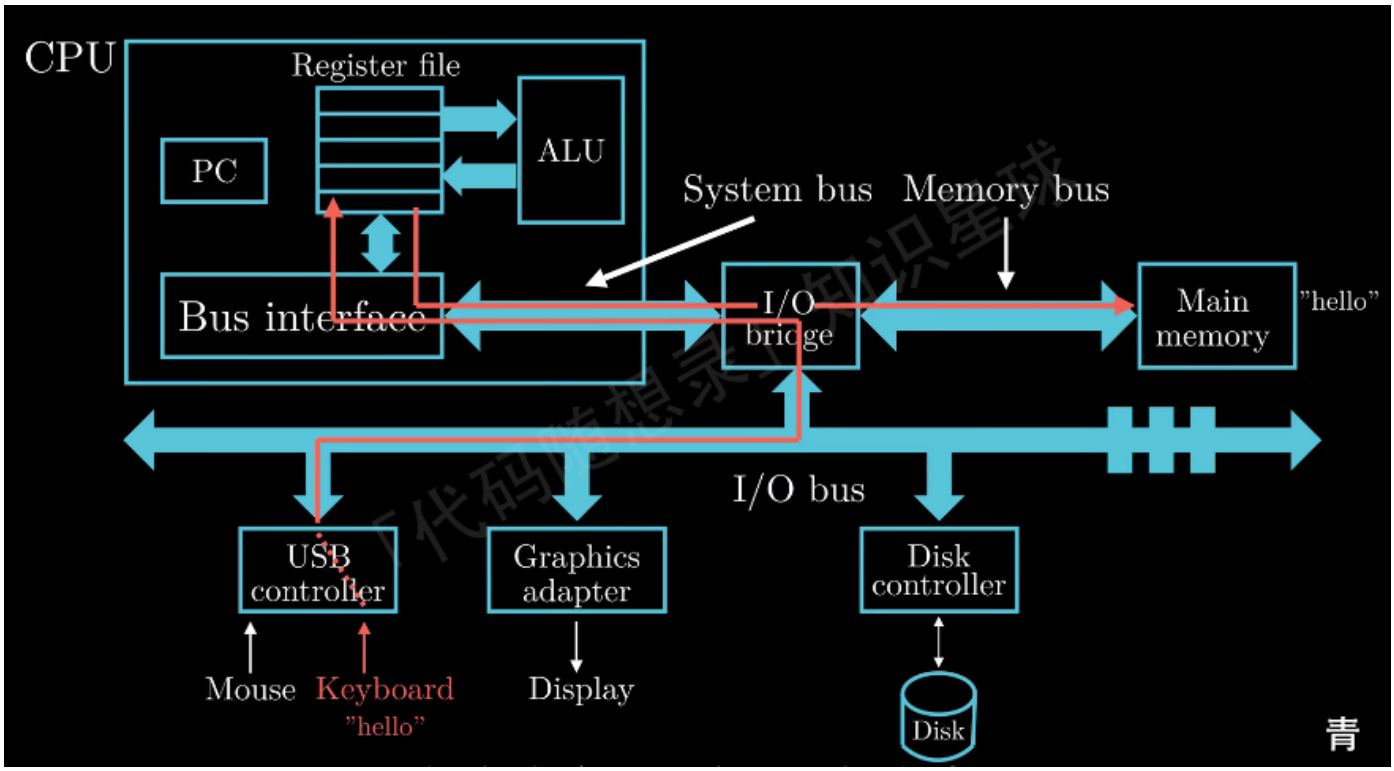
```
linux> ./hello
```

shell 是一个命令解释程序，它输出一个提示符 > 来等待一个命令行的输入，然后执行这个命令，如果该命令行的第一个单词不是内置的 shell 命令，那么 shell 就会假设这是一个可执行文件的名字，对这个文件进行加载并运行。

在这个例子中，shell 加载并且运行 hello 程序，屏幕上显示 hello,world 内容，hello 程序运行结束并退出，shell 继续等待下一个命令的输入。

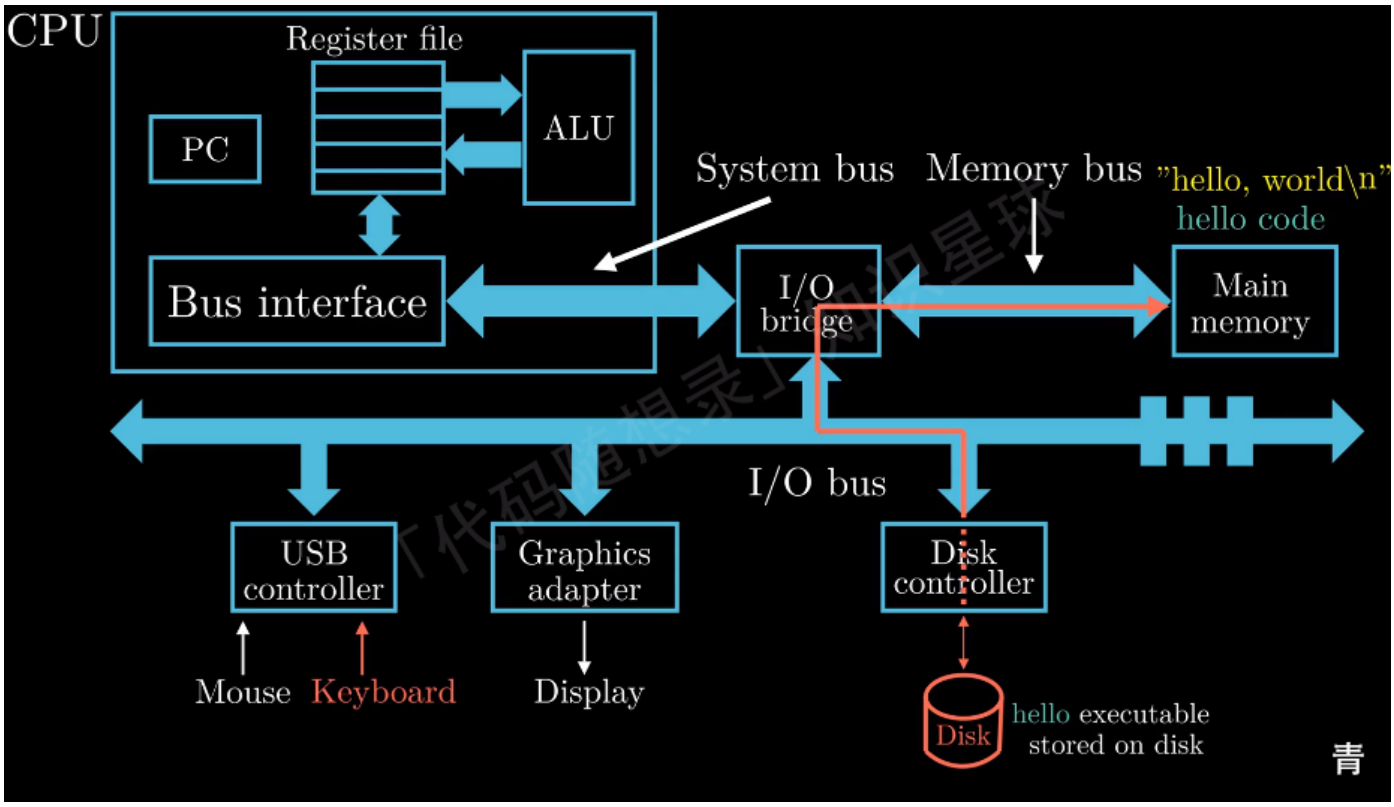
程序执行流程

首先我们通过键盘输入"./hello" 的字符串，shell 程序会将输入的字符逐一读入寄存器，处理器会把 hello 这个字符串放入内存中。

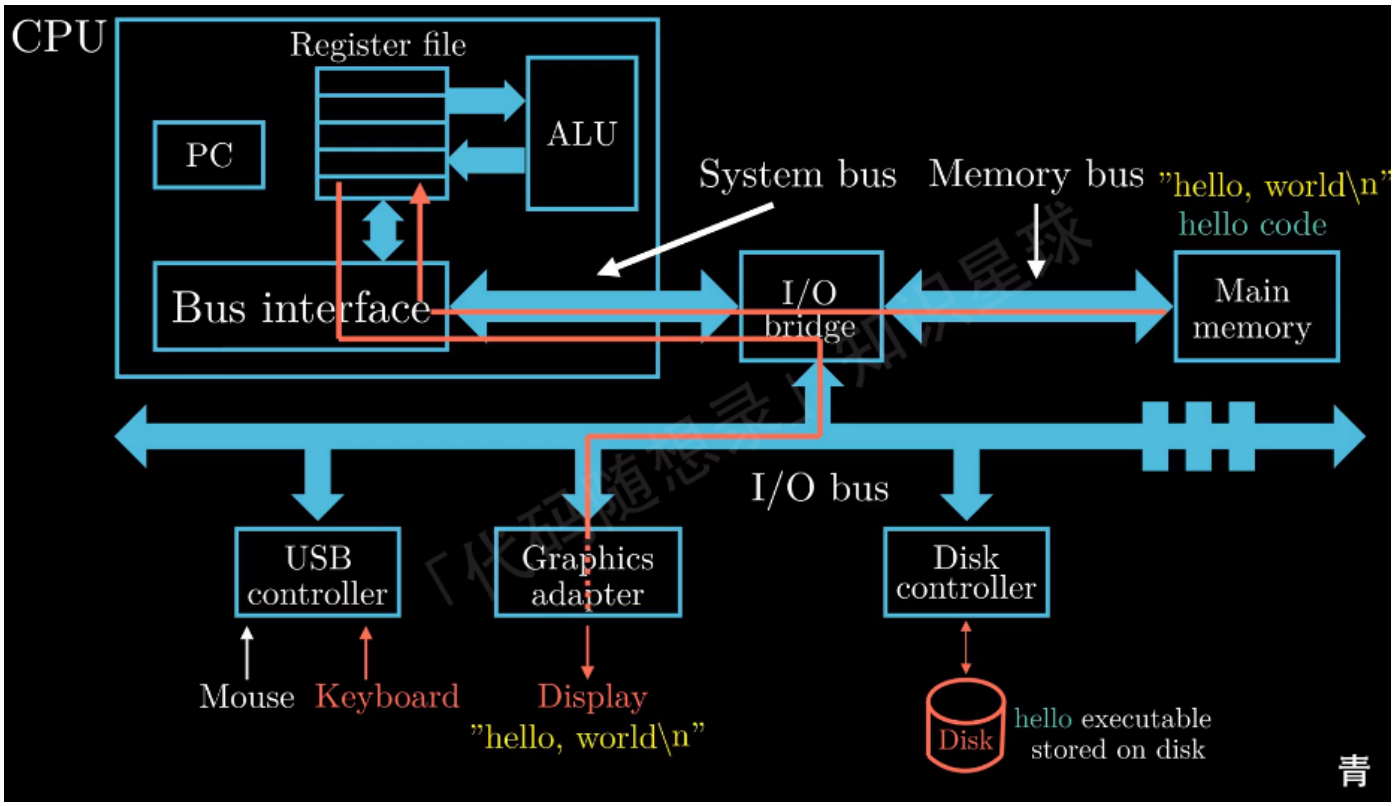


当我们完成输入，按下回车键时，shell 程序就知道我们已经完成了命令的输入，然后执行一系列的指令来来加载可执行文件 hello ，这些指令将 hello 中的数据和代码从磁盘复制到内存。

数据就是我们要显示输出的 "hello , world\n" ，这个复制的过程将利用DMA（Direct Memory Access）技术，数据可以不经过处理器，从磁盘直接到达内存。当可执行文件 hello 中的代码和数据被加载到内存中，处理器就开始执行 main 函数中的代码，main 函数非常简单，只有一个打印功能。



CPU会将 "hello , world\n" 这个字符串从内存复制到寄存器文件。然后再从寄存器文件复制到显示设备，最终hello , world显示在屏幕上。



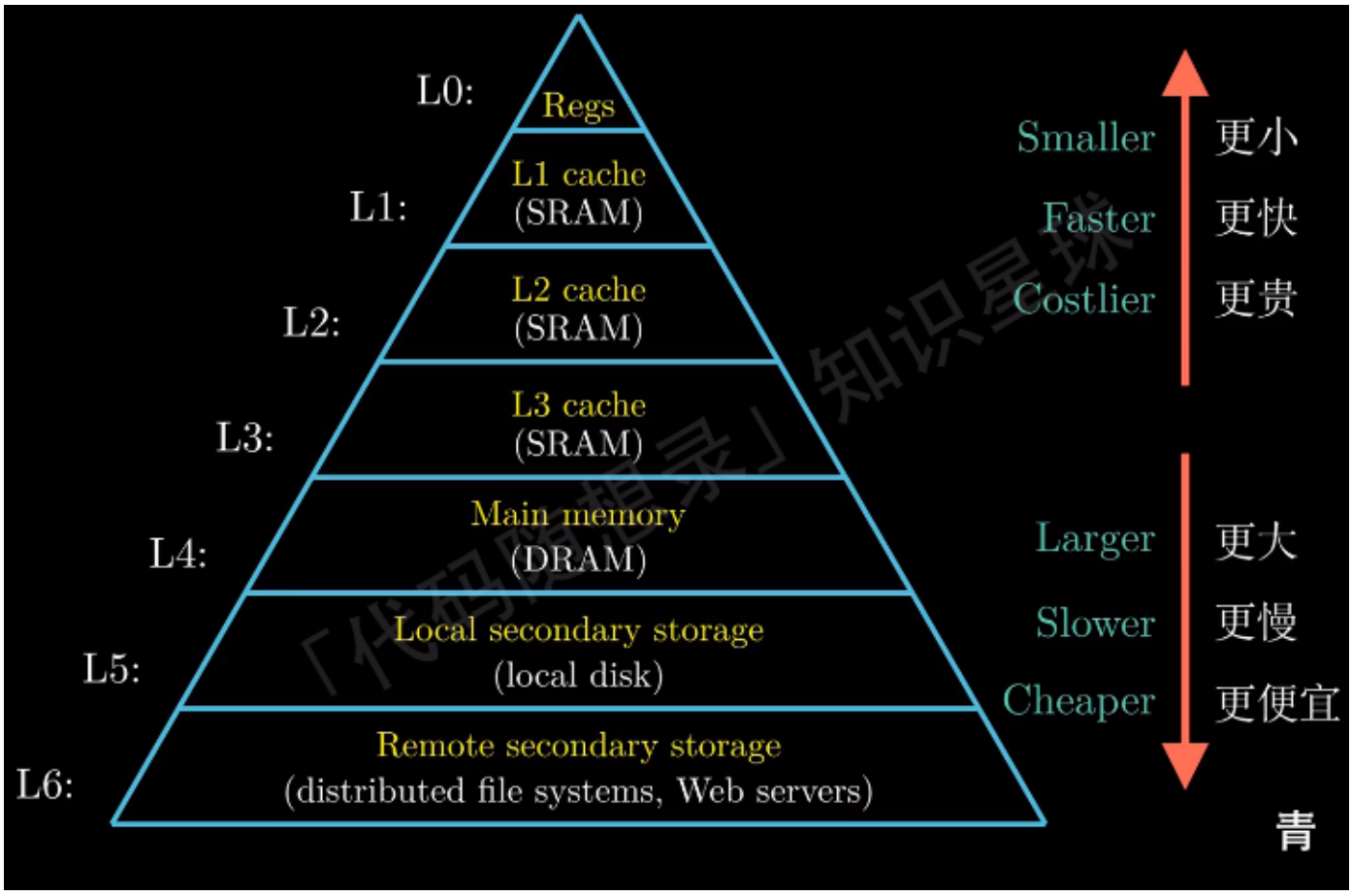
从 hello 程序执行的过程来看，系统即使执行如此简单的程序，数据信息仍旧需要在磁盘、内存、处理器以及 IO 设备之间进行搬运。

数据从一个地方搬运到另外一个地方需要花费时间，系统设计人员的一个主要任务就是缩短信息搬运所花费的时间。

存储设备

高速缓存

整个计算机系统的信息存储可以用一个层次结构来表示。

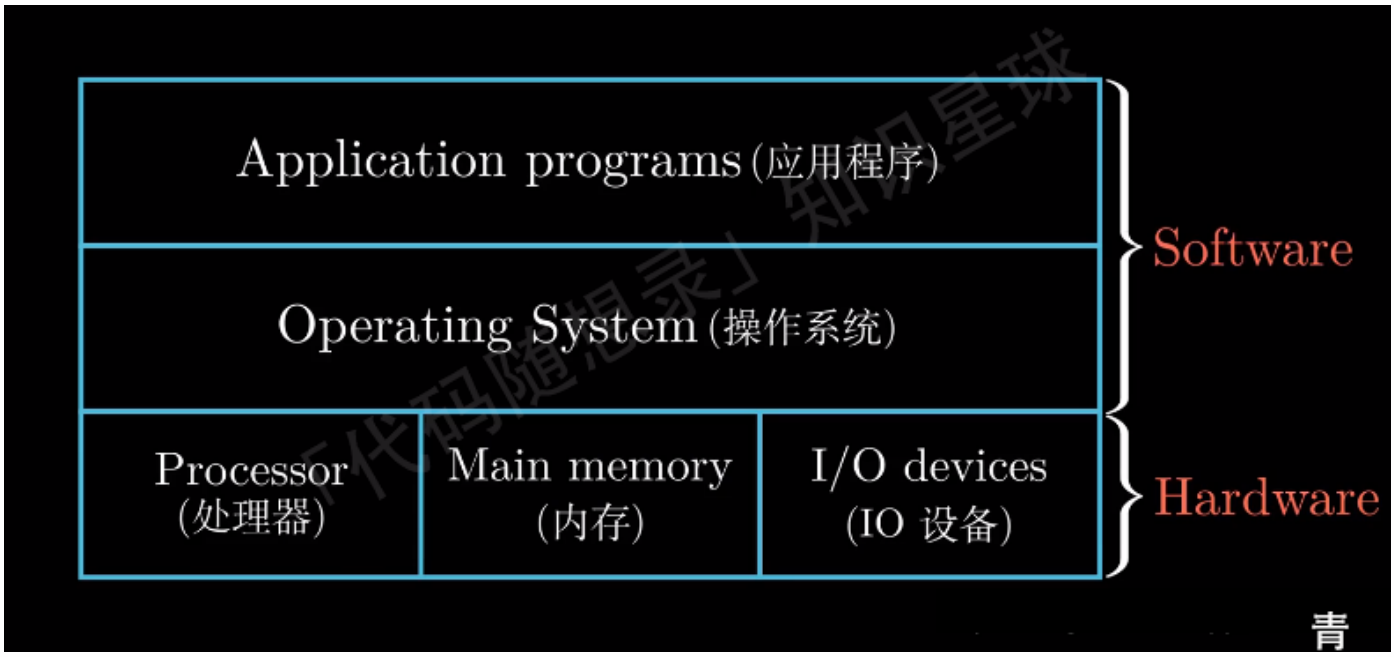


程序员深入理解计算机系统中的高速缓存，可以利用存储器的层次结构来提升程序的性能

操作系统管理硬件

操作系统的作用

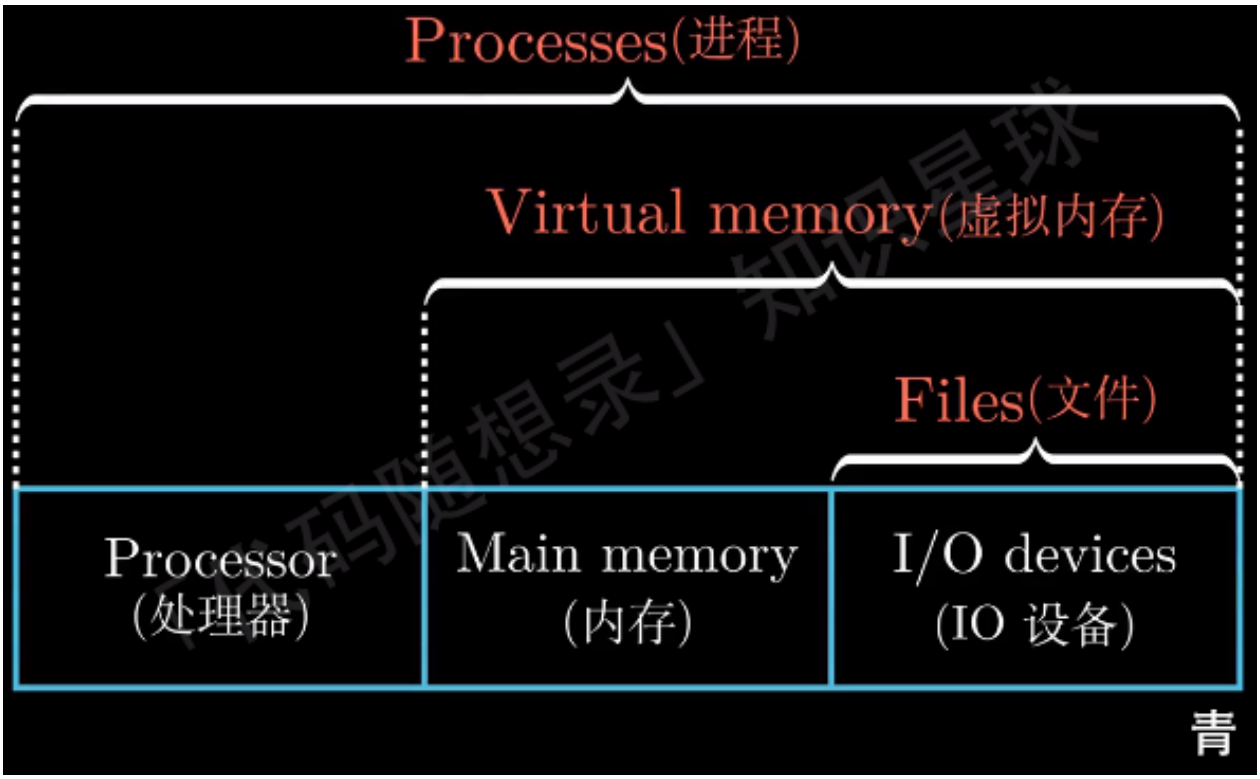
无论是 shell 程序还是 hello 程序都没有直接访问键盘、显示器、磁盘这些硬件设备，**真正操作硬件的是操作系统**，我们可以把操作系统看成是应用程序和硬件之间的中间层，所有的应用程序对硬件的操作必须通过操作系统来完成。



这样设计的目的主要有两个：一是防止硬件被失控的应用程序滥用；另外一个目的就是操作系统提供统一的机制来控制这些复杂的底层硬件。

为了实现上述的功能，操作系统引人了几个抽象的概念。

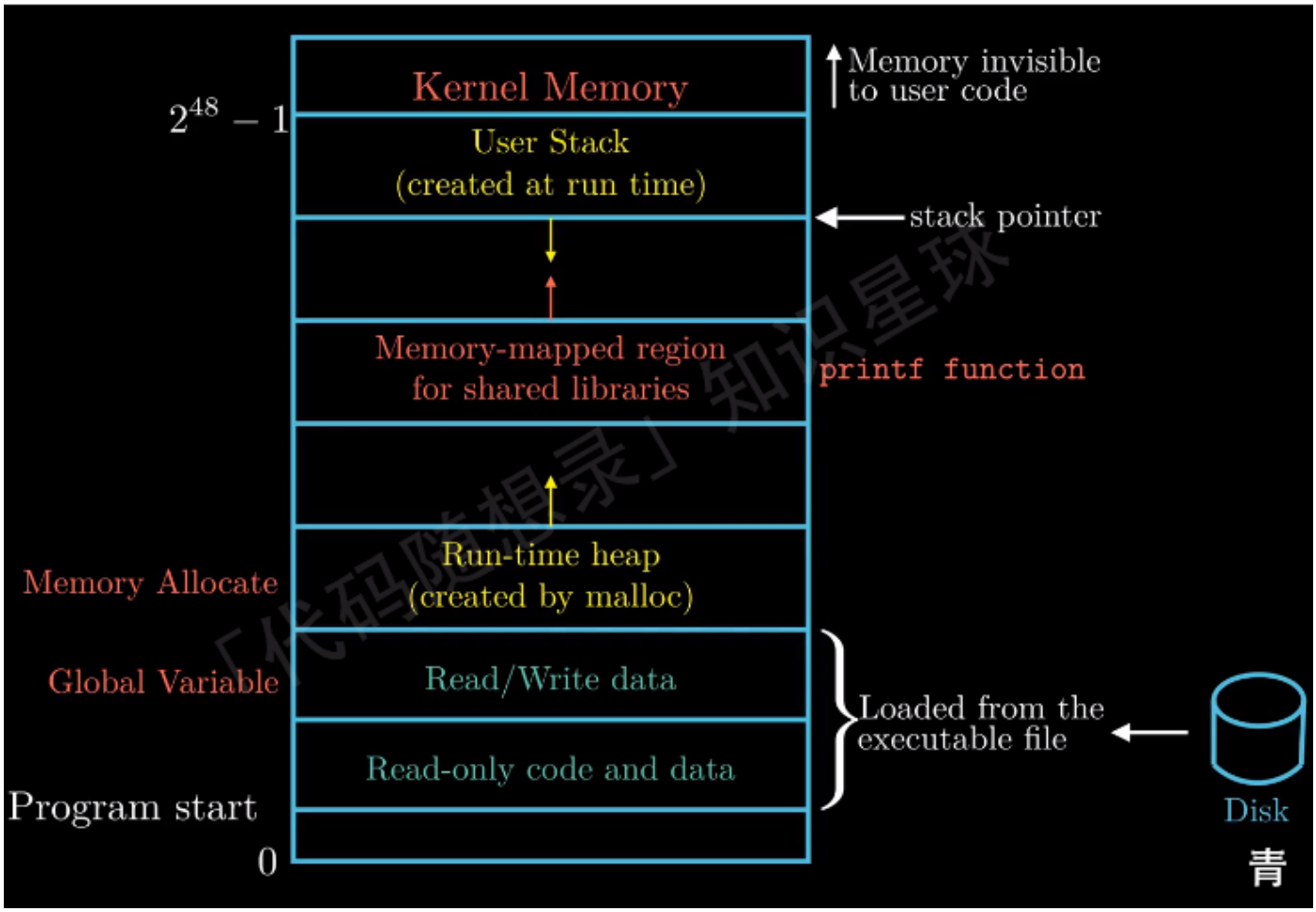
例如： 文件是对 IO 设备的抽象；虚拟内存是对内存和磁盘 IO 的抽象；进程是对处理器、内存以及 IO 设备的抽象。



进程

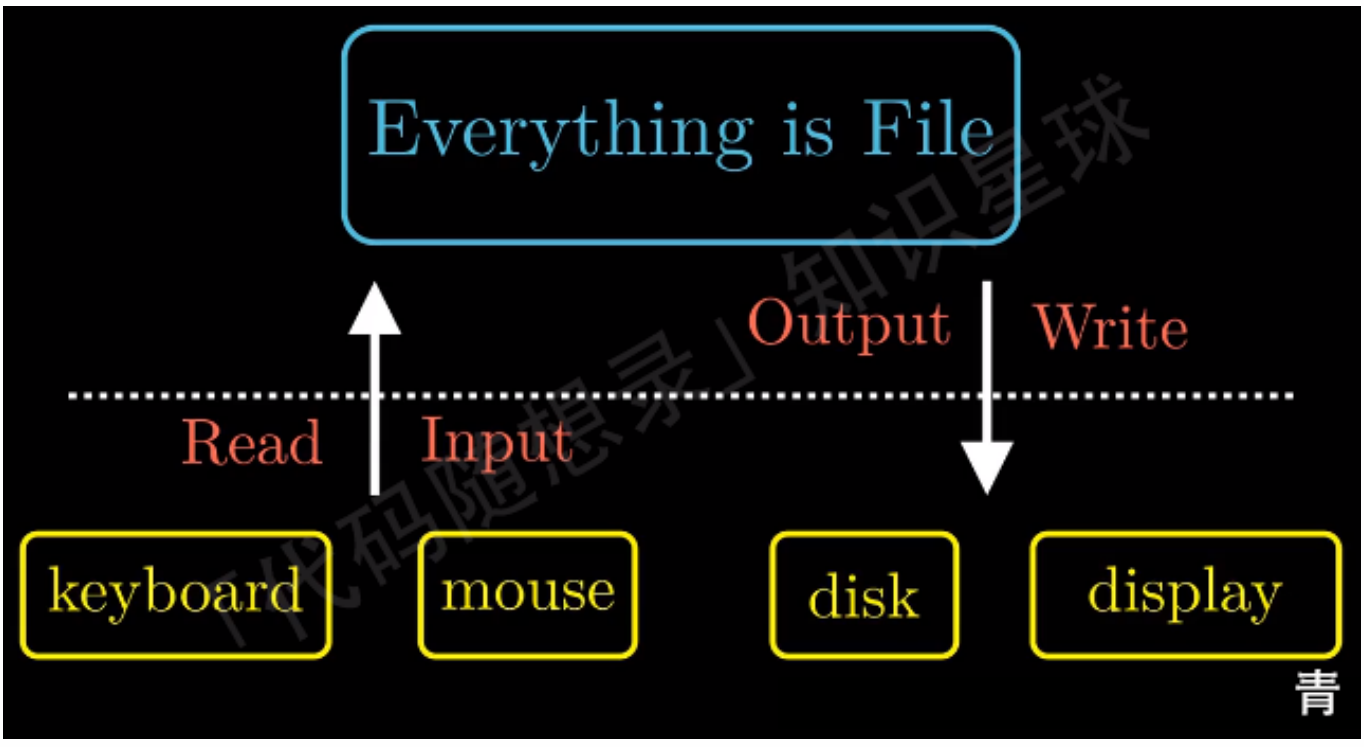
1、虚拟内存

操作系统为每个进程提供了一个假象，就是每个进程都在独自占用整个内存空间，每个进程看到的内存都是一样的，我们称之为虚拟地址空间。



2、文件

Linux 系统的哲学思想是：**一切皆为文件**。

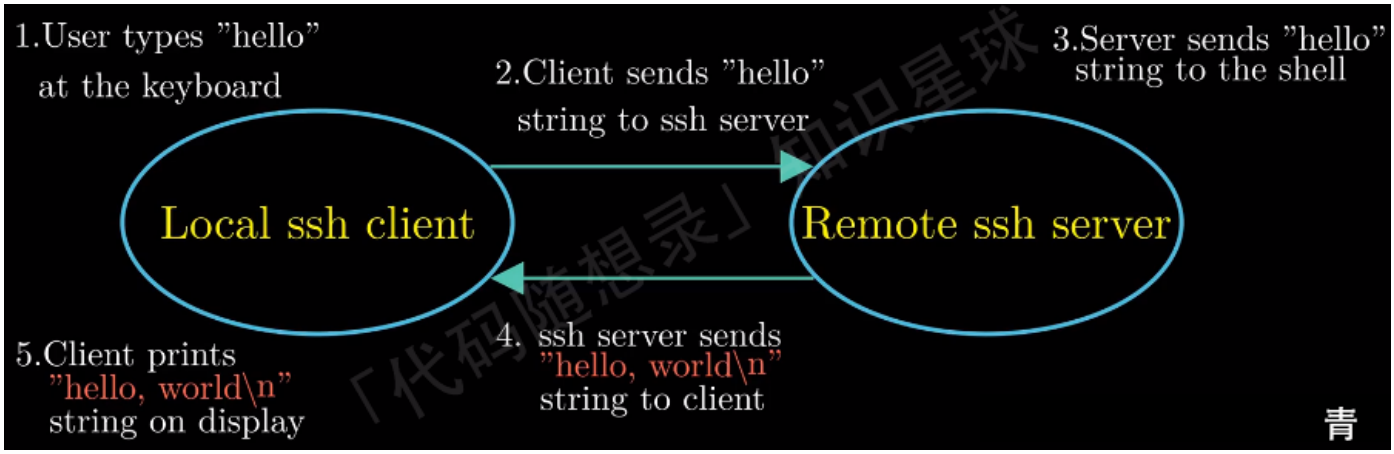


所有的 IO 设备，包括键盘，磁盘，显示器，甚至网络，这些都可以看成文件，系统中所有的输人和输出都可以通过读写文件来完成。

虽然文件的概念非常简单，但却非常强大。例如：当程序员需要处理读写磁盘上的文件时，他们不需要了解具体的磁盘技术，同一个程序，可以在不同磁盘技术上的不同系统上运行（存疑）

系统之间的网络通信

系统来看，网络也可以视为一个 IO 设备。



系统加速

加速比

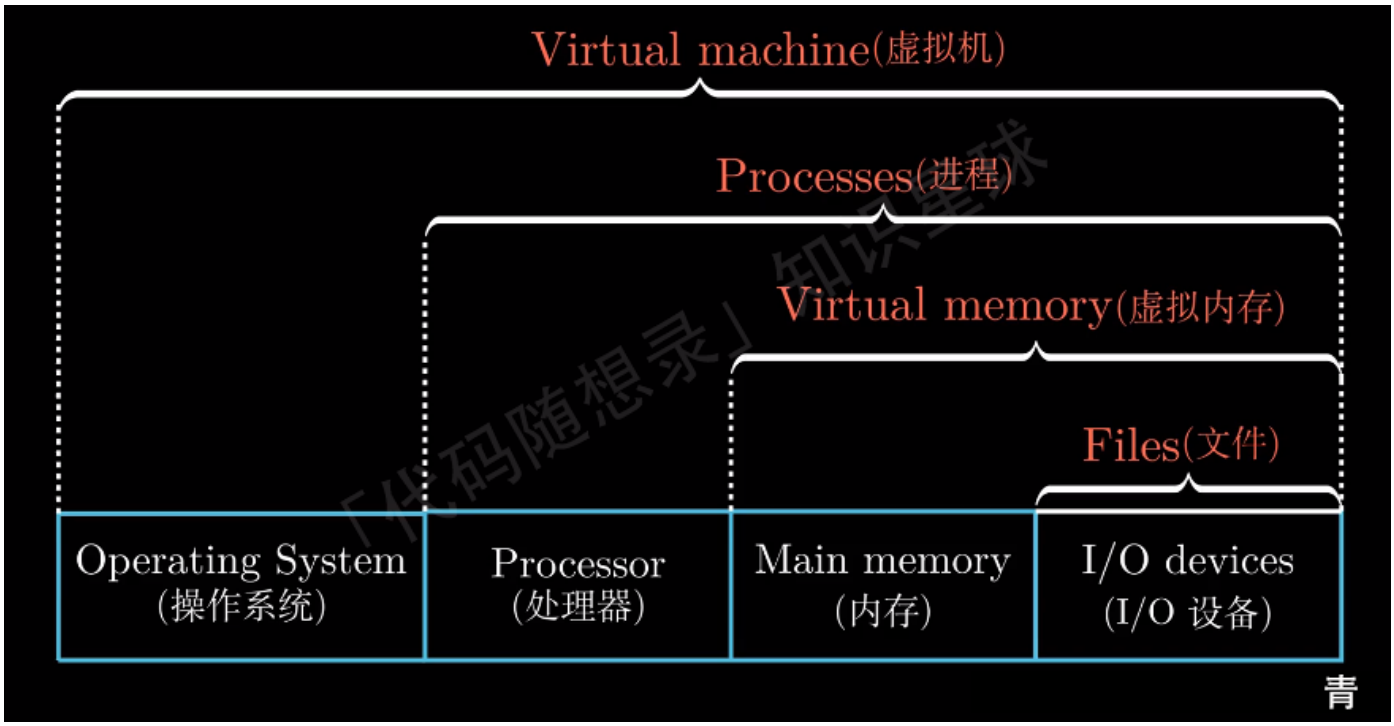
在计算机的世界里，如果我们需要把系统的性能提高2倍或者更多，只有通过优化大部分的组件才能获得。

并发和并行——存疑

如何获得更高的计算能力呢？可以通过以下三种途径：

1. 线程级并发
2. 指令级并行
3. 单指令多数据并行

虚拟机



什么是操作系统

1、学习操作系统的不同层次

（1）初级：了解操作系统提供给应用软件的接口

1. 使用显示器：printf；
2. 使用CPU：fork；
3. 使用文件：open、read
4. 如果只是看书上讲的原理，也只能算初级阶段

（2）中级：从应用软件出发进入操作系统，了解操作系统的原理

1. 例如一段文字是如何写入磁盘的，操作系统级别的代码是如何实现的？
2. 能够改动操作系统、扩展操作系统的功能，例如给一个硬件设备开发驱动程序

（3）高级：给一个硬件板子，设计并实现操作系统

2、操作系统接口

操作系统接口的概念，有3种：

命令行、图形按钮、应用程序

命令行的接口本质是什么？

就是一段程序

程序编译后变成可执行程序，就可以在命令行以命令的方式调用，这些程序中包含一些语句，就是对操作系统接口的调用

一般在系统初始化代码的最后会启动一个shell程序

shell 本身也是一段程序，它的代码很简单，就是一个死循环，不断的读入命令并执行，后面讲完进程管理可以来看这段代码的细节

其中 fork 和 exec 是真正的操作系统接口，可以实现进程管理，调用了 CPU

scanf 也是真正的操作系统接口，可以实现从键盘读入信息，调用了键盘输入

另外 printf 也是，可以调用显示器

```
int main(int argc, char * argv[])
{  char cmd[20];
    while(1)  { scanf("%s", cmd);
            if(!fork()) {exec(cmd);}
            else {wait();}  }  //while(1)
}
```

图形按钮又是怎么回事？

图形按钮基于一套消息机制

当鼠标点击、键盘按下后，通过中断，这一事件被放到消息队列中，操作系统的内核中，有一个应用程序会不断的从这个消息队列中取出并处理这些事件，处理事件的代码中，就有对操作系统接口的调用

总结接口概念，并引出系统调用的概念，操作系统接口就是系统调用

linux内核是用汇编代码和C语言写的，所以通常情况下，系统调用的接口就是C函数头

系统调用有哪些?

不需要记住，要知道从哪里去查这些系统调用，例如POSIX手册

系统调用接口需要有统一的规范，以适配不同的实现，例如POSIX：Portable Operating System Interface of Unix（IEEE制定的一个标准族）